

[Text to speech pricing](#) 

Help and feedback



REFERENCE

[Support and help options](#)

What is text to speech?

08/07/2025

In this overview, you learn about the benefits and capabilities of the text to speech feature of the Speech service, which is part of Azure AI services.

Text to speech enables your applications, tools, or devices to convert text into human like synthesized speech. The text to speech capability is also known as speech synthesis. Use human like standard voices out of the box, or create a custom voice that's unique to your product or brand. For a full list of supported voices, languages, and locales, see [Language and voice support for the Speech service](#).

Core features

Text to speech includes the following features:

 Expand table

Feature	Summary	Demo
Standard voice (called <i>Neural</i> on the pricing page)	Highly natural out-of-the-box voices. Create an Azure subscription and Speech resource, and then use the Speech SDK or visit the Speech Studio portal and select standard voices to get started. Check the pricing details .	Check the Voice Gallery and determine the right voice for your business needs.
Custom voice	Easy-to-use self-service for creating a natural brand voice, with limited access for responsible use. Create an Azure subscription and Azure AI Foundry resource and then apply to use custom voice . After you're granted access, go to the professional voice fine-tuning documentation to get started. Check the pricing details .	Check the voice samples .

More about neural text to speech features

Text to speech uses deep neural networks to make the voices of computers nearly indistinguishable from the recordings of people. With the clear articulation of words, neural text to speech significantly reduces listening fatigue when users interact with AI systems.

The patterns of stress and intonation in spoken language are called *prosody*. Traditional text to speech systems break down prosody into separate linguistic analysis and acoustic prediction steps governed by independent models. That can result in muffled, buzzy voice synthesis.

Here's more information about neural text to speech features in the Speech service, and how they overcome the limits of traditional text to speech systems:

- **Real-time speech synthesis:** Use the [Speech SDK](#) or [REST API](#) to convert text to speech by using [standard voices](#) or [custom voices](#).
- **Asynchronous synthesis of long audio:** Use the [batch synthesis API](#) to asynchronously synthesize text to speech files longer than 10 minutes (for example, audio books or lectures). Unlike synthesis performed via the Speech SDK or Speech to text REST API, responses aren't returned in real-time. The expectation is that requests are sent asynchronously, responses are polled for, and synthesized audio is downloaded when the service makes it available.
- **Standard voices:** Azure AI Speech uses deep neural networks to overcome the limits of traditional speech synthesis regarding stress and intonation in spoken language. Prosody prediction and voice synthesis happen simultaneously, which results in more fluid and natural-sounding outputs. Each standard voice model is available at 24 kHz and high-fidelity 48 kHz. You can use neural voices to:
 - Make interactions with chatbots and voice assistants more natural and engaging.
 - Convert digital texts such as e-books into audiobooks.
 - Enhance in-car navigation systems.

For a full list of standard Azure AI Speech neural voices, see [Language and voice support for the Speech service](#).

- **Improve text to speech output with SSML:** Speech Synthesis Markup Language (SSML) is an XML-based markup language used to customize text to speech outputs. With SSML, you can adjust pitch, add pauses, improve pronunciation, change speaking rate, adjust volume, and attribute multiple voices to a single document.

You can use SSML to define your own lexicons or switch to different speaking styles. With the [multilingual voices](#), you can also adjust the speaking languages via SSML. To improve the voice output for your scenario, see [Improve synthesis with Speech Synthesis Markup Language](#) and [Speech synthesis with the Audio Content Creation tool](#).

- **Visemes:** [Visemes](#) are the key poses in observed speech, including the position of the lips, jaw, and tongue in producing a particular phoneme. Visemes have a strong correlation with voices and phonemes.

By using viseme events in Speech SDK, you can generate facial animation data. This data can be used to animate faces in lip-reading communication, education, entertainment, and customer service. Viseme is currently supported only for the `en-US` (US English) [neural voices](#).

ⓘ Note

In addition to Azure AI Speech neural (non HD) voices, you can also use [Azure AI Speech high definition \(HD\) voices](#) and [Azure OpenAI neural \(HD and non HD\) voices](#). The HD voices provide a higher quality for more versatile scenarios.

Some voices don't support all [Speech Synthesis Markup Language \(SSML\)](#) tags. This includes neural text to speech HD voices, personal voices, and embedded voices.

- For Azure AI Speech high definition (HD) voices, check the SSML support [here](#).
- For personal voice, you can find the SSML support [here](#).
- For embedded voices, check the SSML support [here](#).

Get started

To get started with text to speech, see the [quickstart](#). Text to speech is available via the [Speech SDK](#), the [REST API](#), and the [Speech CLI](#).

💡 Tip

To convert text to speech with a no-code approach, try the [Audio Content Creation](#) tool in [Speech Studio](#) [↗](#).

Sample code

Sample code for text to speech is available on GitHub. These samples cover text to speech conversion in most popular programming languages:

- [Text to speech samples \(SDK\)](#) [↗](#)
- [Text to speech samples \(REST\)](#) [↗](#)

Custom voice

In addition to standard voices, you can create custom voices that are unique to your product or brand. Custom voice is an umbrella term that includes professional voice fine-tuning and personal voice. All it takes to get started is a handful of audio files and the associated transcriptions. For more information, see the [professional voice fine-tuning documentation](#).

Pricing note

Billable characters

When you use the text to speech feature, you're billed for each character that's converted to speech, including punctuation. Although the SSML document itself isn't billable, optional elements that are used to adjust how the text is converted to speech, like phonemes and pitch, are counted as billable characters. Here's a list of what's billable:

- Text passed to the text to speech feature in the SSML body of the request
- All markup within the text field of the request body in the SSML format, except for `<speak>` and `<voice>` tags
- Letters, punctuation, spaces, tabs, markup, and all white-space characters
- Every code point defined in Unicode

For detailed information, see [Speech service pricing](#).

Important

Each Chinese character is counted as two characters for billing, including kanji used in Japanese, hanja used in Korean, or hanzi used in other languages.

Model training and hosting time for custom voice

Custom voice training and hosting are both calculated by hour and billed per second. For the billing unit price, see [Speech service pricing](#).

Professional voice fine-tuning time is measured by "compute hour" (a unit to measure machine running time). Typically, when training a voice model, two computing tasks are running in parallel. So, the calculated compute hours are longer than the actual training time. For professional voice fine-tuning, it usually takes 20 to 40 compute hours to train a single-style voice, and around 90 compute hours to train a multi-style voice. The professional voice fine-tuning time is billed with a cap of 96 compute hours. So in the case that a voice model is trained in 98 compute hours, you'll only be charged with 96 compute hours.

Custom voice endpoint hosting is measured by the actual time (hour). The hosting time (hours) for each endpoint is calculated at 00:00 UTC every day for the previous 24 hours. For example, if the endpoint has been active for 24 hours on day one, it's billed for 24 hours at 00:00 UTC the second day. If the endpoint is newly created or suspended during the day, it's billed for its accumulated running time until 00:00 UTC the second day. If the endpoint isn't currently hosted, it isn't billed. In addition to the daily calculation at 00:00 UTC each day, the billing is

also triggered immediately when an endpoint is deleted or suspended. For example, for an endpoint created at 08:00 UTC on December 1, the hosting hour will be calculated to 16 hours at 00:00 UTC on December 2 and 24 hours at 00:00 UTC on December 3. If the user suspends hosting the endpoint at 16:30 UTC on December 3, the duration (16.5 hours) from 00:00 to 16:30 UTC on December 3 will be calculated for billing.

Personal voice

When you use the personal voice feature, you're billed for both profile storage and synthesis.

- **Profile storage:** After a personal voice profile is created, it will be billed until it's removed from the system. The billing unit is per voice per day. If voice storage lasts for less than 24 hours, it's still billed as one full day.
- **Synthesis:** Billed per character. For details on billable characters, see the above [billable characters](#).

Text to speech avatar

When you use the text-to-speech avatar feature, charges are billed per second based on the length of video output. However, for the real-time avatar, charges are billed per second based on the time when the avatar is active, regardless of whether it's speaking or remaining silent. To optimize costs for real-time avatar usage, refer to the "Use Local Video for Idle" tips provided in the [avatar chat sample code](#) [↗](#).

Custom text to speech avatar training time is measured by "compute hour" (machine running time) and billed per second. Training duration varies depending on how much data you use. It normally takes 20-40 compute hours on average to train a custom avatar. The avatar training time is billed with a cap of 96 compute hours. So in the case that an avatar model is trained in 98 compute hours, you're only charged for 96 compute hours.

Avatar hosting is billed per second per endpoint. You can suspend your endpoint to save costs. If you want to suspend your endpoint, you can delete it directly. To use it again, redeploy the endpoint.

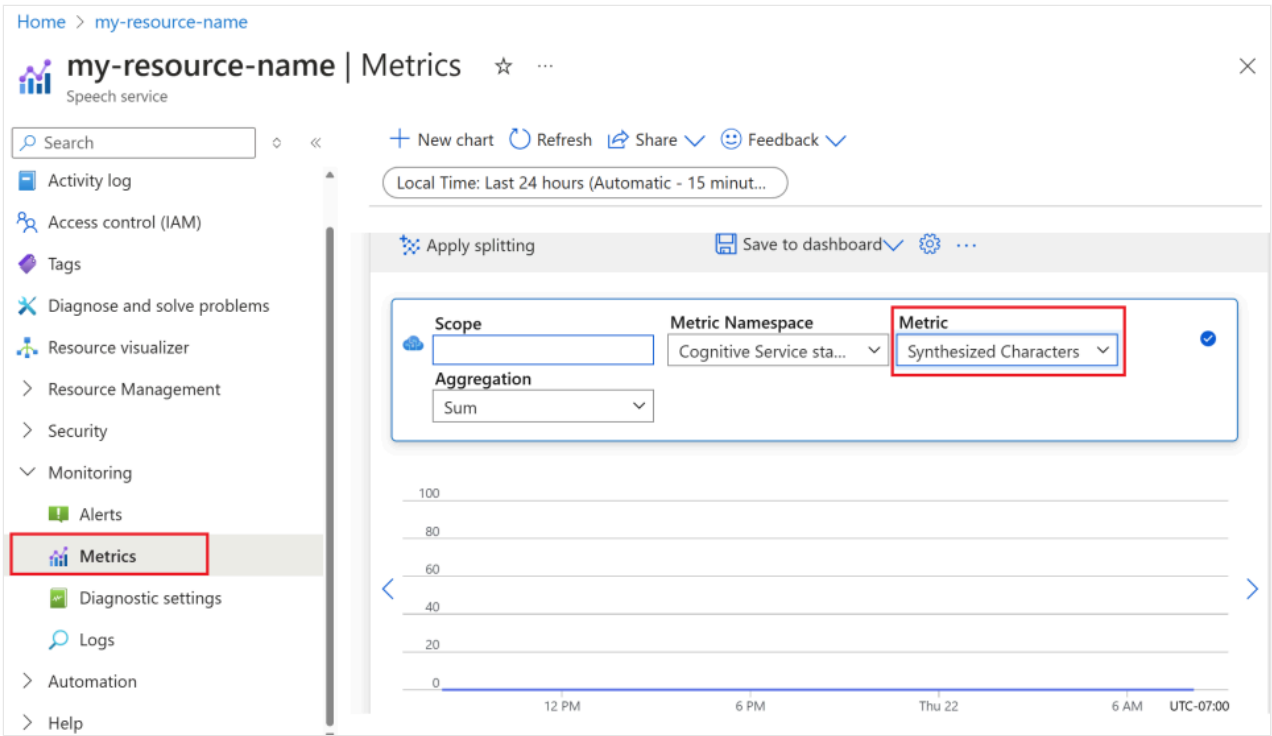
Monitor Azure text to speech metrics

Monitoring key metrics associated with text to speech services is crucial for managing resource usage and controlling costs. This section guides you on how to find usage information in the Azure portal and provide detailed definitions of the key metrics. For more information on Azure monitor metrics, see [Azure Monitor Metrics overview](#).

How to find usage information in the Azure portal

To effectively manage your Azure resources, it's essential to access and review usage information regularly. Here's how to find the usage information:

- 1. Go to the [Azure portal](#) and sign in with your Azure account.
- 2. Navigate to **Resources** and select your resource you wish to monitor.
- 3. Select **Metrics** under **Monitoring** from the left-hand menu.



- 4. Customize metric views.

You can filter data by resource type, metric type, time range, and other parameters to create custom views that align with your monitoring needs. Additionally, you can save the metric view to dashboards by selecting **Save to dashboard** for easy access to frequently used metrics.

- 5. Set up alerts.

To manage usage more effectively, set up alerts by navigating to the **Alerts** tab under **Monitoring** from the left-hand menu. Alerts can notify you when your usage reaches specific thresholds, helping to prevent unexpected costs.

Definition of metrics

Here's a table summarizing the key metrics for Azure text to speech.

Metric name	Description
Synthesized Characters	Tracks the number of characters converted into speech, including standard voice and custom voice. For details on billable characters, see Billable characters .
Video Seconds Synthesized	Measures the total duration of video synthesized, including batch avatar synthesis, real-time avatar synthesis, and custom avatar synthesis.
Avatar Model Hosting Seconds	Tracks the total time in seconds that your custom avatar model is hosted.
Voice Model Hosting Hours	Tracks the total time in hours that your custom voice model is hosted.
Voice Model Training Minutes	Measures the total time in minutes for training your custom voice model.

Reference docs

- [Speech SDK](#)
- [REST API: Text to speech](#)

Responsible AI

An AI system includes not only the technology, but also the people who use it, the people who are affected by it, and the environment in which it's deployed. Read the transparency notes to learn about responsible AI use and deployment in your systems.

- [Transparency note and use cases for custom voice](#)
- [Characteristics and limitations for using custom voice](#)
- [Limited access to custom voice](#)
- [Guidelines for responsible deployment of synthetic voice technology](#)
- [Disclosure for voice talent](#)
- [Disclosure design guidelines](#)
- [Disclosure design patterns](#)
- [Code of Conduct for Text to speech integrations](#)
- [Data, privacy, and security for custom voice](#)

Next steps

- [Text to speech quickstart](#)

- [Get the Speech SDK](#)

Quickstart: Convert text to speech

07/17/2025

[Reference documentation](#) | [Package \(NuGet\)](#) | [Additional samples on GitHub](#)

With Azure AI Speech, you can run an application that synthesizes a human-like voice to read text. You can change the voice, enter text to be spoken, and listen to the output on your computer's speaker.

Tip

You can try text to speech in the [Speech Studio Voice Gallery](#) without signing up or writing any code.

Tip

Try out the [Azure AI Speech Toolkit](#) to easily build and run samples on Visual Studio Code.

Prerequisites

- ✓ An Azure subscription. You can [create one for free](#).
- ✓ [Create an AI Services resource for Speech](#) in the Azure portal.
- ✓ Get the Speech resource key and endpoint. After your Speech resource is deployed, select **Go to resource** to view and manage keys.

Set up the environment

The Speech SDK is available as a [NuGet package](#) that implements .NET Standard 2.0. Install the Speech SDK later in this guide by using the console. For detailed installation instructions, see [Install the Speech SDK](#).

Set environment variables

You need to authenticate your application to access Azure AI services. This article shows you how to use environment variables to store your credentials. You can then access the environment variables from your code to authenticate your application. For production, use a more secure way to store and access your credentials.

Important

We recommend Microsoft Entra ID authentication with [managed identities for Azure resources](#) to avoid storing credentials with your applications that run in the cloud.

Use API keys with caution. Don't include the API key directly in your code, and never post it publicly. If using API keys, store them securely in Azure Key Vault, rotate the keys regularly, and restrict access to Azure Key Vault using role based access control and network access restrictions. For more information about using API keys securely in your apps, see [API keys with Azure Key Vault](#).

For more information about AI services security, see [Authenticate requests to Azure AI services](#).

To set the environment variables for your Speech resource key and endpoint, open a console window, and follow the instructions for your operating system and development environment.

- To set the `SPEECH_KEY` environment variable, replace *your-key* with one of the keys for your resource.
- To set the `ENDPOINT` environment variable, replace *your-endpoint* with one of the endpoints for your resource.

Windows

Console

```
setx SPEECH_KEY your-key  
setx ENDPOINT your-endpoint
```

Note

If you only need to access the environment variables in the current console, you can set the environment variable with `set` instead of `setx`.

After you add the environment variables, you might need to restart any programs that need to read the environment variables, including the console window. For example, if you're using Visual Studio as your editor, restart Visual Studio before you run the example.

Create the application

Follow these steps to create a console application and install the Speech SDK.

1. Open a command prompt window in the folder where you want the new project. Run this command to create a console application with the .NET CLI.

```
.NET CLI
```

```
dotnet new console
```

The command creates a *Program.cs* file in the project directory.

2. Install the Speech SDK in your new project with the .NET CLI.

```
.NET CLI
```

```
dotnet add package Microsoft.CognitiveServices.Speech
```

3. Replace the contents of *Program.cs* with the following code.

```
C#
```

```
using System;
using System.IO;
using System.Threading.Tasks;
using Microsoft.CognitiveServices.Speech;
using Microsoft.CognitiveServices.Speech.Audio;

class Program
{
    // This example requires environment variables named "SPEECH_KEY" and
    "END_POINT"
    static string speechKey =
Environment.GetEnvironmentVariable("SPEECH_KEY");
    static string endpoint = Environment.GetEnvironmentVariable("END_POINT");

    static void OutputSpeechSynthesisResult(SpeechSynthesisResult
speechSynthesisResult, string text)
    {
        switch (speechSynthesisResult.Reason)
        {
            case ResultReason.SynthesizingAudioCompleted:
                Console.WriteLine($"Speech synthesized for text: [{text}]");
                break;
            case ResultReason.Canceled:
                var cancellation =
SpeechSynthesisCancellationDetails.FromResult(speechSynthesisResult);
                Console.WriteLine($"CANCELED: Reason={cancellation.Reason}");

                if (cancellation.Reason == CancellationReason.Error)
                {
```



```

        Console.WriteLine($"CANCELED: ErrorCode=
{cancellation.ErrorCode}");
        Console.WriteLine($"CANCELED: ErrorDetails=
[{cancellation.ErrorDetails}]");
        Console.WriteLine($"CANCELED: Did you set the speech
resource key and endpoint values?");
    }
    break;
default:
    break;
}
}

async static Task Main(string[] args)
{
    var speechConfig = SpeechConfig.FromEndpoint(speechKey, endpoint);

    // The neural multilingual voice can speak different languages based
    on the input text.
    speechConfig.SpeechSynthesisVoiceName = "en-US-
AvaMultilingualNeural";

    using (var speechSynthesizer = new SpeechSynthesizer(speechConfig))
    {
        // Get text from the console and synthesize to the default
        speaker.
        Console.WriteLine("Enter some text that you want to speak >");
        string text = Console.ReadLine();

        var speechSynthesisResult = await
speechSynthesizer.SpeakTextAsync(text);
        OutputSpeechSynthesisResult(speechSynthesisResult, text);
    }

    Console.WriteLine("Press any key to exit...");
    Console.ReadKey();
}
}

```

4. To change the speech synthesis language, replace `en-US-AvaMultilingualNeural` with another [supported voice](#).

All neural voices are multilingual and fluent in their own language and English. For example, if the input text in English is *I'm excited to try text to speech* and you set `es-ES-ElviraNeural` as the language, the text is spoken in English with a Spanish accent. If the voice doesn't speak the language of the input text, the Speech service doesn't output synthesized audio.

5. Run your new console application to start speech synthesis to the default speaker.

```
dotnet run
```

Important

Make sure that you set the `SPEECH_KEY` and `END_POINT` [environment variables](#). If you don't set these variables, the sample fails with an error message.

6. Enter some text that you want to speak. For example, type *I'm excited to try text to speech*. Select the **Enter** key to hear the synthesized speech.

Console

```
Enter some text that you want to speak >  
I'm excited to try text to speech
```

Remarks

More speech synthesis options

This quickstart uses the `SpeakTextAsync` operation to synthesize a short block of text that you enter. You can also use long-form text from a file and get finer control over voice styles, prosody, and other settings.

- See [how to synthesize speech](#) and [Speech Synthesis Markup Language \(SSML\) overview](#) for information about speech synthesis from a file and finer control over voice styles, prosody, and other settings.
- See [batch synthesis API for text to speech](#) for information about synthesizing long-form text to speech.

OpenAI text to speech voices in Azure AI Speech

OpenAI text to speech voices are also supported. See [OpenAI text to speech voices in Azure AI Speech](#) and [multilingual voices](#). You can replace `en-US-AvaMultilingualNeural` with a supported OpenAI voice name such as `en-US-FableMultilingualNeural`.

Clean up resources

You can use the [Azure portal](#) or [Azure Command Line Interface \(CLI\)](#) to remove the Speech resource you created.

Next step

[Learn more about speech synthesis](#)

How to synthesize speech from text

08/07/2025

[Reference documentation](#) | [Package \(NuGet\)](#) [↗] | [Additional samples on GitHub](#) [↗]

In this how-to guide, you learn common design patterns for doing text to speech synthesis.

For more information about the following areas, see [What is text to speech?](#)

- Getting responses as in-memory streams.
- Customizing output sample rate and bit rate.
- Submitting synthesis requests by using Speech Synthesis Markup Language (SSML).
- Using neural voices.
- Subscribing to events and acting on results.

Select synthesis language and voice

The text to speech feature in the Speech service supports more than 400 voices and more than 140 languages and variants. You can get the [full list](#) or try them in the [Voice Gallery](#) [↗].

Specify the language or voice of `SpeechConfig` to match your input text and use the specified voice. The following code snippet shows how this technique works:

C#

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
        "YourSpeechRegion");
    // Set either the `SpeechSynthesisVoiceName` or `SpeechSynthesisLanguage`.
    speechConfig.SpeechSynthesisLanguage = "en-US";
    speechConfig.SpeechSynthesisVoiceName = "en-US-AvaMultilingualNeural";
}
```

All neural voices are multilingual and fluent in their own language and English. For example, if the input text in English, is "I'm excited to try text to speech," and you select `es-ES-ElviraNeural`, the text is spoken in English with a Spanish accent.

If the voice doesn't speak the language of the input text, the Speech service doesn't create synthesized audio. For a full list of supported neural voices, see [Language and voice support for the Speech service](#).

 **Note**

The default voice is the first voice returned per locale from the [Voice List API](#).

The voice that speaks is determined in order of priority as follows:

- If you don't set `SpeechSynthesisVoiceName` or `SpeechSynthesisLanguage`, the default voice for `en-US` speaks.
- If you only set `SpeechSynthesisLanguage`, the default voice for the specified locale speaks.
- If both `SpeechSynthesisVoiceName` and `SpeechSynthesisLanguage` are set, the `SpeechSynthesisLanguage` setting is ignored. The voice that you specify by using `SpeechSynthesisVoiceName` speaks.
- If the voice element is set by using [Speech Synthesis Markup Language \(SSML\)](#), the `SpeechSynthesisVoiceName` and `SpeechSynthesisLanguage` settings are ignored.

In summary, the order of priority can be described as:

[Expand table](#)

<code>SpeechSynthesisVoiceName</code>	<code>SpeechSynthesisLanguage</code>	SSML	Outcome
X	X	X	Default voice for <code>en-US</code> speaks
X	✓	X	Default voice for specified locale speaks.
✓	✓	X	The voice that you specify by using <code>SpeechSynthesisVoiceName</code> speaks.
✓	✓	✓	The voice that you specify by using SSML speaks.

Synthesize speech to a file

Create a [SpeechSynthesizer](#) object. This object shown in the following snippets runs text to speech conversions and outputs to speakers, files, or other output streams. `SpeechSynthesizer` accepts as parameters:

- The [SpeechConfig](#) object that you created in the previous step.
 - An [AudioConfig](#) object that specifies how output results should be handled.
1. Create an `AudioConfig` instance to automatically write the output to a `.wav` file by using the `FromWavFileOutput()` function. Instantiate it with a `using` statement.

```
C#
```

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
"YourSpeechRegion");
    using var audioConfig =
AudioConfig.FromWavFileOutput("path/to/write/file.wav");
}
```

A `using` statement in this context automatically disposes of unmanaged resources and causes the object to go out of scope after disposal.

2. Instantiate a `SpeechSynthesizer` instance with another `using` statement. Pass your `speechConfig` object and the `audioConfig` object as parameters. To synthesize speech and write to a file, run `SpeakTextAsync()` with a string of text.

C#

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
"YourSpeechRegion");
    using var audioConfig =
AudioConfig.FromWavFileOutput("path/to/write/file.wav");
    using var speechSynthesizer = new SpeechSynthesizer(speechConfig,
audioConfig);
    await speechSynthesizer.SpeakTextAsync("I'm excited to try text to speech");
}
```

When you run the program, it creates a synthesized `.wav` file, which is written to the location that you specify. This result is a good example of the most basic usage. Next, you can customize output and handle the output response as an in-memory stream for working with custom scenarios.

Synthesize to speaker output

To output synthesized speech to the current active output device such as a speaker, omit the `AudioConfig` parameter when you're creating the `SpeechSynthesizer` instance. Here's an example:

C#

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
"YourSpeechRegion");
```

```
using var speechSynthesizer = new SpeechSynthesizer(speechConfig);
await speechSynthesizer.SpeakTextAsync("I'm excited to try text to speech");
}
```

Get a result as an in-memory stream

You can use the resulting audio data as an in-memory stream rather than directly writing to a file. With in-memory stream, you can build custom behavior:

- Abstract the resulting byte array as a seekable stream for custom downstream services.
- Integrate the result with other APIs or services.
- Modify the audio data, write custom .wav headers, and do related tasks.

You can make this change to the previous example. First, remove the `AudioConfig` block, because you manage the output behavior manually from this point onward for increased control. Pass `null` for `AudioConfig` in the `SpeechSynthesizer` constructor.

ⓘ Note

Passing `null` for `AudioConfig`, rather than omitting it as in the previous speaker output example, doesn't play the audio by default on the current active output device.

Save the result to a `SpeechSynthesisResult` variable. The `AudioData` property contains a `byte []` instance for the output data. You can work with this `byte []` instance manually, or you can use the `AudioDataStream` class to manage the in-memory stream.

In this example, you use the `AudioDataStream.FromResult()` static function to get a stream from the result:

C#

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
        "YourSpeechRegion");
    using var speechSynthesizer = new SpeechSynthesizer(speechConfig, null);

    var result = await speechSynthesizer.SpeakTextAsync("I'm excited to try text
to speech");
    using var stream = AudioDataStream.FromResult(result);
}
```

At this point, you can implement any custom behavior by using the resulting `stream` object.

Customize audio format

You can customize audio output attributes, including:

- Audio file type
- Sample rate
- Bit depth

To change the audio format, you use the `SetSpeechSynthesisOutputFormat()` function on the `SpeechConfig` object. This function expects an `enum` instance of type [SpeechSynthesisOutputFormat](#). Use the `enum` to select the output format. For available formats, see the [list of audio formats](#).

There are various options for different file types, depending on your requirements. By definition, raw formats like `Raw24Khz16BitMonoPcm` don't include audio headers. Use raw formats only in one of these situations:

- You know that your downstream implementation can decode a raw bitstream.
- You plan to manually build headers based on factors like bit depth, sample rate, and number of channels.

This example specifies the high-fidelity RIFF format `Riff24Khz16BitMonoPcm` by setting `SpeechSynthesisOutputFormat` on the `SpeechConfig` object. Similar to the example in the previous section, you use [AudioDataStream](#) to get an in-memory stream of the result, and then write it to a file.

C#

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
        "YourSpeechRegion");

    speechConfig.SetSpeechSynthesisOutputFormat(SpeechSynthesisOutputFormat.Riff24Khz16BitMonoPcm);

    using var speechSynthesizer = new SpeechSynthesizer(speechConfig, null);
    var result = await speechSynthesizer.SpeakTextAsync("I'm excited to try text to speech");

    using var stream = AudioDataStream.FromResult(result);
    await stream.SaveToWaveFileAsync("path/to/write/file.wav");
}
```

When you run the program, it writes a `.wav` file to the specified path.

Use SSML to customize speech characteristics

You can use SSML to fine-tune the pitch, pronunciation, speaking rate, volume, and other aspects in the text to speech output by submitting your requests from an XML schema. This section shows an example of changing the voice. For more information, see [Speech Synthesis Markup Language overview](#).

To start using SSML for customization, you make a minor change that switches the voice.

1. Create a new XML file for the SSML configuration in your root project directory.

XML

```
<speak version="1.0" xmlns="https://www.w3.org/2001/10/synthesis"
xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    When you're on the freeway, it's a good idea to use a GPS.
  </voice>
</speak>
```

In this example, the file is *ssml.xml*. The root element is always `<speak>`. Wrapping the text in a `<voice>` element allows you to change the voice by using the `name` parameter. For the full list of supported neural voices, see [Supported languages](#).

2. Change the speech synthesis request to reference your XML file. The request is mostly the same, but instead of using the `SpeakTextAsync()` function, you use `SpeakSsmlAsync()`. This function expects an XML string. First, load your SSML configuration as a string by using `File.ReadAllText()`. From this point, the result object is exactly the same as previous examples.

ⓘ Note

If you're using Visual Studio, your build configuration likely won't find your XML file by default. Right-click the XML file and select **Properties**. Change **Build Action** to **Content**. Change **Copy to Output Directory** to **Copy always**.

C#

```
public static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
        "YourSpeechRegion");
    using var speechSynthesizer = new SpeechSynthesizer(speechConfig, null);

    var ssml = File.ReadAllText("./ssml.xml");
```

```
var result = await speechSynthesizer.SpeakSsmlAsync(ssml);

using var stream = AudioDataStream.FromResult(result);
await stream.SaveToWaveFileAsync("path/to/write/file.wav");
}
```

ⓘ Note

To change the voice without using SSML, you can set the property on `SpeechConfig` by using `SpeechConfig.SpeechSynthesisVoiceName = "en-US-AvaMultilingualNeural";`.

Subscribe to synthesizer events

You might want more insights about the text to speech processing and results. For example, you might want to know when the synthesizer starts and stops, or you might want to know about other events encountered during synthesis.

While using the [SpeechSynthesizer](#) for text to speech, you can subscribe to the events in this table:

 Expand table

Event	Description	Use case
<code>BookmarkReached</code>	Signals that a bookmark was reached. To trigger a bookmark reached event, a <code>bookmark</code> element is required in the SSML . This event reports the output audio's elapsed time between the beginning of synthesis and the <code>bookmark</code> element. The event's <code>Text</code> property is the string value that you set in the bookmark's <code>mark</code> attribute. The <code>bookmark</code> elements aren't spoken.	You can use the <code>bookmark</code> element to insert custom markers in SSML to get the offset of each marker in the audio stream. The <code>bookmark</code> element can be used to reference a specific location in the text or tag sequence.
<code>SynthesisCanceled</code>	Signals that the speech synthesis was canceled.	You can confirm when synthesis is canceled.
<code>SynthesisCompleted</code>	Signals that speech synthesis is complete.	You can confirm when synthesis is complete.
<code>SynthesisStarted</code>	Signals that speech synthesis started.	You can confirm when synthesis started.

Event	Description	Use case
<code>Synthesizing</code>	Signals that speech synthesis is ongoing. This event fires each time the SDK receives an audio chunk from the Speech service.	You can confirm when synthesis is in progress.
<code>VisemeReceived</code>	Signals that a viseme event was received.	Visemes are often used to represent the key poses in observed speech. Key poses include the position of the lips, jaw, and tongue in producing a particular phoneme. You can use visemes to animate the face of a character as speech audio plays.
<code>WordBoundary</code>	Signals that a word boundary was received. This event is raised at the beginning of each new spoken word, punctuation, and sentence. The event reports the current word's time offset, in ticks, from the beginning of the output audio. This event also reports the character position in the input text or SSML immediately before the word that's about to be spoken.	This event is commonly used to get relative positions of the text and corresponding audio. You might want to know about a new word, and then take action based on the timing. For example, you can get information that can help you decide when and for how long to highlight words as they're spoken.

ⓘ Note

Events are raised as the output audio data becomes available, which is faster than playback to an output device. The caller must appropriately synchronize streaming and real-time.

Here's an example that shows how to subscribe to events for speech synthesis.

ⓘ Important

Use API keys with caution. Don't include the API key directly in your code, and never post it publicly. If you use an API key, store it securely in Azure Key Vault. For more information about using API keys securely in your apps, see [API keys with Azure Key Vault](#).

For more information about AI services security, see [Authenticate requests to Azure AI services](#).

You can follow the instructions in the [quickstart](#), but replace the contents of that *Program.cs* file with the following C# code:

C#

```
using Microsoft.CognitiveServices.Speech;

class Program
{
    // This example requires environment variables named "SPEECH_KEY" and
    "SPEECH_REGION"
    static string speechKey = Environment.GetEnvironmentVariable("SPEECH_KEY");
    static string speechRegion =
Environment.GetEnvironmentVariable("SPEECH_REGION");

    async static Task Main(string[] args)
    {
        var speechConfig = SpeechConfig.FromSubscription(speechKey, speechRegion);

        var speechSynthesisVoiceName = "en-US-AvaMultilingualNeural";
        var ssm1 = @$"<speak version='1.0' xml:lang='en-US'
xmlns='http://www.w3.org/2001/10/synthesis'
xmlns:mstts='http://www.w3.org/2001/mstts'>
    <voice name='{speechSynthesisVoiceName}'>
        <mstts:viseme type='redlips_front'>
            The rainbow has seven colors: <bookmark
mark='colors_list_begin'/>Red, orange, yellow, green, blue, indigo, and violet.
<bookmark mark='colors_list_end'/>.
        </voice>
    </speak>";

        // Required for sentence-level WordBoundary events

speechConfig.SetProperty(PropertyId.SpeechServiceResponse_RequestSentenceBoundary,
"true");

        using (var speechSynthesizer = new SpeechSynthesizer(speechConfig))
        {
            // Subscribe to events

            speechSynthesizer.BookmarkReached += (s, e) =>
            {
                Console.WriteLine($"BookmarkReached event:" +
                    $"\\r\\n\\tAudioOffset: {(e.AudioOffset + 5000) / 10000}ms" +
                    $"\\r\\n\\tText: \"{e.Text}\".");
            };

            speechSynthesizer.SynthesisCanceled += (s, e) =>
            {
                Console.WriteLine("SynthesisCanceled event");
            };

            speechSynthesizer.SynthesisCompleted += (s, e) =>
            {
                Console.WriteLine($"SynthesisCompleted event:" +
                    $"\\r\\n\\tAudioData: {e.Result.AudioData.Length} bytes" +
                    $"\\r\\n\\tAudioDuration: {e.Result.AudioDuration}");
            };
        }
    }
}
```

```

};

speechSynthesizer.SynthesisStarted += (s, e) =>
{
    Console.WriteLine("SynthesisStarted event");
};

speechSynthesizer.Synthesizing += (s, e) =>
{
    Console.WriteLine($"Synthesizing event:" +
        $"{e.Result.AudioData.Length} bytes");
};

speechSynthesizer.VisemeReceived += (s, e) =>
{
    Console.WriteLine($"VisemeReceived event:" +
        $"{e.AudioOffset + 5000} / 10000ms" +
        $"{e.VisemeId}");
};

speechSynthesizer.WordBoundary += (s, e) =>
{
    Console.WriteLine($"WordBoundary event:" +
        // Word, Punctuation, or Sentence
        $"{e.BoundaryType}" +
        $"{e.AudioOffset + 5000} / 10000ms" +
        $"{e.Duration}" +
        $"{e.Text}\"" +
        $"{e.TextOffset}" +
        $"{e.WordLength}");
};

// Synthesize the SSML
Console.WriteLine($"SSML to synthesize: \r\n{ssml}");
var speechSynthesisResult = await
speechSynthesizer.SpeakSsmlAsync(ssml);

// Output the results
switch (speechSynthesisResult.Reason)
{
    case ResultReason.SynthesizingAudioCompleted:
        Console.WriteLine("SynthesizingAudioCompleted result");
        break;
    case ResultReason.Canceled:
        var cancellation =
SpeechSynthesisCancellationDetails.FromResult(speechSynthesisResult);
        Console.WriteLine($"CANCELED: Reason={cancellation.Reason}");

        if (cancellation.Reason == CancellationReason.Error)
        {
            Console.WriteLine($"CANCELED: ErrorCode=
{cancellation.ErrorCode}");
            Console.WriteLine($"CANCELED: ErrorDetails=
[{cancellation.ErrorDetails}]");
            Console.WriteLine($"CANCELED: Did you set the speech

```

```

resource key and region values?");
    }
    break;
default:
    break;
}
}

Console.WriteLine("Press any key to exit...");
Console.ReadKey();
}
}

```

You can find more text to speech samples at [GitHub](#).

Use a custom endpoint

The custom endpoint is functionally identical to the standard endpoint used for text to speech requests.

One difference is that the `EndpointId` must be specified to use your custom voice via the Speech SDK. You can start with the [text to speech quickstart](#) and then update the code with the `EndpointId` and `SpeechSynthesisVoiceName`.

C#

```

var speechConfig = SpeechConfig.FromSubscription(speechKey, speechRegion);
speechConfig.SpeechSynthesisVoiceName = "YourCustomVoiceName";
speechConfig.EndpointId = "YourEndpointId";

```

To use a custom voice via [Speech Synthesis Markup Language \(SSML\)](#), specify the model name as the voice name. This example uses the `YourCustomVoiceName` voice.

XML

```

<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="YourCustomVoiceName">
    This is the text that is spoken.
  </voice>
</speak>

```

Run and use a container

Speech containers provide websocket-based query endpoint APIs that are accessed through the Speech SDK and Speech CLI. By default, the Speech SDK and Speech CLI use the public

Speech service. To use the container, you need to change the initialization method. Use a container host URL instead of key and region.

For more information about containers, see [Install and run Speech containers with Docker](#).

Next steps

- [Try the text to speech quickstart](#)
- [Get started with custom voice](#)
- [Improve synthesis with SSML](#)

Batch synthesis API for text to speech

08/07/2025

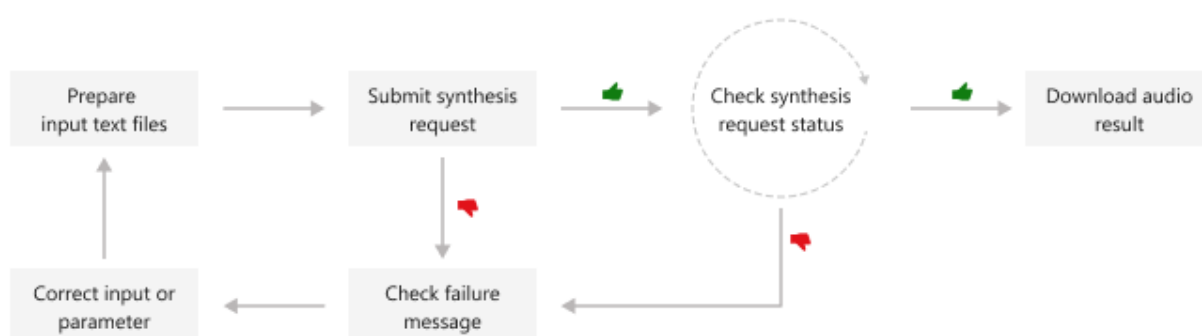
The Batch synthesis API can synthesize a large volume of text input (long and short) asynchronously. Publishers and audio content platforms can create long audio content in a batch. For example: audio books, news articles, and documents. The batch synthesis API can create synthesized audio longer than 10 minutes.

Important

The Batch synthesis API is generally available. The Long Audio API will be retired on April 1st, 2027. For more information, see [Migrate to batch synthesis API](#).

The batch synthesis API is asynchronous and doesn't return synthesized audio in real-time. You submit text files to be synthesized, poll for the status, and download the audio output when the status indicates success. The text inputs must be plain text or [Speech Synthesis Markup Language \(SSML\)](#) text.

This diagram provides a high-level overview of the workflow.



Tip

You can also use the [Speech SDK](#) to create synthesized audio longer than 10 minutes by iterating over the text and synthesizing it in chunks. For a C# example, see [GitHub](#).

You can use the following REST API operations for batch synthesis:

 Expand table

Operation	Method	REST API call
Create batch synthesis	PUT	texttospeech/batchsyntheses/YourSynthesisId
Get batch synthesis	GET	texttospeech/batchsyntheses/YourSynthesisId
List batch synthesis	GET	texttospeech/batchsyntheses
Delete batch synthesis	DELETE	texttospeech/batchsyntheses/YourSynthesisId

For code samples, see [GitHub](#).

Create batch synthesis

To submit a batch synthesis request, construct the HTTP PUT request path and body according to the following instructions:

- Set the required `inputKind` property.
- If the `inputKind` property is set to "PlainText", then you must also set the `voice` property in the `synthesisConfig`. In the following example, the `inputKind` is set to "SSML", so the `synthesisConfig` isn't set.
- Optionally you can set the `description`, `timeToLiveInHours`, and other properties. For more information, see [batch synthesis properties](#).

ⓘ Note

The maximum JSON payload size that's accepted is 2 megabytes.

Set the required `YourSynthesisId` in path. The `YourSynthesisId` must be unique. It must be 3-64 long, contains only numbers, letters, hyphens, underscores and dots, starts and ends with a letter or number.

Make an HTTP PUT request using the URI as shown in the following example. Replace `YourSpeechKey` with your Speech resource key, replace `YourSpeechRegion` with your Speech resource region, and set the request body properties as previously described.

Azure CLI

```
curl -v -X PUT -H "Ocp-Apim-Subscription-Key: YourSpeechKey" -H "Content-Type: application/json" -d '{
  "description": "my ssml test",
  "inputKind": "SSML",
  "inputs": [
    {
```

```

        "content": "<speak version=\"1.0\" xml:lang=\"en-US\"><voice
name=\"en-US-JennyNeural\">The rainbow has seven colors.</voice></speak>"
    },
    "properties": {
        "outputFormat": "riff-24khz-16bit-mono-pcm",
        "wordBoundaryEnabled": false,
        "sentenceBoundaryEnabled": false,
        "concatenateResult": false,
        "decompressOutputFiles": false
    }
}'
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses/
YourSynthesisId?api-version=2024-04-01"

```

You should receive a response body in the following format:

JSON

```

{
  "id": "YourSynthesisId",
  "status": "Running",
  "createdDateTime": "2024-03-12T07:23:18.0097387Z",
  "lastActionDateTime": "2024-03-12T07:23:18.0097388Z",
  "inputKind": "SSML",
  "customVoices": {},
  "properties": {
    "timeToLiveInHours": 168,
    "outputFormat": "riff-24khz-16bit-mono-pcm",
    "concatenateResult": false,
    "decompressOutputFiles": false,
    "wordBoundaryEnabled": false,
    "sentenceBoundaryEnabled": false
  }
}

```

The `status` property should progress from `Running` status to `Succeeded` or `Failed`. You can call the [GET batch synthesis API](#) periodically until the returned status is `Succeeded` or `Failed`.

Get batch synthesis

To get the status of the batch synthesis job, make an HTTP GET request using the URI as shown in the following example. Replace `YourSpeechKey` with your Speech resource key, and replace `YourSpeechRegion` with your Speech resource region.

Azure CLI

```
curl -v -X GET
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses/
YourSynthesisId?api-version=2024-04-01" -H "Ocp-Apim-Subscription-Key:
YourSpeechKey"
```

You should receive a response body in the following format:

JSON

```
{
  "id": "YourSynthesisId",
  "status": "Succeeded",
  "createdDateTime": "2024-03-12T07:23:18.0097387Z",
  "lastActionDateTime": "2024-03-12T07:23:18.7979669",
  "inputKind": "SSML",
  "customVoices": {},
  "properties": {
    "timeToLiveInHours": 168,
    "outputFormat": "riff-24khz-16bit-mono-pcm",
    "concatenateResult": false,
    "decompressOutputFiles": false,
    "wordBoundaryEnabled": false,
    "sentenceBoundaryEnabled": false,
    "sizeInBytes": 120000,
    "succeededAudioCount": 1,
    "failedAudioCount": 0,
    "durationInMilliseconds": 2500,
    "billingDetails": {
      "neuralCharacters": 29
    }
  },
  "outputs": {
    "result": "https://stttssvcuse.blob.core.windows.net/batchsynthesis-
output/29f2105f997c4bfea176d39d05ff201e/YourSynthesisId/results.zip?SAS_Token"
  }
}
```

From `outputs.result`, you can download a ZIP file that contains the audio (such as `0001.wav`), summary, and debug details. For more information, see [batch synthesis results](#).

List batch synthesis

To list all batch synthesis jobs for the Speech resource, make an HTTP GET request using the URI as shown in the following example. Replace `YourSpeechKey` with your Speech resource key and replace `YourSpeechRegion` with your Speech resource region. Optionally, you can set the `skip` and `maxpagesize` (up to 100) query parameters in URL. The default value for `skip` is 0 and the default value for `maxpagesize` is 100.

Azure CLI

```
curl -v -X GET
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses?
api-version=2024-04-01&skip=1&maxpagesize=2" -H "Ocp-Apim-Subscription-Key:
YourSpeechKey"
```

You should receive a response body in the following format:

JSON

```
{
  "value": [
    {
      "id": "my-job-03",
      "status": "Succeeded",
      "createdDateTime": "2024-03-12T07:28:32.5690441Z",
      "lastActionDateTime": "2024-03-12T07:28:33.0042293",
      "inputKind": "SSML",
      "customVoices": {},
      "properties": {
        "timeToLiveInHours": 168,
        "outputFormat": "riff-24khz-16bit-mono-pcm",
        "concatenateResult": false,
        "decompressOutputFiles": false,
        "wordBoundaryEnabled": false,
        "sentenceBoundaryEnabled": false,
        "sizeInBytes": 120000,
        "succeededAudioCount": 1,
        "failedAudioCount": 0,
        "durationInMilliseconds": 2500,
        "billingDetails": {
          "neuralCharacters": 29
        }
      }
    },
    {
      "result": "https://stttssvcuse.blob.core.windows.net/batchsynthesis-
output/29f2105f997c4bfea176d39d05ff201e/my-job-03/results.zip?SAS_Token"
    }
  ],
  {
    "id": "my-job-02",
    "status": "Succeeded",
    "createdDateTime": "2024-03-12T07:28:29.6418211Z",
    "lastActionDateTime": "2024-03-12T07:28:30.0910306",
    "inputKind": "SSML",
    "customVoices": {},
    "properties": {
      "timeToLiveInHours": 168,
      "outputFormat": "riff-24khz-16bit-mono-pcm",
      "concatenateResult": false,
      "decompressOutputFiles": false,
      "wordBoundaryEnabled": false,
```

```

    "sentenceBoundaryEnabled": false,
    "sizeInBytes": 120000,
    "succeededAudioCount": 1,
    "failedAudioCount": 0,
    "durationInMilliseconds": 2500,
    "billingDetails": {
      "neuralCharacters": 29
    }
  },
  "outputs": {
    "result": "https://stttssvcuse.blob.core.windows.net/batchsynthesis-
output/29f2105f997c4bfea176d39d05ff201e/my-job-02/results.zip?SAS_Token"
  }
},
"nextLink":
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses?
skip=3&maxpagesize=2&api-version=2024-04-01"
}

```

From `outputs.result`, you can download a ZIP file that contains the audio (such as `0001.wav`), summary, and debug details. For more information, see [batch synthesis results](#).

The `value` property in the json response lists your synthesis requests. The list is paginated, with a maximum page size of 100. The `"nextLink"` property is provided as needed to get the next page of the paginated list.

Delete batch synthesis

Delete the batch synthesis job history after you retrieved the audio output results. The Speech service keeps batch synthesis history for 168 hours (7 days) by default. Alternatively, you can specify this retention period using the `timeToLiveInHours` property, up to 744 hours (31 days). The date and time of automatic deletion (for synthesis jobs with a status of "Succeeded" or "Failed") is equal to the `lastActionDateTime` + `timeToLiveInHours` properties.

To delete a batch synthesis job, make an HTTP DELETE request using the URI as shown in the following example. Replace `YourSynthesisId` with your batch synthesis ID, replace `YourSpeechKey` with your Speech resource key, and replace `YourSpeechRegion` with your Speech resource region.

Azure CLI

```

curl -v -X DELETE
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses/
YourSynthesisId?api-version=2024-04-01" -H "Ocp-Apim-Subscription-Key:
YourSpeechKey"

```

The response headers include `HTTP/1.1 204 No Content` if the delete request was successful.

Batch synthesis results

After you [get a batch synthesis job](#) with `status` of "Succeeded", you can download the audio output results. Use the URL from the `outputs.result` property of the [batch synthesis GET](#) response.

To get the batch synthesis results file, make an HTTP GET request using the URL as shown in the following example. Replace `YourOutputsResultUrl` with the URL from the `outputs.result` property of the [batch synthesis GET](#) response. Replace `YourSpeechKey` with your Speech resource key.

Azure CLI

```
curl -v -X GET "YourOutputsResultUrl" -H "Ocp-Apim-Subscription-Key: YourSpeechKey" > results.zip
```

The results are in a ZIP file that contains the audio (such as `0001.wav`), summary, and debug details. The numbered prefix of each filename (shown below as `[nnnn]`) is in the same order as the text inputs used when you created the batch synthesis.

ⓘ Note

The `[nnnn].debug.json` file contains the synthesis result ID and other information that might help with troubleshooting. The properties that it contains might change, so you shouldn't take any dependencies on the JSON format.

The summary file contains the synthesis results for each text input. Here's an example `summary.json` file:

JSON

```
{
  "jobID": "7ab84171-9070-4d3b-88d4-1b8cc1cb928a",
  "status": "Succeeded",
  "results": [
    {
      "contents": ["<speak version=\"1.0\" xml:lang=\"en-US\"><voice name=\"en-US-JennyNeural\">The rainbow has seven colors.</voice></speak>"],
      "status": "Succeeded",
      "audioFileName": "0001.wav",
      "properties": {
        "sizeInBytes": "120000",
```

```

        "durationInMilliseconds": "2500"
    }
}
]
}

```

If sentence boundary data was requested (`"sentenceBoundaryEnabled": true`), then a corresponding `[nnnn].sentence.json` file is included in the results. Likewise, if word boundary data was requested (`"wordBoundaryEnabled": true`), then a corresponding `[nnnn].word.json` file is included in the results.

Here's an example word data file with both audio offset and duration in milliseconds:

JSON

```

[
  {
    "Text": "The",
    "AudioOffset": 50,
    "Duration": 137
  },
  {
    "Text": "rainbow",
    "AudioOffset": 200,
    "Duration": 350
  },
  {
    "Text": "has",
    "AudioOffset": 562,
    "Duration": 175
  },
  {
    "Text": "seven",
    "AudioOffset": 750,
    "Duration": 300
  },
  {
    "Text": "colors",
    "AudioOffset": 1062,
    "Duration": 625
  },
  {
    "Text": ".",
    "AudioOffset": 1700,
    "Duration": 100
  }
]

```

Batch synthesis latency and best practices

When using batch synthesis for generating synthesized speech, it's important to consider the latency involved and follow best practices for achieving optimal results.

Latency in batch synthesis

The latency in batch synthesis depends on various factors, including the complexity of the input text, the number of inputs in the batch, and the processing capabilities of the underlying hardware.

The latency for batch synthesis is as follows (approximately):

- The latency of 50% of the synthesized speech outputs is within 10-20 seconds.
- The latency of 95% of the synthesized speech outputs is within 120 seconds.

Best practices

When considering batch synthesis for your application, it's recommended to assess whether the latency meets your requirements. If the latency aligns with your desired performance, batch synthesis can be a suitable choice. However, if the latency doesn't meet your needs, you might consider using real-time API.

HTTP status codes

The section details the HTTP response codes and messages from the batch synthesis API.

HTTP 200 OK

HTTP 200 OK indicates that the request was successful.

HTTP 201 Created

HTTP 201 Created indicates that the batch synthesis create request (via HTTP PUT) was successful.

HTTP 204 error

An HTTP 204 error indicates that the request was successful, but the resource doesn't exist. For example:

- You tried to get or delete a synthesis job that doesn't exist.

- You successfully deleted a synthesis job.

HTTP 400 error

Here are examples that can result in the 400 error:

- The `outputFormat` is unsupported or invalid. Provide a valid format value, or leave `outputFormat` empty to use the default setting.
- The number of requested text inputs exceeded the limit of 10,000.
- You tried to use an invalid deployment ID or a custom voice that isn't successfully deployed. Make sure the Speech resource has access to the custom voice, and the custom voice is successfully deployed. You must also ensure that the mapping of `{"your-custom-voice-name": "your-deployment-ID"}` is correct in your batch synthesis request.
- You tried to use a *F0* Speech resource, but the region only supports the *Standard* Speech resource pricing tier.

HTTP 404 error

The specified entity can't be found. Make sure the synthesis ID is correct.

HTTP 429 error

There are too many recent requests. Each client application can submit up to 100 requests per 10 seconds for each Speech resource. Reduce the number of requests per second.

HTTP 500 error

HTTP 500 Internal Server Error indicates that the request failed. The response body contains the error message.

HTTP error example

Here's an example request that results in an HTTP 400 error, because the `inputs` property is required to create a job.

Console

```
curl -v -X PUT -H "Ocp-Apim-Subscription-Key: YourSpeechKey" -H "Content-Type: application/json" -d '{
  "inputKind": "SSML"
}'
```

```
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses/  
YourSynthesisId?api-version=2024-04-01"
```

In this case, the response headers include `HTTP/1.1 400 Bad Request`.

The response body resembles the following JSON example:

JSON

```
{  
  "error": {  
    "code": "BadRequest",  
    "message": "The inputs is required."  
  }  
}
```

Next steps

- [Speech Synthesis Markup Language \(SSML\)](#)
- [Batch synthesis properties](#)
- [Migrate to batch synthesis](#)

Batch synthesis properties for text to speech

08/07/2025

 Important

The Batch synthesis API is generally available. The Long Audio API will be retired on April 1st, 2027. For more information, see [Migrate to batch synthesis API](#).

The Batch synthesis API can synthesize a large volume of text input (long and short) asynchronously. Publishers and audio content platforms can create long audio content in a batch. For example: audio books, news articles, and documents. The batch synthesis API can create synthesized audio longer than 10 minutes.

Some properties in JSON format are required when you create a new batch synthesis job. Other properties are optional. The batch synthesis response includes other properties to provide information about the synthesis status and results. For example, the `outputs.result` property contains the location of the batch synthesis result files with audio output and logs.

Batch synthesis properties

Batch synthesis properties are described in the following table.

 Expand table

Property	Description
<code>createdDateTime</code>	The date and time when the batch synthesis job was created. This property is read-only.
<code>customVoices</code>	The map of a custom voice name and its deployment ID. For example: <code>"customVoices": {"your-custom-voice-name": "502ac834-6537-4bc3-9fd6-140114daa66d"}</code> You can use the voice name in your <code>synthesisConfig.voice</code> (when the <code>inputKind</code> is set to <code>"PlainText"</code>) or within the SSML text of <code>inputs</code> (when the <code>inputKind</code> is set to <code>"SSML"</code>). This property is required to use a custom voice. If you try to

Property	Description
	use a custom voice that isn't defined here, the service returns an error.
<code>description</code>	The description of the batch synthesis. This property is optional.
<code>id</code>	The batch synthesis job ID you passed in path. This property is required in path.
<code>inputs</code>	The plain text or SSML to be synthesized. When the <code>inputKind</code> is set to "PlainText", provide plain text as shown here: <code>"inputs": [{"content": "The rainbow has seven colors."}]</code>. When the <code>inputKind</code> is set to "SSML", provide text in the Speech Synthesis Markup Language (SSML) as shown here: <code>"inputs": [{"content": "<speak version='1.0' xml:lang='en-US'><voice xml:lang='en-US' xml:gender='Female' name='en-US-AvaMultilingualNeural'>The rainbow has seven colors.</voice></speak>"}]</code>. Include up to 1,000 text objects if you want multiple audio output files. Here's example input text that should be synthesized to two audio output files: <code>"inputs": [{"content": "synthesize this to a file"}, {"content": "synthesize this to another file"}]</code>. However, if the <code>properties.concatenateResult</code> property is set to <code>true</code>, then each synthesized result is written to the same audio output file. You don't need separate text inputs for new paragraphs. Within any of the (up to 1,000) text inputs, you can specify new paragraphs using the <code>"\r\n"</code> (newline) string. Here's example input text with two paragraphs that should be synthesized to the same audio output file: <code>"inputs": [{"content": "synthesize this to a file\r\nsynthesize this to another paragraph in the same file"}]</code> There are no paragraph limits, but the maximum JSON payload size (including all text inputs and other properties) is 2 megabytes. This property is required when you create a new batch synthesis job. This property isn't included in the response when you get the synthesis job.

Property	Description
<code>lastActionDateTime</code>	The most recent date and time when the <code>status</code> property value changed. This property is read-only.
<code>outputs.result</code>	The location of the batch synthesis result files with audio output and logs. This property is read-only.
<code>properties</code>	A defined set of optional batch synthesis configuration settings.
<code>properties.sizeInBytes</code>	The audio output size in bytes. This property is read-only.
<code>properties.billingDetails</code>	The number of words that were processed and billed by <code>customNeuralCharacters</code> (custom voice) versus <code>neuralCharacters</code> (standard voice). This property is read-only.
<code>properties.concatenateResult</code>	Determines whether to concatenate the result. This optional <code>bool</code> value ("true" or "false") is "false" by default.
<code>properties.decompressOutputFiles</code>	Determines whether to unzip the synthesis result files in the destination container. This property can only be set when the <code>destinationContainerUrl</code> property is set. This optional <code>bool</code> value ("true" or "false") is "false" by default.
<code>properties.destinationContainerUrl</code>	The batch synthesis results can be stored in a writable Azure container. If you don't specify a container URI with shared access signatures (SAS) token, the Speech service stores the results in a container managed by Microsoft. SAS with stored access policies isn't supported. When the synthesis job is deleted, the result data is also deleted. This optional property isn't included in the response when you get the synthesis job.
<code>properties.destinationPath</code>	The prefix path for storing batch synthesis results. If no prefix path is provided, a system-generated path will be used. This property is optional and can only be set when the <code>destinationContainerUrl</code> property is specified.
<code>properties.durationInMilliseconds</code>	The audio output duration in milliseconds.

Property	Description
	This property is read-only.
<code>properties.failedAudioCount</code>	The count of batch synthesis inputs to audio output failed. This property is read-only.
<code>properties.outputFormat</code>	The audio output format. For information about the accepted values, see audio output formats. The default output format is <code>riff-24khz-16bit-mono-pcm</code>.
<code>properties.sentenceBoundaryEnabled</code>	Determines whether to generate sentence boundary data. This optional <code>bool</code> value ("true" or "false") is "false" by default. If sentence boundary data is requested, then a corresponding <code>[nnnn].sentence.json</code> file is included in the results data ZIP file.
<code>properties.succeededAudioCount</code>	The count of batch synthesis inputs to audio output succeeded. This property is read-only.
<code>properties.timeToLiveInHours</code>	A duration in hours after the synthesis job is completed, when the synthesis results will be automatically deleted. This optional setting is <code>168</code> (7 days) by default. The maximum time to live is <code>744</code> (31 days). The date and time of automatic deletion (for synthesis jobs with a status of "Succeeded" or "Failed") is equal to the <code>lastActionDateTime</code> + <code>timeToLiveInHours</code> properties. Otherwise, you can call the delete synthesis method to remove the job sooner.
<code>properties.wordBoundaryEnabled</code>	Determines whether to generate word boundary data. This optional <code>bool</code> value ("true" or "false") is "false" by default. If word boundary data is requested, then a corresponding <code>[nnnn].word.json</code> file is included in the results data ZIP file.
<code>status</code>	The batch synthesis processing status. The status should progress from "Running" to either "Succeeded" or "Failed". This property is read-only.

Property	Description
<code>synthesisConfig</code>	The configuration settings to use for batch synthesis of plain text. This property is only applicable when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>synthesisConfig.backgroundAudio</code>	The background audio for each audio output. This optional property is only applicable when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>synthesisConfig.backgroundAudio.fadein</code>	The duration of the background audio fade-in as milliseconds. The default value is <code>0</code>, which is the equivalent to no fade in. Accepted values: <code>0</code> to <code>10000</code> inclusive. For information, see the attributes table under add background audio in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>synthesisConfig.backgroundAudio.fadeout</code>	The duration of the background audio fade-out in milliseconds. The default value is <code>0</code>, which is the equivalent to no fade out. Accepted values: <code>0</code> to <code>10000</code> inclusive. For information, see the attributes table under add background audio in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>synthesisConfig.backgroundAudio.src</code>	The URI location of the background audio file. For information, see the attributes table under add background audio in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This property is required when <code>synthesisConfig.backgroundAudio</code> is set.
<code>synthesisConfig.backgroundAudio.volume</code>	The volume of the background audio file. Accepted values: <code>0</code> to <code>100</code> inclusive. The default value is <code>1</code>. For information, see the attributes table under add background audio in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored.

Property	Description
	This optional property is only applicable when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>synthesisConfig.pitch</code>	The pitch of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>synthesisConfig.rate</code>	The rate of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>synthesisConfig.role</code>	For some voices, you can adjust the speaking role-play. The voice can imitate a different age and gender, but the voice name isn't changed. For example, a male voice can raise the pitch and change the intonation to imitate a female voice, but the voice name isn't changed. If the role is missing or isn't supported for your voice, this attribute is ignored. For information about the available styles per voice, see voice styles and roles. This optional property is only applicable when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>synthesisConfig.speakerProfileId</code>	The speaker profile ID of a personal voice. For information about available personal voice base model names, see integrate personal voice. For information about how to get the speaker profile ID, see language and voice support. This property is required when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>synthesisConfig.style</code>	For some voices, you can adjust the speaking style to express different emotions like cheerfulness, empathy, and calm. You can optimize the voice for different scenarios like customer

Property	Description
	service, newscast, and voice assistant. For information about the available styles per voice, see voice styles and roles. This optional property is only applicable when <code>synthesisConfig.style</code> is set.
<code>synthesisConfig.styleDegree</code>	The intensity of the speaking style. You can specify a stronger or softer style to make the speech more expressive or subdued. The range of accepted values are: 0.01 to 2 inclusive. The default value is 1, which means the predefined style intensity. The minimum unit is 0.01, which results in a slight tendency for the target style. A value of 2 results in a doubling of the default style intensity. If the style degree is missing or isn't supported for your voice, this attribute is ignored. For information about the available styles per voice, see voice styles and roles. This optional property is only applicable when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>synthesisConfig.voice</code>	The voice that speaks the audio output. For information about the available standard voices, see language and voice support. To use a custom voice, you must specify a valid custom voice and deployment ID mapping in the <code>customVoices</code> property. To use a personal voice, you need to specify the <code>synthesisConfig.speakerProfileId</code> property. This property is required when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>synthesisConfig.volume</code>	The volume of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when <code>inputKind</code> is set to <code>"PlainText"</code>.
<code>inputKind</code>	Indicates whether the <code>inputs</code> text property should be plain text or SSML. The possible case-insensitive values are <code>"PlainText"</code> and <code>"SSML"</code>. When the <code>inputKind</code> is set to

Property	Description
	<code>"PlainText"</code> , you must also set the <code>synthesisConfig</code> voice property. This property is required.

Batch synthesis latency and best practices

When using batch synthesis for generating synthesized speech, it's important to consider the latency involved and follow best practices for achieving optimal results.

Latency in batch synthesis

The latency in batch synthesis depends on various factors, including the complexity of the input text, the number of inputs in the batch, and the processing capabilities of the underlying hardware.

The latency for batch synthesis is as follows (approximately):

- The latency of 50% of the synthesized speech outputs is within 10-20 seconds.
- The latency of 95% of the synthesized speech outputs is within 120 seconds.

Best practices

When considering batch synthesis for your application, it's recommended to assess whether the latency meets your requirements. If the latency aligns with your desired performance, batch synthesis can be a suitable choice. However, if the latency doesn't meet your needs, you might consider using real-time API.

HTTP status codes

The section details the HTTP response codes and messages from the batch synthesis API.

HTTP 200 OK

HTTP 200 OK indicates that the request was successful.

HTTP 201 Created

HTTP 201 Created indicates that the create batch synthesis request (via HTTP POST) was successful.

HTTP 204 error

An HTTP 204 error indicates that the request was successful, but the resource doesn't exist. For example:

- You tried to get or delete a synthesis job that doesn't exist.
- You successfully deleted a synthesis job.

HTTP 400 error

Here are examples that can result in the 400 error:

- The `outputFormat` is unsupported or invalid. Provide a valid format value, or leave `outputFormat` empty to use the default setting.
- The number of requested text inputs exceeded the limit of 10,000.
- You tried to use an invalid deployment ID or a custom voice that isn't successfully deployed. Make sure the Speech resource has access to the custom voice, and the custom voice is successfully deployed. You must also ensure that the mapping of `{"your-custom-voice-name": "your-deployment-ID"}` is correct in your batch synthesis request.
- You tried to use a *F0* Speech resource, but the region only supports the *Standard* Speech resource pricing tier.

HTTP 404 error

The specified entity can't be found. Make sure the synthesis ID is correct.

HTTP 429 error

There are too many recent requests. Each client application can submit up to 100 requests per 10 seconds for each Speech resource. Reduce the number of requests per second.

HTTP 500 error

HTTP 500 Internal Server Error indicates that the request failed. The response body contains the error message.

HTTP error example

Here's an example request that results in an HTTP 400 error, because the `inputs` property is required to create a job.

Console

```
curl -v -X PUT -H "Ocp-Apim-Subscription-Key: YourSpeechKey" -H "Content-Type: application/json" -d '{
  "inputKind": "SSML"
}'
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses/YourSynthesisId?api-version=2024-04-01"
```

In this case, the response headers include `HTTP/1.1 400 Bad Request`.

The response body resembles the following JSON example:

JSON

```
{
  "error": {
    "code": "BadRequest",
    "message": "The inputs is required."
  }
}
```

Next steps

- [Speech Synthesis Markup Language \(SSML\)](#)
- [Text to speech quickstart](#)
- [Migrate to batch synthesis](#)

Speech Synthesis Markup Language (SSML) overview

08/07/2025

Speech Synthesis Markup Language (SSML) is an XML-based markup language that you can use to fine-tune your text to speech output attributes such as pitch, pronunciation, speaking rate, volume, and more. It gives you more control and flexibility than plain text input.



Tip

You can hear voices in different styles and pitches reading example text by using the [Voice Gallery](#).

Use case scenarios

SSML is designed to give you flexibility in how you want your speech output to sound, and it provides different properties for how you can customize that output. You can use SSML to:

- [Define the input text structure](#) that determines the structure, content, and other characteristics of your text to speech output. For example, you can use SSML to define a paragraph, a sentence, a break or a pause, or silence. You can wrap text with event tags, like a bookmark or viseme, that your application can process later. A viseme is the visual description of a phoneme, the individual speech sounds, in spoken language.
- [Choose the voice](#), language, name, style, and role. You can use multiple voices in a single SSML document. You can also adjust the emphasis, speaking rate, pitch, and volume. SSML can also insert prerecorded audio, such as a sound effect or a musical note.
- [Control pronunciation](#) of the output audio. For example, you can use SSML with phonemes and a custom lexicon to improve pronunciation. You can also use SSML to define how a word or mathematical expression is pronounced.

Ways to work with SSML

SSML functionality is available in various tools that might fit your use case.

Important

You're billed for each character that's converted to speech, including punctuation. Although the SSML document itself isn't billable, the service counts optional elements that

you use to adjust how the text is converted to speech, like phonemes and pitch, as billable characters. For more information, see the [pricing note](#).

You can use SSML in the following ways:

- [The audio content creation](#) [↗](#) tool lets you author plain text and SSML in Speech Studio. You can listen to the output audio and adjust the SSML to improve speech synthesis. For more information, see [Speech synthesis with the Audio Content Creation tool](#).
- [The batch synthesis API](#) accepts SSML via the `inputs` property.
- [The Speech CLI](#) accepts SSML via the `spx synthesize --ssml SSML` command line argument.
- [The Speech SDK](#) accepts SSML via the "speak" SSML method across the different supported languages.

Next steps

- [SSML document structure and events](#)
- [Voice and sound with SSML](#)
- [Pronunciation with SSML](#)
- [Language and voice support for the Speech service](#)

SSML document structure and events

08/07/2025

The Speech Synthesis Markup Language (SSML) with input text determines the structure, content, and other characteristics of the text to speech output. For example, you can use SSML to define a paragraph, a sentence, a break or a pause, or silence. You can wrap text with event tags such as bookmark or viseme that can be processed later by your application.

Refer to the sections below for details about how to structure elements in the SSML document.

ⓘ Note

In addition to Azure AI Speech neural (non HD) voices, you can also use [Azure AI Speech high definition \(HD\) voices](#) and [Azure OpenAI neural \(HD and non HD\) voices](#). The HD voices provide a higher quality for more versatile scenarios.

Some voices don't support all [Speech Synthesis Markup Language \(SSML\)](#) tags. This includes neural text to speech HD voices, personal voices, and embedded voices.

- For Azure AI Speech high definition (HD) voices, check the SSML support [here](#).
- For personal voice, you can find the SSML support [here](#).
- For embedded voices, check the SSML support [here](#).

Document structure

The Speech service implementation of SSML is based on the World Wide Web Consortium's [Speech Synthesis Markup Language Version 1.0](#). The elements supported by the Speech can differ from the W3C standard.

Each SSML document is created with SSML elements or tags. These elements are used to adjust the voice, style, pitch, prosody, volume, and more.

Here's a subset of the basic structure and syntax of an SSML document:

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="string">
  <mstts:backgroundaudio src="string" volume="string" fadein="string"
fadeout="string"/>
  <mstts:voiceconversion url="string"/>
  <voice name="string" effect="string">
    <audio src="string"></audio>
```

```

<bookmark mark="string"/>
<break strength="string" time="string" />
<emphasis level="value"></emphasis>
<lang xml:lang="string"></lang>
<lexicon uri="string"/>
<math xmlns="http://www.w3.org/1998/Math/MathML"></math>
<mstts:audioduration value="string"/>
<mstts:ttseembedding speakerProfileId="string"></mstts:ttseembedding>
<mstts:express-as style="string" styledegree="value" role="string">
</mstts:express-as>
  <mstts:silence type="string" value="string"/>
  <mstts:viseme type="string"/>
  <p></p>
  <phoneme alphabet="string" ph="string"></phoneme>
  <prosody pitch="value" contour="value" range="value" rate="value"
volume="value"></prosody>
  <s></s>
  <say-as interpret-as="string" format="string" detail="string"></say-as>
  <sub alias="string"></sub>
</voice>
</speak>

```

Some examples of contents that are allowed in each element are described in the following list:

- **audio**: The body of the **audio** element can contain plain text or SSML markup that's spoken if the audio file is unavailable or unplayable. The **audio** element can also contain text and the following elements: **audio**, **break**, **p**, **s**, **phoneme**, **prosody**, **say-as**, and **sub**.
- **bookmark**: This element can't contain text or any other elements.
- **break**: This element can't contain text or any other elements.
- **emphasis**: This element can contain text and the following elements: **audio**, **break**, **emphasis**, **lang**, **phoneme**, **prosody**, **say-as**, and **sub**.
- **lang**: This element can contain all other elements except **mstts:backgroundaudio**, **voice**, and **speak**.
- **lexicon**: This element can't contain text or any other elements.
- **math**: This element can only contain text and MathML elements.
- **mstts:audioduration**: This element can't contain text or any other elements.
- **mstts:backgroundaudio**: This element can't contain text or any other elements.
- **<mstts:voiceconversion>**: This element can't contain text or any other elements. It specifies the source audio URL for the voice conversion.
- **mstts:embedding**: This element can contain text and the following elements: **audio**, **break**, **emphasis**, **lang**, **phoneme**, **prosody**, **say-as**, and **sub**.
- **mstts:express-as**: This element can contain text and the following elements: **audio**, **break**, **emphasis**, **lang**, **phoneme**, **prosody**, **say-as**, and **sub**.
- **mstts:silence**: This element can't contain text or any other elements.

- `mstts:viseme`: This element can't contain text or any other elements.
- `p`: This element can contain text and the following elements: `audio`, `break`, `phoneme`, `prosody`, `say-as`, `sub`, `mstts:express-as`, and `s`.
- `phoneme`: This element can only contain text and no other elements.
- `prosody`: This element can contain text and the following elements: `audio`, `break`, `p`, `phoneme`, `prosody`, `say-as`, `sub`, and `s`.
- `s`: This element can contain text and the following elements: `audio`, `break`, `phoneme`, `prosody`, `say-as`, `mstts:express-as`, and `sub`.
- `say-as`: This element can only contain text and no other elements.
- `sub`: This element can only contain text and no other elements.
- `speak`: The root element of an SSML document. This element can contain the following elements: `mstts:backgroundaudio` and `voice`.
- `voice`: This element can contain all other elements except `mstts:backgroundaudio` and `speak`.

The Speech service automatically handles punctuation as appropriate, such as pausing after a period, or using the correct intonation when a sentence ends with a question mark.

Special characters

To use the characters `&`, `<`, and `>` within the SSML element's value or text, you must use the entity format. Specifically you must use `&` in place of `&`, use `<` in place of `<`, and use `>` in place of `>`. Otherwise the SSML isn't parsed correctly.

For example, specify `green & yellow` instead of `green & yellow`. The following SSML is parsed as expected:

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
    My favorite colors are green &amp; yellow.
  </voice>
</speak>
```

Special characters such as quotation marks, apostrophes, and brackets, must be escaped. For more information, see [Extensible Markup Language \(XML\) 1.0: Appendix D](#).

Double or single quotation marks must enclose the attribute values. For example, `<prosody volume="90">` and `<prosody volume='90'>` are well-formed, valid elements, but `<prosody volume=90>` isn't recognized.

Speak root element

The `speak` element contains information such as version, language, and the markup vocabulary definition. The `speak` element is the root element that's required for all SSML documents. You must specify the default language within the `speak` element, whether or not the language is adjusted elsewhere such as within the `lang` element.

Here's the syntax for the `speak` element:

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xml:lang="string"></speak>
```

 Expand table

Attribute	Description	Required or optional
<code>version</code>	Indicates the version of the SSML specification used to interpret the document markup. The current version is "1.0".	Required
<code>xml:lang</code>	The language of the root document. The value can contain a language code such as <code>en</code> (English), or a locale such as <code>en-US</code> (English - United States).	Required
<code>xmlns</code>	The URI to the document that defines the markup vocabulary (the element types and attribute names) of the SSML document. The current URI is "http://www.w3.org/2001/10/synthesis".	Required

The `speak` element must contain at least one [voice element](#).

speak examples

The supported values for attributes of the `speak` element were [described previously](#).

Single voice example

This example uses the `en-US-AvaNeural` voice. For more examples, see [voice examples](#).

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
    This is the text that is spoken.
  </voice>
</speak>
```

```
</voice>
</speak>
```

Add a break

Use the `break` element to override the default behavior of breaks or pauses between words. Otherwise the Speech service automatically inserts pauses.

Usage of the `break` element's attributes are described in the following table.

[Expand table](#)

Attribute	Description	Required or optional
<code>strength</code>	The relative duration of a pause by using one of the following values: • x-weak • weak • medium (default) • strong • x-strong	Optional
<code>time</code>	The absolute duration of a pause in seconds (such as <code>2s</code>) or milliseconds (such as <code>500ms</code>). Valid values range from 0 to 20000 milliseconds. If you set a value greater than the supported maximum, the service uses <code>20000ms</code> . If the <code>time</code> attribute is set, the <code>strength</code> attribute is ignored.	Optional

Here are more details about the `strength` attribute.

[Expand table](#)

Strength	Relative duration
X-weak	250 ms
Weak	500 ms
Medium	750 ms
Strong	1,000 ms
X-strong	1,250 ms

Break examples

The supported values for attributes of the `break` element were [described previously](#). The following three ways all add 750 ms breaks.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
    Welcome <break /> to text to speech.
    Welcome <break strength="medium" /> to text to speech.
    Welcome <break time="750ms" /> to text to speech.
  </voice>
</speak>
```

Add silence

Use the `mstts:silence` element to insert pauses before or after text, or between two adjacent sentences.

One of the differences between `mstts:silence` and `break` is that a `break` element can be inserted anywhere in the text. Silence only works at the beginning or end of input text or at the boundary of two adjacent sentences.

The silence setting is applied to all input text within its enclosing `voice` element. To reset or change the silence setting again, you must use a new `voice` element with either the same voice or a different voice.

Usage of the `mstts:silence` element's attributes are described in the following table.

[Expand table](#)

Attribute	Description	Required or optional
type	Specifies where and how to add silence. The following silence types are supported: <code>Leading</code> – Extra silence at the beginning of the text. The value that you set is added to the natural silence before the start of text. <code>Leading-exact</code> – Silence at the beginning of the text. The value is an absolute silence length. <code>Tailing</code> – Extra silence at the end of text. The value that you set is added to the natural silence after the last word. <code>Tailing-exact</code> – Silence at the end of the text. The value is an absolute silence length.	Required

Attribute	Description	Required or optional
	<code>Sentenceboundary</code> – Extra silence between adjacent sentences. The actual silence length for this type includes the natural silence after the last word in the previous sentence, the value you set for this type, and the natural silence before the starting word in the next sentence. <code>Sentenceboundary-exact</code> – Silence between adjacent sentences. The value is an absolute silence length. <code>Comma-exact</code> – Silence at the comma in half-width or full-width format. The value is an absolute silence length. <code>Semicolon-exact</code> – Silence at the semicolon in half-width or full-width format. The value is an absolute silence length. <code>Enumerationcomma-exact</code> – Silence at the enumeration comma in full-width format. The value is an absolute silence length. An absolute silence type (with the <code>-exact</code> suffix) replaces any otherwise natural leading or trailing silence. Absolute silence types take precedence over the corresponding non-absolute type. For example, if you set both <code>Leading</code> and <code>Leading-exact</code> types, the <code>Leading-exact</code> type takes effect. The WordBoundary event takes precedence over punctuation-related silence settings including <code>Comma-exact</code>, <code>Semicolon-exact</code>, or <code>Enumerationcomma-exact</code>. When you use both the <code>WordBoundary</code> event and punctuation-related silence settings, the punctuation-related silence settings don't take effect.	
Value	The duration of a pause in seconds (such as <code>2s</code>) or milliseconds (such as <code>500ms</code>). Valid values range from 0 to 20000 milliseconds. If you set a value greater than the supported maximum, the service uses <code>20000ms</code> .	Required

mstts silence examples

The supported values for attributes of the `mstts:silence` element were [described previously](#).

In this example, `mstts:silence` is used to add 200 ms of silence between two sentences.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="http://www.w3.org/2001/mstts" xml:lang="en-US">
<voice name="en-US-AvaNeural">
<mstts:silence type="Sentenceboundary" value="200ms"/>
```

If we're home schooling, the best we can do is roll with what each day brings and try to have fun along the way.

A good place to start is by trying out the slew of educational apps that are helping children stay happy and smash their schooling at the same time.

```
</voice>
</speak>
```

In this example, `mstts:silence` is used to add 50 ms of silence at the comma, 100 ms of silence at the semicolon, and 150 ms of silence at the enumeration comma.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="http://www.w3.org/2001/mstts" xml:lang="zh-CN">
<voice name="zh-CN-YunxiNeural">
<mstts:silence type="comma-exact" value="50ms"/><mstts:silence type="semicolon-
exact" value="100ms"/><mstts:silence type="enumerationcomma-exact" value="150ms"/>
你好呀，云希、晓晓；你好呀。
</voice>
</speak>
```

Specify paragraphs and sentences

The `p` and `s` elements are used to denote paragraphs and sentences, respectively. In the absence of these elements, the Speech service automatically determines the structure of the SSML document.

Paragraph and sentence examples

The following example defines two paragraphs that each contain sentences. In the second paragraph, the Speech service automatically determines the sentence structure, since they aren't defined in the SSML document.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
    <p>
      <s>Introducing the sentence element.</s>
      <s>Used to mark individual sentences.</s>
    </p>
    <p>
      Another simple paragraph.
      Sentence structure in this paragraph is not explicitly marked.
    </p>
  </voice>
</speak>
```

Bookmark element

You can use the `bookmark` element in SSML to reference a specific location in the text or tag sequence. Then you use the Speech SDK and subscribe to the `BookmarkReached` event to get the offset of each marker in the audio stream. The `bookmark` element isn't spoken. For more information, see [Subscribe to synthesizer events](#).

Usage of the `bookmark` element's attributes are described in the following table.

 Expand table

Attribute	Description	Required or optional
<code>mark</code>	The reference text of the <code>bookmark</code> element.	Required

Bookmark examples

The supported values for attributes of the `bookmark` element were [described previously](#).

As an example, you might want to know the time offset of each flower word in the following snippet:

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
    We are selling <bookmark mark='flower_1' />roses and <bookmark
mark='flower_2' />daisies.
  </voice>
</speak>
```

Next steps

- [SSML overview](#)
- [Voice and sound with SSML](#)
- [Language support: Voices, locales, languages](#)

Customize voice and sound with SSML

07/09/2025

You can use Speech Synthesis Markup Language (SSML) to specify the text to speech voice, language, name, style, and role for your speech output. You can also use multiple voices in a single SSML document, and adjust the emphasis, speaking rate, pitch, and volume. In addition, SSML features the ability to insert prerecorded audio, such as a sound effect or a musical note.

The article shows you how to use SSML elements to specify voice and sound. For more information about SSML syntax, see [SSML document structure and events](#).

Use voice elements

At least one `voice` element must be specified within each SSML `speak` element. This element determines the voice that's used for text to speech.

You can include multiple `voice` elements in a single SSML document. Each `voice` element can specify a different voice. You can also use the same voice multiple times with different settings, such as when you [change the silence duration](#) between sentences.

The following table describes the usage of the `voice` element's attributes:

 Expand table

Attribute	Description	Required or optional
<code>name</code>	The voice used for text to speech output. For a complete list of supported standard voices, see Language support .	Required
<code>effect</code>	The audio effect processor that's used to optimize the quality of the synthesized speech output for specific scenarios on devices. For some scenarios in production environments, the auditory experience might be degraded due to the playback distortion on certain devices. For example, the synthesized speech from a car speaker might sound dull and muffled due to environmental factors such as speaker response, room reverberation, and background noise. The passenger might have to turn up the volume to hear more clearly. To avoid manual operations in such a scenario, the audio effect processor can make the sound clearer by compensating the distortion of playback. The following values are supported:	Optional

Attribute	Description	Required or optional
	<code>eq_car</code> – Optimize the auditory experience when providing high-fidelity speech in cars, buses, and other enclosed automobiles. <code>eq_telecomhp8k</code> – Optimize the auditory experience for narrowband speech in telecom or telephone scenarios. You should use a sampling rate of 8 kHz. If the sample rate isn't 8 kHz, the auditory quality of the output speech isn't optimized. If the value is missing or invalid, this attribute is ignored and no effect is applied.	

Voice examples

For information about the supported values for attributes of the `voice` element, see [Use voice elements](#).

Single voice example

This example uses the `en-US-AvaMultilingualNeural` voice.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    This is the text that is spoken.
  </voice>
</speak>
```

Multiple voices example

Within the `speak` element, you can specify multiple voices for text to speech output. These voices can be in different languages. For each voice, the text must be wrapped in a `voice` element.

This example alternates between the `en-US-AvaMultilingualNeural` and `en-US-AndrewMultilingualNeural` voices. The neural multilingual voices can speak different languages based on the input text.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    Good morning!
  </voice>
  <voice name="en-US-AndrewMultilingualNeural">
    Good morning to you too Ava!
  </voice>
</speak>
```

Custom voice example

To use your [custom voice](#), specify the model name as the voice name in SSML.

This example uses a custom voice named **my-custom-voice**.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="my-custom-voice">
    This is the text that is spoken.
  </voice>
</speak>
```

Audio effect example

You use the `effect` attribute to optimize the auditory experience for scenarios such as cars and telecommunications. The following SSML example uses the `effect` attribute with the configuration in car scenarios.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural" effect="eq_car">
    This is the text that is spoken.
  </voice>
</speak>
```

Multi-talker voice example

Multi-talker voices enable natural, dynamic conversations with multiple distinct speakers. This innovation enhances the realism of synthesized dialogues by preserving contextual flow, emotional consistency, and natural speech patterns.

Use this capability to generate engaging, podcast-style speech or conversational exchanges with seamless transitions between speakers. Unlike single-talker models, which synthesize each turn in isolation, multi-talker voices maintain coherence across dialogue, ensuring a more authentic and immersive listening experience.

For `en-US-MultiTalker-Ava-Andrew:DragonHDLatestNeural`, within the `<mstts:dialog>` element, you can specify each turn for the text to speech output, with the format below to alternates between the speaker `ava` and `andrew` for each turn.

XML

```
<speak version='1.0' xmlns='http://www.w3.org/2001/10/synthesis'
xmlns:mstts='https://www.w3.org/2001/mstts' xml:lang='en-US'>
  <voice name='en-US-MultiTalker-Ava-Andrew:DragonHDLatestNeural'>
    <mstts:dialog>
      <mstts:turn speaker="ava">Hello, Andrew! How's your day going?
    </mstts:turn>
      <mstts:turn speaker="andrew">Hey Ava! It's been great, just exploring
some AI advancements in communication.</mstts:turn>
      <mstts:turn speaker="ava">That sounds interesting! What kind of
projects are you working on?</mstts:turn>
      <mstts:turn speaker="andrew">Well, we've been experimenting with text-
to-speech applications, including turning emails into podcasts.</mstts:turn>
      <mstts:turn speaker="ava">Wow, that could really improve content
accessibility! Are you looking for collaborators?</mstts:turn>
      <mstts:turn speaker="andrew">Absolutely! We're open to testing new
ideas and seeing how AI can enhance communication.</mstts:turn>
    </mstts:dialog>
  </voice>
</speak>
```

For supported voices, see the [Language support](#) documentation.

Use speaking styles and roles

By default, neural voices have a neutral speaking style. You can adjust the speaking style, style degree, and role at the sentence level.

ⓘ Note

The Speech service supports styles, style degree, and roles for a subset of neural voices as described in the [voice styles and roles](#) documentation. To determine the supported styles and roles for each voice, you can also use the [list voices](#) API and the [audio content creation](#) [web application](#).

The following table describes the usage of the `mstts:express-as` element's attributes:

 Expand table

Attribute	Description	Required or optional
<code>style</code>	The voice-specific speaking style. You can express emotions like cheerfulness, empathy, and calmness. You can also optimize the voice for different scenarios like customer service, newscast, and voice assistant. If the style value is missing or invalid, the entire <code>mstts:express-as</code> element is ignored and the service uses the default neutral speech. For custom voice styles, see the custom voice style example .	Required
<code>styledegree</code>	The intensity of the speaking style. You can specify a stronger or softer style to make the speech more expressive or subdued. The range of accepted values are: <code>0.01</code> to <code>2</code> inclusive. The default value is <code>1</code> , which means the predefined style intensity. The minimum unit is <code>0.01</code> , which results in a slight tendency for the target style. A value of <code>2</code> results in a doubling of the default style intensity. If the style degree is missing or isn't supported for your voice, this attribute is ignored.	Optional
<code>role</code>	The speaking role-play. The voice can imitate a different age and gender, but the voice name isn't changed. For example, a male voice can raise the pitch and change the intonation to imitate a female voice, but the voice name isn't changed. If the role is missing or isn't supported for your voice, this attribute is ignored.	Optional

The following table describes each supported `style` attribute:

 Expand table

Style	Description
<code>style="advertisement_upbeat"</code>	Expresses an excited and high-energy tone for promoting a product or service.
<code>style="affectionate"</code>	Expresses a warm and affectionate tone, with higher pitch and vocal energy. The speaker is in a state of attracting the attention of the listener. The personality of the speaker is often endearing in nature.
<code>style="angry"</code>	Expresses an angry and annoyed tone.
<code>style="assistant"</code>	Expresses a warm and relaxed tone for digital assistants.
<code>style="calm"</code>	Expresses a cool, collected, and composed attitude when speaking. Tone, pitch, and prosody are more uniform compared to other

Style	Description
	types of speech.
<code>style="chat"</code>	Expresses a casual and relaxed tone.
<code>style="cheerful"</code>	Expresses a positive and happy tone.
<code>style="customerservice"</code>	Expresses a friendly and helpful tone for customer support.
<code>style="depressed"</code>	Expresses a melancholic and despondent tone with lower pitch and energy.
<code>style="disgruntled"</code>	Expresses a disdainful and complaining tone. Speech of this emotion displays displeasure and contempt.
<code>style="documentary-narration"</code>	Narrates documentaries in a relaxed, interested, and informative style suitable for documentaries, expert commentary, and similar content.
<code>style="embarrassed"</code>	Expresses an uncertain and hesitant tone when the speaker is feeling uncomfortable.
<code>style="empathetic"</code>	Expresses a sense of caring and understanding.
<code>style="envious"</code>	Expresses a tone of admiration when you desire something that someone else has.
<code>style="excited"</code>	Expresses an upbeat and hopeful tone. It sounds like something great is happening and the speaker is happy about it.
<code>style="fearful"</code>	Expresses a scared and nervous tone, with higher pitch, higher vocal energy, and faster rate. The speaker is in a state of tension and unease.
<code>style="friendly"</code>	Expresses a pleasant, inviting, and warm tone. It sounds sincere and caring.
<code>style="gentle"</code>	Expresses a mild, polite, and pleasant tone, with lower pitch and vocal energy.
<code>style="hopeful"</code>	Expresses a warm and yearning tone. It sounds like something good is expected to happen to the speaker.
<code>style="lyrical"</code>	Expresses emotions in a melodic and sentimental way.
<code>style="narration-professional"</code>	Expresses a professional, objective tone for content reading.
<code>style="narration-relaxed"</code>	Expresses a soothing and melodious tone for content reading.
<code>style="newscast"</code>	Expresses a formal and professional tone for narrating news.
<code>style="newscast-casual"</code>	Expresses a versatile and casual tone for general news delivery.

Style	Description
<code>style="newscast-formal"</code>	Expresses a formal, confident, and authoritative tone for news delivery.
<code>style="poetry-reading"</code>	Expresses an emotional and rhythmic tone while reading a poem.
<code>style="sad"</code>	Expresses a sorrowful tone.
<code>style="serious"</code>	Expresses a strict and commanding tone. Speaker often sounds stiffer and much less relaxed with firm cadence.
<code>style="shouting"</code>	Expresses a tone that sounds as if the voice is distant or in another location and making an effort to be clearly heard.
<code>style="sports_commentary"</code>	Expresses a relaxed and interested tone for broadcasting a sports event.
<code>style="sports_commentary_excited"</code>	Expresses an intensive and energetic tone for broadcasting exciting moments in a sports event.
<code>style="whispering"</code>	Expresses a soft tone that's trying to make a quiet and gentle sound.
<code>style="terrified"</code>	Expresses a scared tone, with a faster pace and a shakier voice. It sounds like the speaker is in an unsteady and frantic status.
<code>style="unfriendly"</code>	Expresses a cold and indifferent tone.

The following table has descriptions of each supported `role` attribute:

[Expand table](#)

Role	Description
<code>role="Girl"</code>	The voice imitates a girl.
<code>role="Boy"</code>	The voice imitates a boy.
<code>role="YoungAdultFemale"</code>	The voice imitates a young adult female.
<code>role="YoungAdultMale"</code>	The voice imitates a young adult male.
<code>role="OlderAdultFemale"</code>	The voice imitates an older adult female.
<code>role="OlderAdultMale"</code>	The voice imitates an older adult male.
<code>role="SeniorFemale"</code>	The voice imitates a senior female.
<code>role="SeniorMale"</code>	The voice imitates a senior male.

mstts express-as examples

For information about the supported values for attributes of the `mstts:express-as` element, see [Use speaking styles and roles](#).

Style and degree example

You use the `mstts:express-as` element to express emotions like cheerfulness, empathy, and calm. You can also optimize the voice for different scenarios like customer service, newscast, and voice assistant.

The following SSML example uses the `<mstts:express-as>` element with a `sad` style degree of 2.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="zh-CN">
  <voice name="zh-CN-XiaomoNeural">
    <mstts:express-as style="sad" styledegree="2">
      快走吧，路上一定要注意安全，早去早回。
    </mstts:express-as>
  </voice>
</speak>
```

Role example

Apart from adjusting the speaking styles and style degree, you can also adjust the `role` parameter so that the voice imitates a different age and gender. For example, a male voice can raise the pitch and change the intonation to imitate a female voice, but the voice name isn't changed.

This SSML snippet illustrates how the `role` attribute is used to change the role-play for `zh-CN-XiaomoNeural`.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="zh-CN">
  <voice name="zh-CN-XiaomoNeural">
    女儿看见父亲走了进来，问道：
    <mstts:express-as role="YoungAdultFemale" style="calm">
      “您来的挺快的，怎么过来的？”
    </mstts:express-as>
    父亲放下手提包，说：
```

```

<mstts:express-as role="OlderAdultMale" style="calm">
    “刚打车过来的，路上还挺顺畅。”
</mstts:express-as>
</voice>
</speak>

```

Custom voice style example

You can train your custom voice to speak with some preset styles such as `cheerful`, `sad`, and `whispering`. You can also [fine-tune a professional voice](#) to speak in a custom style as determined by your training data. To use your custom voice style in SSML, specify the style name that you previously entered in Speech Studio.

This example uses a custom voice named **my-custom-voice**. The custom voice speaks with the `cheerful` preset style and style degree of `2`, and then with a custom style named **my-custom-style** and style degree of `0.01`.

XML

```

<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
  <voice name="my-custom-voice">
    <mstts:express-as style="cheerful" styledegree="2">
      That'd be just amazing!
    </mstts:express-as>
    <mstts:express-as style="my-custom-style" styledegree="0.01">
      What's next?
    </mstts:express-as>
  </voice>
</speak>

```

Speaker profile ID

You use the `mstts:ttseembedding` element to specify the `speakerProfileId` property for a [personal voice](#). Personal voice is a custom voice trained on your own voice or your customer's voice. For more information, see [create a personal voice](#).

The following SSML example uses the `<mstts:ttseembedding>` element with a voice name and speaker profile ID.

XML

```

<speak version='1.0' xmlns='http://www.w3.org/2001/10/synthesis'
xmlns:mstts='http://www.w3.org/2001/mstts' xml:lang='en-US'>
  <voice xml:lang='en-US' xml:gender='Male' name='PhoenixV2Neural'>

```



```
<mstts:ttembedding speakerProfileId='your speaker profile ID here'>
```

I'm happy to hear that you find me amazing and that I have made your trip planning easier and more fun. 我很高兴听到你觉得我很了不起，我让你的旅行计划更轻松、更有趣。 Je suis heureux d'apprendre que vous me trouvez incroyable et que j'ai rendu la planification de votre voyage plus facile et plus amusante.

```
</mstts:ttembedding>
```

```
</voice>
```

```
</speak>
```

Adjust speaking languages

By default, multilingual voices can autodetect the language of the input text and speak in the language of the default locale of the input text without using SSML. Optionally, you can use the `<lang xml:lang>` element to adjust the speaking language for these voices to set the preferred accent such as `en-GB` for British English. You can adjust the speaking language at both the sentence level and word level. For information about the supported languages for multilingual voice, see [Multilingual voices with the lang element](#) for a table showing the `<lang>` syntax and attribute definitions.

The following table describes the usage of the `<lang xml:lang>` element's attributes:

[Expand table](#)

Attribute	Description	Required or optional
<code>xml:lang</code>	The language that you want the neural voice to speak.	Required to adjust the speaking language for the neural voice. If you're using <code>lang xml:lang</code> , the locale must be provided.

ⓘ Note

The `<lang xml:lang>` element is incompatible with the `prosody` and `break` elements. You can't adjust pause and prosody like pitch, contour, rate, or volume in this element.

Non-multilingual voices don't support the `<lang xml:lang>` element by design.

Multilingual voices with the lang element

Use the [multilingual voices section](#) to determine which speaking languages the Speech service supports for each neural voice, as demonstrated in the following example table. If the voice doesn't speak the language of the input text, the Speech service doesn't output synthesized audio.

Voice	Auto-detected language number	Auto-detected language (locale)	All locales number	All languages (locale) supported from SSML
en-US- AndrewMultilingualNeural ¹ (Male) en-US- AvaMultilingualNeural ¹ (Female) en-US- BrianMultilingualNeural ¹ (Male) en-US- EmmaMultilingualNeural ¹ (Female)	77	Afrikaans (af-ZA), Albanian (sq-AL), Amharic (am-ET), Arabic (ar-EG), Armenian (hy-AM), Azerbaijani (az-AZ), Bahasa Indonesian (id-ID), Bangla (bn-BD), Basque (eu-ES), Bengali (bn-IN), Bosnian (bs-BA), Bulgarian (bg-BG), Burmese (my-MM), Catalan (ca-ES), Chinese Cantonese (zh-HK), Chinese Mandarin (zh-CN), Chinese Taiwanese (zh-TW), Croatian (hr-HR), Czech (cs-CZ), Danish (da-DK), Dutch (nl-NL), English (en-US), Estonian (et-EE), Filipino (fil-PH), Finnish (fi-FI), French (fr-FR), Galician (gl-ES), Georgian (ka-GE), German (de-DE), Greek (el-GR), Hebrew (he-IL), Hindi (hi-IN), Hungarian (hu-HU), Icelandic (is-IS), Irish (ga-IE), Italian (it-IT), Japanese (ja-JP), Javanese (jv-ID), Kannada (kn-IN), Kazakh (kk-KZ), Khmer (km-KH),	91	Afrikaans (South Africa) (af-ZA), Albanian (Albania) (sq-AL), Amharic (Ethiopia) (am-ET), Arabic (Egypt) (ar-EG), Arabic (Saudi Arabia) (ar-SA), Armenian (Armenia) (hy-AM), Azerbaijani (Azerbaijan) (az-AZ), Basque (Basque) (eu-ES), Bengali (India) (bn-IN), Bosnian (Bosnia and Herzegovina) (bs-BA), Bulgarian (Bulgaria) (bg-BG), Burmese (Myanmar) (my-MM), Catalan (Spain) (ca-ES), Chinese (Cantonese, Traditional) (zh-HK), Chinese (Mandarin, Simplified) (zh-CN), Chinese (Taiwanese Mandarin) (zh-TW), Croatian (Croatia) (hr-HR), Czech (Czech) (cs-CZ), Danish (Denmark) (da-DK), Dutch (Belgium) (nl-BE), Dutch (Netherlands) (nl-NL), English (Australia) (en-AU), English (Canada) (en-CA), English (Hong Kong SAR) (en-HK), English (India) (en-IN), English (Ireland) (en-IE), English (United Kingdom) (en-GB), English (United States) (en-US), Estonian (Estonia) (et-EE), Filipino (Philippines) (fil-PH), Finnish (Finland) (fi-FI), French (Belgium) (fr-BE), French (Canada) (fr-CA), French (France) (fr-FR), French (Switzerland) (fr-CH), Galician (Galician) (gl-ES), Georgian (Georgia) (ka-GE),

Voice	Auto-detected language number	Auto-detected language (locale)	All locales number	All languages (locale) supported from SSML
		Korean (ko-KR), Lao (lo-LA), Latvian (lv-LV), Lithuanian (lt-LT), Macedonian (mk-MK), Malay (ms-MY), Malayalam (ml-IN), Maltese (mt-MT), Mongolian (mn-MN), Nepali (ne-NP), Norwegian Bokmål (nb-NO), Pashto (ps-AF), Persian (fa-IR), Polish (pl-PL), Portuguese (pt-BR), Romanian (ro-RO), Russian (ru-RU), Serbian (sr-RS), Sinhala (si-LK), Slovak (sk-SK), Slovene (sl-SI), Somali (so-SO), Spanish (es-ES), Sundanese (su-ID), Swahili (sw-KE), Swedish (sv-SE), Tamil (ta-IN), Telugu (te-IN), Thai (th-TH), Turkish (tr-TR), Ukrainian (uk-UA), Urdu (ur-PK), Uzbek (uz-UZ), Vietnamese (vi-VN), Welsh (cy-GB), Zulu (zu-ZA)		German (Austria) (de-AT), German (Germany) (de-DE), German (Switzerland) (de-CH), Greek (Greece) (el-GR), Hebrew (Israel) (he-IL), Hindi (India) (hi-IN), Hungarian (Hungary) (hu-HU), Icelandic (Iceland) (is-IS), Indonesian (Indonesia) (id-ID), Irish (Ireland) (ga-IE), Italian (Italy) (it-IT), Japanese (Japan) (ja-JP), Javanese (Indonesia) (jv-ID), Kannada (India) (kn-IN), Kazakh (Kazakhstan) (kk-KZ), Khmer (Cambodia) (km-KH), Korean (Korea) (ko-KR), Lao (Laos) (lo-LA), Latvian (Latvia) (lv-LV), Lithuanian (Lithuania) (lt-LT), Macedonian (North Macedonia) (mk-MK), Malay (Malaysia) (ms-MY), Malayalam (India) (ml-IN), Maltese (Malta) (mt-MT), Mongolian (Mongolia) (mn-MN), Nepali (Nepal) (ne-NP), Norwegian (Bokmål, Norway) (nb-NO), Pashto (Afghanistan) (ps-AF), Persian (Iran) (fa-IR), Polish (Poland) (pl-PL), Portuguese (Brazil) (pt-BR), Portuguese (Portugal) (pt-PT), Romanian (Romania) (ro-RO), Russian (Russia) (ru-RU), Serbian (Cyrillic, Serbia) (sr-RS), Sinhala (Sri Lanka) (si-LK), Slovak (Slovakia) (sk-SK), Slovenian (Slovenia) (sl-SI), Somali (Somalia) (so-SO), Spanish (Mexico) (es-MX), Spanish (Spain) (es-ES), Sundanese (Indonesia) (su-ID), Swahili (Kenya) (sw-

Voice	Auto-detected language number	Auto-detected language (locale)	All locales number	All languages (locale) supported from SSML
				KE), Swedish (Sweden) (sv-SE), Tamil (India) (ta-IN), Telugu (India) (te-IN), Thai (Thailand) (th-TH), Turkish (Türkiye) (tr-TR), Ukrainian (Ukraine) (uk-UA), Urdu (Pakistan) (ur-PK), Uzbek (Uzbekistan) (uz-UZ), Vietnamese (Vietnam) (vi-VN), Welsh (United Kingdom) (cy-GB), Zulu (South Africa) (zu-ZA)

¹ Those are neural multilingual voices in Azure AI Speech. All multilingual voices can speak in the language in default locale of the input text without [using SSML](#). However, you can still use the `<lang xml:lang>` element to adjust the speaking accent of each language to set preferred accent such as British accent (en-GB) for English. The prefix in each voice name indicates its primary locale; for example, the primary locale for en-US-AndrewMultilingualNeural is en-US.

ⓘ Note

Multilingual voices don't fully support certain SSML elements, such as break, emphasis, silence, and sub.

Lang examples

For information about the supported values for attributes of the lang element, see [Adjust speaking language](#).

You must specify en-US as the default language within the speak element, whether or not the language is adjusted elsewhere. In this example, the primary language for en-US-AvaMultilingualNeural is en-US.

This SSML snippet shows how to use <lang xml:lang> to speak de-DE with the en-US-AvaMultilingualNeural neural voice.

```
XML
```

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    <lang xml:lang="de-DE">
      Wir freuen uns auf die Zusammenarbeit mit Ihnen!
    </lang>
  </voice>
</speak>
```

Within the `speak` element, you can specify multiple languages including `en-US` for text to speech output. For each adjusted language, the text must match the language and be wrapped in a `voice` element. This SSML snippet shows how to use `<lang xml:lang>` to change the speaking languages to `es-MX`, `en-US`, and `fr-FR`.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    <lang xml:lang="es-MX">
      ¡Esperamos trabajar con usted!
    </lang>
    <lang xml:lang="en-US">
      We look forward to working with you!
    </lang>
    <lang xml:lang="fr-FR">
      Nous avons hâte de travailler avec vous!
    </lang>
  </voice>
</speak>
```

Adjust prosody

You can use the `prosody` element to specify changes to pitch, contour, range, rate, and volume for the text to speech output. The `prosody` element can contain text and the following elements: `audio`, `break`, `p`, `phoneme`, `prosody`, `say-as`, `sub`, and `s`.

Because prosodic attribute values can vary over a wide range, the speech recognizer interprets the assigned values as a suggestion of what the actual prosodic values of the selected voice should be. Text to speech limits or substitutes values that aren't supported. Examples of unsupported values are a pitch of 1 MHz or a volume of 120.

The following table describes the usage of the `prosody` element's attributes:

Attribute	Description	Required or optional
<code>contour</code>	Contour represents changes in pitch. These changes are represented as an array of targets at specified time positions in the speech output. Sets of parameter pairs define each target. For example: <pre><prosody contour="(0%,+20Hz) (10%,-2st) (40%,+10Hz)"></pre> The first value in each set of parameters specifies the location of the pitch change as a percentage of the duration of the text. The second value specifies the amount to raise or lower the pitch by using a relative value or an enumeration value for pitch (see <code>pitch</code>). Pitch contour doesn't work on single words and short phrases. It's recommended to adjust the pitch contour on whole sentences or long phrases.	Optional
<code>pitch</code>	Indicates the baseline pitch for the text. Pitch changes can be applied at the sentence level. The pitch changes should be within 0.5 to 1.5 times the original audio. You can express the pitch as: - An absolute value: Expressed as a number followed by "Hz" (Hertz). For example, <code><prosody pitch="600Hz">some text</prosody></code>. - A relative value: - As a relative number: Expressed as a number preceded by "+" or "-" and followed by "Hz" or "st" that specifies an amount to change the pitch. For example: <code><prosody pitch="+80Hz">some text</prosody></code> or <code><prosody pitch="-2st">some text</prosody></code>. The "st" indicates the change unit is semitone, which is half of a tone (a half step) on the standard diatonic scale. - As a percentage: Expressed as a number preceded by "+" (optionally) or "-" and followed by "%", indicating the relative change. For example: <code><prosody pitch="50%">some text</prosody></code> or <code><prosody pitch="-50%">some text</prosody></code>. - A constant value: <code>x-low</code> (equivalently 0.55, -45%) <code>low</code> (equivalently 0.8, -20%) <code>medium</code> (equivalently 1, default value) <code>high</code> (equivalently 1.2, +20%) <code>x-high</code> (equivalently 1.45, +45%)	Optional
<code>range</code>	A value that represents the range of pitch for the text. You can express <code>range</code> by using the same absolute values, relative values, or enumeration values used to describe <code>pitch</code>.	Optional
<code>rate</code>	Indicates the speaking rate of the text. Speaking rate can be applied at the word or sentence level. The rate changes should be within 0.5 to 2 times the original audio. You can express <code>rate</code> as:	Optional

Attribute	Description	Required or optional
	- A relative value: - As a relative number: Expressed as a number that acts as a multiplier of the default. For example, a value of <code>1</code> results in no change in the original rate. A value of <code>0.5</code> results in a halving of the original rate. A value of <code>2</code> results in twice the original rate. - As a percentage: Expressed as a number preceded by "+" (optionally) or "-" and followed by "%", indicating the relative change. For example: <code><prosody rate="50%">some text</prosody></code> or <code><prosody rate="-50%">some text</prosody></code>. - A constant value: <code>x-slow</code> (equivalently 0.5, -50%) <code>slow</code> (equivalently 0.64, -46%) <code>medium</code> (equivalently 1, default value) <code>fast</code> (equivalently 1.55, +55%) <code>x-fast</code> (equivalently 2, +100%)	
<code>volume</code>	Indicates the volume level of the speaking voice. Volume changes can be applied at the sentence level. You can express the volume as: - An absolute value: Expressed as a number in the range of <code>0.0</code> to <code>100.0</code>, from <i>quietest</i> to <i>loudest</i>, such as <code>75</code>. The default value is <code>100.0</code>. - A relative value: - As a relative number: Expressed as a number preceded by "+" or "-" that specifies an amount to change the volume. Examples are <code>+10</code> or <code>-5.5</code>. - As a percentage: Expressed as a number preceded by "+" (optionally) or "-" and followed by "%", indicating the relative change. For example: <code><prosody volume="50%">some text</prosody></code> or <code><prosody volume="+3%">some text</prosody></code>. - A constant value: <code>silent</code> (equivalently 0) <code>x-soft</code> (equivalently 0.2) <code>soft</code> (equivalently 0.4) <code>medium</code> (equivalently 0.6) <code>loud</code> (equivalently 0.8) <code>x-loud</code> (equivalently 1, default value)	Optional

Prosody examples

For information about the supported values for attributes of the `prosody` element, see [Adjust prosody](#).

Change speaking rate example

This SSML snippet illustrates how the `rate` attribute is used to change the speaking rate to 30% greater than the default rate.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    <prosody rate="+30.00%">
      Enjoy using text to speech.
    </prosody>
  </voice>
</speak>
```

Change volume example

This SSML snippet illustrates how the `volume` attribute is used to change the volume to 20% greater than the default volume.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    <prosody volume="+20.00%">
      Enjoy using text to speech.
    </prosody>
  </voice>
</speak>
```

Change pitch example

This SSML snippet illustrates how the `pitch` attribute is used so that the voice speaks in a high pitch.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    Welcome to <prosody pitch="high">Enjoy using text to speech.</prosody>
  </voice>
</speak>
```

Change pitch contour example

This SSML snippet illustrates how the `contour` attribute is used to change the contour.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    <prosody contour="(60%,-60%) (100%,+80%)" >
      Were you the only person in the room?
    </prosody>
  </voice>
</speak>
```

Adjust emphasis

You can use the optional `emphasis` element to add or remove word-level stress for the text. This element can only contain text and the following elements: `audio`, `break`, `emphasis`, `lang`, `phoneme`, `prosody`, `say-as`, `sub`, and `voice`.

ⓘ Note

The word-level emphasis tuning is only available for these neural voices: `en-US-GuyNeural`, `en-US-DavisNeural`, and `en-US-JaneNeural`.

For words that have low pitch and short duration, the pitch might not be raised enough to be noticed.

The following table describes the `emphasis` element's attributes:

⌵ Expand table

Attribute	Description	Required or optional
<code>level</code>	Indicates the strength of emphasis to be applied: <code>reduced</code> <code>none</code> <code>moderate</code> <code>strong</code> When the <code>level</code> attribute isn't specified, the default level is <code>moderate</code>. For details on each attribute, see emphasis element.	Optional

Emphasis examples

For information about the supported values for attributes of the `emphasis` element, see [Adjust emphasis](#).

This SSML snippet demonstrates how you can use the `emphasis` element to add moderate level emphasis for the word "meetings."

XML

```
<speaking version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
  <voice name="en-US-AndrewMultilingualNeural">
    I can help you join your <emphasis level="moderate">meetings</emphasis> fast.
  </voice>
</speaking>
```

Add recorded audio

The `audio` element is optional. You can use it to insert prerecorded audio into an SSML document. The body of the `audio` element can contain plain text or SSML markup spoken if the audio file is unavailable or unplayable. The `audio` element can also contain text and the following elements: `audio`, `break`, `p`, `s`, `phoneme`, `prosody`, `say-as`, and `sub`.

Any audio included in the SSML document must meet these requirements:

- The audio file must be valid `*.mp3`, `*.wav`, `*.opus`, `*.ogg`, `*.flac`, or `*.wma` files.
- The combined total time for all text and audio files in a single response can't exceed 600 seconds.
- The audio must not contain any customer-specific or other sensitive information.

ⓘ Note

The `audio` element isn't supported by the [Long Audio API](#). For long-form text to speech, use the [batch synthesis API](#) instead.

The following table describes the usage of the `audio` element's attributes:

[↗](#) Expand table

Attribute	Description	Required or optional
<code>src</code>	The URI location of the audio file. The audio must be hosted on an internet-accessible HTTPS endpoint. HTTPS is required. The domain hosting the file must present a valid, trusted TLS/SSL certificate. You should put the audio file into Blob Storage in the same Azure region as the text to speech endpoint to minimize the latency.	Required

Audio examples

For information about the supported values for attributes of the `audio` element, see [Add recorded audio](#).

This SSML snippet illustrates how to use `src` attribute to insert audio from two .wav files.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    <p>
      <audio src="https://contoso.com/opinionprompt.wav"/>
        Thanks for offering your opinion. Please begin speaking after the
        beep.
      <audio src="https://contoso.com/beep.wav">
        Could not play the beep, please voice your opinion now.
      </audio>
    </p>
  </voice>
</speak>
```

Adjust the audio duration

Use the `mstts:audioduration` element to set the duration of the output audio. Use this element to help synchronize the timing of audio output completion. The audio duration can be decreased or increased between `0.5` to `2` times the rate of the original audio. The original audio is the audio without any other rate settings. The speaking rate is slowed down or sped up accordingly based on the set value.

The audio duration setting applies to all input text within its enclosing `voice` element. To reset or change the audio duration setting again, you must use a new `voice` element with either the same voice or a different voice.

The following table describes the usage of the `mstts:audioduration` element's attributes:

Attribute	Description	Required or optional
<code>value</code>	The requested duration of the output audio in either seconds, such as <code>2s</code>, or milliseconds, such as <code>2000ms</code>. The maximum value for output audio duration is 300 seconds. This value should be within <code>0.5</code> to <code>2</code> times the original audio without any other rate settings. For example, if the requested duration of your audio is <code>30s</code>, then the original audio must otherwise be between 15 and 60 seconds. If you set a value outside of these boundaries, the duration is set according to the respective minimum or maximum multiple. For output audio longer than 300 seconds, first generate the original audio without any other rate settings, then calculate the rate to adjust using the prosody rate to achieve the desired duration.	Required

mstts audio duration examples

For information about the supported values for attributes of the `mstts:audioduration` element, see [Adjust the audio duration](#).

In this example, the original audio is around 15 seconds. The `mstts:audioduration` element is used to set the audio duration to 20 seconds or `20s`.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="http://www.w3.org/2001/mstts" xml:lang="en-US">
<voice name="en-US-AvaMultilingualNeural">
<mstts:audioduration value="20s"/>
```

If we're home schooling, the best we can do is roll with what each day brings and try to have fun along the way.

A good place to start is by trying out the slew of educational apps that are helping children stay happy and smash their schooling at the same time.

```
</voice>
</speak>
```

Add background audio

You can use the `mstts:backgroundaudio` element to add background audio to your SSML documents or mix an audio file with text to speech. With `mstts:backgroundaudio`, you can loop an audio file in the background, fade in at the beginning of text to speech, and fade out at the end of text to speech.

If the background audio provided is shorter than the text to speech or the fade out, it loops. If it's longer than the text to speech, it stops when the fade out is finished.

Only one background audio file is allowed per SSML document. You can intersperse `audio` tags within the `voice` element to add more audio to your SSML document.

Note

The `mstts:backgroundaudio` element should be put in front of all `voice` elements. If specified, it must be the first child of the `speak` element.

The `mstts:backgroundaudio` element isn't supported by the [Long Audio API](#). For long-form text to speech, use the [batch synthesis API](#) (Preview) instead.

The following table describes the usage of the `mstts:backgroundaudio` element's attributes:

[Expand table](#)

Attribute	Description	Required or optional
<code>src</code>	The URI location of the background audio file.	Required
<code>volume</code>	The volume of the background audio file. Accepted values: <code>0</code> to <code>100</code> inclusive. The default value is <code>1</code> .	Optional
<code>fadein</code>	The duration of the background audio fade-in as milliseconds. The default value is <code>0</code> , which is the equivalent to no fade in. Accepted values: <code>0</code> to <code>10000</code> inclusive.	Optional
<code>fadeout</code>	The duration of the background audio fade-out in milliseconds. The default value is <code>0</code> , which is the equivalent to no fade out. Accepted values: <code>0</code> to <code>10000</code> inclusive.	Optional

mstss backgroundaudio examples

For information about the supported values for attributes of the `mstts:backgroundaudi` element, see [Add background audio](#).

XML

```
<speak version="1.0" xml:lang="en-US" xmlns:mstts="http://www.w3.org/2001/mstts">
  <mstts:backgroundaudio src="https://contoso.com/sample.wav" volume="0.7"
    fadein="3000" fadeout="4000"/>
  <voice name="en-US-AvaMultilingualNeural">
```

The text provided in this document are spoken over the background audio.

```
</voice>
</speak>
```

Viseme element

A viseme is the visual description of a phoneme in spoken language. It defines the position of the face and mouth while a person is speaking. You can use the `mstts:viseme` element in SSML to request viseme output. For more information, see [Get facial position with viseme](#).

The viseme setting is applied to all input text within its enclosing `voice` element. To reset or change the viseme setting again, you must use a new `voice` element with either the same voice or a different voice.

Usage of the `viseme` element's attributes are described in the following table.

[Expand table](#)

Attribute	Description	Required or optional
<code>type</code>	The type of viseme output. <code>redlips_front</code> – lip-sync with viseme ID and audio offset output <code>FacialExpression</code> – blend shapes output	Required

⚠ Note

Currently, `redlips_front` only supports neural voices in `en-US` locale, and `FacialExpression` supports neural voices in `en-US` and `zh-CN` locales.

Viseme examples

The supported values for attributes of the `viseme` element were [described previously](#).

This SSML snippet illustrates how to request blend shapes with your synthesized speech.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="http://www.w3.org/2001/mstts" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
```

```
<mstts:viseme type="FacialExpression"/>
Rainbow has seven colors: Red, orange, yellow, green, blue, indigo, and
violet.
</voice>
</speak>
```

Voice conversion element

Voice conversion (preview) is the process of transforming the voice characteristics of a given audio to a target voice speaker. After voice conversion, the resulting audio reserves source audio's linguistic content and prosody while the voice timbre sounds like the target speaker. For more information, see [voice conversion](#).

Use the `<mstts:voiceconversion>` tag via Speech Synthesis Markup Language (SSML) to specify the source audio URL and the target voice for the conversion. For a complete list of supported target voices, see [supported voices for voice conversion](#).

The following table describes the usage of the `mstts:voiceconversion` element's attributes:

 Expand table

Attribute	Description	Required or optional
<code>url</code>	The URL of the source audio file that provides linguistic content and prosody for the synthesized speech. The <code>url</code> must be accessible via HTTPS URL. For example, <code>https://example.com/source.wav</code> Input audio must be under 100 MB.	Required

Here's how the voice conversion works:

- The source audio is a prerecorded audio file that contains the spoken words and prosody.
 - Text content: The final synthesized speech follows the spoken words in the source audio.
 - Prosody and rhythm: The speech maintains the timing and intonation from the source.
- The `<voice>` tag specifies the target voice used for the output audio. For information about the supported target voices, see [supported voices for voice conversion](#).
- The output audio keeps the timbre (tone and voice quality) of the target voice, but follows the text and speaking style of the source audio.

ⓘ Note

All SSML elements related to prosody and pronunciation such as `<prosody>` or `<mstts:express-as>` are ignored.

Text input is optional and any text included in the SSML are ignored during rendering.

mstss voiceconversion examples

The following example demonstrates how to use `<mstts:voiceconversion>` to synthesize speech using a **target neural voice** while **matching both the content and prosody** of a given source audio:

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
  <voice xml:lang="en-US" xml:gender="Female" name="en-US-
AvaMultilingualNeural">
    <mstts:voiceconversion
url="https://your.blob.core.windows.net/sourceaudio.wav"/>
  </voice>
</speak>
```

Next steps

- [SSML overview](#)
- [SSML document structure and events](#)
- [Language and voice support for the Speech service](#)

Pronunciation with SSML

09/26/2025

You can use Speech Synthesis Markup Language (SSML) with text to speech to specify how the speech is pronounced. For example, you can use SSML with phonemes and a custom lexicon to improve pronunciation. You can also use SSML to define how a word or mathematical expression is pronounced.

Refer to the following sections for details about how to use SSML elements to improve pronunciation. For more information about SSML syntax, see [SSML document structure and events](#).

phoneme element

The `phoneme` element is used for phonetic pronunciation in SSML documents. Always provide human-readable speech as a fallback.

Phonetic alphabets are composed of phones, which are made up of letters, numbers, or characters, sometimes in combination. Each phone describes a unique sound of speech. The phonetic alphabet is in contrast to the Latin alphabet, where any letter might represent multiple spoken sounds. Consider the different `en-US` pronunciations of the letter "c" in the words "candy" and "cease" or the different pronunciations of the letter combination "th" in the words "thing" and "those."

ⓘ

Note

For a list of locales that support phonemes, see footnotes in the [language support](#) table.

Usage of the `phoneme` element's attributes are described in the following table.

 Expand table

Attribute	Description	Required or optional
<code>alphabet</code>	The phonetic alphabet to use when you synthesize the pronunciation of the string in the <code>ph</code> attribute. The string that specifies the alphabet must be specified in lowercase letters. The following options are the possible alphabets that you can specify: <code>ipa</code> – See SSML phonetic alphabets <code>sapi</code> – See SSML phonetic alphabets	Optional

Attribute	Description	Required or optional
	• <code>ups</code> – See Universal Phone Set ↗ • <code>x-sampa</code> – See SSML phonetic alphabets The alphabet applies only to the <code>phoneme</code> in the element.	
<code>ph</code>	A string containing phones that specify the pronunciation of the word in the <code>phoneme</code> element. If the specified string contains unrecognized phones, text to speech rejects the entire SSML document and produces none of the speech output specified in the document. For <code>ipa</code>, to stress one syllable by placing stress symbol before this syllable, you need to mark all syllables for the word. Or else, the syllable before this stress symbol is stressed. For <code>sapi</code>, if you want to stress one syllable, you need to place the stress symbol after this syllable, whether or not all syllables of the word are marked.	Required

phoneme examples

The supported values for attributes of the `phoneme` element were [described previously](#). In the first two examples, the values of `ph="tə.'meɪ.tou"` or `ph="təmeɪ'tou"` are specified to stress the syllable `meɪ`.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
    <phoneme alphabet="ipa" ph="tə.'meɪ.tou"> tomato </phoneme>
  </voice>
</speak>
```

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
    <phoneme alphabet="ipa" ph="təmeɪ'tou"> tomato </phoneme>
  </voice>
</speak>
```

XML

```
<speak version="1.0" xmlns="https://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
```

```
<phoneme alphabet="sapi" ph="iy eh n y uw eh s"> en-US </phoneme>
</voice>
</speak>
```

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
    <s>His name is Mike <phoneme alphabet="ups" ph="JH AU"> Zhou </phoneme>
  </s>
</voice>
</speak>
```

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
    <phoneme alphabet='x-sampa' ph='he."lou'>hello</phoneme>
  </voice>
</speak>
```

Custom lexicon

You can define how single entities (such as company, a medical term, or an emoji) are read in SSML by using the [phoneme](#) and [sub](#) elements. To define how multiple entities are read, create an XML structured custom lexicon file. Then you upload the custom lexicon XML file and reference it with the SSML `lexicon` element.

ⓘ Note

For a list of locales that support custom lexicon, see footnotes in the [language support](#) table.

The `lexicon` element isn't supported by the [Long Audio API](#). For long-form text to speech, use the [batch synthesis API](#) (Preview) instead.

Usage of the `lexicon` element's attributes are described in the following table.

[Expand table](#)

Attribute	Description	Required or optional
<code>uri</code>	The URI of the publicly accessible custom lexicon XML file with either the <code>.xml</code> or <code>.pls</code> file extension. Using Azure Blob Storage is recommended but not required. For more information about the custom lexicon file, see Pronunciation Lexicon Specification (PLS) Version 1.0 .	Required

Custom lexicon examples

The supported values for attributes of the `lexicon` element were [described previously](#).

After you publish your custom lexicon, you can reference it from your SSML. The following SSML example references a custom lexicon that was uploaded to `https://www.example.com/customlexicon.xml`. We support lexicon URLs from Azure Blob Storage. However, note that other public URLs may not be compatible.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
  xmlns:mstts="http://www.w3.org/2001/mstts"
  xml:lang="en-US">
  <voice name="en-US-AvaNeural">
    <lexicon uri="https://www.example.com/customlexicon.xml"/>
    BTW, we will be there probably at 8:00 tomorrow morning.
    Could you help leave a message to Robert Benigni for me?
  </voice>
</speak>
```

Custom lexicon file

To define how multiple entities are read, you can define them in a custom lexicon XML file with either the `.xml` or `.pls` file extension.

ⓘ Note

The custom lexicon file is a valid XML document, but it can't be used as an SSML document.

Here are some limitations of the custom lexicon file:

- **File size:** The custom lexicon file size is limited to a maximum of 100 KB. If the file size exceeds the 100-KB limit, the synthesis request fails. You can split your lexicon into

multiple lexicons and include them in SSML if the file size exceeds 100 KB.

- **Lexicon cache refresh:** The custom lexicon is cached with the URI as the key on text to speech when it's first loaded. The lexicon with the same URI isn't reloaded within 15 minutes, so the custom lexicon change needs to wait 15 minutes at the most to take effect.

The supported elements and attributes of a custom lexicon XML file are described in the [Pronunciation Lexicon Specification \(PLS\) Version 1.0](#). Here are some examples of the supported elements and attributes:

- The `lexicon` element contains at least one `lexeme` element. Lexicon contains the necessary `xml:lang` attribute to indicate which locale it should be applied for. One custom lexicon is limited to one locale by design, so if you apply it for a different locale, it doesn't work. The `lexicon` element also has an `alphabet` attribute to indicate the alphabet used in the lexicon. The possible values are `ipa` and `x-microsoft-sapi`.
- Each `lexeme` element contains at least one `grapheme` element and one or more `grapheme`, `alias`, and `phoneme` elements. The `lexeme` element is case sensitive in the custom lexicon. For example, if you only provide a phoneme for the `lexeme` "Hello", it doesn't work for the `lexeme` "hello".
- The `grapheme` element contains text that describes the [orthography](#).
- The `alias` elements are used to indicate the pronunciation of an acronym or an abbreviated term.
- The `phoneme` element provides text that describes how the `lexeme` is pronounced. The syllable boundary is '.' in the IPA alphabet. The `phoneme` element can't contain white space when you use the IPA alphabet.
- When the `alias` and `phoneme` elements are provided with the same `grapheme` element, `alias` has higher priority.

Microsoft provides a [validation tool for the custom lexicon](#) that helps you find errors (with detailed error messages) in the custom lexicon file. Using the tool is recommended before you use the custom lexicon XML file in production with the Speech service.

Custom lexicon file examples

The following XML example (not SSML) would be contained in a custom lexicon `.xml` file.

When you use this custom lexicon, "BTW" is read as "By the way." "Benigni" is read with the provided IPA "be'ni:nji."

```

<?xml version="1.0" encoding="UTF-8"?>
<lexicon version="1.0"
  xmlns="http://www.w3.org/2005/01/pronunciation-lexicon"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.w3.org/2005/01/pronunciation-lexicon
    http://www.w3.org/TR/2007/CR-pronunciation-lexicon-20071212/pls.xsd"
  alphabet="ipa" xml:lang="en-US">
  <lexeme>
    <grapheme>BTW</grapheme>
    <alias>By the way</alias>
  </lexeme>
  <lexeme>
    <grapheme>Benigni</grapheme>
    <phoneme>bɛ'ni:nji</phoneme>
  </lexeme>
  <lexeme>
    <grapheme>😊</grapheme>
    <alias>test emoji</alias>
  </lexeme>
</lexicon>

```

You can't directly set the pronunciation of a phrase by using the custom lexicon. If you need to set the pronunciation for an acronym or an abbreviated term, first provide an `alias`, and then associate the `phoneme` with that `alias`. For example:

XML

```

<?xml version="1.0" encoding="UTF-8"?>
<lexicon version="1.0"
  xmlns="http://www.w3.org/2005/01/pronunciation-lexicon"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.w3.org/2005/01/pronunciation-lexicon
    http://www.w3.org/TR/2007/CR-pronunciation-lexicon-20071212/pls.xsd"
  alphabet="ipa" xml:lang="en-US">
  <lexeme>
    <grapheme>Scotland MV</grapheme>
    <alias>ScotlandMV</alias>
  </lexeme>
  <lexeme>
    <grapheme>ScotlandMV</grapheme>
    <phoneme>'skɒtlənd.'mi:diəm.wɛrv</phoneme>
  </lexeme>
</lexicon>

```

You could also directly provide your expected `alias` for the acronym or abbreviated term. For example:

XML

```

<lexeme>
  <grapheme>Scotland MV</grapheme>
  <alias>Scotland Media Wave</alias>
</lexeme>

```

The preceding custom lexicon XML file examples use the IPA alphabet, which is also known as the IPA phone set. We suggest that you use the IPA because it's the international standard. For some IPA characters, they're the "precomposed" and "decomposed" version when they're being represented with Unicode. The custom lexicon only supports the decomposed Unicode.

The Speech service defines a phonetic set for these locales: `en-US`, `fr-FR`, `de-DE`, `es-ES`, `ja-JP`, `zh-CN`, `zh-HK`, and `zh-TW`. For more information on the detailed Speech service phonetic alphabet, see the [Speech service phonetic sets](#).

You can use the `x-microsoft-sapi` as the value for the `alphabet` attribute with custom lexicons as demonstrated here:

XML

```

<?xml version="1.0" encoding="UTF-8"?>
<lexicon version="1.0"
  xmlns="http://www.w3.org/2005/01/pronunciation-lexicon"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.w3.org/2005/01/pronunciation-lexicon
    http://www.w3.org/TR/2007/CR-pronunciation-lexicon-20071212/pls.xsd"
  alphabet="x-microsoft-sapi" xml:lang="en-US">
  <lexeme>
    <grapheme>BTW</grapheme>
    <alias> By the way </alias>
  </lexeme>
  <lexeme>
    <grapheme> Benigni </grapheme>
    <phoneme> b eh 1 - n iy - n y iy </phoneme>
  </lexeme>
</lexicon>

```

say-as element

The `say-as` element indicates the content type, such as number or date, of the element's text. This element provides guidance to the speech synthesis engine about how to pronounce the text.

Usage of the `say-as` element's attributes are described in the following table.

[Expand table](#)

Attribute	Description	Required or optional
<code>interpret-as</code>	Indicates the content type of an element's text. For a list of types, see the following table.	Required
<code>format</code>	Provides additional information about the precise formatting of the element's text for content types that might have ambiguous formats. SSML defines formats for content types that use them. See the following table.	Optional
<code>detail</code>	Indicates the level of detail to be spoken. For example, this attribute might request that the speech synthesis engine pronounce punctuation marks. There are no standard values defined for <code>detail</code> .	Optional

The following content types are supported for the `interpret-as` and `format` attributes. Include the `format` attribute only if `format` column isn't empty in this table.

ⓘ Note

The `characters` and `spell-out` values for the `interpret-as` attribute are supported for all [text to speech locales](#). Other `interpret-as` attribute values are supported for all locales of the following languages: Arabic, Catalan, Chinese, Danish, Dutch, English, French, Finnish, German, Hindi, Italian, Japanese, Korean, Norwegian, Polish, Portuguese, Russian, Spanish, and Swedish.

[Expand table](#)

<code>interpret-as</code>	<code>format</code>	Interpretation
<code>characters</code> , <code>spell-out</code>		The text is spoken as individual letters (spelled out). The speech synthesis engine pronounces: <pre><say-as interpret-as="characters">test</say-as></pre> As "T E S T."
<code>cardinal</code> , <code>number</code>	None	The text is spoken as a cardinal number. The speech synthesis engine pronounces: <pre>There are <say-as interpret-as="cardinal">10</say-as> options</pre> As "There are ten options."

interpret-as	format	Interpretation
<code>ordinal</code>	None	The text is spoken as an ordinal number. The speech synthesis engine pronounces: <pre>Select the <say-as interpret-as="ordinal">3rd</say-as> option</pre> As "Select the third option."
<code>number_digit</code>	None	The text is spoken as a sequence of individual digits. The speech synthesis engine pronounces: <pre><say-as interpret-as="number_digit">123456789</say-as></pre> As "1 2 3 4 5 6 7 8 9."
<code>fraction</code>	None	The text is spoken as a fractional number. The speech synthesis engine pronounces: <pre><say-as interpret-as="fraction">3/8</say-as> of an inch</pre> As "three eighths of an inch."
<code>date</code>	dmy, mdy, ymd, ydm, ym, my, md, dm, d, m, y	The text is spoken as a date. The <code>format</code> attribute specifies the date's format (<i>d=day, m=month, and y=year</i>). The speech synthesis engine pronounces: <pre>Today is <say-as interpret-as="date">10-12-2016</say-as></pre> As "Today is October twelfth two thousand sixteen." Pronounces: <pre>Today is <say-as interpret-as="date" format="dmy">10-12-2016</say-as></pre> As "Today is December tenth two thousand sixteen."
<code>time</code>	hms12, hms24	The text is spoken as a time. The <code>format</code> attribute specifies whether the time is specified by using a 12-hour clock (hms12) or a 24-hour clock (hms24). Use a colon to separate numbers representing hours, minutes, and seconds. Here are some valid time examples: 12:35, 1:14:32, 08:15, and 02:50:45. The speech synthesis engine pronounces: <pre>The train departs at <say-as interpret-as="time" format="hms12">4:00am</say-as></pre> As "The train departs at four A M."

interpret-as	format	Interpretation
duration	hms, hm, ms	The text is spoken as a duration. The <code>format</code> attribute specifies the duration's format (<i>h=hour, m=minute, and s=second</i>). The speech synthesis engine pronounces: <pre><say-as interpret-as="duration">01:18:30</say-as></pre> As "one hour eighteen minutes and thirty seconds". Pronounces: <pre><say-as interpret-as="duration" format="ms">01:18</say-as></pre> As "one minute and eighteen seconds". This tag is only supported on English and Spanish.
telephone	None	The text is spoken as a telephone number. The speech synthesis engine pronounces: <pre>The number is <say-as interpret-as="telephone">(888) 555-1212</say-as></pre> As "My number is area code eight eight eight five five five one two one two."
currency	None	The text is spoken as a currency. The speech synthesis engine pronounces: <pre><say-as interpret-as="currency">99.9 USD</say-as></pre> As "ninety-nine US dollars and ninety cents."
address	None	The text is spoken as an address. The speech synthesis engine pronounces: <pre>I'm at <say-as interpret-as="address">150th CT NE, Redmond, WA</say-as></pre> As "I'm at 150th Court Northeast Redmond Washington."
name	None	The text is spoken as a person's name. The speech synthesis engine pronounces: <pre><say-as interpret-as="name">ED</say-as></pre> As [æd]. In Chinese names, some characters pronounce differently when they appear in a family name. For example, the speech synthesis engine says 仇 in

interpret-as	format	Interpretation
		<code><say-as interpret-as="name">仇先生</say-as></code>
		As [qiú] instead of [chóu].

say-as examples

The supported values for attributes of the `say-as` element were [described previously](#).

The speech synthesis engine speaks the following example as "Your first request was for one room on October nineteenth twenty ten with early arrival at twelve thirty five PM."

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    <p>
      Your <say-as interpret-as="ordinal"> 1st </say-as> request was for <say-as
interpret-as="cardinal"> 1 </say-as> room
      on <say-as interpret-as="date" format="mdy"> 10/19/2010 </say-as>, with
early arrival at <say-as interpret-as="time" format="hms12"> 12:35pm </say-as>.
    </p>
  </voice>
</speak>
```

sub element

Use the `sub` element to indicate that the alias attribute's text value should be pronounced instead of the element's enclosed text. In this way, the SSML contains both a spoken and written form.

Usage of the `sub` element's attributes are described in the following table.

[Expand table](#)

Attribute	Description	Required or optional
<code>alias</code>	The text value that should be pronounced instead of the element's enclosed text.	Required

sub examples

The supported values for attributes of the `sub` element were [described previously](#).

The speech synthesis engine speaks the following example as "World Wide Web Consortium."

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    <sub alias="World Wide Web Consortium">W3C</sub>
  </voice>
</speak>
```

Mathematical expressions reading

There are two ways to read a mathematical expression:

- With Math domain element,

Embed the plain text mathematical expression directly in SSML and specify the math domain using `<mstts:prompt domain="Math" />`.

See the section: [Reading plain text mathematical expressions](#)

- With MathML elements

Represent the mathematical expression with MathML elements.

See the section: [Reading mathematical expressions with MathML](#)

ⓘ Note

The two features are currently supported in the following locales: de-DE, en-AU, en-GB, en-US, all the sibling locales of English, es-ES, es-MX, all the sibling locales of Spanish, fr-CA, fr-FR, it-IT, ja-JP, ko-KR, pt-BR, and zh-CN.

Reading plain text mathematical expressions

To enable complex mathematical expression reading, you can add `<mstts:prompt domain="Math" />` element to enable math-specific pronunciation rules.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
```

```

<voice name="en-US-AvaMultilingualNeural">
  <mstts:prompt domain="Math" />
  x = (-b ± √(b² - 4ac)) / 2a
</voice>
</speak>

```

By default, parentheses aren't read out in mathematical expressions. If you'd like the parentheses read out, you can specify `<mstts:mathspeechverbosity level="verbose" />` in SSML

XML

```

<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    <mstts:prompt domain="Math" /><mstts:mathspeechverbosity level="verbose" />
    x = (-b ± √(b² - 4ac)) / 2a
  </voice>
</speak>

```

If you'd like the expression read out in other language with a multilingual voice, specify lang element in SSML.

XML

```

<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    <mstts:prompt domain="Math" />
    <lang xml:lang="es-ES">x = (-b ± √(b² - 4ac)) / 2a</lang>
  </voice>
</speak>

```

Reading mathematical expressions with MathML

The Mathematical Markup Language (MathML) is an XML-compliant markup language that describes mathematical content and structure. The Speech service can use the MathML as input text to properly pronounce mathematical notations in the output audio.

All elements from the [MathML 2.0](#) and [MathML 3.0](#) specifications are supported, except the MathML 3.0 [Elementary Math](#) elements.

Take note of these MathML elements and attributes:

- The `xmlns` attribute in `<math xmlns="http://www.w3.org/1998/Math/MathML">` is optional.

- The `semantics`, `annotation`, and `annotation-xml` elements don't output speech, so they're ignored.
- If an element isn't recognized, it'll be ignored, but the child elements within it will still be processed.

The XML syntax doesn't support the MathML entities, so you must use the corresponding [unicode characters](#) to represent the entities, for example, the entity `©` should be represented by its unicode characters `©`, otherwise an error occurs.

MathML examples

The text to speech output for this example is "a squared plus b squared equals c squared".

XML

```
< speak version='1.0' xmlns='http://www.w3.org/2001/10/synthesis'
xmlns:mstts='http://www.w3.org/2001/mstts' xml:lang='en-US' >
  < voice name='en-US-JennyNeural' >
    < math xmlns='http://www.w3.org/1998/Math/MathML' >
      < msup >
        < mi >a</mi>
        < mn>2</mn>
      </msup>
      < mo>+</mo>
      < msup >
        < mi >b</mi>
        < mn>2</mn>
      </msup>
      < mo>=</mo>
      < msup >
        < mi >c</mi>
        < mn>2</mn>
      </msup>
    </math>
  </voice>
</speak>
```

Next steps

- [SSML overview](#)
- [SSML document structure and events](#)
- [Language support: Voices, locales, languages](#)

SSML phonetic alphabets

08/07/2025

Phonetic alphabets are used with the [Speech Synthesis Markup Language \(SSML\)](#) to improve the pronunciation of text to speech voices. To learn when and how to use each alphabet, see [Use phonemes to improve pronunciation](#).

Speech service supports the [International Phonetic Alphabet \(IPA\)](#) [↗] suprasegmentals that are listed here. You set `ipa` as the `alphabet` in [SSML](#).

 Expand table

<code>ipa</code>	Symbol	Note
<code>'</code>	Primary stress	Don't use single quote (<code>`</code> or <code>'</code>) though they look similar.
<code>,</code>	Secondary stress	Don't use comma (<code>,</code>) though it looks similar.
<code>.</code>	Syllable boundary	
<code>:</code>	Long	Don't use colon (<code>:</code> or <code>:</code>) though they look similar.
<code>~</code>	Linking	

Tip

You can use [the international phonetic alphabet keyboard](#) [↗] to create the correct `ipa` suprasegmentals.

For some locales, Speech service defines its own phonetic alphabets, which ordinarily map to the [International Phonetic Alphabet \(IPA\)](#) [↗]. The eight locales that support the Microsoft Speech API (SAPI, or `sapi`) are en-US, fr-FR, de-DE, es-ES, ja-JP, zh-CN, zh-HK, and zh-TW. For those eight locales, you set `sapi` or `ipa` as the `alphabet` in [SSML](#).

See the sections in this article for the phonemes that are specific to each locale.

Note

The following tables list viseme IDs corresponding to phonemes for different locales. When viseme ID is 0, it indicates silence.

ar-EG/ar-SA

Vowels for ar-EG

[Expand table](#)

ipa	viseme	Example 1	Example 2	Example 3
a	2		حرب	شَرَدَ
a:	2		لِسانَهُ	أَذْيَانَهَا
i	6		رَجُلٌ	يَهْ
i:	6		كَبِيرٌ	أُزْدَوَازِي
u	7		أَجْرٌ	مُنْدُ
u:	7		آخِرُونَ	أَعَاقُوا

Consonant for ar-EG

[Expand table](#)

ipa	viseme	Example 1	Example 2	Example 3
b	21	بازارَ	أَبَ	أَعْطَابَ
d	19	دَابَّةَ	أَبَادَ	أَفْسَدَ
g	20	جَمَلٌ	رَجَبٌ	مَرَجَ
k	20	كَأَخَرَ	أَكْلٌ	أَوْرَاكَ
t	19	تَأَخَى	مُرْتَبٌ	أُخْتُ
dʕ	19	ضَاعِنَ	مُضِيرٌ	مَرَضَ
q	20	قَمَرٌ	أَتَعَاقَدَ	أَرْقَ
tʕ	19	طَالِبَانِ	أَحَاطَوْهُ	رَبَطَ
ʔ	19	أُفْقِي	فَأَزَ	السَّمَاءَ
f	18	فَنٌّ	يَفْرُ	شَرَفَ
h	12	هَيَّاجَ	أَحَبَّتْهُ	الْأَبْلَهَ

ipa	viseme	Example 1	Example 2	Example 3
ħ	12	حُرّ	رَحِمَ	الْبَلَح
s	15	سِرّ	قَسَمَ	الْجَالِس
θ	19	ثَرَى	أَحْدَثَ	الْحَارِث
z	15	زِر	قَزَمَ	الْخُبْز
ðˤ	17	ظَنَّ	أَظْهَرَ	قَبِيط
ð	17	ذَلِكَ	أَذَى	أُنْقِذْ
ɣ	20	غَنَى	أَذْمَعَهُ	دِمَاغ
x	12	خَبَرٌ	رَخُوْ	أَخْ
ʃ	16	شَاقَتْ	خَشِيَهُ	وَحْش
sˤ	15	صِلَةٌ	وَصَلَ	غَوْص
j	6	يَوْمٌ	أَذْوَبَةٌ	يَمْدِيرِيْ
w	7	وَلَدٌ	لَوْنٌ	مَكاو
l	14	لَيْسَ	عَلِمَ	عَمَل
m	21	مَجْدٌ	تَمَنُ	قَلَم
n	19	نَائِبٌ	أَذْيَانًا	أَمْن
r	13	رَكْبٌ	قَرْنٌ	يَمْكَرُّ
ʕ	12	عَامًا	لَعِبُ	بَيْع

bg-BG

Vowels for bg-BG

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
i	6	искат	сценаристката	лечими
ɛ	4	елен	цена	оспорените

ipa	viseme	Example 1	Example 2	Example 3
ɔ	3	оправя	което	колело
a	2	ангел	докосват	цена
u	7	уклончивата	хубавичка	табу
ja	6,2	ябълка	оял	залая
ɣ	1	ъгъл	ъгъл	имамбаялдъ
ju	6,7	ютия	отвоювал	стою

Consonant for bg-BG

[Expand table](#)

ipa	viseme	Example 1	Example 2	Example 3
n	19	неразбираемия	преразпределените	искан
ʒ	16	жълт	тъжен	таралеж
k	20	камък	вкарвал	петък
ts	19,15	цар	меценат	царевец
t	19	тайна	натъжени	прост
p	21	приказката	натъпкан	прилеп
r	13	разлика	прозорец	хитър
s	15	сърна	месец	процес
d	19	дом	меден	джаред
x	12	хора	доход	цех
zʲ	15	зян	Замразявайки	
lʲ	14	любов	влюбване	
l	14	лебед	блед	хмел
nʲ	19	някога	понякога	
v	18	време	навреме	лъвов

ipa	viseme	Example 1	Example 2	Example 3
m	21	море	време	корем
b	21	баба	риба	ястреб
g	20	гарван	наруган	драг
dʒ	19,16	Джорджева	тютюнджийския	пледж
f	18	филм	реформа	релеф
m ^j	21	мях	смях	
t ^j	19	тяхна	стяга	
r ^j	13	рядка	впрягайки	
p ^j	21	пяхме	спя	
d ^j	19	дядовите	задявайки	
j	6	йод	пейо	ручей
v ^j	18	вяло	посивяло	
s ^j	15	сякаш	всяка	
b ^j	21	бяла	Избягваме	
k ^j	20	кюпа	прокюрдският	
g ^j	20	гюргево	панагюрското	
f ^j	18	фючърс	Парфюмерия	
z	15	зима	изразходват	разказ
ʃ	16	шума	машина	изпечеш
tʃ	19,16	чак	случаи	меч
dʒ	19,15	дзифт	скръндза	Лодз

ca-ES

Vowels for ca-ES

ipa	viseme	Example 1	Example 2	Example 3
a	2	amen	amaro	està
ɔ	3	odre	ofertori	microtò
ə	1	estan	seré	aigua
e	4	érem	feta	seré
ɛ	4	ecosistema	incorrecta	haver
i	6	itinerants	itinerants	zombi
o	8	ombra	retondre	omissió
u	7	universitaris	candidatures	crono

Consonant for ca-ES

Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	baba	dobla	
β	21	vià	baba	
tʃ	19,16	txadià	matxucs	faig
d	19	dedicada	conduïa	navidad
ð	17	The_Sun	dedicada	trinidad
f	18	facilitades	afectarà	àgraf
g	20	gracia	congratula	
ɣ	20		aigua	
j	6	hiena	esplaia	cofoi
dʒ	19,16	djakarta	compostatge	george
k	20	curós	dodecà	doblec
l	14	laberint	miolar	preval
ʎ	14	lligada	millorarà	perbull

ipa	viseme	Example 1	Example 2	Example 3
m	21	macadàmies	femar	sublim
n	19	necessaris	sanitaris	alterament
ŋ	20		algonquí	albenc
ɲ	19	nyasa	remenjar	alemany
p	21	pegues	estepa	cap
ɾ	19		caro	càrter
r	13	rabada	carro	lofòfor
s	15	ceri	cursar	cus
ʃ	16	xacar	microxip	midraix
t	19	tabacaires	estratifica	debatut
θ	19	ceará	vecinos	Álvarez
w	7	westfalià	inaugurar	inscriu
x	12	juanita	mujeres	heinrich
z	15	zelar	brasils	alianze
ʒ	16	gebrada	ascogènic	orange

cs-CZ

Vowels for cs-CZ

Expand table

ipa	viseme	Example 1	Example 2	Example 3
ɪ	6	ideální	handicapované	giganty
ɛ	4	efektní	grotesce	hygieně
a	2	agens	infarktu	extra
o	8	oáz	hřbitovy	čisto

ipa	viseme	Example 1	Example 2	Example 3
u	7	ubrání	komponuje	grilu
i:	6	íránské	jedinými	generační
ɛ:	4	éry	jednoduchého	gumové
a:	2	árijské	brán	hořká
o:	8	ódu	famózního	jó
u:	7	úbočí	petúnie	grilů
oʊ	8,4	ouha	fanouška	hanbou
aʊ	2,4	auto	hydrauliky	
ɛʊ	4,4	euforie	terapeutická	
ə	1			

Consonant for cs-CZ

[Expand table](#)

ipa	viseme	Example 1	Example 2	Example 3
p	21	padá	hyperaktivní	handicap
b	21	balon	hraběte	
t	19	tabák	étos	agent
d	19	disco	čidel	
c	16	tuhýk	kandidáti	kaprad'
ʃ	16	d'ábel	hrdiny	
k	20	kaligrafie	důkazní	grafik
g	20	gangstera	hygienici	
ʦ	19,15	civilistům	evoluce	klávesnic
ʤ	19,15		leckde	
ʧ	19,16	čepice	hlídači	klíč

ipa	viseme	Example 1	Example 2	Example 3
ɖ͡ʒ	19,16	džusy	kilojouľů	
f	18	fádní	grafice	graf
v	18	vabank	exkluzivita	
s	15	sekundě	grimasami	impuls
z	15	záhadným	gruzínské	
ɾ	13	řad	hořícím	
ʃ	16	šálků	grošů	kéž
ʒ	16	žab	loupeže	
j	6	já	čihající	línej
x	12	chlapcem	lichotí	domorodých
ɦ	12	hektar	historického	
r	13	rabat	guru	faktor
l	14	ladu	eliminovat	netajil
m	21	macešsky	amatérský	cením
n	19	následkům	cenné	gurmán
ŋ	20		kolonky	
ɲ	19	ňadra	komorně	kolegyň
ɱ	21		komfort	
ɹ̥	13	třída	extratřídy	komentář

da-DK

Vowels for da-DK

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
a	2	abitur	kortfattet	marina
ɑ	2	abnorme	picaresk	varer
ɑ:	2	ara	pirat	vandkar
ɛ	4	edderkop	andægtig	tomatpuré
ɛ:	4	eventyr	dyrlæge	
ɔ	3	onde	dysfunktion	Suså
ɒ	2	ånd	abehand	akkumulator
ɒ:	2	årbog	fyrreårig	rickshaw
ɔ:	3	åbne	modstående	husarblå
e	4		buntmagere	bortgifter
æ:	1	abegilde	flagdage	hovedfag
e	4	editere	klarheden	kanapé
ø	1	ødem	mælkebøtte	miljø
ø:	1	øde	bortførere	lindø
ə	1	Eyolf	meneders	æde
e:	4	edelweiss	placebo	Culmsee
i	6	iaften	lipizzaner	bankkonti
i:	6	ilinger	industrien	bagi
o	8	oase	udelød	bravo
œ	4	øm	varmerør	bøh
œ:	4	Earl	arbejdsprøves	Edinburgh
o:	8	oberst	bevatrons	congo
u	7	udøver	spektakulær	endnu
u:	7	uge	spidsbue	eisuu
y	4	ybsalon	underkrydder	menu

ipa	viseme	Example 1	Example 2	Example 3
y:	4	yde	vidsyn	dødssyg

Consonant for da-DK

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	blandt	gabende	dåb
d	19	då	hævder	grædt
ð	17	thousand	baaden	grad
f	18	fabrik	spøgefugle	boligstof
g	20	gaader	fåborgenser	fik
h	12	håb	trægheden	
j	6	hjorte	miljø	bagtøj
k ^h	20	kabale	bortkomme	Cuc
l	14	lå	romanblad	fatal
m	21	maya	taxametret	calcium
n	19	nå	togene	bredden
ŋ	20		funktion	klarering
p ^h	21	pa	culpa	
ʔ	19		affyre	bortgå
ʀ	13	rå	areal	
ɹ	4		jonglørkunst	bjørnebær
s	15	så	person	belønnes
ʃ	16	Sjælland	benzinstation	affiche
t	19	toge	sidetal	administrationsapparat
v	18	vaagner	evaluere	dåkalv

ipa	viseme	Example 1	Example 2	Example 3
w	7	walkman	prøve	meterlov

de-DE/de-CH/de-AT

Suprasegmentals for German

[Expand table](#)

Example 1 (Onset for consonant, word-initial for vowel)	Example 2 (Intervocalic for consonant, word-medial nucleus for vowel)	Example 3 (Coda for consonant, word-final for vowel)	Comments
anders /a 1 n - d ax r s/	Multiplikationszeichen /m uh l - t iy - p l iy - k a - ts y ow 1 n s - ts ay - c n/	Biologie /b iy - ow - l ow - g iy 1/	Speech service phone set put stress after the vowel of the stressed syllable
Allgemeinwissen /a 2 l - g ax - m ay 1 n - v ih - s n/	Abfallentsorgungsfirma /a 1 p - f a l - ^ eh n t - z oh 2 ax r - g uh ng s - f ih ax r - m a/	Computertomographie /k oh m - p y uw 1 - t ax r - t ow - m ow - g r a - f iy 2/	The Speech service phone set puts stress after the vowel of the sub-stressed syllable

Vowels for German

[Expand table](#)

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
a:	a:	2	Aber	Maßstab	Schema
a	a	2	Abfall	Bach	Agatha
oh	ɔ	3	Osten	Pfosten	
eh:	ɛ:	4	Ähnlichkeit	Bär	Fasciae ¹
eh	ɛ	4	ändern	Prozent	Amygdalae
ax	ə	1	'verstauen ²	Aachen	Frage

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
iy	i:	6	Iran	abbiegt	Relativitätstheorie
ih	ɪ	6	Innung	singen	Woody
eu	ø:	1	Ösen	ablösen	Malmö
ow	o, o:	8	ohne	Balkon	Treptow
oe	æ	4	Öffnung	befördern	
ey	e, e:	4	Eberhard	abfegt	b
uw	u:	7	Udo	Hut	Akku
uh	ʊ	4	Unterschiedes	bunt	
ue	y:	4	Übermut	pflügt	Menü
uy	ʏ	7	üppig	System	

- 1 Only in words of foreign origin, such as *Fasciae*.
- 2 Word-initial only in words of foreign origin, such as *Appointment*. Syllable-initial in 'verstauen.

Diphthong for German

[Expand table](#)

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
ay	aɪ	2,6	einsam	Unabhängigkeit	Abtei
aw	au	2,7	außen	abbaust	Stau
oy	ɔy, ɔʏ	3,4	Euphorie	träumt	scheu

Semivowels for German

[Expand table](#)

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
ax r	ɐ	4		abändern	locker

Consonants for German

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
b	b	21	Bank		Pub ¹
d	d	19	danken	Lendl ²	Claude ³
jh	ɖʒ	16	Jeff	gemanagt	Change ⁴
f	f	18	Fahrtdauer	angriffslustig	abbruchreif
g	g	20	gut	Greg ⁵	
h	h	12	Hausanbau		
y	j	6	Jod	Reaktion	hui
k	k	20	Koma	Aspekt	Fleck
l	l	14	lau	ähneln	zuviel
m	m	21	Mut	Amt	Lehm
n	n	19	nun	und	Huhn
ng	ŋ	20	Nguyen ⁶	Schwank	Ring
p	p	21	Partner	abrupt	Tip
pf	pf	21,18	Pferd	dampft	Topf
r	ʀ, r, ʁ	13	Reise	knurrt	Haar
s	s	15	Staccato ⁷	bist	mies
sh	ʃ	16	Schule	mischt	lappisch
t	t	19	Traum	Straße	Mut
ts	ts	19,15	Zug	Arzt	Witz
ch	tʃ	19,16	Tschechien	aufgeputscht	bundesdeutsch
v	v	18	winken	Qualle	Groove ⁸
x	x ⁹ , ç ¹⁰	12	Bacherach ¹¹	Macht möglichst	Schmach 'ich
z	z	15	super		
zh	ʒ	16	Genre	Breezinski	Edvige

- 1 Only in words of foreign origin, such as *Pub*.
- 2 Only in words of foreign origin, such as *Lendl*.
- 3 Only in words of foreign origin, such as *Claude*.
- 4 Only in words of foreign origin, such as *Change*.
- 5 Word-terminally only in words of foreign origin, such as *Greg*.
- 6 Only in words of foreign origin, such as *Nguyen*.
- 7 Only in words of foreign origin, such as *Staccato*.
- 8 Only in words of foreign origin, such as *Groove*.
- 9 The IPA **x** is a hard "ch" after all non-front vowels (a, aa, oh, ow, uh, uw, and the diphthong aw).
- 10 The IPA **ç** is a soft "ch" after front vowels (ih, iy, eh, ae, uy, ue, oe, eu, and diphthongs ay, oy) and consonants.
- 11 Word-initial only in words of foreign origin, such as *Juan*. Syllable-initial also in words such as *Bacherach*.

Oral consonants for German

[Expand table](#)

sapi	ipa	VisemeID	Example
^	ʔ	19	beachtlich /b ax - ^ a 1 x t - l ih c/

ⓘ Note

We need to add a [gs] phone between two distinct vowels, except when the two vowels are a genuine diphthong. This oral consonant is a glottal stop. For more information, see [glottal stop](#). Besides, de-CH, de-AT locales don't support SAPI phones now.

el-GR

Vowels for el-GR

[Expand table](#)

ipa	viseme	Example 1	Example 2	Example 3
a	2	άκουσμα	μαγαζάκι	ουδέτερα
e	4	αίτιο	κατέχω	ουδέποτε

ipa	viseme	Example 1	Example 2	Example 3
i	6	εισπνοή	ξυρισμένο	εαρινή
o	8	όαση	δώρο	δυ**ο
u	7	συγγρικό	παπούτσι	μαϊμού

Consonant for el-GR

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	μπορώ	φάμπρικα	παμπ
c	16	και	σακί	
ç	12	χέρι	μοναχή	
d	19	ντύνει	πέντε	οφ-σάιντ
ð	17	δρόμος	κραδασμοί	
ɖz	19,15	τζάμι	φλάντζα	
f	18	φεύγω	καφετέρια	ουφ
g	20	γκρέμισε	όγκος	πινγκ-πονγκ
ɣ	20	γαστρονομική	μεγαλόπνοα	
ʒ	16	γκισέ	φαράγγι	
j	6			
ɟ	12	γέρος	κραγιόν	
k	20	κάρτα	ιππικό	κρακ
l	14	λόγος	μιλώ	προφίλ
m	21	μιλώ	δραματική	πριμ
n	19	νόμιμο	Δανέζα	πριν
p	21	πίνω	απέτυχαν	σελοτέιπ
r	19	ρούχα	εαρινή	σέντερ

ipa	viseme	Example 1	Example 2	Example 3
s	15	σελίδα	μέσο	πλάτος
t	19	τότε	τότε	κιτ
θ	19	θέλω	φορολογηθείς	μαμούθ
ts	19,15	τσάγια	χαράτσι	ματς
v	18	βράδυ	διαβάζω	μοβ
x	12	χρόνος	μονάχα	αχ
z	15	ζέστη	χιονίζει	πλαζ

en-GB/en-IE/en-AU

Vowels for en-GB

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
ɑː	2		fast	bra
æ	1		fat	
ʌ	1		bug	
ɛə	4,1			hair
aʊ	2,4	out	mouth	how
ə	1	a		driver
aɪ	2,6		five	
ɛ	4	egg	dress	
ɜː	5	ernest	shirt	fur
eɪ	4,6	ailment	lake	pay
ɪ	6		adding	
ɪə	6,1		beard	hear

ipa	viseme	Example 1	Example 2	Example 3
i:	6	eat	seed	see
ɒ	2		pod	
ɔ:	3		dawn	
əʊ	1,4		code	pillow
ɔɪ	3,6		point	boy
ʊ	4		look	
ʊə	4,1			tour
u:	7		food	two

Consonant for en-GB

[↗](#) Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	bike	ribbon	rib
tʃ	19,16	challenge	nature	rich
d	19	date	caddy	slid
ð	17	this	father	breathe
f	18	face	laughing	enough
g	20	gold	bragging	beg
h	12	hurry	ahead	
j	6	yes		
dʒ	19,16	gin	badger	bridge
k	20	cat	lucky	truck
l	14	left	gallon	fill
m	21	mile	limit	ham
n	19	nose	phonetic	tin

ipa	viseme	Example 1	Example 2	Example 3
ŋ	20		singer	long
p	21	price	super	tip
ɹ	13	rate	very	
s	15	say	sissy	pass
ʃ	16	shop	cashier	leash
t	19	top	kitten	bet
θ	19	theatre	mathematics	breath
v	18	very	liver	have
w	7	will		
z	15	zero	blizzard	rose
ʒ	16		vision	beige

en-US/en-CA

Suprasegmentals for English

 Expand table

Example 1 (onset for consonant, word-initial for vowel)	Example 2 (intervocalic for consonant, word-medial nucleus for vowel)	Example 3 (coda for consonant, word-final for vowel)	Comments
burger /b er 1 r - g ax r/	falafel /f ax - l aa 1 - f ax l/	guitar /g ih - t aa 1 r/	The Speech service phone set puts stress after the vowel of the stressed syllable.
inopportune /ih 2 - n aa - p ax r - t uw 1 n/	dissimilarity /d ih - s ih 2- m ax - l eh 1 - r ax - t iy/	workforce /w er 1 r k - f ao 2 r s/	The Speech service phone set puts stress after the vowel of the sub-stressed syllable.

Vowels for English

[Expand table](#)

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
iy	i	6	eat	feel	valley
ih	ɪ	6	if	fill	
ey	eɪ	4,6	ate	gate	day
eh	ɛ	4	every	pet	meh (rare word-final)
ae	æ	1	active	cat	nah (rare word-final)
aa	ɑ	2	obstinate	poppy	rah (rare word-final)
ao	ɔ	3	orange	cause	Utah
uh	ʊ	4	book		
ow	oʊ	8,4	old	clone	go
uw	u	7	Uber	boost	too
ah	ʌ	1	uncle	cut	
ay	aɪ	11	ice	bite	fly
aw	aʊ	9	out	south	cow
oy	ɔɪ	10	oil	join	toy
y uw	ju	6,7	Yuma	human	few
ax	ə	1	ago	woman	area

R-colored vowels for English

[Expand table](#)

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
ih r	ɪɹ	6,13	ears	tiramisu	near
eh r	ɛɹ	4,13	airplane	apparently	scare
uh r	ʊɹ	4,13			cure
ay r	aɪɹ	11,13	Ireland	fireplace	choir

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
aw r	auɹ	9,13	hours	powerful	sour
ao r	ɔɹ	3,13	orange	moral	soar
aa r	ɑɹ	2,13	artist	start	car
er r	ə	5	earth	bird	fur
ax r	ə	1		allergy	supper

Semivowels for English

[Expand table](#)

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
w	w	7	with, suede	always	
y	j	6	yard, few	onion	

Aspirated oral stops for English

[Expand table](#)

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
p	p	21	put	happen	flap
b	b	21	big	number	crab
t	t	19	talk	capital	sought
d	d	19	dig	random	rod
k	k	20	cut	slacker	Iraq
g	g	20	go	ago	drag

Nasal stops for English

[Expand table](#)

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
m	m	21	mat, smash	camera	room
n	n	19	no, snow	tent	chicken
ng	ŋ	20		link	sing

Fricatives for English

 Expand table

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
f	f	18	fork	left	half
v	v	18	value	event	love
th	θ	19	thin	empathy	month
dh	ð	17	then	mother	smooth
s	s	15	sit	risk	facts
z	z	15	zap	busy	kids
sh	ʃ	16	she	abbreviation	rush
zh	ʒ	16	Jacques	pleasure	garage
h	h	12	help	enhance	a-ha!

Affricates for English

 Expand table

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
ch	tʃ	19,16	chin	future	attach
jh	dʒ	19,16	joy	original	orange

Approximants for English

 Expand table

sapi	ipa	VisemeID	Example 1	Example 2	Example 3
l	l	14	lid, glad	palace	chill
r	ɹ	13	red, bring	borrow	tar

ⓘ Note

en-CA locale doesn't support SAPI phones.

es-ES

Vowels for es-ES

⌵ Expand table

sapi	ipa	viseme	Example 1	Example 2	Example 3
a	a	2	alto	cantar	casa
i	i	6	ibérica	avispa	taxi
e	e	4	elefante	atento	elefante
o	o	8	ocaso	encontrar	ocaso
u	u	7	usted	punta	Juanlu

Consonant for es-ES

⌵ Expand table

sapi	ipa	viseme	Example 1	Example 2	Example 3
b	b	21	baobab	cambio	amb
	β	21		baobab	baobab
ch	tʃ	19,16	cheque	coche	Marraquech
d	d	19	dedo	candado	portland
	ð	17		dedo	verdad

sapi	ipa	viseme	Example 1	Example 2	Example 3
f	f	18	fácil	elefante	puf
g	g	20	ganga	ganga	dóping
	ɣ	20		agua	tuareg
j	j	6	iodo	caliente	rey
jj	ɟɟ	6,6		villa	
k	k	20	coche	boca	titánic
l	l	14	lápiz	ala	cordel
ll	ʎ	14	llave	conllevar	
m	m	21	morder	amar	álbum
n	n	19	nada	cena	ratón
nj	ɲ	19	ñaña	arañazo	
p	p	21	poca	topo	stop
r	r	19		cara	abrir
rr	ɾ	13	radio	corre	purr
s	s	15	saco	vaso	pelos
t	t	19	toldo	atar	disquet
th	θ	19	zebra	azul	lápiz
w	w	7	hueso	agua	guau
x	x	12	jota	ajo	reloj

💡 Tip

The **es-ES** Speech service phone set doesn't support the following Spanish IPA: **β**, **ð**, and **ɣ**. If they're needed, consider using the IPA directly.

es-MX

Vowels for es-MX

[Expand table](#)

ipa	VisemeID	Example 1	Example 2	Example 3
a	2	azúcar	tomate	ropa
e	4	eso	remero	amé
i	6	hilo	liquido	olí
o	8	hogar	olote	caso
u	7	uno	ninguno	tabú

Consonants for es-MX

[Expand table](#)

ipa	VisemeID	Example 1	Example 2	Example 3
b	21	bote		
β	21	órbita	envolvente	
tʃ	19,16	chico	hacha	
d	19	dátil		
ð	17	orden	oda	
f	18	foco	oficina	
g	20	gajo		
ɣ	20	agua	hoguera	
j	6	iodo	caliente	rey
ʝ	6,6		olla	
k	20	casa	ácaro	
l	14	loco	ala	
ʎ	14	llave	enyugo	
m	21	mata	amar	

ipa	VisemeID	Example 1	Example 2	Example 3
n	19	nada	ano	
ɲ	19	ñoño	año	
p	21	papa	papa	
ɾ	19		aro	
r	13	rojo	perro	
s	15	silla	asa	
t	19	tomate		soft
w	7	huevo		
x	12	jarra	hoja	

fi-FI

Vowels for fi-FI

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
ɑ	2	avautuu	vaihtuvan	pouta
ɑ̃	2,6	aika	vaihtuu	lauantai
ɑ̃u	2,7	aura	uloskirjaudu	Passau
ɑ:	2	ay-väen	neutraali	poutaa
æ	1	äveriäs	öljyjätin	pöytä
æ̃	1,6	äiti	iäkkäiden	täi
æ̃y	1,4	äyrin	täytyy	käy
æ:	1	ääriryhmiä	häädetään	päivää
e	4	enköhän	terve	me
ẽ	4,6	ei	vaihteita	hei

ipa	viseme	Example 1	Example 2	Example 3
ø	1	öljyalan	ulkonäön	tiedä ^{nk} ö
ø̞i	1,6	öisin	töitä	viittilöi
ø̞y	1,4	öylätti	pöytä	
ø:	1	Öölanti	ulkoministeriöön	Bodø
eu	4,7	eurot	kyläseura	leu
ey	4,4	Eysturoy	keskeytyä	
e:	4	eesti	kyljelleen	aiheuttanee
i	6	iäkästä	viha	Berliini
i̞e	6,4	ientaskun	kieli	lie
i̞u	6,7		viulu	
i̞y	6,4		vihkiytynyt	
i:	6	lida	siika	solmii
o	8	oksa	asuintaloja	spekulaatio
oi	8,6	oivia	koittaa	spekuloi
ou	8,7	outo	autokoulu	window
o:	8	ok	koostaa	yo
u	7	ufoista	Bärlund	jätmaksu
ui	7,6	ui	multa	epäonnistui
uo	7,8	Uolevi	Suomi	Hilavuo
u:	7	url	innokkuus	kiikkuu
y	4	ydin	ökyrikas	kesy
yø	4,1	yö	työtä	järjestötyö
y̞i	4,6	Yichangin	syitä	järjestäytyi
y:	4	yo	ryyppy	iskeytyy

Consonant for fi-FI

ipa	viseme	Example 1	Example 2	Example 3
b	21	baareissa	Urban	Jakob
d	19	dementia	ladot	jugend
f	18	face	afgaani	Alf
g	20	gaalassa	fagotti	Aslög
h	12	ha	astumahan	Dietrich
j	6	jää	öljyä	Kaj
k	20	Kajaanin	epätarkat	idyllic
l	14	Lyytikäinen	euroseteleiden	mail
m	21	mä	roimat	späm
n	19	nää	baanalle	iäkkään
ŋ	20		nähdäänkin	planning
p	21	pa	epäsuoria	backup
r	13	risteilyn	baari	Player
s	15	sä	öljyisiä	Bärnäs
ʃ	16	Schauman	Bangladeshin	cash
t	19	tä	eurosta	epäsuorat
u	18	vaadi	innostava	Kiev

fr-FR/fr-CA/fr-CH

Suprasegmentals for French

The Speech service phone set puts stress after the vowel of the stressed syllable. However, the **fr-FR** Speech service phone set doesn't support the IPA substress '̣'. If the IPA substress is needed, you should use the IPA directly.

Vowels for French

[Expand table](#)

sapi	ipa	viseme	Example 1	Example 2	Example 3
ae	a	2	arbre	patte	ira
af	ɑ	2		pâte	pas
an	ā	2	enfant	enfant	temps
ax	ə	1		petite	le
eh	ε	4	elle	perdu	était
eu	ø	1	œufs	creuser	queu
ey	e	4	ému	crétin	ôté
in	ē	4	important	peinture	matin
iy	i	6	idée	petite	ami
oe	œ	4	œuf	peur	
oh	ɔ	3	obstacle	corps	
on	ō	3	onze	rondeur	bon
ow	o	8	auditeur	beaucoup	pô
un	œ	4	un	lundi	brun
uw	u	7	outrage	introuvable	ou
uy	y	4	une	punir	élu

Consonant for French

[Expand table](#)

sapi	ipa	viseme	Example 1	Example 2	Example 3
b	b	21	bête	habille	robe
d	d	19	dire	rondeur	chaude
f	f	18	femme	suffixe	bof
g	g	20	gauche	égale	bague

sapi	ipa	viseme	Example 1	Example 2	Example 3
gn	ɲ	19			peigne
hw	ɥ	7	huile	nuire	
k	k	20	carte	écaille	bec
l	l	14	long	élire	bal
m	m	21	madame	aimer	pomme
n	n	19	nous	tenir	bonne
ng	ŋ	20			parking
p	p	21	patte	repas	cap
r	ʁ	13	rat	chariot	sentir
s	s	15	sourir	assez	passe
sh	ʃ	16	chanter	machine	poche
t	t	19	tête	ôter	net
v	v	18	vent	inventer	rêve
w	w	7	oui	fouine	
y	j	6	yod	piétiner	Marseille
z	z	15	zéro	raisonner	rose
	n̥	19			un arbre
	t̥	19			quand
	z̥	15			corps

1 Only for some foreign words.

💡 Tip

The fr-FR Speech service phone set doesn't support the following French liaisons, n̥, t̥, and z̥. If they are needed, you should consider using the IPA directly.

⚠ Note

fr-CA, fr-CH locales don't support SAPI phones now.

he-IL

Vowels for he-IL

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
i	6	איש	סיר	כלי
e	4	אש	כן	פה
a	2	אף	קו	מה
o	8	אות	יום	לא
u	7	עוגה	כרוב	הגיעו

Consonant for he-IL

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
p	21	פס	קליפה	טיפ
b	21	בד	סבא	פאב
t	19	טיפה	מתנה	חוט
d	19	דבר	אדום	תמיד
k	20	כתב	בוקר	חלק
g	20	גדול	אגף	דג
ʔ	19	אָבִיב	שיעור	
f	18	פילטר	סופר	סוף
v	18	וילון	כבד	לב
s	15	שמלה	כסף	פנס

ipa	viseme	Example 1	Example 2	Example 3
z	15	זאב	מזל	אגוז
ʃ	16	שולחן	פשוט	כביש
x	12	חתול	אוכל	פרח
h	12	הולך	זהב	בה
ʦs	19,15	צד	עצם	מומלץ
m	21	מאוד	סימן	חלום
n	19	נפש	תינוק	אבן
l	14	לשון	מילה	דגל
ʁ	13	ראשון	מורה	חיבור
j	6	ילד	מצוין	כדאי
ʒ	16	ז'אנר	מז'ור	'בז
tʃ	19,16	צ'יפ	קפוצ'ון	'סנדוויץ
dʒ	19,16	ג'ונגל	פיג'מה	'קוטג

hr-HR

Vowels for hr-HR

Expand table

ipa	viseme	Example 1	Example 2	Example 3
e	4	Egipat	Games	drveće
e:	4	eri	brijegom	de1taljne
i	6	ispada	želimo	javnosti
i:	6	iako	list	kompletni
u	7	ubacio	konkurentne	jednu
u:	7	Una	funta	Yu

ipa	viseme	Example 1	Example 2	Example 3
a	2	američke	kovačić	kredita
a:	2	anđela	gradila	Divulja
o	8	oaza	nanosi	do
o:	8	Olgu	kiselog	to

Consonant for hr-HR

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
d	19	dakle	evidenciji	kod
v	18	volja	građevinskog	imperativ
s	15	sabor	informisanja	interes
t	19	tad	informirati	ispit
n	19	na	žene	jeftin
l	14	logor	konstatirali	kapital
ʎ	14	ljudski	košulju	kralj
ʦ	19,15	carina	krivice	pravac
ʧ	19,16	četvorica	kritičan	osnivač
j	6	jednostavan	kuju	ovaj
x	12	hrvatskog	mahala	ovakvih
z	15	znanstvenom	mehanizacije	prijelaz
ʒ	16	žalbu	mladeži	crtež
r	13	red	moraju	dar
k	20	kažu	nakani	dnevnik
m	21	Mađara	napadima	dobrim
p	21	Poljska	napadnut	kamp

ipa	viseme	Example 1	Example 2	Example 3
g	20	gore	negativna	kaznenog
ʦ	16	ćelija	neisplaćene	mladić
f	18	fabula	nostrifikaciji	šef
b	21	Belgija	oba	sukob
dʒ	19,16	džempera	Ilidže	George
ɲ	19	nje	emitiranja	stupanj
ʦ	16	đakovačkim	gađati	vođ
ʃ	16	šef	stišati	Glavaš

hu-HU

Vowels for hu-HU

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
ø	1	ördög	ördög	huszonkettő
ø:	1	ők	önműködően	öntöző
a	2		Schumacher	
a:	2	árut	bizonyára	burzsua
ɛ	4	előzni	kisbéresnek	hogyne
e:	4	Édes	komédia	fölfelé
i	6	idegen	omnibuszok	kollégiumi
i:	6	ígyen	áhítat	
o	8	oldali	komor	Figo
o	2	atyját	olvasni	Olga
o:	8	ólmot	históriát	fénymásoló

ipa	viseme	Example 1	Example 2	Example 3
u	7	ugyanis	ezután	falu
u:	7	úrrá	fékút	számú
y	4	üdítőt	átsütve	alsóbbrendű
y:	4	űrállomás	gépjárművek	idejű

Consonant for hu-HU

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	busz	hiába	
b:	21		hasá bb urgonya	rövide bb
d	19	dévédé	fapados	szaporítanád
ɟ	16	gyártó	franciaágyas	huszonegy
d:	19		Goddess	hadd
ɟ:	16		ebédjén	hagyj
ɖʒ	19,16	Dzsó	menedzselési	college
ɖʒ:	19,16		Baggio	
dz	19,15	dzémsz	edző	McDonalds
dz:	19,15		eddeni	
f	18	figura	kofa	golf
f:	18		koffer	seriff
g	20	gondolom	számítógép	rúg
g:	20		faggatta	függ
h	12	hit	ruhában	ah
h:	12		ehhez	
j	6	János	olyan	karéj

ipa	viseme	Example 1	Example 2	Example 3
ɲ	19	nyakán	pecsenyét	példány
j:	6	Ljeszkovai	majmolják	állj
ɲ:	19		pihenjen	
k	20	kofa	ruhákat	ruhák
k:	20		megkondult	makk
l	14	lent	rúla	evvel
l:	14		kolléga	áll
m	21	magyar	számít	órám
m:	21		anyámmal	kilogramm
n	19	népies	szótlantul	hiszen
ŋ	20		angel	
n:	19		onnan	fenn
p	21	Pál	tapogatódzó	számítógép
p:	21		beröppent	befejezéseképp
r	13	rág	óraára	órakor
r:	13		amerre	forr
s	15	számára	fölveszi	fölmész
ʃ	16	saját	förtelmesen	főorvos
s:	15		halasszuk	hazajössz
ʃ:	16		házassága	keress
t	19	Tata	rútak	hit
c	16		egyházmegye	dirty
t:	19		hitte	vágodott
c:	16		bátyja	Pretty
ts	19,15	címe	biciklis	huszonnyolc
tʃ	19,16	csigán	húgocskám	Gregorics

ipa	viseme	Example 1	Example 2	Example 3
ʦ:	19,15		játszad	játsz
ʧ̞:	19,16		barátságos	futballmeccs
v	18	varr	Olívia	Dönöv
v:	18		evvel	
x	12	hrabovszki	ihletével	
ɯ	20		alapján	Kapj
z	15	zúgást	csizmám	csimpánz
ʒ	16	zsûrivel	félmázsás	Balázs
z:	15		húzza	fékezz
ʒ:	16		garázssal	

id-ID

Vowels for id-ID

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
ə	1		benar	komite
a	2		babat	engga
ai̯	2,6	air	raikage	pantai
au̯	2,4	aurat	ataupun	pisau
e	4	energi	feri	tempe
ɛ	4	enrique	nenek	
i	6	indah	yakin	key
i	6	irama	biar	deflasi
ɔ	3	off	esok	law

ipa	viseme	Example 1	Example 2	Example 3
o	8	obat	bobot	domino
ɔɪ	3,6		reboisasi	sepoi-sepoi
u	7	umur	buah	linu
ʊ	4		duduk	

Consonant for id-ID

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
ʔ	19	amores	maaf	pencak
b	21	babat	nebak	dishub
d	19	deder	pedang	cloud
ɖʒ	19,16	jarum	penjajah	buruj
f	18	flat	aversi	genitif
g	20	gabus	dagel	hamburg
h	12	hama	tuhan	entah
ɲ	19	nyaman	menyuci	
j	6	yakin	jaya	baduy
k	20	ketan	aku	polsek
l	14	labu	talenta	vital
m	21	masuk	namanya	sinom
n	19	nada	sekunar	proton
ŋ	20	ngengat	kenanga	abang
p	21	pacar	hampa	cakap
r	13	rabu	dikurang	nasar
s	15	sabuk	tetesan	jenius

ipa	viseme	Example 1	Example 2	Example 3
ʃ	16	syarat	isyarat	british
t	19	tabir	adaptasi	durat
tʃ	19,16	cakap	dicari	
w	7	wajah	yuwana	
x	12	khusuk	akhirnya	barzakh
z	15	zakat	pezina	mahfuz

it-IT

Vowels for it-IT

Expand table

ipa	viseme	Example 1	Example 2	Example 3
a	2	amo	sano	scorta
ai	2,6	aics	abbaino	mai
au	2,7	audio	rauco	bau
e	4	eroico	venti	sapore
ɛ	4	elle	avvento	lacchè
ɛj	4,6	eira	email	lei
ɛu	4,7	euro	neuro	
ei	4,6		aseità	scultorei
eu	4,7	europeo	feudale	
i	6	italiano	vino	solì
u	7	unico	luna	zebù
o	8	obesità	straordinari	amico
ɔ	3	otto	botte	però

ipa	viseme	Example 1	Example 2	Example 3
ɔj	3,6		oppioidi	
oi	8,6	oibò	intellettualoide	Gameboy
ou	8,7		show	talkshow

Consonant for it-IT

Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	bene	ebanista	Euroclub
b:	21		gobba	
tʃ	16	cenare	acido	french
tʃ:	19,16		braccio	
k:	20		pacco	Innsbruck
d	19	dente	adorare	interland
d:	19		cadde	
ʒ	15	zero	orzo	
ʒ:	15		mezzo	
f	18	fame	afa	alef
f:	18		beffa	bluff
ʒ	16	gente	agire	beige
ʒ:	16		oggi	
g	20	gara	alghe	smog
g:	20		fugga	Zuegg
ʎ	14	gli	ammiragli	
ʎ:	14		foggia	
ɲ:	19		bagno	

ipa	viseme	Example 1	Example 2	Example 3
ɲ	19	gnocco	padrigno	Montaigne
j	6	ieri	piede	freewifi
k	20	caro	anche	tic
l	14	lana	alato	col
l:	14		colla	full
m	21	mano	amare	Adam
m:	21		grammo	
n	19	naso	lana	non
n:	19		panna	
p	21	pane	epico	stop
p:	21		coppa	
r	19	rana	motore	per
r:	13		carro	Starr
s	15	sano	cascata	lapis
s:	15		cassa	cordless
ʃ	16	scemo	Gramsci	slash
ʃ:	16		ascia	fiches
t	19	tana	eterno	alt
t:	19		zitto	
ʈ	15	tsunami	turbolenza	subtests
ʈ:	15		bozza	
v	18	vento	avaro	Asimov
v:	18		bevvi	
w	7	uovo	duomo	Marlowe
z	15	smodato	casa	elections

ja-JP

The Speech service phone set for `ja-JP` is based on the native phone [Kana](#) set.

Please see the following tables for kana and corresponding viseme in parentheses.

Katakana for ja-JP

[Expand table](#)

Katakana	ア	イ	ウ	エ	オ
ア	ア (19,2)	イ (6,6)	ウ (7,6)	エ (19,4)	オ (19,8)
カ	カ (20,2)	キ (20,6)	ク (20,6)	ケ (20,4)	コ (20,8)
サ	サ (15,2)	シ (16,6)	ス (15,6)	セ (15,4)	ソ (15,8)
タ	タ (19,2)	チ (16,6)	ツ (19,15,6)	テ (19,4)	ト (19,8)
ナ	ナ (19,2)	ニ (19,6)	ヌ (19,6)	ネ (19,4)	ノ (19,8)
ハ	ハ (12,2)	ヒ (12,6)	フ (12,6)	ヘ (12,4)	ホ (12,8)
マ	マ (21,2)	ミ (21,6)	ム (21,6)	メ (21,4)	モ (21,8)
ヤ	ヤ (6,2)	n/a	ユ (6,6)	n/a	ヨ (6,8)
ラ	ラ (19,2)	リ (19,6)	ル (19,6)	レ (19,4)	ロ (19,8)
ワ	ワ (7,2)	n/a	n/a	n/a	ヲ (19,8)
	ン (19)	n/a	n/a	n/a	n/a

Katakana diacritics for ja-JP

[Expand table](#)

Katakana diacritics	ア	イ	ウ	エ	オ
ガ	ガ (20,2)	ギ (20,6)	グ (20,6)	ゲ (20,4)	ゴ (20,8)
ザ	ザ (15,2)	ジ (16,6)	ズ (15,6)	ゼ (15,4)	ゾ (15,8)
ダ	ダ (19,2)	ヂ (16,6)	ヅ (15,6)	デ (19,4)	ド (19,8)
バ	バ (21,2)	ビ (21,6)	ブ (21,6)	ベ (21,4)	ボ (21,8)

Katakana diacritics	ア	イ	ウ	エ	オ
パ	パ (21,2)	ピ (21,6)	プ (21,6)	ペ (21,4)	ポ (21,8)

Katakana Yōon for ja-JP

[Expand table](#)

Katakana Yōon	ヤ	ユ	ヨ
キ	キヤ(20,6,2)	キユ(20,6,6)	キヨ(20,6,8)
シ	シヤ(16,6,2)	シユ(16,6,6)	シヨ(16,6,8)
チ	チヤ(16,6,2)	チユ(16,6,6)	チヨ(16,6,8)
ニ	ニヤ(19,6,2)	ニユ(19,6,6)	ニヨ(19,6,8)
ヒ	ヒヤ(12,6,2)	ヒユ(12,6,6)	ヒヨ(12,6,8)
ミ	ミヤ(21,6,2)	ミユ(21,6,6)	ミヨ(21,6,8)
リ	リヤ(19,6,2)	リユ(19,6,6)	リヨ(19,6,8)
ギ	ギヤ(20,6,2)	ギユ(20,6,6)	ギヨ(20,6,8)
ジ	ジヤ(16,6,2)	ジユ(16,6,6)	ジヨ(16,6,8)
ヂ	ヂヤ(16,6,2)	ヂユ(16,6,6)	ヂヨ(16,6,8)
ビ	ビヤ(21,6,2)	ビユ(21,6,6)	ビヨ(21,6,8)
ピ	ピヤ(21,6,2)	ピユ(21,6,6)	ピヨ(21,6,8)

Examples for ja-JP

[Expand table](#)

Character	sapi	ipa
合成	ゴ'ウセ	go'wuseji
所有者	ショユ'ウ?ヤ	ɕjoju'wuɕja
最適化	サイテキカ+	sajitecika,

ko-KR

Vowels for ko-KR

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
a	2	아가씨	강	하다
ɛ	4	애국가	백	세번째
e	4	에너지	가겠구나	가게
ʊ	6	으레	으쓱해	가스
i	6	이거	아직까지	어미
ʌ	1	어미	차선변경	가시겠어
o	8	오래	의혹을	차고
u	7	우간다	은둔자의	가시나무
ɰi	20,6	의자	배추흰나비가	가요계의
ø	1	외가댁	배치된다	하되
ʷa	7,2	와글와글	가치관과	가지와
ʷɛ	7,4	왜관	호쾌가	안돼
ʷe	7,4	웨딩드레스		
ʷi	7,6	위계적	구조위원회가	한가위
ʷʌ	7,1	워낙	가까워서	가까워
ja	6,2	야구	기술집약도가	끌려가야
jɛ	6,4	애가	가수애기예요	
je	6,4	예감	적대관계가	의례
jʌ	6,1	여가	감정평가사	이리하여
jo	6,8	요구가	사용중지	가거든요
ju	6,7	유가적	경제교류가	소유

Consonant for ko-KR

ipa	viseme	Example 1	Example 2	Example 3
b	21	바가		밥
p	21	빠른가	막부정부가	
b	21		사범학교가	
t͡ɕ^h	19,16	참가비는	아침과	
ɖ	19	동네가		바깥
t	19	따라가지	깎두기가	
d	19		산도가	
ɡ	20	가쵸		각
k	20	까마귀가	젓가락으로	
g	20		단추가	
h	12		손가락질하거나	
ɦ	12	하기가	손가락질하며	
ɖ͡ʑ	19,16		손잡이가	
ɖ͡ʑ̚	19,16	자유가		
t͡ɕ	19,16	짜가면서	손가락질할	
k^h	20	키가	아킬레스건	
l	14			국가체제를
m	21	마다가스카르	통나무가	기침
n	19	나가서는	아느냐	따라가다보면
ŋ	20		강아지	한강
p^h	21	파티가	아파트	
r	19	라디오가	아름답게	
s^h	15	사고가	아스팔트	
s	15	쌍둥이가	멕시코가	

ipa	viseme	Example 1	Example 2	Example 3
t ^h	19	택시가	여타의	

ms-MY

Vowels for ms-MY

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
i	6	ibu	iklim	ahli
u	7	uang	buah	bahu
ə	1		kerja	nasionalisme
e	4	edar	aktres	kue
o	8	orang	anggota	pidato
a	2	anjing	anak	ada
ai	2,6			ceraai
au	2,7	auto	akaun	bakau
oi	8,6			amboi

Consonant for ms-MY

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
p	21	pekat	sekeping	cakap
b	21	banjir	lebih	jawab
t	19	tidak	peta	sikit
d	19	dekat	adakah	akad
k	20	ketat	akak	book

ipa	viseme	Example 1	Example 2	Example 3
g	20	gabung	bogel	dialog
ʔ	19	abang	kepercayaan	letak
tʃ	19,16	cepat	baca	
dʒ	19,16	jabatan	aja	kolej
m	21	memang	laman	malam
n	19	negeri	tanam	taman
ɲ	19	nyanyi	tanya	
ŋ	20		mangga	sayang
f	18	filem	artifak	aktif
v	18	vaksin	aktiviti	
s	15	sahabat	akses	tumis
z	15	zaman	lazat	
ʃ	16	syarikat	bersyarat	
x	12	khabar	akhir	tarikh
r	13	racun	merah	lebar
h	12	hingga	aduhai	boleh
j	6	yang	ayah	
w	7	walaupun	bawah	
l	14	lidah	alam	katil

nb-NO

Vowels for nb-NO

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
ɑ	2	annonse	betrakte	hoppa
æ	1	ergre	Palermo	
æ:	1	ærlig	belære	bæ
ɑ:	2	are	betale	bedra
ɛ	4	energi	kadetten	hoppe
ø:	1	øre	behøve	adjø
e:	4	ener	berede	distre
ɪ	6	ikke	setningen	taxi
i:	6	Eagle	bevise	konditori
ɔ	3	åtte	kontrollen	altså
œ	4	ønske	belønning	Mossø
o:	8	år	område	begå
u	7	okse	økokrim	ego
u:	7	ord	telefonen	bidro
ʏ	7	ytterst	benytte	Ally
ʊ	6	under	forundret	jaggu
ʊ:	6	ule	umulig	intervju
y:	4	yte	belyse	paraply
æɪ	1,6	eiendom**	utleide	snarvei
æʊ	1,6	aura	Litauen	fortau
ɑɪ	2,6	aibel**	Aserbajdsjan	Dubai
œʏ	4,7	øyer	ablegøyer	syltetøy
ɔʏ	3,7	Oilers	boikotten	konvoi
ʊɪ	6,6		Bruins	Mitsui

Consonant for nb-NO

ipa	viseme	Example 1	Example 2	Example 3
p	21	pil	ape	lapp
t	19	tall	matte	matt
k	20	kall	jakke	takk
b	21	bil	klubbe	lobb
d	19	dal	lide	gadd
g	20	gås	sagen	lag
f	18	fil	klaffe	klaff
h	12	hall	beholde	
s	15	sil	vise	viss
ʃ	15	sju	maskin	dusj
ç	12	tjukk	bekjenne	Korch
v	18	vår	leve	lov
m	21	mil	komme	lam
n	19	nål	minnes	søvn
ŋ	20		penger	lang
l	14	løs	måle	tal
r	13	ris	karre	tørr
j	6	jag	utjevne	detalj
ɖ	19		burde	ferd
ʎ	14		farlig	jarl
ɳ	19		barnet	jern
t	19		skjorte	gjort

nl-NL/nl-BE

Vowels for nl-NL

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
ɑ	2	af	bak	bavarois
a:	2	aan	maal	ja
ā	2	enfin	manchet	croissant
au	2,7	oud	bout	hou
ɛ	4	en	weg	hè
e:	4	één	heet	nee
ɛ:	4	airbag	blèr	
ɛi	4,6	eis	wijn	zij
ē	4		lingerie	elektricien
ø:	1	euro	deur	milieu
ɪ	6	ik	ding	
i	6	iets	sliep	drie
ɔ	3	op	slot	joh
u	7	oefen	hoed	doe
ɔ:	3		roze	
ō	3			Macron
o:	8	ook	boom	zo
ʏ	7	urn	dus	
ə	1	een	trommel	de
œy	4,4	uil	juist	bui
œ	4	oeuvre	service	
y	4	uur	tuur	nu

Consonant for nl-NL

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	boos	fobie	
d	19	dat	kudde	
f	18	fiets	gefeest	blaf
χ	12	ga	magie	hoog
ʔ	19		beamen	
ɦ	12	hoek	behaard	
g	20	gujarati	again	drug
j	6	jij	boeien	haai
k	20	kat	haken	zwak
l	14	land	familie	kool
m	21	man	demon	raam
n	19	niks	kannon	pan
ŋ	20		brenge	zing
p	21	poer	rapen	heb
ʀ	13	romp	waarom	kier
s	15	soms	precies	heus
ʃ	16	sjaal	vaasje	lunch
t	19	tot	laten	groot
w	7		flauwe	follow
v	18	voor	haven	
u	18	wat	fusiewet	
z	15	zal	lezen	
ʒ	16	jus	beige	

pl-PL

Vowels for pl-PL

[Expand table](#)

ipa	viseme	Example 1	Example 2	Example 3
a	2	autor	kolację	karasia
ɛ	4	elbląska	reflektor	osiągnięcie
ɛ̃	4		nieczęsto	skorupę
i	6	informatycznym	powiśle	gadali
ĩ	6		cudzymi	rodziły
ɔ	3	osobniki	uboju	rogatko
ɔ̃	3		mąż	intelektualistą
u	7	unosimy	arkuszy	przeznaczeniu

Consonant for pl-PL

[Expand table](#)

ipa	viseme	Example 1	Example 2	Example 3
b	21	bliska	wyśrubowane	
bʲ	21	biuro	zapobiegać	
t͡ɕ	19,16	ciągników	uczuciowych	suchość
t͡ɕ̞	19,15	częstkowe	odpoczynek	niszcz
c	16	kije	lisikiewicz	
d	19	drogowy	porodówkę	bastard
dʲ	19	dializy	studiuję	
d͡z	19,15	dzwonnica	wrodzony	
d͡z̞	19,16	dziurawe	pochodziłam	

ipa	viseme	Example 1	Example 2	Example 3
f	18	wprost	długofalowy	konserwantów
f ^j	18	filmoteki	grafiką	
g	20	geometrią	nawigacja	
ʒ	16	gitarzystą	religijnym	
ɖʒ	19,15	dżunglę	menedżerskie	
k	20	królestwa	naukowo	matematyk
l	14	latający	populacje	handel
l ^j	14	lisek	okolica	
m	21	majątkowy	wytłumaczenia	błagam
m ^j	21	mieszkającej	dynamicznie	
n	19	napędu	poczynaniach	balkon
ŋ	20		ciągłość	
ɲ	19	niewidoczna	zmieniała	poznań
p	21	potoczne	terapeuci	odstęp
p ^j	21	pijawek	skupieniu	
r	13	regionu	operową	administrator
r ^j	13	ripostuje	imperialnej	
s	15	solone	przetrasowania	biogaz
ɕ	16	sierpień	doniesieniem	mogłaś
ʃ	16	szanowanych	wpatrzeniu	skręcasz
t	19	talentom	kwaterze	dowód
t ^j	19	tirami	marketingiem	
ʈs	19,15	cyfrą	agencyjne	pałac
v	18	wysłaniu	przeprowadzają	
v ^j	18	winiarstwa	bukowianka	
w	7	łączenie	prałata	drukował

ipa	viseme	Example 1	Example 2	Example 3
x	12	hamulcem	zdychają	kostiumach
x ^j	12	hiszpańscy	psychice	
j	6	jeździła	popijałam	najważniejszej
z	15	zaskakują	partyzanckich	wiz
z	16	ziemniaki	zgryzienia	
ʒ	16	żyrandol	nowożytniej	

pt-BR

Vowels for pt-BR

Expand table

ipa	viseme	Example 1	Example 2	Example 3
i	6	ilha	ficar	comi
ĩ	6	intacto	pintar	aberdeen
a	2	água	dada	má
ɔ	3	ora	porta	cipó
u	7	ufanista	mula	peru
ũ	7	uns	pungente	kuhn
o	8	ortopedista	fofo	avô
e	4	elefante	elefante	você
ẽ	4	anta	canta	amanhã
ə	1	aqui	amaciar	dada
ɛ	4	ela	serra	até
ē	4	endorfina	pender	
õ	8	ontologia	conto	

Consonant for pt-BR

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
ɰ	7			atualização
w	7	washington	água	usou
p	21	pato	capital	
b	21	bola	cabeça	
t	19	tato	rato	
d	19	dado	amado	
g	20	gato	maragato	
m	21	mato	comer	
n	19	no	ano	
ɲ	19	nhoque	ninho	
f	18	faca	afago	
v	18	vaca	cavar	
r	19		para	amar
s	15	satisfeito	amassado	casados
z	15	zebra	azar	
ʃ	16	cheirar	machado	
ʒ	16	jaca	injusta	
x	12	rota	carreta	
tʃ	19,16	tirar	atirar	
dʒ	19,16	dia	adiar	
l	14	lata	aleto	
ʎ	14	lhama	malhado	
ɣ	6		inabalavelmente	hífen

ipa	viseme	Example 1	Example 2	Example 3
j	6		caixa	sai
k	20	casa	ensacado	

pt-PT

Vowels for pt-PT

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
a	2	ábdito	consular	medirá
e	4	abacaxi	domação	longa
ej	4,6	eidético	direita	detectei
ẽ	4	anverso	viajante	afã
ẽj	4,6	angels	viagens	também
ẽw	4,7	hã	significaçãozinha	gabão
ew	4,7		saudar	hello
aj	2,6	airosa	culturais	vai
ɔ	3	hora	depósito	ló
ɔj	3,6	óis	heróico	dói
aw	2,7	outlook	incauto	pau
ə	1	extremo	sapremar	noite
e	4	eclipse	haver	buffet
ɛ	4	eco	hibérnios	paté
ɛw	4,7		pirinéus	escarcéu
ẽ	4	embaçado	dirimente	ámen
ew	4,7	eu	deus	bebeu

ipa	viseme	Example 1	Example 2	Example 3
i	6	igreja	aplaudido	escrevi
ĩ	6	impaciente	espinçar	manequim
ɪw	6,7		niue	garantiu
o	8	ofir	consumidor	stacatto
oɟ	8,6	oirar	noite	foi
õ	8	ombrão	barronda	dom
õɟ	8,6		ocupações	expõe
u	7	ubi	facultativo	fado
uɟ	7,6	uivar	arruivado	fui
ũ	7	umbilical	funcionar	fórum
ũɟ	7,6		muito	

Consonant for pt-PT

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	bacalhau	tabaco	club
d	19	dado	dado	band
r	19	rename	verás	chutar
f	18	fim	eficácia	golf
g	20	gadinho	apego	blog
j	6	iode	desassociado	substitui
k	20	kiwi	traficado	snack
l	14	laborar	pelada	full
ɭ	14		polvo	brasil
ʎ	14	lhanamente	antilhas	

ipa	viseme	Example 1	Example 2	Example 3
m	21	maça	amanhã	modem
n	19	nutritivo	campana	scan
ɲ	19	nhambu-grande	toalhinha	penh
p	21	pai	crápula	laptop
ʀ	13	recordar	guerra	chauffeur
s	15	seco	grosseira	boss
ʃ	16	chuva	duchar	médios
t	19	tabaco	pelota	input
v	18	vaca	combatível	pavlov
w	7	waffle	restituir	katofio
z	15	zâmbia	prazer	jazz
ʒ	16	gelada	infligir	cuj

ro-RO

Vowels for ro-RO

Expand table

ipa	viseme	Example 1	Example 2	Example 3
ə	1	ăsta	abătut	fizică
ɨ	6	înspre	hotărâre	îhî
a	2	absolut	prematur	Praga
e	4	educație	tablete	alianțe
ɛa	4,2		studentească	badea
eo	4,8		bleumarin	vreo
i	6	Italia	aripi	aberații

ipa	viseme	Example 1	Example 2	Example 3
o	8	oricum	catacombe	radio
ɔa	8,2	oară	închisoare	șamoa
u	7	umble	gradul	Alexandru

Consonant for ro-RO

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	băi	vizibil	arab
b ^j	21			microbi
d	19	doctor	video	miliard
ɖʒ	19,16	ger	vegetație	
ɖʒ ^j	19,16			colegi
f	18	furtună	efort	ecograf
f ^j	18			filosofi
g	20	galeria	egal	filolog
g ^j	20		neghină	unghi
h	12	hexagon	arhitect	monarh
j	6	ieri	băiat	ditai
k	20	cadru	baricadat	pitic
k ^j	20			urechi
l	14	laptop	alint	Paul
l ^j	14			circuli
m	21	mandarine	camera	atom
m ^j	21			consumi
n	19	nepot	Canada	Eden

ipa	viseme	Example 1	Example 2	Example 3
ŋ	20		banca	plâng
n ^j	19			dragoni
p	21	pai	opera	Filip
p ^j	21			ocupi
r	13	real	mere	distribuitor
r ^j	13			palmieri
s	15	sertar	căsătorit	exclus
ʃ	16	șine	cușetă	greș
ʃ ^j	16			groși
t	19	teracota	material	abonament
t ^j	19			foști
ts	19,15	țar	cuțit	vorbăreț
tʃ	19,16	circa	meciuri	
ts ^j	19,15			uscați
tʃ ^j	19,16			indici
v	18	vaccin	gravidă	fugitiv
v ^j	18			nervi
w	7	uau	cuantificare	pliu
z	15	zoologică	frază	parbriz
ʒ	16	jar	abajur	pasaj
z ^j	15			semnezi
ʒ ^j	16			dârji

ru-RU

Vowels for ru-RU

[Expand table](#)

ipa	viseme	Example 1	Example 2	Example 3
a	2	адрес	радость	беда
л	1	облаков	застенчивость	внучка
ə	1		яблочного	
ε	4	эпос	белка	кафе
i	6	иней	лист	соловьи
ɪ	6	игра	медведь	мгновение
ɨ	6	энергия	лысый	весы
ɔ	3	окрик	мот	весло
u	7	ужин	куст	пойду

Consonant for ru-RU

[Expand table](#)

ipa	viseme	Example 1	Example 2	Example 3
p	21	профессор	поплавок	укроп
pʲ	21	Петербург	ослепительно	степь
b	21	большой	собака	
bʲ	21	белый	убедить	
t	19	тайна	старенький	твид
tʲ	19	тепло	учитель	синеть
d	19	доверчиво	недалеко	
dʲ	19	дядя	единица	
k	20	крыло	кукуруза	кустарник
kʲ	20	кипяток	неяркий	
g	20	гроза	немного	

ipa	viseme	Example 1	Example 2	Example 3
g ^j	20	герань	помогите	
x	12	хороший	поход	дух
x ^j	12	хилый	хихиканье	
f	18	фантазия	шкафах	кров
f ^j	18	фестиваль	кофе	верфь
v	18	внучка	синева	
v ^j	18	вертеть	свет	
s	15	сказочник	лесной	карапуз
s ^j	15	сеять	посередине	зажглись
z	15	заяц	звезда	
z ^j	15	земляника	созерцал	
ʃ	15	шуметь	пшено	мышь
ʒ	15	жилище	кружевной	
t͡s	19,15	целитель	Венеция	незнакомец
t͡ʃ	19,16	часы	очарование	мяч
ʧ:	16	щелчок	ощущать	лещ
m	21	молодежь	несмотря	том
m ^j	21	меч	дымить	семь
n	19	начало	оконце	сон
n ^j	19	небо	линялый	тюлень
l	14	лужа	долгожитель	мел
l ^j	14	лицо	недалеко	соль
r	13	радость	сорока	двор
r ^j	13	рябина	набережная	дверь
j	6	есть	маяк	игрушечный

sk-SK

Vowels for sk-SK

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
i	6	idea	efektivita	éry
e	4	edícia	absencia	farme
a	2	abnormálne	eskapáda	ária
o	8	oba	banková	esperanto
u	7	udial	február	babu
ɤ	6		mäsité	
i:	6	ílu	edícii	druhý
e:	4	éra	anamnézy	ázijské
a:	2	áno	animácia	druhová
o:	8	ódy	bilión	haló
u:	7	úbočí	absolútna	druhú
ia	6,2		piatkové	maškrtia
ie	6,4		domnienka	námestie
iu	6,7			väčšiu
uo	7,8	ôsma	jahôd	malinô
au	2,7	audio	aplaudovalo	sredau
ou	8,7			
ə	1			

Consonant for sk-SK

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
p	21	pád	apelu	cap
b	21	babička	abiogenézy	dub
t	19	tabak	foto	art
d	19	delta	editor	backhand
c	16	ťahač	docenti	farnost'
ʃ	16	d'alej	strede	lod'
k	20	kabaret	bábkový	chudáčik
g	20	galón	demagógia	mozog
ʦ	19,15	certifikácie	cicavce	bác
ʤ	19,15		medzí	
tʃ	19,16	čajový	frčí	boháč
dʒ	19,16	džúsu	brandže	
f	18	fabrík	biografie	fotograf
v	18	vokály	evanjelické	
s	15	sekunda	esá	algoritmus
z	15	zábal	fazuľu	aníz
ʃ	16	šabl'a	duševná	chápeš
ʒ	16	žaba	veži	kaluž
x	12	charakter	biochémie	bohatých
h	12	háčik	bohmi	
r	13	rabat	eróziou	éter
ɾ	13		chatrče	leicester
ɾː	13		vrtal	
l	14	lampa	električka	čakal
ɫ	14		dĺhý	nóbl
ɫː	14		jabl'k	

ipa	viseme	Example 1	Example 2	Example 3
ʎ	14	l'ad	citel'né	byl'
m	21	meter	emisný	aktom
ɱ	21		amfiteáter	
n	19	nábeh	colnému	fajn
ɳ	19		slovinský	
ŋ	20		banket	
ɲ	19	ňom	ani	jačmeň
ɥ	7		cestovní	aktív
ɨ	6		fajka	chatovej
j	6	ja	eseje	
w	7	vzbudí	krivdí	

sl-SI

Vowels for sl-SI

Expand table

ipa	viseme	Example 1	Example 2	Example 3
ə	1	rjava	pisemski	december
a	2	azilantov	hišam	delniška
a:	2	avto	marketplace	lucia
ɛ	4	edicija	pridem	ničle
e:	4	ebola	pridevnik	prepove
ɛ:	4	ena	producenta	janže
i	6	ideja	poudarim	poudariti
i:	6	igla	ilirska	jedmi

ipa	viseme	Example 1	Example 2	Example 3
ɔ	3	oba	morfološke	Marko
ɔ:	3	oče	črnomorskem	
o:	8	občina	reformam	seno
u	7	ulova	mamut	mandatu
u:	7	ura	dramaturgom	intervju

Consonant for sl-SI

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	brez	bober	lebdeča
d	19	dajal	degradacija	navedbah
d ¹	19	dleto		navedla
dn	19,19	dna		dohodne
ďž	19,16	džezovske	bridža	menedžment
ďž	19,15	odzivajo		Kocbek
f	18	fagota	fotografa	golf
ɱ	21			nimfa
ɣ	20			ahdeloja
g	20	gaber	lagal	ragbi
ɪ	6		pojdi	emisij
j	6	jadra	kajak	najostreje
k	20	kabel	akademija	alkoholik
l	14	labirint	olajša	šal
l ^j	14			poljski
m	21	maček	omara	potem

ipa	viseme	Example 1	Example 2	Example 3
ŋ	20	Ngaliemski_slapovi		banka
n	19	nabave	obarvana	obarvan
nʲ	19			konjska
p	21	pada	sapa	sestop
r	13	rabila	sorazmerna	spor
s	15	srajca	virusa	virus
ʃ	16	šah	sušijo	tovariš
t	19	tabela	gojiti	flavtist
t¹	19	tla	škatla	desetletne
tn	19,19			devetnajst
tʃ	19,16	čaj	gneča	hlač
ts	19,15	car	raca	rejec
u	7			avto
v	18	veja	avanture	
w	7	vgradila		slabokrvnost
m	7	vstati		
x	12	hiša	jahači	jamah
ʒ	16	žaba	ježa	možgani
z	15	zmaj	doza	izvoza

sv-SE

Vowels for sv-SE

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
a	2	andas	betrakta	gryta
æ	1	ärter	smärta	
æ:	1	ärliga	besvära	
ɑ:	2	avig	betala	bedra
ɔ	3	åtta	kontrollen	jaså
a_u	2,7	aura	pauser	Olau
ə	1		äpplena	pojke
e	4	energi	servetten	Arjepluovve
ɛ	4	äpple	berätta	Siviä
ɛ:	4	äta	beträda	trä
e:	4	eka	konkreta	café
œ	8	ört	störta	
œ:	4	ören	beröra	
ø	4	öppen	Alströmer	Päiviö
ø:	1	öl	belöning	adjö
ɪ	6	idé	vitsippa	kiwi
i:	6	ivrig	kris	parti
ʊ	4	oas	betrodda	konto
u:	7	oro	förtroende	bero
o:	8	åtala	telefonen	nivå
ə	1	uppenbar	förundrad	farstu
ʊ:	6	ute	bestulen	intervju
y	4	ytterst	rykte	Tommy
y:	4	yta	förtydliga	paraply

Consonant for sv-SE

ipa	viseme	Example 1	Example 2	Example 3
p	21	pil	apa	topp
t	19	tal	matta	akut
k	20	kål	jacka	tak
b	21	bil	klubba	jobb
d	19	dal	lida	stad
g	20	gås	såga	lag
f	18	fil	klaffa	klaff
h	12	hal	behålla	
s	15	sil	visa	viss
ʃ	16	sjuk	maskin	dusch
ç	16	tjock	åkkänslan	coach
v	18	vår	leva	torv
m	21	mil	kamma	arm
n	19	nål	minnas	sömn
ŋ	20		ringa	ung
l	14	lös	måla	tal
r	13	ris	kärra	borr
j	6	jag	tröja	haj
ɖ	19		borda	bord
ʎ	14		porlande	kärl
ɳ	19		gärna	barn
ʂ	15		forsa	fors
ʈ	19		skjorta	gjort

Vowels for th-TH

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
a	2		นะ	
a:	2		หาด	หา
e	4		เตะ	
e:	4		เท	ได้
i	6		ปิด	
i:	6		ปีก	ปี
ia	6,2		เจียบ	
o	8		บด	
o:	8		แลกโตส	ฉาวโฉ่
ə	1		ใบฝักเงิน	
ə:	1		เกิน	เหม่อ
u	7		จุด	
u:	7		ดูด	ดู
ua	7,2		กวน	รัว
ʊ	6		ยัด	
ʊ:	6		มิด	มีอ
ʊa	6,2		เรื่อน	เรื่อ
ɛ	4		คู่แข่ง	
ɛ:	4		แบน	คำแปล
ɔ	3		น็อค	
ɔ:	3		นอน	เจ็บคอ

Consonant for th-TH

[Expand table](#)

ipa	viseme	Example 1	Example 2	Example 3
b	21	บิน	กระบี่	ยมน
t̚	19,16	เจ็ด	กระโจม	
t̚ʰ	19,16	เข้าๆ	วันอาสาฬหบูชา	
d	19	เดช	วิดีโอ	
f	18	ฟ้า	เฟื่องฟู	
h	12	หา	เศษอาหาร	
j	6	ยา	เรียดนด	ยาย
k	20	กา	เนื้อไก่	มาก
kʰ	20	คา	เลขา	
l	14	ลา	หูลลาม	
m	21	มา	เสมอ	รัดกุม
n	19	นา	แอมโมเนีย	กาล
ŋ	20	งา	แต่งงาน	แหล่ง
p	21	ปา	โดยทั่วไปแล้ว	กับ
pʰ	21	พา	ลัทธิ	
r	13	รา	ไข่มาลาเรีย	
s	15	สาม	ไม่มีสติ	
t	19	ตา	กตึกา	อาจ
tʰ	19	ทา	คุณธรรม	
w	7	วัง	กรี	แก้ว
ʔ	19	อา	สะอาด	

Tone for th-TH

[Expand table](#)

ipa	Description	Example
˥	Mid	ไป bpai ^M (to go)
˨	Low	ไข่ khai ^L (egg)
˥˩	Falling	ใช่ chai ^F (yes; agreement)
˥˥	High	ครับ khrap ^H ([spoken by a male] yes)
˨˨˩	Rising	หนัง nang ^R (cinema film)

tr-TR

Vowels for tr-TR

[Expand table](#)

ipa	viseme	Example 1	Example 2	Example 3
a	2	armut	fal	elma
ɑː	2	ağrı	kağrı	dağ
e	4	erik	kel	akide
eː	4	eğri	değnek	yeğ
œ	4	ördek	göl	banliyö
œ̞	4,16	öğlen	açıköğretim	
i	6	ilaç	kil	kedi
i̞	6,16	iğne	mevkiine	tebliğ
o	8	orman	kol	vazo
o̞	8,16	oğlan	doğru	doğ
u	7	uçak	kuş	koku
u̞	7,16	uğur	buğra	başbuğ
ʊ	6	ıhlamur	tıp	kazı
ʊ̞	6,16	ığdır	sıgılık	tığ

ipa	viseme	Example 1	Example 2	Example 3
y	4	ülke	gül	öykü
ʏ̞	4,16		düğme	

Consonant for tr-TR

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	balık	ebe	sertab
c	16	keçi	ekşi	bölük
tʃ	19,16	çöp	uçak	ilgeç
d	19	dere	badem	ad
f	18	fil	kefen	çarşaf
g	20	gaz	yorgun	diyalog
ɣ	20		ağır	
ʒ	16	gavur	kevgir	
h	12	halat	ahır	külâh
j	6	yer	ayva	olay
dʒ	19,16	cezve	evcimen	hac
k	20	kabak	bakla	çocuk
l	14	leylek	delik	kil
ʙ	6	lala	kalın	pul
m	21	muz	yemek	kalem
n	19	nar	inek	sorun
ŋ	20		mangal	ring
p	21	para	kapı	rakip
r	19	renk	iri	minder

ipa	viseme	Example 1	Example 2	Example 3
s	15	sal	kısa	bahis
ʃ	16	şekil	koşu	afiş
t	19	terlik	kutup	adalet
v	18	vadi	tava	ev
w	7		tavuk	
z	15	zemin	gezi	kaz
ʒ	16	jöle	angajman	refüj

vi-VN

Vowels for vi-VN

 Expand table

ipa	viseme	Example 1	Example 2	Example 3
a	2		ban	ma
ɛ	4		hẹp	mẹ
i	6			chí
ɔ	3		học	tò
u	7		hung	thù
ua	7,2			chúa
aj	2,6			tài
ɛj	4,6			lấy
əj	1,6			chơi
o	8			xô
iew	6,4,7			riêu
ɪə	6,1		khướt	

ipa	viseme	Example 1	Example 2	Example 3
ɔ̌i	3,6			mọi
ə	1		phần	
ie	6,4		biển	
ǔj	7,6			mùi
āw	2,7			bảo
ɪ	6		chứng	tử
e	4		nhặt	
ǎw	2,7			màu
ǎj	2,6			ngày
ɪə̌j	6,1,6			lưới
ǒj	8,6			hồi
ə:	1		lợn	sở
e	4		sên	tế
ɔ̌w	3,7			tấu
ēw	4,7			béo
l̄w	6,7			tịu
ɪ̄w	6,7			tựu
ěj	4,6			bêu
ɪ̄^w	6,1,7			bươu
ɪ̄j	6,6			củi
ɪ	6	inh	tinh	
iə	6,1			kìa
ā̌j	2	ach	họach	

Consonant for vi-VN

ipa	viseme	Example 1	Example 2	Example 3
b	21	ba		
k	20	canh		diệc
z	15	diễn		
j	6	giặc		
ɟ	13	róc		
f	18	phụ		
ɣ	20	gà		
h	12	hoa		
l	14	làm		
m	21	mười		nắm
n	19	núi		tiền
p	21	pin		pháp
s	15	xào		
ʃ	15	son		
t	19	tuổi		hết
v	18	vàng		
ɖ	19	đón		
ŋ	20	ngủn		lưng
x	12	khùng		
ɲ	19	nhàm		sanh
tʰ	19	thông		
ʈ	19	trùng		
tʃ	19,16	chim		chỉ
w	7		hoang	

Tone for vi-VN

[Expand table](#)

ipa	Description	Example
	tone ngang	xem
	tone huyền	làm
	tone sắc	sống
	tone hỏi	phải
	tone ngã	cũng
	tone nặng	một

zh-CN

The Speech service phone set for **zh-CN** is based on the native phone [Pinyin](#) [↗](#) set.

Pinyin Initials for zh-CN

[Expand table](#)

Pinyin	viseme	Character example	sapi example
b	21	玻	bo 1
p	21	坡	po 1
m	21	摸	mo 1
f	18	佛	fo 2
d	19	得	de 2
t	19	特	te 4
n	19	呢	ne 5
l	14	乐	le 4
g	20	哥	ge 1
k	20	科	ke 1

Pinyin	viseme	Character example	sapi example
h	12	喝	he 1
j	16	基	ji 1
q	16	欺	qi 1
x	16	希	xi 1
zh	19, 15	知	zhi 1
ch	19, 15	吃	chi 1
sh	15	诗	shi 1
r	15	日	ri 4
z	15	资	zi 1
c	15	此	ci 3
s	15	思	si 1
y	6	衣	yi 1
w	7	屋	wu 1

Pinyin Finals for zh-CN

 Expand table

Pinyin	viseme	Character example	sapi example
a	2, 2 (19, 2, 2 for no initials)	法	fa 3
o	7, 8, 8 (19, 8, 8 for no initials)	泼	po 1
e	1, 1 (19, 1, 1 for no initials)	歌	ge 1
i	6, 6, 6	理	li 3
u	7, 7, 7	步	bu 4
v	7, 4, 4	女	nv 3
ai	2, 4 (19, 2, 4 for no initials)	百	bai 3
ei	4, 6 (19, 4, 6 for no initials)	北	bei 3
ui	7, 4, 6	对	dui 4

Pinyin	viseme	Character example	sapi example
ao	2, 8 (19, 2, 8 for no initials)	号	hao 4
ou	8, 7 (19, 8, 7 for no initials)	走	zou 3
iu	6, 8, 7	牛	niu 2
ie	6, 4, 4	谢	xie 4
ue	7, 4, 4	略	lue 4
er	19, 1, 1	耳	er 3
an	2, 19 (19, 2, 19 for no initials)	喊	han 3
en	1, 19 (19, 1, 19 for no initials)	肯	ken 3
in	6, 6, 19	宾	bin 1
un	7, 1, 19	尊	zun 1
ang	2, 20 (19, 2, 20 for no initials)	朗	lang 3
eng	1, 20	恒	heng 2
ing	6, 1, 20	赢	ying 2
ong	7, 7, 20	红	hong 2
ia	6, 2, 2	家	jia 1
ian	6, 2, 19	面	mian 4
iang	6, 2, 20	象	xiang 4
iao	6, 2, 8	表	biao 3
iong	7, 7, 20	兄	xiong 1
ua	7, 2, 2	花	hua 1
uai	7, 2, 4	帅	shuai 4
uan	7, 2, 19	短	duan 3
uang	7, 2, 20	广	guang 3
uo	7, 8, 8	多	duo 1

Pinyin Whole syllable for zh-CN

Pinyin	viseme	Character example	sapi example
zhi	19, 15, 6, 6	知	zhi 1
chi	19, 15, 6, 6	吃	chi 1
shi	15, 6, 6	诗	shi 1
ri	15, 6, 6	日	ri 1
zi	15, 6, 6	资	zi 1
ci	15, 6, 6	词	ci 2
si	15, 6, 6	斯	si 1
yi	6, 6, 6	衣	yi 1
wu	7, 7, 7	屋	wu 1
yu	7, 4, 4	鱼	yu 2
ye	6, 4, 4	叶	ye 4
yue	7, 4, 4	月	yue 4
yuan	7, 2, 19	圆	yuan 2
yin	6, 6, 19	音	yin 1
yun	7, 1, 19	云	yun 1
ying	6, 1, 20	英	ying 1
a	19, 2, 2	啊	a 1
o	19, 8, 8	噢	o 1
e	19, 1, 1	鹅	e 2
ai	19, 2, 4	爱	ai 4
ei	19, 4, 6	欸	ei 1
ao	19, 2, 8	奥	ao 4
ou	19, 8, 7	偶	ou 3
an	19, 2, 19	安	an 1
en	19, 1, 19	恩	en 1

Pinyin	viseme	Character example	sapi example
ang	19, 2, 20	昂	ang 2

Pinyin Tone for zh-CN

[Expand table](#)

Pinyin	Character example	sapi example
bā	八	ba 1
bá	拔	ba 2
bǎ	把	ba 3
bà	坝	ba 4
ba	吧	ba 5

Example for zh-CN

[Expand table](#)

Character	Speech service
组织关系	zu 3 - zhi 1 - guan 1 - xi 5
累进	lei 3 - jin 4
西宅巷	xi 1 - zhai 2 - xiang 4
一会儿	yi 2 - hui r 4

zh-HK

The Speech service phone set for **zh-HK** is based on the native phone [Jyutping](#) [↗](#) set.

Jyutping Initials for zh-HK

[Expand table](#)

Jyutping	Character example	sapi example	VisemeID
p	怕	paa 3	21
b	巴	baa 1	21
t	他	taa 1	19
d	打	daa 2	19
k	卡	kaa 1	20
g	家	gaa 1	20
f	花	faa 1	18
s	沙	saa 1	15
h	蝦	haa 1	12
m	媽	maa 1	21
n	那	naa 5	19
ng	牙	ngaa 4	20
c	叉	caa 1	19 15
z	渣	zaa 1	19 15
l	啦	laa 1	14
kw	誇	kwaa 1	20
gw	瓜	gwaa 1	20
w	蛙	waa 1	7
j	廿	jaa 6	6

Jyutping Middle for zh-HK

 Expand table

Jyutping	Character example	sapi example	VisemeID
aa	沙	saa 1	2
e	些	se 1	4
i	詩	si 1	6

Jyutping	Character example	sapi example	VisemeID
o	疏	so 1	3 In [ou], [o] corresponding viseme is 8
u	夫	fu 1	7
oe	鋸	goe 3	4
yu	書	syu 1	4
a	新	san 1	4
eo	律	leot 6	1

Jyutping Ending for zh-HK

Expand table

Jyutping	Character example	sapi example	VisemeID
i	西	sai 1	6 (In [eoi], [i] corresponding viseme is 4)
u	收	sau 1	7
p	夾	gep 6	21
t	不	bat 1	19
k	策	caak 3	20
m	心	sam 1	21
n	新	san 1	19
ng	敬	ging 3	20

Jyutping Tone for zh-HK

Expand table

Tone number	Description	Character example	sapi example
1	High level/High falling or Entering High Level	詩	si 1
2	Mid Rising	史	si 2
3	Mid Level or Entering Mid Level	試	si 3

Tone number	Description	Character example	sapi example
4	Low Falling	時	si 4
5	Low Rising	市	si 5
6	Low Level or Entering Low Level	是	si 6

zh-TW

The Speech service phone set for zh-TW is based on the native phone [Bopomofo](#) [↗](#) set.

Bopomofo Initials for zh-TW

[↗](#) Expand table

Bopomofo	Pinyin	sapi Example (Bopomofo, Pinyin)
ㄅ	b	玻 (ㄅㄛ, bo 1)
ㄆ	p	坡 (ㄆㄛ, po 1)
ㄇ	m	摸 (ㄇㄛ, mo 1)
ㄈ	f	佛 (ㄈㄛˊ, fo 2)
ㄉ	d	得 (ㄉㄛˊ, de 2)
ㄊ	t	特 (ㄊㄛˋ, te 4)
ㄋ	n	呢 (ㄋㄛˋ, ne 5)
ㄌ	l	樂 (ㄌㄛˋ, le 4)
ㄍ	g	哥 (ㄍㄛ, ge 1)
ㄎ	k	科 (ㄎㄛ, ke 1)
ㄏ	h	喝 (ㄏㄛ, he 1)
ㄐ	j	基 (ㄐㄧ, ji 1)
ㄑ	q	欺 (ㄑㄧ, qi 1)
ㄒ	x	希 (ㄒㄧ, xi 1)
ㄓ	zh	知 (ㄓ, zhi 1)

Bopomofo	Pinyin	sapi Example (Bopomofo, Pinyin)
ㄷ	ch	吃 (ㄷ, chi 1)
ㄕ	sh	詩 (ㄕ, shi 1)
ㄖ	r	日 (ㄖ, ri 4)
ㄗ	z	資 (ㄗ, zi 1)
ㄘ	c	此 (ㄘ, ci 3)
ㄙ	s	思 (ㄙ, si 1)
ㄌ	y	衣 (ㄌ, yi 1)
ㄨ	w	屋 (ㄨ, wu 1)

Bopomofo Finals for zh-TW

 Expand table

Bopomofo	Pinyin	sapi Example (Bopomofo, Pinyin)
ㄚ	a	法 (ㄈㄚ, fa 3)
ㄛ	o	潑 (ㄆㄛ, po 1)
ㄜ	e	歌 (ㄍㄜ, ge 1)
ㄝ	i	理 (ㄌㄝ, li 3)
ㄨ	u	步 (ㄅㄨ, bu 4)
ㄩ	v	女 (ㄋㄩ, nv 3)
ㄞ	ai	百 (ㄅㄞ, bai 3)
ㄟ	ei	北 (ㄅㄟ, bei 3)
ㄠ	ui	對 (ㄉㄠ, dui 4)
ㄡ	ao	號 (ㄏㄡ, hao 4)
ㄣ	ou	走 (ㄘㄣ, zou 3)
ㄤ	iu	牛 (ㄋㄤ, niu 2)
ㄥ	ie	謝 (ㄒㄥ, xie 4)

Bopomofo	Pinyin	sapi Example (Bopomofo, Pinyin)
ㄌㄝˊ	ue	略 (ㄌㄌㄝˊ, lue 4)
ㄏㄢˋ	an	喊 (ㄏㄢˋ, han 3)
ㄎㄣˋ	en	肯 (ㄎㄎㄣˋ, ken 3)
ㄅㄧㄣˊ	in	賓 (ㄅㄧㄣˊ, bin 1)
ㄗㄨㄣˊ	un	尊 (ㄗㄗㄨㄣˊ, zun 1)
ㄌㄤˋ	ang	朗 (ㄌㄌㄤˋ, lang 3)
ㄏㄥˊ	eng	恆 (ㄏㄥˊ, heng 2)
ㄧㄥˊ	ing	贏 (ㄧㄥˊ, ying 2)
ㄏㄨㄥˊ	ong	紅 (ㄏㄨㄥˊ, hong 2)
ㄐㄧㄚˊ	ia	家 (ㄐㄐㄧㄚˊ, jia 1)
ㄇㄧㄢˋ	ian	麵 (ㄇㄇㄧㄢˋ, mian 4)
ㄒㄧㄤˋ	iang	象 (ㄒㄧㄤˋ, xiang 4)
ㄅㄧㄠˋ	iao	表 (ㄅㄧㄠˋ, biao 3)
ㄒㄩㄥˊ	iong	兄 (ㄒㄩㄥˊ, xiong 1)
ㄏㄨㄚˊ	ua	花 (ㄏㄨㄚˊ, hua 1)
ㄕㄨㄞˋ	uai	帥 (ㄕㄕㄨㄞˋ, shuai 4)
ㄉㄨㄢˋ	uan	短 (ㄉㄉㄨㄢˋ, duan 3)
ㄍㄨㄤˋ	uang	廣 (ㄍㄍㄨㄤˋ, guang 3)
ㄉㄨㄛˊ	uo	多 (ㄉㄉㄨㄛˊ, duo 1)

Bopomofo Whole syllable zh-TW

[Expand table](#)

Bopomofo	Pinyin	sapi Example (Bopomofo, Pinyin)
ㄓ	zhi	知 (ㄓ, zhi 1)
ㄔ	chi	吃 (ㄔ, chi 1)

Bopomofo	Pinyin	sapi Example (Bopomofo, Pinyin)
尸	shi	诗 (尸, shi 1)
日	ri	日 (日, ri 1)
ㄗ	zi	资 (ㄗ, zi 1)
ㄘ	ci	词 (ㄘ, ci 2)
ㄌ	si	斯 (ㄌ, si 1)
ㄣ	yi	衣 (ㄣ, yi 1)
ㄨㄛ	wo	我 (ㄨㄛ, wo 3)
ㄨ	wu	屋 (ㄨ, wu 1)
ㄣ	yu	鱼 (ㄣ, yu 2)
ㄣ ㄝ	ye	叶 (ㄣ ㄝ, ye 4)
ㄣ ㄝ	yue	月 (ㄣ ㄝ, yue 4)
ㄣ	er	耳 (ㄣ, er 3)
ㄣ ㄢ	yuan	圆 (ㄣ ㄢ, yuan 2)
ㄣ ㄣ	yin	音 (ㄣ ㄣ, yin 1)
ㄣ ㄣ	yun	云 (ㄣ ㄣ, yun 1)
ㄣ ㄥ	ying	英 (ㄣ ㄥ, ying 1)
ㄣ ㄥ	yong	拥 (ㄣ ㄥ, yong 1)
ㄚ	a	啊 (ㄚ, a 1)
ㄛ	o	噢 (ㄛ, o 1)
ㄜ	e	鹅 (ㄜ, e 2)
ㄞ	ai	爱 (ㄞ, ai 4)
ㄟ	ei	欸 (ㄟ, ei 1)
ㄠ	ao	奥 (ㄠ, ao 4)
ㄡ	ou	偶 (ㄡ, ou 3)
ㄢ	an	安 (ㄢ, an 1)
ㄣ	en	恩 (ㄣ, en 1)

Bopomofo	Pinyin	sapi Example (Bopomofo, Pinyin)
ㄤ	ang	昂 (ㄤˊ, ang 2)
ㄥ	eng	鞞 (ㄥ, eng 1)
ㄝ	ê	ㄝ (ㄝ, ê 1)

Bopomofo tone zh-TW

[↗](#) Expand table

Bopomofo	Tone number	sapi Example (Bopomofo, Pinyin)
ˊ	1	八 (ㄅㄚˊ or ㄅㄚˇ, ba 1)
ˊˊ	2	拔 (ㄅㄚˊˊ, ba 2)
ˇ	3	把 (ㄅㄚˇ, ba 3)
ˋ	4	坝 (ㄅㄚˋ, ba 4)
ˊˋ	5	吧 (ㄅㄚˊˋ, ba 5)

Map X-SAMPA to IPA

When using X-SAMPA phone symbols `'` and `"`, it's important to avoid conflicts with the wrapper symbol. If the X-SAMPA string contains phone `'`, we recommend using phone `_j` instead. If you don't want to replace the phone `'`, you need to use double quotes `"` as a wrapper. Similarly, if the X-SAMPA string contains phone `"`, then double quotes shouldn't be used as a wrapper, and you must use the single quote `'` as a wrapper. Otherwise, it will cause an error.

The table below shows a mapping relationship between X-SAMPA (Extended Speech Assessment Methods Phonetic Alphabet) and IPA alphabets. The X-SAMPA symbols are shown at left, with the corresponding IPA symbols to the right.

txt	
x-sampa (L) ipa (R) a a b b b_< b̥	

c
d
d`
d_<
e
f
g
g_<
h
h\
i
j
j\
k
l
l`
l\
m
n
n`
o
p
p\
q
r
r`
r\
r\`
s
s`
s\
t
t`
u
v
P
v\
w
x
x\
y
z
z`
z\
A
B
B\
C
D
E
F
G
G\
G_<
H

c
d
d
d
e
f
g
g
h
h
i
j
j
k
l
l
J
m
n
n
o
p
p
q
r
r
J
J
s
s
t
t
u
v
u
u
w
x
h
y
z
z
z
a
b
B
c
d
e
e
Y
G
G
y

H\	H
I	I
I\	ı
J	ƶ
J\	Ʒ
J_<	ƚ
K	ƙ
K\	ķ
L	ℒ
L\	ℓ
M	ℳ
M\	℣
N	ℕ
N\	ℕ
O	℔
O\	℔
Q	ℙ
R	℞
R\	℞
S	ℑ
T	℔
U	℔
U\	℔
V	ℒ
W	ℳ
X	ℒ
X\	ℒ
Y	ℒ
Z	℔
.	.
„	„
%	%
_j	ı
ı	ı
:	:
:\	ı
@	ə
@\	ə
@`	ə
{	æ
}	æ
1	ı
2	ø
3	ε
3\	ε
4	ı
5	ı
6	ε
7	ı
8	ø
9	œ
&	œ
?	ı
?\	ı
<\	ı

_w
_X
_X

w
~
x

Lower speech synthesis latency using Speech SDK

08/07/2025

In this article, we introduce the best practices to lower the text to speech synthesis latency and bring the best performance to your end users.

Normally, we measure the latency by `first byte latency` and `finish latency`, as follows:

 Expand table

Latency	Description	<code>SpeechSynthesisResult</code> property key
<code>first byte client latency</code>	Indicates the time delay between the synthesis starts and the first audio chunk is received on the client including network latency.	<code>SpeechServiceResponse_SynthesisFirstByteLatencyMs</code>
<code>finish client latency</code>	Indicates the time delay between the synthesis starts and the whole synthesized audio is received on the client including network latency.	<code>SpeechServiceResponse_SynthesisFinishLatencyMs</code>
<code>network latency</code>	The network latency between the client and Azure TTS service.	<code>SpeechServiceResponse_SynthesisNetworkLatencyMs</code>
<code>first byte service latency</code>	Indicates the time delay between Azure TTS service received synthesis request and the first audio chunk is returned.	<code>SpeechServiceResponse_SynthesisServiceLatencyMs</code>

The Speech SDK puts the latency durations in the Properties collection of `SpeechSynthesisResult`. The following sample code shows these values.

C#

```
var result = await synthesizer.SpeakTextAsync(text);
Console.WriteLine($"first byte client latency:
\t{result.Properties.GetProperty(PropertyId.SpeechServiceResponse_SynthesisFirstByteLatencyMs)} ms");
Console.WriteLine($"finish client latency:
\t{result.Properties.GetProperty(PropertyId.SpeechServiceResponse_SynthesisFinishLatencyMs)} ms");
Console.WriteLine($"network latency:
\t{result.Properties.GetProperty(PropertyId.SpeechServiceResponse_SynthesisNetworkLatencyMs)} ms");
Console.WriteLine($"first byte service latency:
```

```
\t{result.Properties.GetProperty(PropertyId.SpeechServiceResponse_SynthesisServiceLatencyMs)} ms");  
// you can also get the result id, and send to us when you need help for diagnosis  
var resultId = result.ResultId;
```

The first byte latency is lower than finish latency in most cases. The first byte latency is independent from text length, while finish latency increases with text length.

Ideally, we want to minimize the user-experienced latency (the latency before user hears the sound) to one network route trip time plus the first audio chunk latency of the speech synthesis service.

Streaming

Streaming is critical to lowering latency. Client code can start playback when the first audio chunk is received. In a service scenario, you can forward the audio chunks immediately to your clients instead of waiting for the whole audio.

You can use the [PullAudioOutputStream](#), [PushAudioOutputStream](#), [Synthesizing event](#), and [AudioDataStream](#) of the Speech SDK to enable streaming.

Taking `AudioDataStream` as an example:

```
C#  
  
using (var synthesizer = new SpeechSynthesizer(config, null as AudioConfig))  
{  
    using (var result = await synthesizer.StartSpeakingTextAsync(text))  
    {  
        using (var audioDataStream = AudioDataStream.FromResult(result))  
        {  
            byte[] buffer = new byte[16000];  
            uint filledSize = 0;  
            while ((filledSize = audioDataStream.ReadData(buffer)) > 0)  
            {  
                Console.WriteLine($"{filledSize} bytes received.");  
            }  
        }  
    }  
}
```

Pre-connect and reuse SpeechSynthesizer

The Speech SDK uses a websocket to communicate with the service. Ideally, the network latency should be one route trip time (RTT). If the connection is newly established, the network

latency includes extra time to establish the connection. The establishment of a websocket connection needs the TCP handshake, SSL handshake, HTTP connection, and protocol upgrade, which introduces time delay. To avoid the connection latency, we recommend pre-connecting and reusing the `SpeechSynthesizer`.

Pre-connect

To pre-connect, establish a connection to the Speech service when you know the connection is needed soon. For example, if you're building a speech bot in client, you can pre-connect to the speech synthesis service when the user starts to talk, and call `SpeakTextAsync` when the bot reply text is ready.

C#

```
using (var synthesizer = new SpeechSynthesizer(uspConfig, null as AudioConfig))
{
    using (var connection = Connection.FromSpeechSynthesizer(synthesizer))
    {
        connection.Open(true);
    }
    await synthesizer.SpeakTextAsync(text);
}
```

ⓘ Note

If the text is available, just call `SpeakTextAsync` to synthesize the audio. The SDK will handle the connection.

Reuse SpeechSynthesizer

Another way to reduce the connection latency is to reuse the `SpeechSynthesizer` so you don't need to create a new `SpeechSynthesizer` for each synthesis. We recommend using object pool in service scenario. See our sample code for [C#](#) and [Java](#).

Transmit compressed audio over the network

When the network is unstable or with limited bandwidth, the payload size also affects latency. Meanwhile, a compressed audio format helps to save the users' network bandwidth, which is especially valuable for mobile users.

We support many compressed formats including `opus`, `webm`, `mp3`, `silk`, and so on, see the full list in [SpeechSynthesisOutputFormat](#). For example, the bitrate of `Riff24Khz16BitMonoPcm` format is 384 kbps, while `Audio24Khz48KBitRateMonoMp3` only costs 48 kbps. The Speech SDK automatically uses a compressed format for transmission when a `pcm` output format is set. For Linux and Windows, `GStreamer` is required to enable this feature. Refer [this instruction](#) to install and configure `GStreamer` for Speech SDK. For Android, iOS, and macOS, no extra configuration is needed starting version 1.20.

Input text streaming

Text streaming allows real-time text processing for rapid audio generation. It's perfect for dynamic text vocalization, such as reading outputs from AI models like GPT in real-time. This feature minimizes latency and improves the fluidity and responsiveness of audio outputs, making it ideal for interactive applications, live events, and responsive AI-driven dialogues.

How to use text streaming

Text streaming is supported in C#, C++ and Python with Speech SDK.

To use the text streaming feature, connect to the websocket V2 endpoint:

```
wss://{region}.tts.speech.microsoft.com/cognitiveservices/websocket/v2
```

See the sample code for setting the endpoint:

C#

```
// IMPORTANT: MUST use the websocket v2 endpoint
var ttsEndpoint =
    $"wss://{Environment.GetEnvironmentVariable("AZURE_TTS_REGION")}.tts.speech.microsoft.com/cognitiveservices/websocket/v2";
var speechConfig = SpeechConfig.FromEndpoint(
    new Uri(ttsEndpoint),
    Environment.GetEnvironmentVariable("AZURE_TTS_API_KEY"));
```

Key steps

1. **Create a text stream request:** Use `SpeechSynthesisRequestInputType.TextStream` to initiate a text stream.
2. **Set global properties:** Adjust settings such as output format and voice name directly, as the feature handles partial text inputs and doesn't support SSML. Refer to the following sample code for instructions on how to set them. OpenAI text to speech voices aren't

supported by the text streaming feature. See this [language table](#) for full language support.

C#

```
// Set output format
speechConfig.SetSpeechSynthesisOutputFormat(SpeechSynthesisOutputFormat.Raw24
Khz16BitMonoPcm);

// Set a voice name
SpeechConfig.SetProperty(PropertyId.SpeechServiceConnection_SynthVoice, "en-
US-AvaMultilingualNeural");
```

3. **Stream your text:** For each text chunk generated from a GPT model, use `request.InputStream.Write(text);` to send the text to the stream.
4. **Close the stream:** Once the GPT model completes its output, close the stream using `request.InputStream.Close();`.

For detailed implementation, see the [sample code on GitHub](#) [↗](#)

Others tips

Cache CRL files

The Speech SDK uses CRL files to check the certification. Caching the CRL files until expired helps you avoid downloading CRL files every time. See [How to configure OpenSSL for Linux](#) for details.

Use latest Speech SDK

We keep improving the Speech SDK's performance, so try to use the latest Speech SDK in your application.

Load test guideline

You can use load test to test the speech synthesis service capacity and latency. Here are some guidelines:

- The speech synthesis service has the ability to autoscale, but takes time to scale out. If the concurrency is increased in a short time, the client might get long latency or 429 error

code (too many requests). So, we recommend you increase your concurrency step by step in load test. [See this article](#) for more details, especially [this example of workload patterns](#).

- You can use our sample using object pool ([C#](#) and [Java](#)) for load test and getting the latency numbers. You can modify the test turns and concurrency in the sample to meet your target concurrency.
- The service has quota limitation based on the real traffic, therefore, if you want to perform load test with the concurrency higher than your real traffic, connect before your test.

Next steps

- [See the samples](#) on GitHub

Get facial position with viseme

08/07/2025

ⓘ Note

To explore the locales supported for viseme ID and blend shapes, refer to [the list of all supported locales](#). Scalable Vector Graphics (SVG) is only supported for the `en-US` locale.

A *viseme* is the visual description of a phoneme in spoken language. It defines the position of the face and mouth while a person is speaking. Each viseme depicts the key facial poses for a specific set of phonemes.

You can use visemes to control the movement of 2D and 3D avatar models, so that the facial positions are best aligned with synthetic speech. For example, you can:

- Create an animated virtual voice assistant for intelligent kiosks, building multi-mode integrated services for your customers.
- Build immersive news broadcasts and improve audience experiences with natural face and mouth movements.
- Generate more interactive gaming avatars and cartoon characters that can speak with dynamic content.
- Make more effective language teaching videos that help language learners understand the mouth behavior of each word and phoneme.
- People with hearing impairment can also pick up sounds visually and "lip-read" speech content that shows visemes on an animated face.

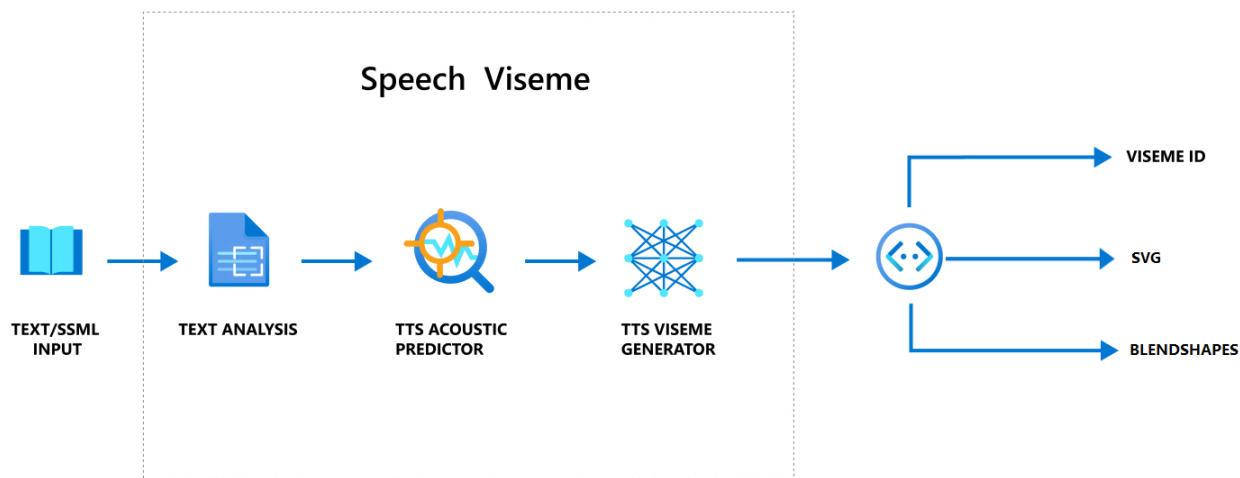
For more information about visemes, view this [introductory video](#) .

<https://www.youtube-nocookie.com/embed/ui9XT47uwxs> 

Overall workflow of producing viseme with speech

Neural Text to speech (Neural TTS) turns input text or SSML (Speech Synthesis Markup Language) into lifelike synthesized speech. Speech audio output can be accompanied by viseme ID, Scalable Vector Graphics (SVG), or blend shapes. Using a 2D or 3D rendering engine, you can use these viseme events to animate your avatar.

The overall workflow of viseme is depicted in the following flowchart:



Viseme ID

Viseme ID refers to an integer number that specifies a viseme. We offer 22 different visemes, each depicting the mouth position for a specific set of phonemes. There's no one-to-one correspondence between visemes and phonemes. Often, several phonemes correspond to a single viseme, because they looked the same on the speaker's face when they're produced, such as **s** and **z**. For more specific information, see the table for [mapping phonemes to viseme IDs](#).

Speech audio output can be accompanied by viseme IDs and **Audio offset**. The **Audio offset** indicates the offset timestamp that represents the start time of each viseme, in ticks (100 nanoseconds).

Map phonemes to visemes

Visemes vary by language and locale. Each locale has a set of visemes that correspond to its specific phonemes. The [SSML phonetic alphabets](#) documentation maps viseme IDs to the corresponding International Phonetic Alphabet (IPA) phonemes. The table in this section shows a mapping relationship between viseme IDs and mouth positions, listing typical IPA phonemes for each viseme ID.

[Expand table](#)

Viseme ID	IPA	Mouth position
0	Silence	
1	æ, ə, ʌ	
2	ɑ	
3	ɔ	

Viseme ID	IPA	Mouth position
4	ɛ, ʊ	
5	ə	
6	ʃ, ɪ, ɪ	
7	w, u	

Viseme ID	IPA	Mouth position
8	o	
9	aʊ	
10	ɔɪ	
11	aɪ	

Viseme ID	IPA	Mouth position
12	h	
13	u	
14	l	
15	s, z	

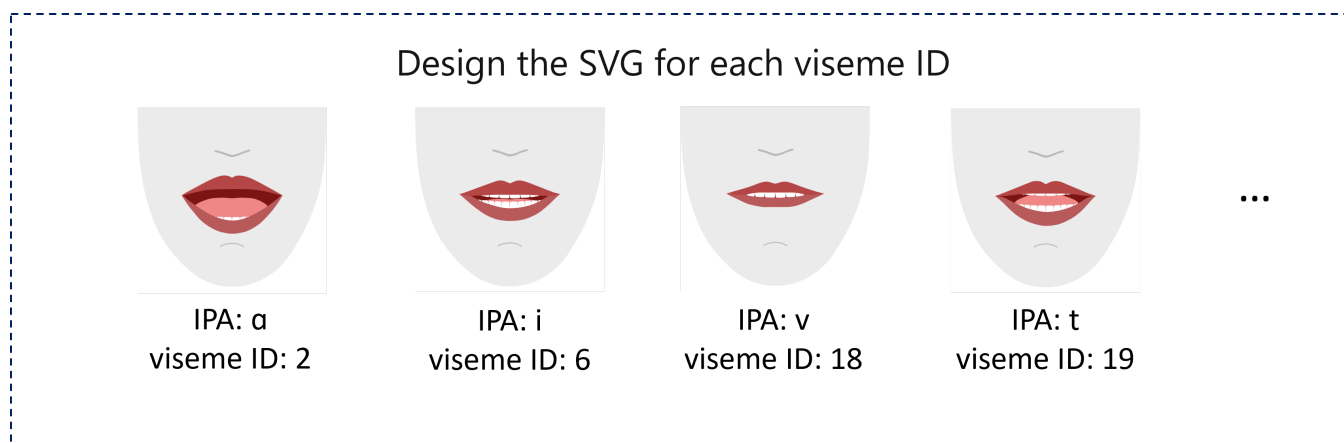
Viseme ID	IPA	Mouth position
16	ʃ, tʃ, dʒ, ʒ	
17	ð	
18	f, v	
19	d, t, n, θ	

Viseme ID	IPA	Mouth position
20	k, g, ŋ	
21	p, b, m	

2D SVG animation

For 2D characters, you can design a character that suits your scenario and use Scalable Vector Graphics (SVG) for each viseme ID to get a time-based face position.

With temporal tags that are provided in a viseme event, these well-designed SVGs is processed with smoothing modifications, and provide robust animation to the users. For example, the following illustration shows a red-lipped character designed for language learning.



3D blend shapes animation

You can use blend shapes to drive the facial movements of a 3D character that you designed.

The blend shapes JSON string is represented as a 2-dimensional matrix. Each row represents a frame. Each frame (in 60 FPS) contains an array of 55 facial positions.

Get viseme events with the Speech SDK

To get viseme with your synthesized speech, subscribe to the `VisemeReceived` event in the Speech SDK.

ⓘ Note

To request SVG or blend shapes output, you should use the `mstts:viseme` element in SSML. For details, see [how to use viseme element in SSML](#).

The following snippet shows how to subscribe to the viseme event:

C#

```
using (var synthesizer = new SpeechSynthesizer(speechConfig, audioConfig))
{
    // Subscribes to viseme received event
    synthesizer.VisemeReceived += (s, e) =>
    {
        Console.WriteLine($"Viseme event received. Audio offset: " +
            $"{e.AudioOffset / 10000}ms, viseme id: {e.VisemeId}.");

        // `Animation` is an xml string for SVG or a json string for blend shapes
        var animation = e.Animation;
    };

    // If VisemeID is the only thing you want, you can also use `SpeakTextAsync()`
    var result = await synthesizer.SpeakSsmlAsync(ssml);
}
```

Here's an example of the viseme output.

Viseme ID

text

(Viseme), Viseme ID: 1, Audio offset: 200ms.

(Viseme), Viseme ID: 5, Audio offset: 850ms.

.....

(Viseme), Viseme ID: 13, Audio offset: 2350ms.

After you obtain the viseme output, you can use these events to drive character animation. You can build your own characters and automatically animate them.

Next steps

- [SSML phonetic alphabets](#)
- [How to improve synthesis with SSML](#)

What are high definition voices?

Azure AI Speech continues to advance in the field of text to speech technology with the introduction of neural text to speech high definition (HD) voices. The HD voices can understand the content, automatically detect emotions in the input text, and adjust the speaking tone in real-time to match the sentiment. HD voices maintain a consistent voice persona from their neural (and non HD) counterparts, and deliver even more value through enhanced features.

Key features of neural text to speech HD voices

The following are the key features of Azure AI Speech HD voices:

 Expand table

Key features	Description
Human-like speech generation	Neural text to speech HD voices can generate highly natural and human-like speech. The model is trained on millions of hours of multilingual data, enabling it to accurately interpret input text and generate speech with the appropriate emotion, pace, and rhythm without manual adjustments.
Conversational	Neural text to speech HD voices can replicate natural speech patterns, including spontaneous pauses and emphasis. When given conversational text, the model can reproduce common phonemes like pauses and filler words. The generated voice sounds as if someone is conversing directly with you.
Prosody variations	Neural text to speech HD voices introduce slight variations in each output to enhance realism. These variations make the speech sound more natural, as human voices naturally exhibit variation.
High fidelity	The primary objective of neural text to speech HD voices is to generate high-fidelity audio. The synthetic speech produced by our system can closely mimic human speech in both quality and naturalness.

Comparison of Azure AI Speech HD voices to other Azure text to speech voices

How do Azure AI Speech HD voices compare to other Azure text to speech voices? How do they differ in terms of features and capabilities?

Here's a comparison of features between Azure AI Speech HD voices, Azure OpenAI HD voices, and Azure AI Speech voices:

Feature	Azure AI Speech HD voices	Azure OpenAI HD voices	Azure AI Speech voices (not HD)
Region	See Speech service regions	See Speech service regions	Available in dozens of regions. See the Speech service regions .
Number of voices	30	6	More than 500
Multilingual	Yes	Yes	Yes (applicable only to multilingual voices)
SSML support	Support for a subset of SSML elements .	Support for a subset of SSML elements .	Support for the full set of SSML in Azure AI Speech.
Development options	Speech SDK, Speech CLI, REST API	Speech SDK, Speech CLI, REST API	Speech SDK, Speech CLI, REST API
Deployment options	Cloud only	Cloud only	Cloud, embedded, hybrid, and containers.
Real-time or batch synthesis	Real-time only	Real-time and batch synthesis	Real-time and batch synthesis
Latency	Less than 300 ms	Greater than 500 ms	Less than 300 ms
Sample rate of synthesized audio	8, 16, 24, and 48 kHz	8, 16, 24, and 48 kHz	8, 16, 24, and 48 kHz
Speech output audio format	opus, mp3, pcm, truesilk	opus, mp3, pcm, truesilk	opus, mp3, pcm, truesilk

Supported Azure AI Speech HD voices

The Azure AI Speech HD voice values are in the format `voicename:basemodel:version`. The name before the colon, such as `en-US-Ava`, is the voice persona name and its original locale. The base model is tracked by versions in subsequent updates.

Currently, `DragonHD` is the only base model available for Azure AI Speech HD voices. To ensure that you're using the latest version of the base model that we provide without having to make a code change, use the `LatestNeural` version.

For example, for the persona `en-US-Ava` you can specify the following HD voice values:

- `en-US-Ava:DragonHDLatestNeural`: Always uses the latest version of the base model that we provide later.

The following table lists the Azure AI Speech HD voices that are currently available.

 Expand table

Voice Name	Gender	Status	Note
de-DE-Florian:DragonHDLatestNeural	Male	GA	
de-DE-Seraphina:DragonHDLatestNeural	Female	GA	
en-US-Adam:DragonHDLatestNeural	Male	GA	
en-US-Alloy:DragonHDLatestNeural	Male	Preview	
en-US-Andrew:DragonHDLatestNeural	Male	GA	
en-US-Andrew2:DragonHDLatestNeural	Male	GA	Optimized for conversational content
en-US-Andrew3:DragonHDLatestNeural	Male	Preview	Optimized for podcast content
en-US-Aria:DragonHDLatestNeural	Female	Preview	
en-US-Ava:DragonHDLatestNeural	Female	GA	
en-US-Ava3:DragonHDLatestNeural	Female	Preview	Optimized for podcast content
en-US-Brian:DragonHDLatestNeural	Male	GA	
en-US-Davis:DragonHDLatestNeural	Male	GA	
en-US-Emma:DragonHDLatestNeural	Female	GA	
en-US-Emma2:DragonHDLatestNeural	Female	GA	Optimized for conversational content
en-US-Jenny:DragonHDLatestNeural	Female	Preview	
en-US-MultiTalker-Ava-Andrew:DragonHDLatestNeural	Male	Preview	
en-US-Nova:DragonHDLatestNeural	Female	Preview	
en-US-Phoebe:DragonHDLatestNeural	Female	Preview	
en-US-Serena:DragonHDLatestNeural	Female	Preview	
en-US-Steffan:DragonHDLatestNeural	Male	GA	
es-ES-Tristan:DragonHDLatestNeural	Male	GA	
es-ES-Ximena:DragonHDLatestNeural	Female	GA	

Voice Name	Gender	Status	Note
fr-FR-Remy:DragonHDLatestNeural	Male	GA	
fr-FR-Vivienne:DragonHDLatestNeural	Female	GA	
ja-JP-Masaru:DragonHDLatestNeural	Male	GA	
ja-JP-Nanami:DragonHDLatestNeural	Female	GA	
zh-CN-Xiaochen:DragonHDLatestNeural	Female	GA	
zh-CN-Yunfan:DragonHDLatestNeural	Male	GA	

How to use Azure AI Speech HD voices

You can use HD voices with the same Speech SDK and REST APIs as the non HD voices.

Here are some key points to consider when using Azure AI Speech HD voices:

- **Voice locale:** The locale in the voice name indicates its original language and region.
- **Base models:**
 - HD voices come with a base model that understands the input text and predicts the speaking pattern accordingly. You can specify the desired model (such as DragonHDLatestNeural) according to the availability of each voice.
- **SSML usage:** To reference a voice in SSML, use the format `voicename:basemodel:version`. The name before the colon, such as `de-DE-Seraphina`, is the voice persona name and its original locale. The base model is tracked by versions in subsequent updates.
- **Temperature parameter:**
 - The temperature value is a float ranging from 0 to 1, influencing the randomness of the output. You can also adjust the temperature parameter to control the variation of outputs. Less randomness yields more stable results, while more randomness offers variety but less consistency.
 - Lower temperature results in less randomness, leading to more predictable outputs. Higher temperature increases randomness, allowing for more diverse outputs. The default temperature is set at 1.0.

Here's an example of how to use Azure AI Speech HD voices in SSML:

ssml

```
<speak version='1.0' xmlns='http://www.w3.org/2001/10/synthesis'
xmlns:mstts='https://www.w3.org/2001/mstts' xml:lang='en-US'>
<voice name='en-US-Ava:DragonHDLatestNeural' parameters='temperature=0.8'>Here is a
```

```
test</voice>
</speak>
```

Supported and unsupported SSML elements for Azure AI Speech HD voices

The Speech Synthesis Markup Language (SSML) with input text determines the structure, content, and other characteristics of the text to speech output. For example, you can use SSML to define a paragraph, a sentence, a break or a pause, or silence. You can wrap text with event tags such as bookmark or viseme that your application processes later.

The Azure AI Speech HD voices don't support all SSML elements or events that other Azure AI Speech voices support. Of particular note, Azure AI Speech HD voices don't support [word boundary events](#).

For detailed information on the supported and unsupported SSML elements for Azure AI Speech HD voices, refer to the following table. For instructions on how to use SSML elements, refer to the [Speech Synthesis Markup Language \(SSML\) documentation](#).

 Expand table

SSML element	Description	Supported in Azure AI Speech HD voices
<voice>	Specifies the voice and optional effects (<code>eq_car</code> and <code>eq_telecomhp8k</code>).	Yes
<mstts:express-as>	Specifies speaking styles and roles.	No
<mstts:ttseembedding>	Specifies the <code>speakerProfileId</code> property for a personal voice.	No
<lang xml:lang>	Specifies the speaking language.	Yes
<prosody>	Adjusts pitch, contour, range, rate, and volume.	No
<emphasis>	Adds or removes word-level stress for the text.	No
<audio>	Embeds prerecorded audio into an SSML document.	No
<mstts:audioduration>	Specifies the duration of the output audio.	No
<mstts:backgroundaudio>	Adds background audio to your SSML documents or mixes an audio file with text to speech.	No

SSML element	Description	Supported in Azure AI Speech HD voices
<code><phoneme></code>	Specifies phonetic pronunciation in SSML documents.	No
<code><lexicon></code>	Defines how multiple entities are read in SSML.	Yes (only supports alias)
<code><say-as></code>	Indicates the content type, such as number or date, of the element's text.	Yes
<code><sub></code>	Indicates that the alias attribute's text value should be pronounced instead of the element's enclosed text.	Yes
<code><math></code>	Uses the MathML as input text to properly pronounce mathematical notations in the output audio.	No
<code><bookmark></code>	Gets the offset of each marker in the audio stream.	No
<code><break></code>	Overrides the default behavior of breaks or pauses between words.	No
<code><mstts:silence></code>	Inserts pause before or after text, or between two adjacent sentences.	No
<code><mstts:viseme></code>	Defines the position of the face and mouth while a person is speaking.	No
<code><p></code>	Denotes paragraphs in SSML documents.	Yes
<code><s></code>	Denotes sentences in SSML documents.	Yes

ⓘ Note

Although a [previous section in this guide](#) also compared Azure AI Speech HD voices to Azure OpenAI HD voices, the SSML elements supported by Azure AI Speech aren't applicable to Azure OpenAI voices.

HD Flash voices

HD voices are currently supported in `eastus`, `westeurope`, and `southeastasia` regions, to provide similar capabilities for customers in China regions (`chinaeast2`, `chinanorth2`, `chinanorth3`), we offer HDFlash versions of selected HD voices. These HDFlash voices deliver

enhanced naturalness compared to standard voices. You also can find those HDFlash voices in `eastus`, `westeurope`, and `southeastasia`.

Below is the complete list of available HD Flash voices:

 Expand table

Voice Name	Gender
zh-CN-Xiaochen:DragonHDFlashLatestNeural	Female
zh-CN-Xiaoxiao:DragonHDFlashLatestNeural	Female
zh-CN-Xiaoxiao2:DragonHDFlashLatestNeural	Female
zh-CN-Yunxia:DragonHDFlashLatestNeural	Male
zh-CN-Yunxiao:DragonHDFlashLatestNeural	Male
zh-CN-Yunye:DragonHDFlashLatestNeural	Male
zh-CN-Yunyi:DragonHDFlashLatestNeural	Male

 Note

HD Flash only support text in `zh-CN` and `en-US`.

Related content

- [Try the text to speech quickstart in Azure AI Speech](#)
- [Learn more about how to use SSML and events](#)

Last updated on 11/06/2025

What is voice conversion? (Preview)

07/10/2025

 Note

This feature is currently in public preview. This preview is provided without a service-level agreement, and is not recommended for production workloads. Certain features might not be supported or might have constrained capabilities. For more information, see [Supplemental Terms of Use for Microsoft Azure Previews](#).

Voice conversion is the process of transforming the voice characteristics of a given audio to a target voice speaker. After voice conversion, the resulting audio reserves source audio's linguistic content and prosody while the voice timbre sounds like the target speaker.

There are 3 reasons users need voice conversion functionality:

- Voice conversion can replicate your content using a different voice identity while maintaining the original prosody and emotion. For instance, in education, teachers can record themselves reading stories, and voice conversion can deliver these stories using a pre-designed cartoon character's voice. This method preserves the expressiveness of the teacher's reading while incorporating the unique timbre of the cartoon character's voice.
- Another application is multilingual dubbing. When localized content is read by different voices, voice conversion can transform them into a uniform voice, ensuring a consistent experience across all languages while keeping the most localized voice characters.
- Voice conversion enhances the control over the expressiveness of a voice. By transforming various speaking styles, such as adopting a unique tone or conveying exaggerated emotions, a voice gains greater versatility in expression and can be more dynamic in different scenarios.

Key capabilities

Voice conversion (or voice changer or speech to speech conversion) is built on state-of-the-art generative models and offers high-quality voice conversion. It delivers the following core capabilities:

 Expand table

Capability	Description
High speaker similarity	Captures the timbre and vocal identity of the target speaker.

Capability	Description
	Generates audio that accurately matches the target voice.
Prosody preservation	Maintains rhythm, stress, and intonation of source audio. Preserves expressive and emotional qualities.
High audio fidelity	Generates realistic, natural-sounding audio. Minimizes artifacts.
Multilingual support	Enables multilingual voice conversion. Supports 91 locales (same as standard text to speech locale support). See supported voices for voice conversion for the complete list.

Use voice conversion

You can use Azure AI Speech voice conversion with either the Speech SDK or text to speech REST APIs.

Use the `<mstts:voiceconversion>` tag via Speech Synthesis Markup Language (SSML) to specify the source audio URL and the target voice for the conversion. For a complete list of supported target voices, see [supported voices for voice conversion](#).

Example SSML

XML

```
< speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
  < voice xml:lang="en-US" xml:gender="Female" name="en-US-AvaMultilingualNeural">
    <mstts:voiceconversion
url="https://your.blob.core.windows.net/sourceaudio.wav"/>
  </voice>
</speak>
```

For details about the SSML structure and usage, see the [Speech Synthesis Markup Language \(SSML\) reference](#) documentation.

Related content

- [Text to speech overview](#)
- [Speech synthesis markup voice](#)

What is custom voice?

08/07/2025

Custom voice is a text to speech feature that lets you create a one-of-a-kind, customized, synthetic voice for your applications. With custom voice, you can build a highly natural-sounding voice for your brand or characters by providing human speech samples as fine-tuning data.

Important

Custom voice access is [limited](#) based on eligibility and usage criteria. Request access on the [intake form](#) .

Out of the box, [text to speech](#) can be used with standard voices for each [supported language](#). The standard voices work well in most text to speech scenarios if a unique voice isn't required.

Custom voice is based on the neural text to speech technology and the multilingual, multi-speaker, universal model. You can create synthetic voices that are rich in speaking styles, or adaptable cross languages. The realistic and natural sounding voice of custom voice can represent brands, personify machines, and allow users to interact with applications conversationally. See the [supported languages](#) for custom voice.

How does it work?

To create a custom voice, use [Speech Studio](#)  to upload the recorded audio and corresponding scripts, train the model, and deploy the voice to a custom endpoint.

Creating a great custom voice requires careful quality control in each step, from voice design and data preparation, to the deployment of the voice model to your system.

Before you get started in Speech Studio, here are some considerations:

- [Design a persona](#) of the voice that represents your brand by using a persona brief document. This document defines elements such as the features of the voice, and the character behind the voice. This helps to guide the process of creating a custom voice model, including defining the scripts, selecting your voice talent, training, and voice tuning.
- [Select the recording script](#) to represent the user scenarios for your voice. For example, you can use the phrases from bot conversations as your recording script if you're creating a customer service bot. Include different sentence types in your scripts, including statements, questions, and exclamations.

Here's an overview of the steps to create a custom voice in Speech Studio:

1. [Create a project](#) to contain your data, voice models, tests, and endpoints. Each project is specific to a country/region and language. If you're going to create multiple voices, it's recommended that you create a project for each voice.
2. [Set up voice talent](#). Before you can fine-tune a professional voice, you must submit a recording of the voice talent's consent statement. The voice talent statement is a recording of the voice talent reading a statement that they consent to the usage of their speech data for professional voice fine-tuning.
3. [Prepare fine-tuning data](#) in the right [format](#). It's a good idea to capture the audio recordings in a professional quality recording studio to achieve a high signal-to-noise ratio. The quality of the voice model depends heavily on your fine-tuning data. Consistent volume, speaking rate, pitch, and consistency in expressive mannerisms of speech are required.
4. [Train your voice model](#). Select at least 300 utterances to create a custom voice. A series of data quality checks are automatically performed when you upload them. To build high-quality voice models, you should fix any errors and submit again.
5. [Test your voice](#). Prepare test scripts for your voice model that cover the different use cases for your apps. It's a good idea to use scripts within and outside the training dataset, so you can test the quality more broadly for different content.
6. [Deploy and use your voice model](#) in your apps.

You can tune, adjust, and use your custom voice, similarly as you would use a standard voice. Convert text into speech in real-time, or generate audio content offline with text input. You use the [REST API](#), the [Speech SDK](#), or the [Speech Studio](#) [↗](#).

Tip

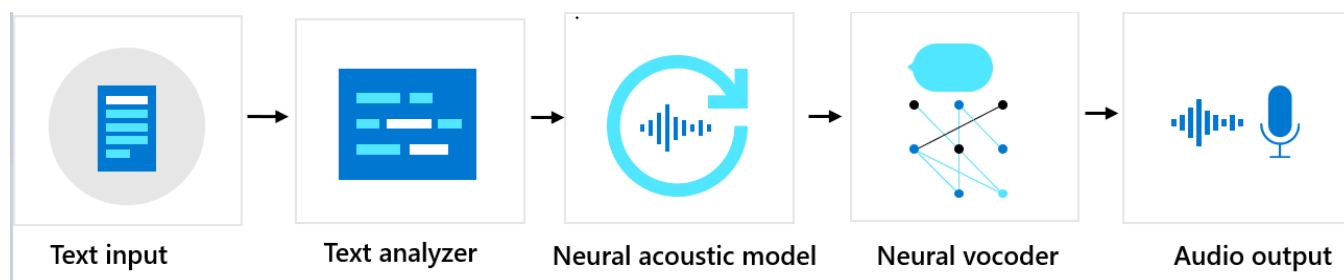
Check out the code samples in the [Speech SDK repository on GitHub](#) [↗](#) to see how to use custom voice in your application.

The style and the characteristics of the trained voice model depend on the style and the quality of the recordings from the voice talent used for training. However, you can make several adjustments by using [SSML \(Speech Synthesis Markup Language\)](#) when you make the API calls to your voice model to generate synthetic speech. SSML is the markup language used to communicate with the text to speech service to convert text into audio. The adjustments you can make include change of pitch, rate, intonation, and pronunciation correction. If the voice model is built with multiple styles, you can also use SSML to switch the styles.

Components sequence

Custom voice consists of three major components: the text analyzer, the neural acoustic model, and the neural vocoder. To generate natural synthetic speech from text, text is first input into the text analyzer, which provides output in the form of phoneme sequence. A *phoneme* is a basic unit of sound that distinguishes one word from another in a particular language. A sequence of phonemes defines the pronunciations of the words provided in the text.

Next, the phoneme sequence goes into the neural acoustic model to predict acoustic features that define speech signals. Acoustic features include the timbre, the speaking style, speed, intonations, and stress patterns. Finally, the neural vocoder converts the acoustic features into audible waves, so that synthetic speech is generated.



Neural text to speech voice models are trained by using deep neural networks based on the recording samples of human voices. For more information, see [this Microsoft blog post](#). To learn more about how a neural vocoder is trained, see [this Microsoft blog post](#).

Responsible AI

An AI system includes not only the technology, but also the people who use it, the people who are affected by it, and the environment in which it's deployed. Read the transparency notes to learn about responsible AI use and deployment in your systems.

- [Transparency note and use cases for custom voice](#)
- [Characteristics and limitations for using custom voice](#)
- [Limited access to custom voice](#)
- [Guidelines for responsible deployment of synthetic voice technology](#)
- [Disclosure for voice talent](#)
- [Disclosure design guidelines](#)
- [Disclosure design patterns](#)
- [Code of Conduct for Text to speech integrations](#)
- [Data, privacy, and security for custom voice](#)

Next steps

- [Create a project](#)
- [Prepare fine-tuning data](#)

- Train model

Professional voice fine-tuning data

08/07/2025

When you're ready to create a custom voice for your application, the first step is to gather audio recordings and associated scripts to start professional voice fine-tuning. "Custom voice" is an umbrella term that includes both professional voice fine-tuning and personal voice. The Speech service uses this data for professional voice fine-tuning, creating a unique voice tuned to match the voice in the recordings. After you fine-tune a professional voice, you can start synthesizing speech in your applications.



Tip

To create a voice for production use, we recommend you use a professional recording studio and voice talent. For more information, see [record voice samples for professional voice fine-tuning](#).

Types of data for professional voice fine-tuning

A dataset for professional voice fine-tuning includes audio recordings and a text file with the associated transcriptions. Each audio file should contain a single utterance (a single sentence or a single turn for a dialog system), and be less than 15 seconds long.

In some cases, you might not have the right dataset ready. You can test professional voice fine-tuning with available audio files, short or long, with or without transcripts.

This table lists data types and how each is used for professional voice fine-tuning.

Expand table

Data type	Description	When to use	Extra processing required	Processed as
Individual utterances + matching transcript	A collection (.zip) of audio files (.wav) as individual utterances. Each audio file should be 15 seconds or less in length, paired with a formatted transcript (.txt).	Professional recordings with matching transcripts	Ready for fine-tuning.	Segmented
Long audio + transcript	A collection (.zip) of long, unsegmented audio files (.wav or .mp3, longer than 20 seconds, at most 1,000 audio	You have audio files and matching transcripts, but they aren't	Segmentation (using batch transcription). Audio format	Segmented, Contextual

Data type	Description	When to use	Extra processing required	Processed as
	files), paired with a collection (.zip) of transcripts that contains all spoken words.	segmented into utterances.	transformation wherever required.	
Audio only (Preview)	A collection (.zip) of audio files (.wav or .mp3, at most 1,000 audio files) without a transcript.	You only have audio files available, without transcripts.	Segmentation + transcript generation (using batch transcription). Audio format transformation wherever required.	Segmented, Contextual

Files should be grouped by type into a dataset and uploaded as a zip file. Each dataset can only contain a single data type.

ⓘ Note

The maximum number of datasets allowed to be imported per subscription is 500 zip files for standard subscription (S0) users.

Processed as Contextual would retain the audio as a whole to keep the contextual information for more natural intonations.

Individual utterances + matching transcript

You can prepare recordings of individual utterances and the matching transcript in two ways. Either [write a script and have it read by a voice talent](#) or use publicly available audio and transcribe it to text. If you do the latter, edit disfluencies from the audio files, such as "um" and other filler sounds, stutters, mumbled words, or mispronunciations.

To produce a good voice model, create the recordings in a quiet room with a high-quality microphone. Consistent volume, speaking rate, speaking pitch, and expressive mannerisms of speech are essential.

For data format examples, refer to the sample dataset on [GitHub](#). The sample dataset includes the sample script and the associated audio.

Audio data for Individual utterances + matching transcript

Each audio file should contain a single utterance (a single sentence or a single turn of a dialog system), less than 15 seconds long. All files must be in the same spoken language. Multi-language custom Text to speech voices aren't supported, except for the Chinese-English bi-lingual. Each audio file must have a unique filename with the filename extension .wav.

Follow these guidelines when preparing audio.

 Expand table

Property	Value
File format	RIFF (.wav), grouped into a .zip file
File name	File name characters supported by Windows OS, with .wav extension. The characters <code>\ / : * ? " < > \ </code> aren't allowed. It can't start or end with a space, and can't start with a dot. No duplicate file names allowed.
Sampling rate	24 KHz and higher required when fine-tuning a professional voice.
Sample format	PCM, at least 16-bit
Audio length	Shorter than 15 seconds
Archive format	.zip
Maximum archive size	2048 MB

 Note

The default sampling rate for professional voice fine-tuning is 24 KHz. Audio files with a sampling rate lower than 16,000 Hz will be rejected. If a .zip file contains .wav files with different sample rates, only those equal to or higher than 16,000 Hz will be imported. Your audio files with a sampling rate higher than 16,000 Hz and lower than 24 KHz will be up-sampled to 24 KHz for fine-tuning. It's recommended that you use a sample rate of 24 KHz and higher for your fine-tuning data.

Transcription data for Individual utterances + matching transcript

The transcription file is a plain text file. Use these guidelines to prepare your transcriptions.

 Expand table

Property	Value
File format	Plain text (.txt)
Encoding format	ANSI, ASCII, UTF-8, UTF-8-BOM, UTF-16-LE, or UTF-16-BE. For zh-CN, ANSI and ASCII encoding aren't supported.
# of utterances per line	One - Each line of the transcription file should contain the name of one of the audio files, followed by the corresponding transcription. You must use a tab (\t) to separate the file name and transcription.
Maximum file size	2048 MB

Here's an example of how the transcripts are organized utterance by utterance in one .txt file:

```
0000000001[tab] This is the waistline, and it's falling.  
0000000002[tab] We have trouble scoring.  
0000000003[tab] It was Janet Maslin.
```

It's important that the transcripts are 100% accurate transcriptions of the corresponding audio. Errors in the transcripts introduce quality loss during the fine-tuning process.

Long audio + transcript (Preview)

ⓘ Note

For **Long audio + transcript (Preview)**, only these languages are supported: Chinese (Mandarin, Simplified), Chinese (Cantonese, Traditional), Chinese (Taiwanese Mandarin), English (India), English (United Kingdom), English (United States), French (France), German (Germany), Hindi (India), Italian (Italy), Japanese (Japan), Portuguese (Brazil), Spanish (Spain) and Spanish (Mexico).

Processed as Contextual is currently only available for Chinese (Mandarin, Simplified) and English (United States).

In some cases, you might not have segmented audio available. The Speech Studio can help you segment long audio files and create transcriptions. The long-audio segmentation service uses the [Batch Transcription API](#) feature of speech to text.

The service offers two processing modes:

- **Segmented:** The default processing mode that works with all supported languages
- **Contextual:** An enhanced mode that retains the audio as a whole to keep the contextual information for more natural intonations.

During the processing of the segmentation, your audio files and the transcripts are also sent to the custom speech service to refine the recognition model so the accuracy can be improved for your data. No data is retained during this process. After the segmentation is done, only the utterances segmented and their mapping transcripts will be stored for your downloading and fine-tuning.

Audio data for Long audio + transcript

Follow these guidelines when preparing audio for segmentation.

 Expand table

Property	Value
File format	RIFF (.wav) or .mp3, grouped into a .zip file
File name	File name characters supported by Windows OS, with .wav extension. The characters <code>\ / : * ? " < > \ </code> aren't allowed. It can't start or end with a space, and can't start with a dot. No duplicate file names allowed.
Sampling rate	24 KHz and higher required when fine-tuning a professional voice.
Sample format	RIFF(.wav): PCM, at least 16-bit. mp3: At least 256 Kbps bit rate.
Audio length	Longer than 30 seconds
Archive format	.zip
Maximum archive size	2048 MB, at most 1,000 audio files included

Note

The default sampling rate for professional voice fine-tuning is 24 KHz. Audio files with a sampling rate lower than 16,000 Hz will be rejected. Your audio files with a sampling rate higher than 16,000 Hz and lower than 24 KHz will be up-sampled to 24 KHz for fine-tuning. It's recommended that you should use a sample rate of 24 KHz and higher for your fine-tuning data.

Segmented utterances should ideally be between 5 and 15 seconds long. For optimal segmentation results, it is recommended to include natural pauses of 0.5 to 1 second every 5 to 15 seconds of speech, preferably at the end of phrases or sentences.

All audio files should be grouped into a zip file. It's OK to put .wav files and .mp3 files into the same zip file. For example, you can upload a 45-second audio file named 'kingstory.wav' and a 200-second long audio file named 'queenstory.mp3' in the same zip file. All .mp3 files will be transformed into the .wav format after processing.

Transcription data for Long audio + transcript

Transcripts must be prepared to the specifications listed in this table. Each audio file must be matched with a transcript.

 Expand table

Property	Value
File format	Plain text (.txt), grouped into a .zip
File name	Use the same name as the matching audio file
Encoding format	ANSI, ASCII, UTF-8, UTF-8-BOM, UTF-16-LE, or UTF-16-BE. For zh-CN, ANSI and ASCII encoding aren't supported.
# of utterances per line	No limit
Maximum file size	2048 MB

All transcripts files in this data type should be grouped into a zip file. For example, you might upload a 45-second audio file named 'kingstory.wav' and a 200-second long audio file named 'queenstory.mp3' in the same zip file. You need to upload another zip file containing the corresponding two transcripts--one named 'kingstory.txt' and the other one named 'queenstory.txt'. Within each plain text file, you provide the full correct transcription for the matching audio.

After your dataset is successfully uploaded, we'll help you segment the audio file into utterances based on the transcript provided. You can check the segmented utterances and the matching transcripts by downloading the dataset. Unique IDs are assigned to the segmented utterances automatically. It's important that you make sure the transcripts you provide are 100% accurate. Errors in the transcripts can reduce the accuracy during the audio segmentation and further introduce quality loss in the fine-tuning phase that comes later.

Audio only (Preview)

ⓘ Note

For **Audio only (Preview)**, only these languages are supported: Chinese (Mandarin, Simplified), Chinese (Cantonese, Traditional), Chinese (Taiwanese Mandarin), English (India), English (United Kingdom), English (United States), French (France), German (Germany), Hindi (India), Italian (Italy), Japanese (Japan), Portuguese (Brazil), Spanish (Spain) and Spanish (Mexico).

Processed as Contextual is currently only available for Chinese (Mandarin, Simplified) and English (United States).

If you don't have transcriptions for your audio recordings, use the **Audio only** option to upload your data. Our system can help you segment and transcribe your audio files.

The service offers two processing modes:

- **Segmented:** The default processing mode that works with all supported languages
- **Contextual:** An enhanced mode that retains the audio as a whole to keep the contextual information for more natural intonations.

Follow these guidelines when preparing audio.

 Expand table

Property	Value
File format	RIFF (.wav) or .mp3, grouped into a .zip file
File name	File name characters supported by Windows OS, with .wav extension. The characters <code>\ / : * ? " < > \ </code> aren't allowed. It can't start or end with a space, and can't start with a dot. No duplicate file names allowed.
Sampling rate	24 KHz and higher required when fine-tuning a professional voice.
Sample format	RIFF(.wav): PCM, at least 16-bit mp3: At least 256 Kbps bit rate.
Audio length	No limit
Archive format	.zip
Maximum archive size	2048 MB, at most 1,000 audio files included

ⓘ Note

The default sampling rate for professional voice fine-tuning is 24 KHz. Your audio files with a sampling rate higher than 16,000 Hz and lower than 24 KHz will be up-sampled to 24 KHz for fine-tuning. It's recommended that you should use a sample rate of 24 KHz and higher for your fine-tuning data.

Segmented utterances should ideally be between 5 and 15 seconds long. For optimal segmentation results, it is recommended to include natural pauses of 0.5 to 1 second every 5 to 15 seconds of speech, preferably at the end of phrases or sentences.

All audio files should be grouped into a zip file. Once your dataset is successfully uploaded, the Speech service helps you segment the audio file into utterances based on our speech batch transcription service. You can select either the Standard or Contextual processing mode, depending on your language and requirements. Unique IDs are assigned to the segmented utterances automatically. Matching transcripts are generated through speech recognition. All .mp3 files will be transformed into the .wav format after processing. You can check the segmented utterances and the matching transcripts by downloading the dataset.

Next steps

- [Train your voice model](#)
- [Deploy and use your voice model](#)
- [How to record voice samples](#)

Recording voice samples for custom voice

08/07/2025

This article provides you with best practices on preparing high-quality voice samples for professional voice fine-tuning. To understand how the data is processed and the minimum requirements for data acceptance, please refer to [upload your data](#).

Creating a high-quality professional voice from scratch isn't a casual undertaking. The central component of a custom voice is a large collection of audio samples of human speech. It's vital that these audio recordings be of high quality. Choose a voice talent who has experience making these kinds of recordings, and have them recorded by a recording engineer using professional equipment.

Before you can make these recordings, though, you need a script: the words are spoken by your voice talent to create the audio samples.

Many small but important details go into creating a professional voice recording. This guide is a roadmap for a process that will help you get good, consistent results.

Tips for preparing data for a high-quality voice

A highly natural custom voice depends on several factors, like the quality and size of your training data.

The quality of your training data is a primary factor. For example, in the same training set, consistent volume, speaking rate, speaking pitch, and speaking style are essential to create a high-quality custom voice. You should also avoid background noise in the recording and make sure the script and recording match. To ensure the quality of your data, you need to follow [script selection criteria](#) and [recording requirements](#).

Regarding the size of the training data, in most cases you can build a reasonable custom voice with 300 utterances. According to our tests, adding more training data in most languages doesn't necessarily improve naturalness of the voice itself (tested using the MOS score), however, with more training data that covers more word instances, you have higher possibility to reduce the ratio of dissatisfactory parts of speech for the voice, such as the glitches. To hear what dissatisfactory parts of speech sound like, refer to [the GitHub examples](#).

In some cases, you might want a voice persona with unique characteristics. For example, a cartoon persona needs a voice with a special speaking style, or a voice that is dynamic in intonation. For such cases, we recommend that you prepare at least 1000 (preferably 2000) utterances, and record them at a professional recording studio. To learn more about how to

improve the quality of your voice model, see [characteristics and limitations for using custom voice](#).

Voice recording roles

There are four basic roles in a custom voice recording project:

 Expand table

Role	Purpose
Voice talent	This person's voice forms the basis of the custom voice.
Recording engineer	Oversees the technical aspects of the recording and operates the recording equipment.
Director	Prepares the script and coaches the voice talent's performance.
Editor	Finalizes the audio files and prepares them for upload to the Speech service.

An individual can fill more than one role. This guide assumes that you are filling the director role and hiring both a voice talent and a recording engineer. If you want to make the recordings yourself, this article includes some information about the recording engineer role. The editor role isn't needed until after the recording session. In the meantime, the director or the recording engineer can fill this role.

Choose your voice talent

Actors with experience in voiceover, voice character work, announcing or news reading make good voice talent. Choose voice talent whose natural voice you like. It's possible to create unique "character" voices, but it's harder for most talent to perform them consistently, and the effort can cause voice strain. The single most important factor for choosing voice talent is consistency. Your recordings for the same voice style should all sound like they were made on the same day in the same room. You can approach this ideal through good recording practices and engineering.

Your voice talent must be able to speak with consistent rate, volume level, pitch, and tone with clear dictation. They also need to be able to control their pitch variation, emotional effect, and speech mannerisms. Recording voice samples can be more fatiguing than other kinds of voice work, so most voice talents can only record for two or three hours a day. Limit sessions to three or four days a week, with a day off in-between if possible.

Work with your voice talent to develop a persona that defines the overall sound and emotional tone of the custom voice. Define the speaking styles for your persona and ask your voice talent to read the script in a way that aligns with your desired styles. Ensure that the speaking style remains consistent throughout the recordings for a set of training data.

For example, a persona with a naturally upbeat personality would carry a note of optimism in their voice. However, this personality should be expressed consistently across all recordings for a set of training data. Listen to existing voices to get a sense of what you're aiming for.

Tip

Usually, you'll want to own the voice recordings you make. Your voice talent should be amenable to a work-for-hire contract for the project.

Create a script

The starting point of any custom voice recording session is the script, which contains the utterances to be spoken by your voice talent. The term "utterances" encompasses both full sentences and shorter phrases. Building a custom voice requires at least 300 recorded utterances as training data.

The utterances in your script can come from anywhere: fiction, non-fiction, transcripts of speeches, news reports, and anything else available in printed form. For a brief discussion of potential legal issues, see the "[Legalities](#)" section. You can also write your own text.

Your utterances don't need to come from the same source, the same kind of source, or have anything to do with each other. However, if you use set phrases (for example, "You have successfully logged in") in your speech application, make sure to include them in your script. It gives your custom voice a better chance of pronouncing those phrases well.

We recommend the recording scripts include both general sentences and domain-specific sentences. For example, if you plan to record 2,000 sentences, 1,000 of them could be general sentences, another 1,000 of them could be sentences from your target domain or the use case of your application.

We provide [sample scripts in the 'General', 'Chat' and 'Customer Service' domains for each language](#)  to help you prepare your recording scripts. You can use these Microsoft shared scripts for your recordings directly or use them as a reference to create your own.

Script selection criteria

Below are some general guidelines that you can follow to create a good corpus (recorded audio samples) for professional voice fine-tuning.

- For most use cases, sentences are recommended to be between 2 and 15 seconds long, containing 5 to 30 words for Latin-based languages or 4 to 80 words for non-Latin languages. Aim to balance your script to include a variety of sentence types and lengths. Ensure your script does not include any duplicate sentences.

If your use case requires a high emphasis on questions, exclamations, or a mix of particularly long and short sentences, it is recommended to include a good portion of sentences as questions or exclamations, along with very short phrases and longer phrases up to 20 seconds in length.

For how to balance the different sentence types, refer to the following table:

 Expand table

Sentence types	Coverage
Statement sentences	Statement sentences should be 70-80% of the script.
Short word/phrase	Short word/phrase scripts should be about 10% of total utterances, with 5 to 7 words per case. Short words or phrases should be separated by commas to help remind voice talent to pause briefly while reading.
Question sentences (Optional)	Question sentences should be about 10%-20% of your domain script, including 5%-10% of rising and 5%-10% of falling tones. These sentences are required if you want the generated voice to accurately convey questions.
Exclamation sentences (Optional)	Exclamation sentences should be about 10%-20% of your script. These sentences are required if you want the generated voice to accurately convey exclamations.

 Note

You can estimate the number of words in a sentence by assuming a speech rate in words per second based on your language.

Best practices include:

- Balanced coverage for Parts of Speech, like verbs, nouns, adjectives, and so on.
- Balanced coverage for pronunciations. Include all letters from A to Z so the Text to speech engine learns how to pronounce each letter in your style.
- Readable, understandable, common-sense scripts for the speaker to read.

- Avoid too many similar patterns for words/phrases, like "easy" and "easier".
- Include different formats of numbers: address, unit, phone, quantity, date, and so on, in all sentence types.
- Include spelling sentences if it's something your custom voice will read. For example, "The spelling of Apple is A P P L E".

ⓘ Note

For Contextual processing mode, which provides more natural intonations and better conversational capabilities:

- Use paragraph-level text rather than sentence-level text for recordings. This approach helps capture natural speech flow between sentences and preserves contextual information.
- Each recording should ideally be longer than 30 seconds (containing more than 60 words for Latin-based languages or 160 words for non-Latin languages).
- A contextual training set with more than 30 minutes of total audio or 300 utterances can be used for training a custom voice.

- Don't put multiple sentences into one line/one utterance. Separate each line by utterance.
- Make sure the sentence is clean. Generally, don't include too many nonstandard words like numbers or abbreviations as they're hard to read. Some applications might require the reading of many numbers or acronyms. In these cases, you can include these words, but normalize them in their spoken form.

Below are some best practices for example:

- For lines with abbreviations, instead of "BTW", write "by the way".
- For lines with digits, instead of "911", write "nine one one".
- For lines with acronyms, instead of "ABC", write "A B C".

With that, make sure your voice talent pronounces these words in an expected way. Keep your script and recordings matched during the training process.

- Your script should include many different words and sentences with different kinds of sentence lengths, structures, and moods.
- Check the script carefully for errors. If possible, have someone else check it too. When you run through the script with your voice talent, you might catch more mistakes.

Difference between voice talent script and training script

The training script can differ from the voice talent script, especially for scripts that contain digits, symbols, abbreviations, date, and time. Scripts prepared for the voice talent must follow native reading conventions, such as 50% and \$45. The scripts used for training must be normalized to match the audio recording, such as *fifty percent* and *forty-five dollars*.

 Note

We provide some example scripts for the voice talent on [GitHub](#)[↗]. To use the example scripts for training, you must normalize them according to the recordings of your voice talent before uploading the file.

The following table shows the difference between scripts for voice talent and the normalized script for training.

 Expand table

Category	Voice talent script example	Training script example (normalized)
Digits	123	one hundred and twenty-three
Symbols	50%	fifty percent
Abbreviation	ASAP	as soon as possible
Date and time	March 3rd at 5:00 PM	March third at five PM

Typical defects of a script

The script's poor quality can adversely affect the training results. To achieve high-quality training results, it's crucial to avoid defects.

Script defects generally fall into the following categories:

 Expand table

Category	Example
Meaningless content.	"Colorless green ideas sleep furiously."
Incomplete sentences.	- "This was my last eve" (no subject, no specific meaning) - "They're already funny (no quote mark in the end, it's not a complete sentence)
Typo in the sentences.	- Start with a lower case - No ending punctuation if needed

Category	Example
	- Misspelling - Lack of punctuation: no period in the end (except news title) - End with symbols, except comma, question, exclamation - Wrong format, such as: - 45\$ (should be \$45) - No space or excess space between word/punctuation
Duplication in similar format, one per each pattern is enough.	- "Now is 1pm in New York" - "Now is 2pm in New York" - "Now is 3pm in New York" - "Now is 1pm in Seattle" - "Now is 1pm in Washington D.C."
Uncommon foreign words: only commonly used foreign words are acceptable in the script.	In English one might use the French word "faux" in common speech, but a French expression such as "coincer la bulle" would be uncommon.
Emoji or any other uncommon symbols	

Script format

The script is for use during recording sessions, so you can set it up any way you find easy to work with. Create the text file that's required by Speech Studio separately.

A basic script format contains three columns:

- The number of the utterance, starting at 1. Numbering makes it easy for everyone in the studio to refer to a particular utterance ("let's try number 356 again"). You can use the Microsoft Word paragraph numbering feature to number the rows of the table automatically.
- A blank column where you write the take number or time code of each utterance to help you find it in the finished recording.
- The text of the utterance itself.



Time Code	Utterance
1	Four score and seven years ago our fathers brought forth on this continent a new nation, conceived in Liberty, and dedicated to the proposition that all men are created equal.
2	Now we are engaged in a great civil war, testing whether that nation, or any nation so conceived and so dedicated, can long endure.
3	We are met on a great battlefield of that war.
4	We have come to dedicate a portion of that field as a final resting place for those who here gave their lives that that nation might live.

ⓘ Note

Most studios record in short segments known as "takes". Each take typically contains 10 to 24 utterances. Just noting the take number is sufficient to find an utterance later. If you're recording in a studio that prefers to make longer recordings, you'll want to note the time code instead. The studio will have a prominent time display.

Leave enough space after each row to write notes. Be sure that no utterance is split between pages. Number the pages, and print your script on one side of the paper.

Print three copies of the script: one for the voice talent, one for the recording engineer, and one for the director (you). Use a paper clip instead of staples: an experienced voice artist separates the pages to avoid making noise as the pages are turned.

Voice talent statement

To train a neural voice, you must [create a voice talent profile](#) with an audio file recorded by the voice talent consenting to the usage of their speech data to fine-tune a professional voice model. When preparing your recording script, make sure you include the statement sentence.

Legalities

Under copyright law, an actor's reading of copyrighted text might be a performance for which the author of the work should be compensated. This performance won't be recognizable in the final product, the custom voice. Even so, the legality of using a copyrighted work for this

purpose isn't well established. Microsoft can't provide legal advice on this issue; consult your own legal counsel.

Fortunately, it's possible to avoid these issues entirely. There are many sources of text you can use without permission or license.

 Expand table

Text source	Description
CMU Arctic corpus 	About 1100 sentences selected from out-of-copyright works specifically for use in speech synthesis projects. An excellent starting point.
Works no longer under copyright	Typically works published prior to 1923. For English, Project Gutenberg  offers tens of thousands of such works. You might want to focus on newer works, as the language is closer to modern English.
Government works	Works created by the United States government aren't copyrighted in the United States, though the government can claim copyright in other countries/regions.
Public domain	Works for which copyright is explicitly disclaimed or dedicated to the public domain. It might not be possible to waive copyright entirely in some jurisdictions.
Permissively licensed works	Works distributed under a license like Creative Commons or the GNU Free Documentation License (GFDL). Wikipedia uses the GFDL. Some licenses, however, may impose restrictions on performance of the licensed content that might affect the creation of a custom voice model, so read the license carefully.

Recording your script

Record your script at a professional recording studio that specializes in voice work. They have a recording booth, the right equipment, and the right people to operate it. It's recommended not to skimp on recording.

Discuss your project with the studio's recording engineer and listen to their advice. The recording should have little or no dynamic range compression (maximum of 4:1). It's critical that the audio has consistent volume and a high signal-to-noise ratio, while being free of unwanted sounds.

Recording requirements

To achieve high-quality training results, follow the following requirements during recording or data preparation:

- Clear and well pronounced

- Natural speed: not too slow or too fast between audio files.
- Appropriate volume, prosody and break: stable within the same sentence or between sentences, correct break for punctuation.
- No noise during recording
- Fit your persona design
- No wrong accent: fit to the target design
- No wrong pronunciation

You can refer to below specification to prepare for the audio samples as best practice.

 Expand table

Property	Value
File format	*.wav, Mono
Sampling rate	24 KHz
Sample format	16 bit, PCM
Peak volume levels	-3 dB to -6 dB
SNR	> 35 dB
Silence	- There should have some silence (recommend 100 ms) at the beginning and ending, but no longer than 200 ms - Silence between words or phrases < -30 dB - Silence in the wave after last word is spoken < -60 dB
Environment noise or echo	- The level of noise at start of the wave before speaking < -70 dB

Note

You can record at higher sampling rate and bit depth, for example in the format of 48 KHz 24 bit PCM. During the professional voice fine-tuning, we'll down sample it to 24 KHz 16 bit PCM automatically.

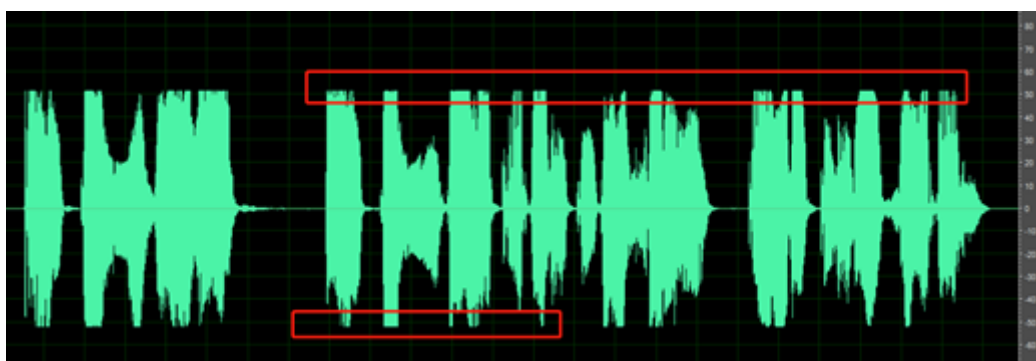
A higher signal-to-noise ratio (SNR) indicates lower noise in your audio. You can typically reach a 35+ SNR by recording at professional studios. Audio with an SNR below 20 can result in obvious noise in your generated voice.

Consider re-recording any utterances with low pronunciation scores or poor signal-to-noise ratios. If you can't re-record, consider excluding those utterances from your data.

Typical audio errors

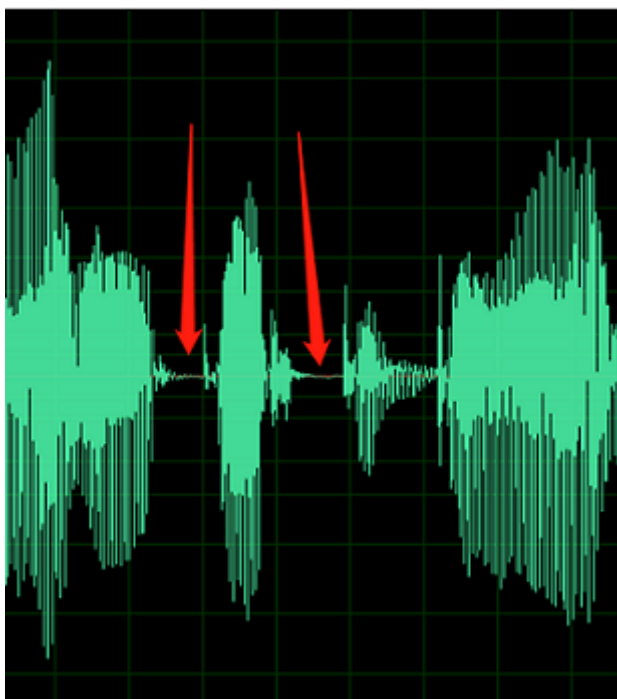
For high-quality training results, avoiding audio errors is highly recommended. Audio errors are usually within following categories:

- Audio file name doesn't match the script ID.
- WAR file has an invalid format and can't be read.
- Audio sampling rate is lower than 16 KHz. It's recommended that the .wav file sampling rate be equal or higher than 24 KHz for high-quality neural voice.
- Volume peak isn't within the range of -3 dB (70% of max volume) to -6 dB (50%).
- Waveform overflow: the waveform is cut at its peak value and is thus not complete.

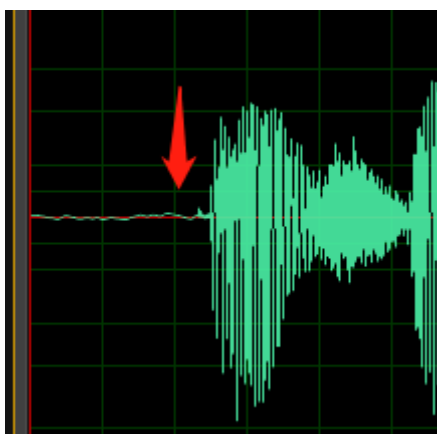


- The silent parts of the recording aren't clean; you can hear sounds such as ambient noise, mouth noise and echo.

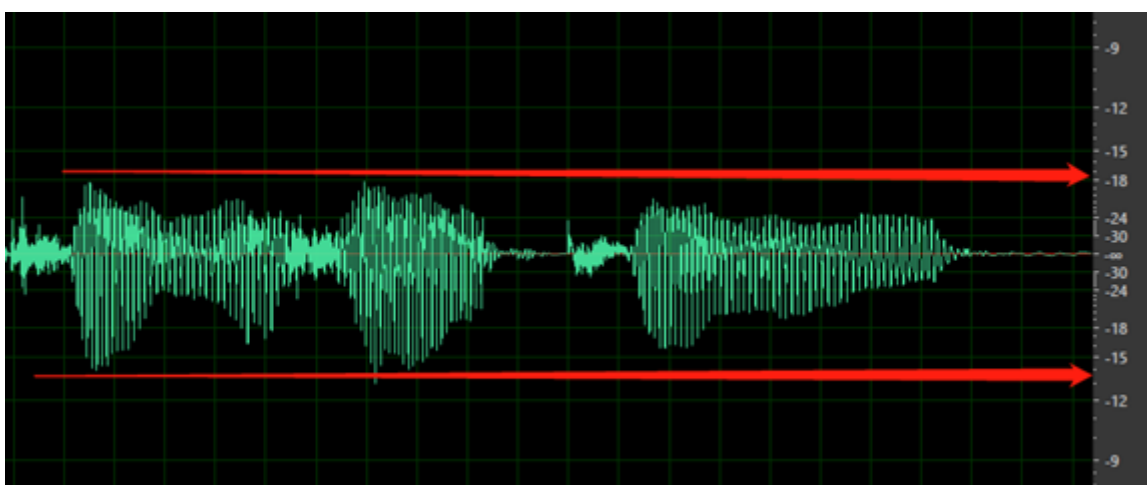
For example, below audio contains the environment noise between speeches.



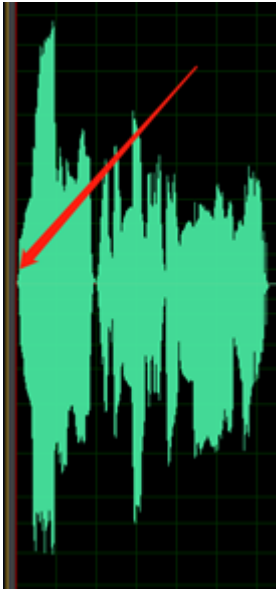
Below sample contains signs of DC offset or echo.



- The overall volume is too low. Your data is tagged as an issue if the volume is lower than -18 dB (10% of max volume). Make sure all audio files should be consistent at the same level of volume.



- No silence before the first word or after the last word. Also, the start or end silence shouldn't be longer than 200 ms or shorter than 100 ms.



Do it yourself

If you want to make the recording yourself, instead of going into a recording studio, here's a short primer. Thanks to the rise of home recording and podcasting, it's easier than ever to find good recording advice and resources online.

Your "recording booth" should be a small room with no noticeable echo or "room tone." It should be as quiet and soundproof as possible. Drapes on the walls can be used to reduce echo and neutralize or "deaden" the sound of the room.

Use a high-quality studio condenser microphone ("mic" for short) intended for recording voice. Sennheiser, AKG, and even newer Zoom mics can yield good results. You can buy a mic, or rent one from a local audio-visual rental firm. Look for one with a USB interface. This type of mic conveniently combines the microphone element, preamp, and analog-to-digital converter into one package, simplifying hookup.

You can also use an analog microphone. Many rental houses offer "vintage" microphones known for their voice character. Professional analog gear uses balanced XLR connectors, rather than the 1/4-inch plug that's used in consumer equipment. If you go analog, you'll also need a preamp and a computer audio interface with these connectors.

Install the microphone on a stand or boom, and install a pop filter in front of the microphone to eliminate noise from "plosive" consonants like "p" and "b." Some microphones come with a suspension mount that isolates them from vibrations in the stand, which is helpful.

The voice talent must stay at a consistent distance from the microphone. Use tape on the floor to mark where they should stand. If the talent prefers to sit, take special care to monitor mic

distance and avoid chair noise.

Use a stand to hold the script. Avoid angling the stand so that it can reflect sound toward the microphone.

The person operating the recording equipment — the recording engineer — should be in a separate room from the talent, with some way to talk to the talent in the recording booth (a *talkback circuit*).

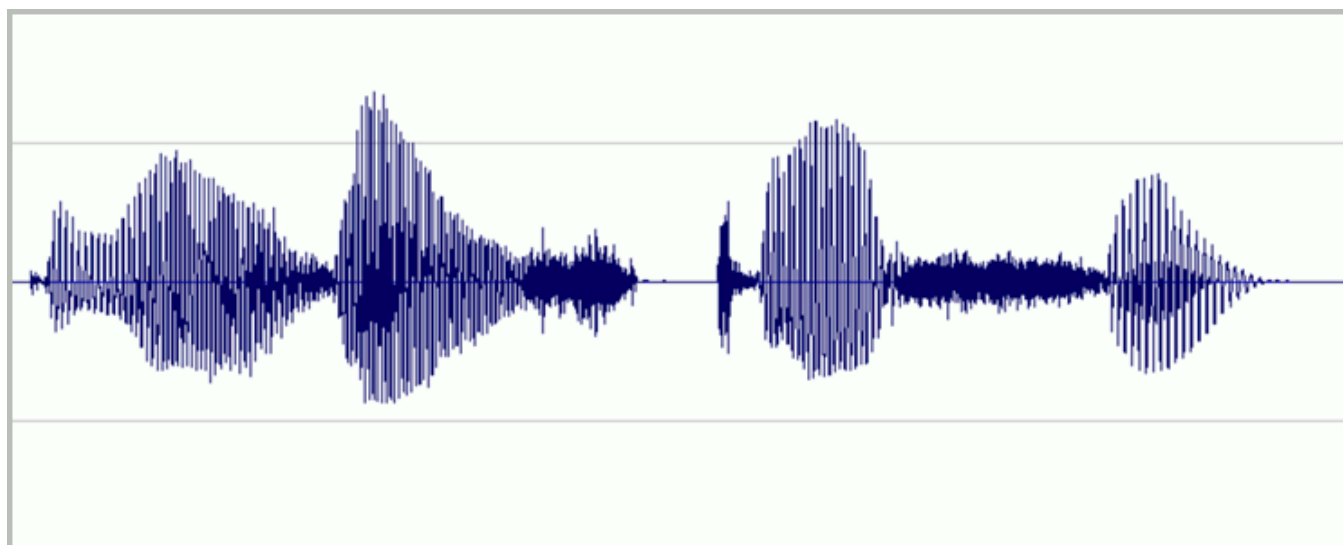
The recording should contain as little noise as possible, with a goal of -80 dB.

Listen closely to a recording of silence in your "booth," figure out where any noise is coming from, and eliminate the cause. Common sources of noise are air vents, fluorescent light ballasts, traffic on nearby roads, and equipment fans (even notebook PCs might have fans). Microphones and cables can pick up electrical noise from nearby AC wiring, usually a hum or buzz. A buzz can also be caused by a *ground loop*, which is caused by having equipment plugged into more than one electrical circuit.

💡 Tip

In some cases, you might be able to use an equalizer or a noise reduction software plug-in to help remove noise from your recordings, although it is always best to stop it at its source.

Set levels so that most of the available dynamic range of digital recording is used without overdriving. That means set the audio loud, but not so loud that it becomes distorted. An example of the waveform of a good recording is shown in the following image:



Here, most of the range (height) is used, but the highest peaks of the signal don't reach the top or bottom of the window. You can also see that the silence in the recording approximates a

thin horizontal line, indicating a low noise floor. This recording has acceptable dynamic range and signal-to-noise ratio.

Record directly into the computer via a high-quality audio interface or a USB port, depending on the mic you're using. For analog, keep the audio chain simple: mic, preamp, audio interface, computer. You can license both [Avid Pro Tools](#) and [Adobe Audition](#) monthly at a reasonable cost. If your budget is extremely tight, try the free [Audacity](#).

Record at 44.1 KHz 16 bit monophonic (CD quality) or better. Current state-of-the-art is 48 KHz 24 bit, if your equipment supports it. You'll down-sample your audio to 24 KHz 16-bit before you submit it to Speech Studio. Still, it pays to have a high-quality original recording in the event that edits are needed.

Ideally, have different people serve in the roles of director, engineer, and talent. Don't try to do it all yourself. In a pinch, one person can be both the director and the engineer.

Before the session

To avoid wasting studio time, run through the script with your voice talent before the recording session. While the voice talent becomes familiar with the text, they can clarify the pronunciation of any unfamiliar words.

 Note

Most recording studios offer electronic display of scripts in the recording booth. In this case, type your run-through notes directly into the script's document. You'll still want a paper copy to take notes on during the session, though. Most engineers will want a hard copy, too. And you'll still want a third printed copy as a backup for the talent in case the computer is down.

Your voice talent might ask which word you want emphasized in an utterance (the "operative word"). Tell them that you want a natural reading with no particular emphasis. Emphasis can be added when speech is synthesized; it shouldn't be a part of the original recording.

Direct the talent to pronounce words distinctly. Every word of the script should be pronounced as written. Sounds shouldn't be omitted or slurred together, as is common in casual speech, *unless they have been written that way in the script*.

Written text	Unwanted casual pronunciation
never going to give you up	never gonna give you up
there are four lights	there're four lights
how's the weather today	how's th' weather today
say hello to my little friend	say hello to my lil' friend

The talent shouldn't add distinct pauses between words. The sentence should still flow naturally, even while sounding a little formal. This fine distinction might take practice to get right.

The recording session

Create a reference recording, or *match file*, of a typical utterance at the beginning of the session. Ask the talent to repeat this line every page or so. Each time, compare the new recording to the reference. This practice helps the talent remain consistent in volume, tempo, pitch, and intonation. Meanwhile, the engineer can use the match file as a reference for levels and overall consistency of sound.

The match file is especially important when you resume recording after a break or on another day. Play it a few times for the talent and have them repeat it each time until they're matching well.

To record a corpus with a specific style, carefully choose scripts that showcase the desired style. During recording, ensure the voice talent maintains consistent in volume, tempo, pitch, and tone to achieve recordings that embody the intended style.

Coach your talent to take a deep breath and pause for a moment before each utterance. Record a couple of seconds of silence between utterances. Words should be pronounced the same way each time they appear, considering context. For example, "record" as a verb is pronounced differently from "record" as a noun.

Record approximately five seconds of silence before the first recording to capture the "room tone". This practice helps Speech Studio compensate for noise in the recordings.

Tip

All you need to capture is the voice talent, so you can make a monophonic (single-channel) recording of just their lines. However, if you record in stereo, you can use the second channel to record the chatter in the control room to capture discussion of

particular lines or takes. Remove this track from the version that's uploaded to Speech Studio.

Listen closely, using headphones, to the voice talent's performance. You're looking for good but natural diction, correct pronunciation, and a lack of unwanted sounds. Don't hesitate to ask your talent to re-record an utterance that doesn't meet these standards.

Tip

If you are using a large number of utterances, a single utterance might not have a noticeable effect on the resultant custom voice. It might be more expedient to simply note any utterances with issues, exclude them from your dataset, and see how your custom voice turns out. You can always go back to the studio and record the missed samples later.

Note the take number or time code on your script for each utterance. Ask the engineer to mark each utterance in the recording's metadata or cue sheet as well.

Take regular breaks and provide a beverage to help your voice talent keep their voice in good shape.

After the session

Modern recording studios run on computers. At the end of the session, you receive one or more audio files, not a tape. These files are probably WAV or AIFF format in CD quality (44.1 KHz 16-bit) or better. 24 KHz 16-bit is common and desirable. The default sampling rate for a custom voice is 24 KHz. It's recommended that you should use a sample rate of 24 KHz and higher for your training data. Higher sampling rates, such as 96 KHz, aren't usually needed.

Speech Studio requires each provided utterance to be in its own file. Each audio file delivered by the studio contains multiple utterances. So the primary post-production task is to split up the recordings and prepare them for submission. The recording engineer might have placed markers in the file (or provided a separate cue sheet) to indicate where each utterance starts.

Use your notes to find the exact takes you want, and then use a sound editing utility, such as [Avid Pro Tools](#), [Adobe Audition](#), or the free [Audacity](#), to copy each utterance into a new file.

Listen to each file carefully. At this stage, you can edit out small unwanted sounds that you missed during recording, like a slight lip smack before a line, but be careful not to remove any actual speech. If you can't fix a file, remove it from your dataset and note that you've done so.

Convert each file to 16 bits and a sample rate of 24 KHz and higher before saving and if you recorded the studio chatter, remove the second channel. Save each file in WAV format, naming the files with the utterance number from your script.

Finally, create the transcript that associates each WAV file with a text version of the corresponding utterance. [Train your voice model](#) includes details of the required format. You can copy the text directly from your script. Then create a Zip file of the WAV files and the text transcript.

Archive the original recordings in a safe place in case you need them later. Preserve your script and notes, too.

Next steps

You're ready to upload your recordings and create your custom voice.

[Train your voice model](#)

Create a project for professional voice

05/20/2025

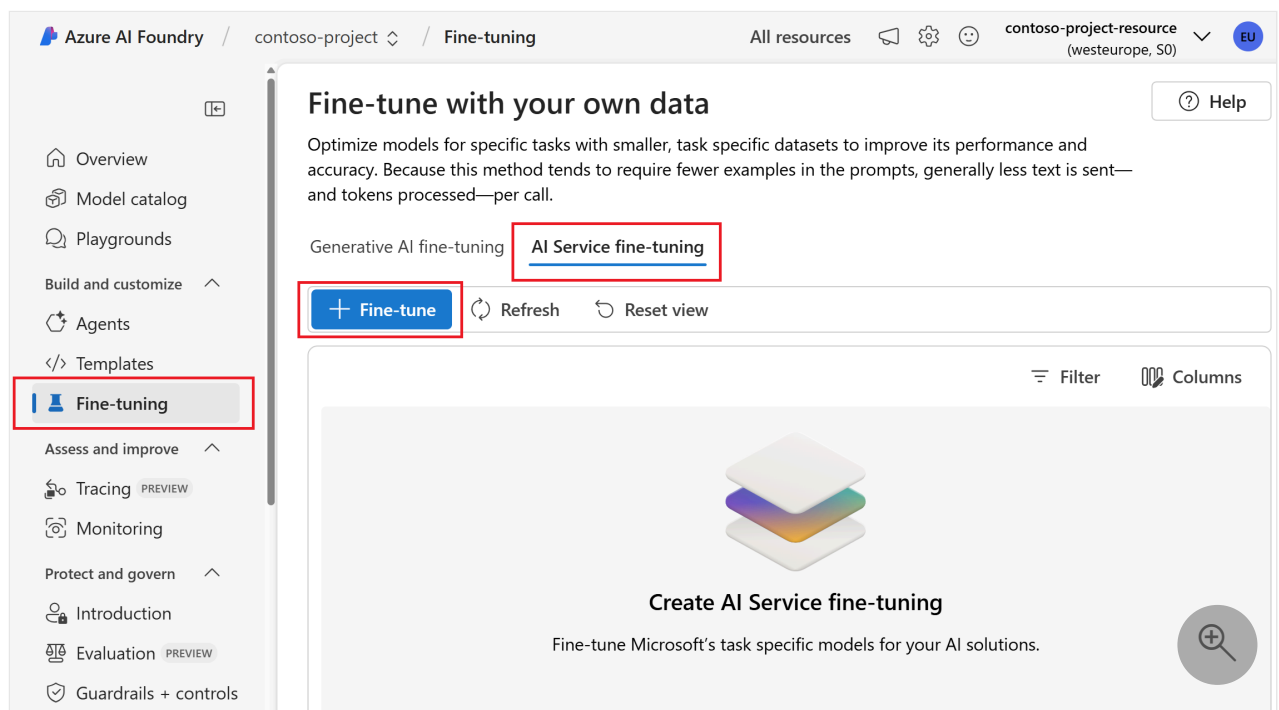
All it takes to get started are a handful of audio files and the associated transcriptions. See if custom voice supports your [language](#) and [region](#).

Start fine-tuning

In the [Azure AI Foundry portal](#), you can fine-tune some Azure AI services models. For example, you can fine-tune a professional voice model.

To fine-tune a professional voice model, follow these steps:

1. Go to your Azure AI Foundry project in the [Azure AI Foundry portal](#). If you need to create a project, see [Create an Azure AI Foundry project](#).
2. Select **Fine-tuning** from the left pane.
3. Select **AI Service fine-tuning** > **+ Fine-tune**.



4. In the wizard, select **Custom voice (professional voice fine-tuning)**.
5. Select **Next**.
6. Follow the instructions provided by the wizard to create your fine-tuning workspace.

Continue fine-tuning

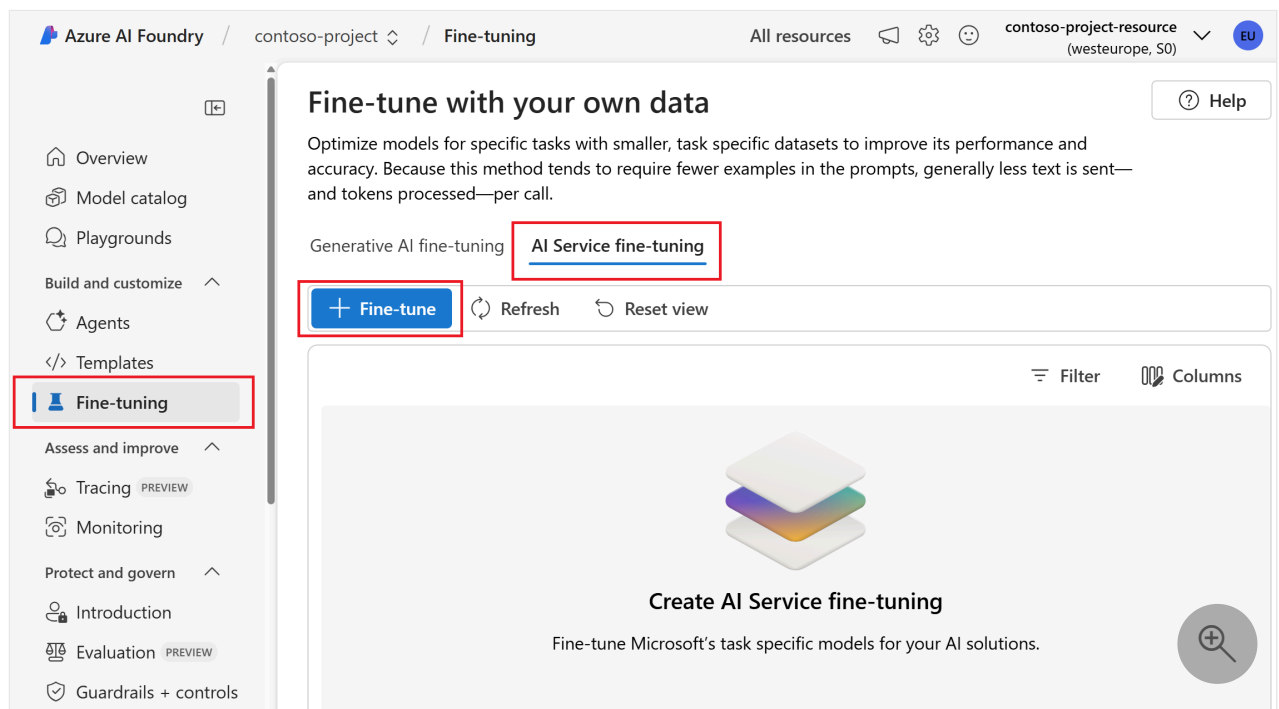
Go to the Azure AI Speech documentation to learn how to continue fine-tuning your professional voice model:

- [Add voice talent consent](#)
- [Add training datasets](#)
- [Train your voice model](#)
- [Deploy your professional voice model as an endpoint](#)

View fine-tuned models

After fine-tuning, you can access your custom voice models and deployments from the **Fine-tuning** page.

1. Sign in to the [Azure AI Foundry portal](#).
2. Select **Fine-tuning** from the left pane.
3. Select **AI Service fine-tuning**. You can view the status of your fine-tuning tasks and the models that were created.



Next steps

Add voice talent consent to the professional voice project.

Add voice talent consent to the professional voice project

05/20/2025

A voice talent is an individual or target speaker whose voices are recorded and used to create neural voice models.

Before you can fine-tune a professional voice, you must submit a recording of the voice talent's consent statement. The voice talent statement is a recording of the voice talent reading a statement that they consent to the usage of their speech data for professional voice fine-tuning. The consent statement is also used to verify that the voice talent is the same person as the speaker in the fine-tuning data.

Tip

Before you get started in Azure AI Foundry portal, define your voice [persona and choose the right voice talent](#).

You can find the verbal consent statement in multiple languages on [GitHub](#). The language of the verbal statement must be the same as your recording. See also the [disclosure for voice talent](#).

Add voice talent

Tip

For a sample consent statement and training data, see the [GitHub repository](#).

To add a voice talent profile and upload their consent statement, follow these steps:

1. Sign in to the [Azure AI Foundry portal](#).
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the professional voice fine-tuning task (by model name) that you [started as described in the create professional voice article](#).
4. Select **Set up voice talent** > **+ Add voice talent**.

5. In the **Add new voice talent** wizard, select the target scenarios for the voice talent. The target scenarios must be consistent with what you provided in the application form. The scenarios are used to help identify the voice talent and to ensure that the voice model is trained for the intended use cases.
6. Optionally in the **Voice characteristics** text box, enter a description of the characteristics of the voice you're going to create.
7. Select **Next**.
8. On the **Upload verbal statement** page, follow the instructions to upload the voice talent statement you recorded beforehand.
 - Enter the voice talent name and company name. The voice talent name must be the name of the person who recorded the consent statement. Enter the name in the same language used in the recorded statement. The company name must match the company name that was spoken in the recorded statement. Ensure the company name is entered in the same language as the recorded statement.
 - Make sure the verbal statement was [recorded](#) with the same settings, environment, and speaking style as your fine-tuning data.

The screenshot shows the 'Add new voice talent' wizard with three steps: 'Voice characteristics' (completed), 'Statement upload' (current), and 'Review and add'. The 'Statement upload' section contains instructions to upload an audio recording. Below the instructions are two text input fields: 'Voice talent name *' with the value 'Sample voice actor' and 'Company name *' with the value 'Contoso'. These fields are highlighted with a red rectangle. Below them is a text area for the verbal statement with the text: "I **Sample voice actor** am aware that recordings of my voice will be used by **Contoso** to create and use a synthetic version of my voice." Another red rectangle highlights the 'Verbal statement file *' label and its corresponding empty file upload area. At the bottom, there are 'Back' and 'Next' buttons, and a floating action button with a plus icon. On the right side, there are 'Add voice talent' and 'Cancel' buttons.

9. Select **Next**.
10. Review the voice talent and persona details, and select **Add voice talent**.

After the voice talent status is *Succeeded*, you can [add fine-tuning data](#).

Next steps

Add training data for professional voice fine-tuning

Add a professional voice training dataset

05/20/2025

When you're ready to create a custom text to speech voice for your application, the first step is to gather audio recordings and associated scripts to start training the voice model. For details on recording voice samples, see [the tutorial](#). The Speech service uses this data to create a unique voice tuned to match the voice in the recordings. After you've trained the voice, you can start synthesizing speech in your applications.

All data you upload must meet the requirements for the data type that you choose. It's important to correctly format your data before it's uploaded, which ensures the data will be accurately processed by the Speech service. To confirm that your data is correctly formatted, see [Training data types](#).

ⓘ Note

- Standard subscription (S0) users can upload five data files simultaneously. If you reach the limit, wait until at least one of your data files finishes importing. Then try again.
- The maximum number of data files allowed to be imported per subscription is 500 .zip files for standard subscription (S0) users. Please see out [Speech service quotas and limits](#) for more details.

Upload your data

💡 Tip

For a sample consent statement and training data, see the [GitHub repository](#).[↗]

When you're ready to upload your data, go to the **Prepare training data** tab to add your first training set and upload data. A *training set* is a set of audio utterances and their mapping scripts used for training a voice model. You can use a training set to organize your training data. The service checks data readiness per each training set. You can import multiple data to a training set.

To upload training data, follow these steps:

1. Sign in to the [Azure AI Foundry portal](#).[↗]

2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the professional voice fine-tuning task (by model name) that you [started as described in the create professional voice article](#).
4. Select **Prepare training data > Upload data**.
5. In the **Upload data** wizard, choose a [data type](#). If you're using the sample data, select **Individual utterances + matching transcript**.

Upload data [Close]

Data type (selected)

Training set selection

Data upload

Name and description

Review and upload

Choose a data type

Individual utterances + matching transcript
A collection (.zip) of audio files (.wav) as individual utterances, recommended to be 15 seconds or shorter, paired with a formatted transcript (.txt) denoting the audio file names.

Long audio + transcript [Preview]
A collection (.zip) of long, unsegmented audio files (.wav or .mp3), longer than 20 seconds, paired with a collection (.zip) of transcripts containing all of the spoken word.

Audio only [Preview]
A collection (.zip) of audio files (.wav or .mp3) without a transcript. A transcription for each spoken utterance will be automatically generated.

[Learn more about training data type](#)

Next [Upload data] [Cancel]

6. Select **Next**.
7. On the **Specify the target training set** page, select **Create new**.
8. Enter a training set name and then select **Create**.

9. Select **Next**.
10. On the **Data upload** page, select a **Recording file** and **Script file** in the respective tiles.
You can select local files from your computer or enter the Azure Blob storage URL to upload data.
11. Select **Next**.
12. Enter a name and description for your data and then select **Next**.
13. Review the upload details, and select **Upload data**.

ⓘ Note

Duplicate IDs aren't accepted. Utterances with the same ID will be removed.

Duplicate audio names are removed from the training. Make sure the data you select don't contain the same audio names within the .zip file or across multiple .zip files. If utterance IDs (either in audio or script files) are duplicates, they're rejected.

Data files are automatically validated when you select **Upload data**. Data validation includes series of checks on the audio files to verify their file format, size, and sampling rate. If there are any errors, fix them and submit again.

After you upload the data, you can check the details in the training set detail view. On the detail page, you can further check the pronunciation issue and the noise level for each of your data. The pronunciation score at the sentence level ranges from 0-100. A score below 70

normally indicates a speech error or script mismatch. Utterances with an overall score lower than 70 will be rejected. A heavy accent can reduce your pronunciation score and affect the generated digital voice.

Resolve data issues online

After upload, you can check the data details of the training set. Before continuing to [train your voice model](#), you should try to resolve any data issues.

Typical data issues

The issues are divided into three types. Refer to the following tables to check the respective types of errors.

Auto-rejected

Data with these errors won't be used for training. Imported data with errors will be ignored, so you don't need to delete them. You can [fix these data errors online](#) or upload the corrected data again for training.

 Expand table

Category	Name	Description
Script	Invalid separator	You must separate the utterance ID and the script content with a Tab character.
Script	Invalid script ID	The script line ID must be numeric.
Script	Duplicated script	Each line of the script content must be unique. The line is duplicated with {}.
Script	Script too long	The script must be less than 1,000 characters.
Script	No matching audio	The ID of each utterance (each line of the script file) must match the audio ID.
Script	No valid script	No valid script is found in this dataset. Fix the script lines that appear in the detailed issue list.
Audio	No matching script	No audio files match the script ID. The name of the .wav files must match with the IDs in the script file.
Audio	Invalid audio format	The audio format of the .wav files is invalid. Check the .wav file format by using an audio tool like SoX .
Audio	Low sampling rate	The sampling rate of the .wav files can't be lower than 16 KHz.

Category	Name	Description
Audio	Too long audio	Audio duration is longer than 30 seconds. Split the long audio into multiple files. It's a good idea to make utterances shorter than 15 seconds.
Audio	No valid audio	No valid audio is found in this dataset. Check your audio data and upload again.
Mismatch	Low scored utterance	Sentence-level pronunciation score is lower than 70. Review the script and the audio content to make sure they match.

Auto-fixed

The following errors are fixed automatically, but you should review and confirm the fixes are made correctly.

[Expand table](#)

Category	Name	Description
Mismatch	Silence auto fixed	The start silence is detected to be shorter than 100 ms, and has been extended to 100 ms automatically. Download the normalized dataset and review it.
Mismatch	Silence auto fixed	The end silence is detected to be shorter than 100 ms, and has been extended to 100 ms automatically. Download the normalized dataset and review it.
Script	Text auto normalized	Text is automatically normalized for digits, symbols, and abbreviations. Review the script and audio to make sure they match.

Manual check required

Unresolved errors listed in the next table affect the quality of training, but data with these errors won't be excluded during training. For higher-quality training, it's a good idea to fix these errors manually.

[Expand table](#)

Category	Name	Description
Script	Non-normalized text	This script contains symbols. Normalize the symbols to match the audio. For example, normalize / to <i>slash</i> .
Script	Not enough question utterances	At least 10 percent of the total utterances should be question sentences. This helps the voice model properly express a questioning tone.

Category	Name	Description
Script	Not enough exclamation utterances	At least 10 percent of the total utterances should be exclamation sentences. This helps the voice model properly express an excited tone.
Script	No valid end punctuation	Add one of the following at the end of the line: full stop (half-width '.' or full-width '。'), exclamation point (half-width '!' or full-width '！'), or question mark (half-width '?' or full-width '？').
Audio	Low sampling rate for neural voice	It's recommended that the sampling rate of your .wav files should be 24 KHz or higher for creating neural voices. If it's lower, it will be automatically raised to 24 KHz.
Volume	Overall volume too low	Volume shouldn't be lower than -18 dB (10 percent of max volume). Control the volume average level within proper range during the sample recording or data preparation.
Volume	Volume overflow	Overflowing volume is detected at {}s. Adjust the recording equipment to avoid the volume overflow at its peak value.
Volume	Start silence issue	The first 100 ms of silence isn't clean. Reduce the recording noise floor level, and leave the first 100 ms at the start silent.
Volume	End silence issue	The last 100 ms of silence isn't clean. Reduce the recording noise floor level, and leave the last 100 ms at the end silent.
Mismatch	Low scored words	Review the script and the audio content to make sure they match, and control the noise floor level. Reduce the length of long silence, or split the audio into multiple utterances if it's too long.
Mismatch	Start silence issue	Extra audio was heard before the first word. Review the script and the audio content to make sure they match, control the noise floor level, and make the first 100 ms silent.
Mismatch	End silence issue	Extra audio was heard after the last word. Review the script and the audio content to make sure they match, control the noise floor level, and make the last 100 ms silent.
Mismatch	Low signal-noise ratio	Audio SNR level is lower than 20 dB. At least 35 dB is recommended.

Category	Name	Description
Mismatch	No score available	Failed to recognize speech content in this audio. Check the audio and the script content to make sure the audio is valid, and matches the script.

Next steps

Train the professional voice

Train your professional voice model

05/20/2025

In this article, you learn how to fine-tune a professional voice through the Azure AI Foundry portal.

Important

Professional voice fine-tuning is currently only available in some regions. After your voice model is trained in a supported region, you can [copy the professional voice model](#) to an Azure AI Foundry resource in another region as needed. For more information, see the footnotes in the [Speech service table](#).

Training duration varies depending on how much data you use. It takes about 40 compute hours on average to fine-tune a professional voice. With an Azure AI Foundry standard (S0) resource, you can train four voices simultaneously. If you reach the limit, wait until at least one of your voice models finishes training, and then try again.

Note

Although the total number of hours required per [training method](#) varies, the same unit price applies to each. For more information, see the [custom neural training pricing details](#) .

Choose a training method

After you validate your data files, use them to build your custom voice model. When you create a custom voice, you can choose to train it with one of the following methods:

- **Neural:** Create a voice in the same language of your training data.
- **Neural - cross lingual:** Create a voice that speaks a different language from your training data. For example, with the `zh-CN` training data, you can create a voice that speaks `en-US`.

The language of the training data and the target language must both be one of the [languages that are supported](#) for cross lingual voice training. You don't need to prepare training data in the target language, but your test script must be in the target language.

- **Neural - multi style:** Create a custom voice that speaks in multiple styles and emotions, without adding new training data. Multiple style voices are useful for video game

characters, conversational chatbots, audiobooks, content readers, and more.

To create a multiple style voice, you need to prepare a set of general training data, at least 300 utterances. Select one or more of the preset target speaking styles. You can also create multiple custom styles by providing style samples, of at least 100 utterances per style, as extra training data for the same voice. The supported preset styles vary according to different languages. See [available preset styles across different languages](#).

- [Neural - multi lingual \(preview\)](#): Create a voice that speaks multiple languages using the single-language training data. For example, with the `en-US` primary training data, you can create a voice that speaks `en-US`, `de-DE`, `zh-CN` etc. secondary languages.

The primary language of the training data and the secondary languages must be in the [languages that are supported](#) for multi lingual voice training. You don't need to prepare training data in the secondary languages.

The language of the training data must be one of the [languages that are supported](#) for custom voice, cross-lingual, or multiple style training.

Train your custom voice model

To create a custom voice in Azure AI Foundry portal, follow these steps for one of the following methods:

Neural

1. Sign in to the [Azure AI Foundry portal](#) [↗](#).
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the professional voice fine-tuning task (by model name) that you [started as described in the create professional voice article](#).
4. Select **Train model** > **+ Train model**.
5. Select **Neural** as the [training method](#) for your model. To use a different training method, see [Neural - cross lingual](#), [Neural - multi style](#), [Neural - multi lingual \(preview\)](#), or [Neural - HD Voice \(preview\)](#).

Train a new model

Training method

- Training method (selected)
- Data and voice talent
- Test script
- Basic information
- Review and train

Select the training method for your model

Neural - Default
 Create a voice in the same language as your training data.

Neural - Cross lingual
 Create a voice that speaks a different language from your training data.

Neural - Multi lingual Preview
 Create a voice that speaks multiple languages using training data from a single language.

Neural - Multi style
 Create a voice that speaks in multiple styles.

Version * ⓘ
V10.1

Version description: Model updated to understand context and automatically adjust speaking tones with contextual training data. 15 compute hours estimated for each training. Start with 300 utterances or 30 minutes of speech data to

Next **Train** **Cancel**

6. Select a version of the training recipe for your model. The latest version is selected by default. The supported features and training time can vary by version. Normally, we recommend the latest version. In some cases, you can choose an earlier version to reduce training time. See [Bilingual training](#) for more information about bilingual training and differences between locales.

7. Select **Next**.

8. Select the data that you want to use for training. Duplicate audio names are removed from the training. Make sure that the data you select doesn't contain the same audio names across multiple .zip files.

You can select only successfully processed datasets for training. If you don't see your training set in the list, check your data processing status.

9. Select a speaker file with the voice talent statement that corresponds to the speaker in your training data.

10. Select **Next**.

11. Select a test script and then select **Next**.

- Each training generates 100 sample audio files automatically to help you test the model with a default script.
- Alternatively, you can select **Add my own test script** and provide your own test script with up to 100 utterances to test the model at no extra cost. The generated audio files are a combination of the automatic test scripts and custom test scripts. For more information, see [test script requirements](#).

12. Enter a **Voice model name**. Choose a name carefully. The model name is used as the voice name in your [speech synthesis request](#) by the SDK and SSML input. Only letters, numbers, and a few punctuation characters are allowed. Use different names for different neural voice models.
13. Optionally, enter the **Description** to help you identify the model. A common use of the description is to record the names of the data that you used to create the model.
14. Select the checkbox to accept the terms of use and then select **Next**.
15. Review the settings and select the box to accept the terms of use.
16. Select **Train** to start training the model.

Bilingual training

If you select the **Neural** training type, you can train a voice to speak in multiple languages. The `zh-CN`, `zh-HK`, and `zh-TW` locales support bilingual training for the voice to speak both Chinese and English. Depending in part on your training data, the synthesized voice can speak English with an English native accent or English with the same accent as the training data.

ⓘ Note

To enable a voice in the `zh-CN` locale to speak English with the same accent as the sample data, you should upload English data to a **Contextual** training set, or choose `Chinese (Mandarin, Simplified)`, `English bilingual` when creating a project or specify the `zh-CN (English bilingual)` locale for the training set data via REST API.

In your contextual training set, include at least 100 sentences or 10 minutes of English content and do not exceed the amount of Chinese content.

The following table shows the differences among the locales:

[↗](#) Expand table

Speech Studio locale	REST API locale	Bilingual support
<code>Chinese (Mandarin, Simplified)</code>	<code>zh-CN</code>	If your sample data includes English, the synthesized voice speaks English with an English native accent, instead of the same accent as the sample data, regardless of the amount of English data.

Speech Studio locale	REST API locale	Bilingual support
Chinese (Mandarin, Simplified), English bilingual	zh-CN (English bilingual)	If you want the synthesized voice to speak English with the same accent as the sample data, we recommend including over 10% English data in your training set. Otherwise, the English speaking accent might not be ideal.
Chinese (Cantonese, Simplified)	zh-HK	If you want to train a synthesized voice capable of speaking English with the same accent as your sample data, make sure to provide over 10% English data in your training set. Otherwise, it defaults to an English native accent. The 10% threshold is calculated based on the data accepted after successful uploading, not the data before uploading. If some uploaded English data is rejected due to defects and doesn't meet the 10% threshold, the synthesized voice defaults to an English native accent.
Chinese (Taiwanese Mandarin, Traditional)	zh-TW	If you want to train a synthesized voice capable of speaking English with the same accent as your sample data, make sure to provide over 10% English data in your training set. Otherwise, it defaults to an English native accent. The 10% threshold is calculated based on the data accepted after successful uploading, not the data before uploading. If some uploaded English data is rejected due to defects and doesn't meet the 10% threshold, the synthesized voice defaults to an English native accent.

Monitor the training process

The **Train model** table displays a new entry that corresponds to this newly created model. The status reflects the process of converting your data to a voice model, as described in this table:

 Expand table

State	Meaning
Processing	Your voice model is being created.
Succeeded	Your voice model has been created and can be deployed.
Failed	Your voice model has failed in training. The cause of the failure might be, for example, unseen data problems or network issues.
Canceled	The training for your voice model was canceled.

While the model status is **Processing**, you can select the model and then select **Cancel training** to cancel training. You're not charged for this canceled training.

Train and review your voice model

Train a new model | Deploy model | Cancel training | Edit | Delete | Copy to | Clone model

Name	Description	Model type	Model language	Status	Engine version	Data size	Training
Contoso fine-tuned voice		Neural V10.1	English (United States)	Processing	2025.02.07.0	300	

After you finish training the model successfully, you can review the model details and [Test your voice model](#).

Rename your model

You have to clone your model to rename it. You can't rename the model directly.

1. Select the model.
2. Select **Clone model** to create a clone of the model with a new name in the current project.
3. Enter the new name on the **Clone voice model** window.
4. Select **Submit**. The text *Neural* is automatically added as a suffix to your new model name.

Test your voice model

After your voice model is successfully built, you can use the generated sample audio files to test it before you deploy it.

ⓘ Note

Neural - multi lingual (preview) and Neural - HD Voice (preview) do not support this type of testing.

The quality of the voice depends on many factors, such as:

- The size of the training data.
- The quality of the recording.
- The accuracy of the transcript file.

- How well the recorded voice in the training data matches the personality of the designed voice for your intended use case.

Select **DefaultTests** under **Testing** to listen to the sample audio files. The default test samples include 100 sample audio files generated automatically during training to help you test the model. In addition to these 100 audio files provided by default, your own test script utterances are also added to **DefaultTests** set. This addition is at most 100 utterances. You're not charged for the testing with **DefaultTests**.

If you want to upload your own test scripts to further test your model, select **Add test scripts** to upload your own test script.

Before you upload test script, check the [Test script requirements](#). You're charged for the extra testing with the batch synthesis based on the number of billable characters. See [Azure AI Speech pricing](#).

Under **Add test scripts**, select **Browse for a file** to select your own script, then select **Add** to upload it.

Test script requirements

The test script must be a `.txt` file that is less than 1 MB. Supported encoding formats include ANSI/ASCII, UTF-8, UTF-8-BOM, UTF-16-LE, or UTF-16-BE.

Unlike the [training transcription files](#), the test script should exclude the utterance ID, which is the filename of each utterance. Otherwise, these IDs are spoken.

Here's an example set of utterances in one `.txt` file:

text

```
This is the waistline, and it's falling.  
We have trouble scoring.  
It was Janet Maslin.
```

Each paragraph of the utterance results in a separate audio. If you want to combine all sentences into one audio, make them a single paragraph.

ⓘ Note

The generated audio files are a combination of the automatic test scripts and custom test scripts.

Update engine version for your voice model

Azure text to speech engines are updated from time to time to capture the latest language model that defines the pronunciation of the language. After you train your voice, you can apply your voice to the new language model by updating to the latest engine version.

- When a new engine is available, you're prompted to update your neural voice model.
- Go to the model details page and follow the on-screen instructions to install the latest engine.
- Alternatively, select **Install the latest engine** later to update your model to the latest engine version. You're not charged for engine update. The previous versions are still kept.
- You can check all engine versions for the model from the **Engine version** list, or remove one if you don't need it anymore.

The updated version is automatically set as default. But you can change the default version by selecting a version from the drop-down list and selecting **Set as default**.

If you want to test each engine version of your voice model, you can select a version from the list, then select **DefaultTests** under **Testing** to listen to the sample audio files. If you want to upload your own test scripts to further test your current engine version, first make sure the version is set as default, then follow the steps in [Test your voice model](#).

Updating the engine creates a new version of the model at no extra cost. After you update the engine version for your voice model, you need to deploy the new version to [create a new endpoint](#). You can only deploy the default version.

After you create a new endpoint, you need to [transfer the traffic to the new endpoint in your product](#).

To learn more about the capabilities and limits of this feature, and the best practice to improve your model quality, see [Characteristics and limitations for using custom voice](#).

Copy your voice model to another project

ⓘ Note

In this context "project" refers to a fine-tuning task rather than an Azure AI Foundry project.

After training you can copy your voice model to another project for the same region or another region.

For example, you can copy a professional voice model that was trained in one region, to a project for another region. Professional voice fine-tuning is currently only [available in some regions](#).

To copy your custom voice model to another project:

1. On the **Train model** tab, select a voice model that you want to copy, and then select **Copy to project**.
2. Select the **Subscription**, **Target region**, **Connected AI Service resource** (AI Foundry resource), and **Target fine-tuning task** where you want to copy the model.
3. Select **Copy to** to copy the model.
4. Select **View model** under the notification message for the successful copying.

Navigate to the project where you copied the model to [deploy the model copy](#).

Next steps

Deploy the professional voice endpoint

Deploy your professional voice model as an endpoint

05/20/2025

After you successfully created and [fine-tuned](#) your professional voice model, you deploy it to a custom voice endpoint.

ⓘ Note

You can create up to 50 endpoints with a standard (S0) Speech resource, each with its own custom voice.

To use your fine-tuned professional voice, you must specify the voice model name, use the custom URI directly in an HTTP request, and use the same Speech resource to pass through the authentication of the text to speech service.

Add a deployment endpoint

To create a professional voice endpoint:

To deploy an endpoint, follow these steps:

1. Sign in to the [Azure AI Foundry portal](#).
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the professional voice fine-tuning task (by model name) that you [started as described in the create professional voice article](#).
4. Select **Deploy model** > **Deploy model**.
5. Select a voice model that you want to associate with this endpoint and then select **Next**.
6. Enter a **Endpoint name** and **Description** for your custom endpoint.
7. Select **Endpoint type** according to your scenario. If your resource is in a supported region, the default setting for the endpoint type is *High performance*. Otherwise, if the resource is in an unsupported region, the only available option is *Fast resume*.
 - **High performance:** Optimized for scenarios with real-time and high-volume synthesis requests, such as conversational AI, call-center bots. It takes around 5 minutes to deploy or resume an endpoint. For information about regions where the *High performance* endpoint type is supported, see the footnotes in the [regions](#) table.
 - **Fast-resume:** Optimized for audio content creation scenarios with less frequent synthesis requests. Easy and quick to deploy or resume an endpoint in under a

minute. The fast-resume endpoint type is supported in all [regions](#) where text to speech is available.

8. Select **Next**.
9. Select the checkbox to accept the terms of use and then select **Next**.
10. Review the settings and select the box to accept the model hosting costs.
11. Select **Deploy** to create your endpoint.

After your endpoint is deployed, the endpoint name appears as a link. Select the link to display information specific to your endpoint, such as the endpoint key, endpoint URL, and sample code. When the status of the deployment is **Succeeded**, the endpoint is ready for use.

Use your custom voice

The custom endpoint is functionally identical to the standard endpoint that's used for text to speech requests.

One difference is that the `EndpointId` must be specified to use the custom voice via the Speech SDK. You can start with the [text to speech quickstart](#) and then update the code with the `EndpointId` and `SpeechSynthesisVoiceName`. For more information, see [use a custom endpoint](#).

To use a custom voice via [Speech Synthesis Markup Language \(SSML\)](#), specify the model name as the voice name. This example uses the `YourCustomVoiceName` voice.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
  <voice name="YourCustomVoiceName">
    This is the text that is spoken.
  </voice>
</speak>
```

Switch to a new voice model in your product

Once you updated your voice model to the latest engine version, or if you want to switch to a new voice in your product, you need to redeploy the new voice model to a new endpoint. Redeploying new voice model on your existing endpoint isn't supported. After deployment, switch the traffic to the newly created endpoint. We recommend that you transfer the traffic to the new endpoint in a test environment first to ensure that the traffic works well, and then transfer to the new endpoint in the production environment. During the transition, you need to keep the old endpoint. If there are some problems with the new endpoint during transition, you can switch back to your old endpoint. If the traffic has been running well on the new endpoint for about 24 hours (recommended value), you can delete your old endpoint.

ⓘ Note

If your voice name is changed and you're using Speech Synthesis Markup Language (SSML), be sure to use the new voice name in SSML.

Suspend and resume an endpoint

You can suspend or resume an endpoint to limit spend and conserve resources that aren't in use. You aren't charged while the endpoint is suspended. When you resume an endpoint, you can continue to use the same endpoint URL in your application to synthesize speech.

ⓘ Note

The suspend operation completes almost immediately. The resume operation completes in about the same amount of time as a new deployment.

This section describes how to suspend or resume a custom voice endpoint in the Azure AI Foundry portal.

Suspend endpoint

To suspend and deactivate your endpoint:

1. Sign in to the [Azure AI Foundry portal](#).
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the professional voice fine-tuning task (by model name) that you [started as described in the create professional voice article](#).
4. Select **Deploy model**.
5. Select the endpoint you want to suspend and then select **Suspend**.

Contoso custom voice

Professional voice | English (United States)

Male

- Getting started
- Set up voice talent
- Prepare training data
- Train model
- Deploy model**

Deploy your model

Deploy model | **Suspend** | Resume | Edit | Delete

Name	Description	Voice name	Model...	Model langua...	Created on	Status
Contoso fine-tuned voice		Contoso fine-tuned voice	Neural V10.1	English (United States)	5/15/2025, 7:15 AM	Succeeded

6. In the dialog box that appears, select **Suspend**. After the endpoint is suspended, the status changes from **Succeeded** to **Suspended**.

Resume endpoint

1. To resume and activate your endpoint, select **Resume** from the **Deploy model** tab in the [Azure AI Foundry portal](#) .
2. In the dialog box that appears, select **Submit**. After you successfully reactivate the endpoint, the status will change from **Suspended** to **Succeeded**.

Next steps

- Learn more about custom voice in the [overview](#).
- Learn more about custom avatar in the [overview](#).

Custom voice lite

06/02/2025

Custom voice lite is a project type where you can demo and evaluate custom voice before investing in professional recordings to create a higher-quality voice. No application is required for demo and evaluation purposes. However, Microsoft restricts and selects the recording and testing samples for use with custom voice lite. You must apply for full access to professional voice fine-tuning in order to deploy and use the custom voice lite model for business purpose. In that case, request access on the [intake form](#).

 Note

Custom voice lite is only available in the [Speech Studio](#). It isn't available through the Azure AI Foundry portal, REST API, or SDKs.

With a custom voice lite project, you record your voice online by reading 20-50 pre-defined scripts provided by Microsoft. After you've recorded at least 20 samples, you can start to train a model. Once the model is trained successfully, you can review the model and check out 20 output samples produced with another set of pre-defined scripts.

See the [supported languages](#) for custom voice.

Compare project types

The following table summarizes key differences between custom voice lite and professional voice fine-tuning.

 **Expand table**

Items	Lite	Professional
Target scenarios	Demonstration or evaluation	Professional scenarios like brand and character voices for chat bots, or audio content reading.
Training data	Record online using Speech Studio	Bring your own data. Recording in a professional studio is recommended.
Scripts for recording	Provided in Speech Studio	Use your own scripts that match the use case scenario. Microsoft provides example scripts for reference.

Items	Lite	Professional
Required data size	20-50 utterances	300-2000 utterances
Training time	Less than one compute hour	Approximately 20-40 compute hours
Voice quality	Moderate quality	High quality
Availability	Anyone can record samples online and train a model for demo and evaluation purpose. Full access to custom voice is required if you want to deploy the custom voice lite model for business use.	Data upload isn't restricted, but you can only fine-tune a professional voice after access is approved. Professional voice fine-tuning is limited based on eligibility and usage criteria. Request access on the intake form .
Pricing	Per unit prices apply equally for both custom voice lite and professional voice fine-tuning. Check the pricing details here .	Per unit prices apply equally for both custom voice lite and professional voice fine-tuning. Check the pricing details here .

Create a custom voice lite project

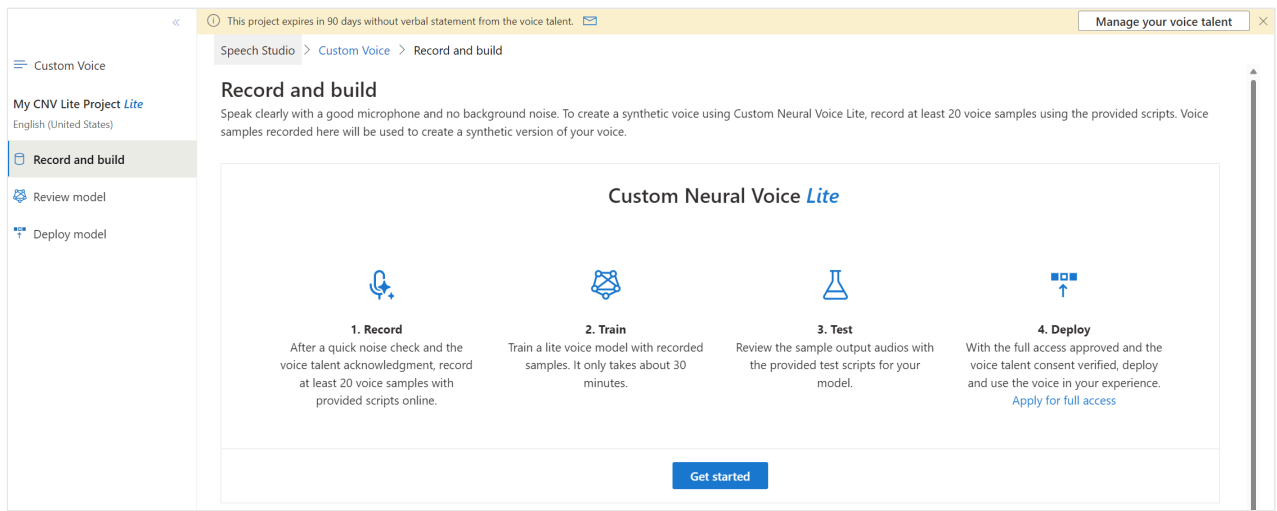
To create a custom voice lite project, follow these steps:

1. Sign in to the [Speech Studio](#).
2. Select the subscription and Speech resource to work with.
3. Select **Custom voice** > **Create a project**.
4. Select **Custom neural voice lite** > **Next**. To create a custom voice professional project instead, see the [professional voice fine-tuning documentation](#).
5. Follow the instructions provided by the wizard to create your project.

Important

The custom voice lite project expires after 90 days unless the [verbal statement](#) recorded by the voice talent is submitted.

6. Select the new project by name or select **Go to project**. You see these menu items in the left panel: **Record and build**, **Review model**, and **Deploy model**.



Record and build a custom voice lite model

Record at least 20 voice samples (up to 50) with provided scripts online. Voice samples recorded here are used to create a synthetic version of your voice.

ⓘ Note

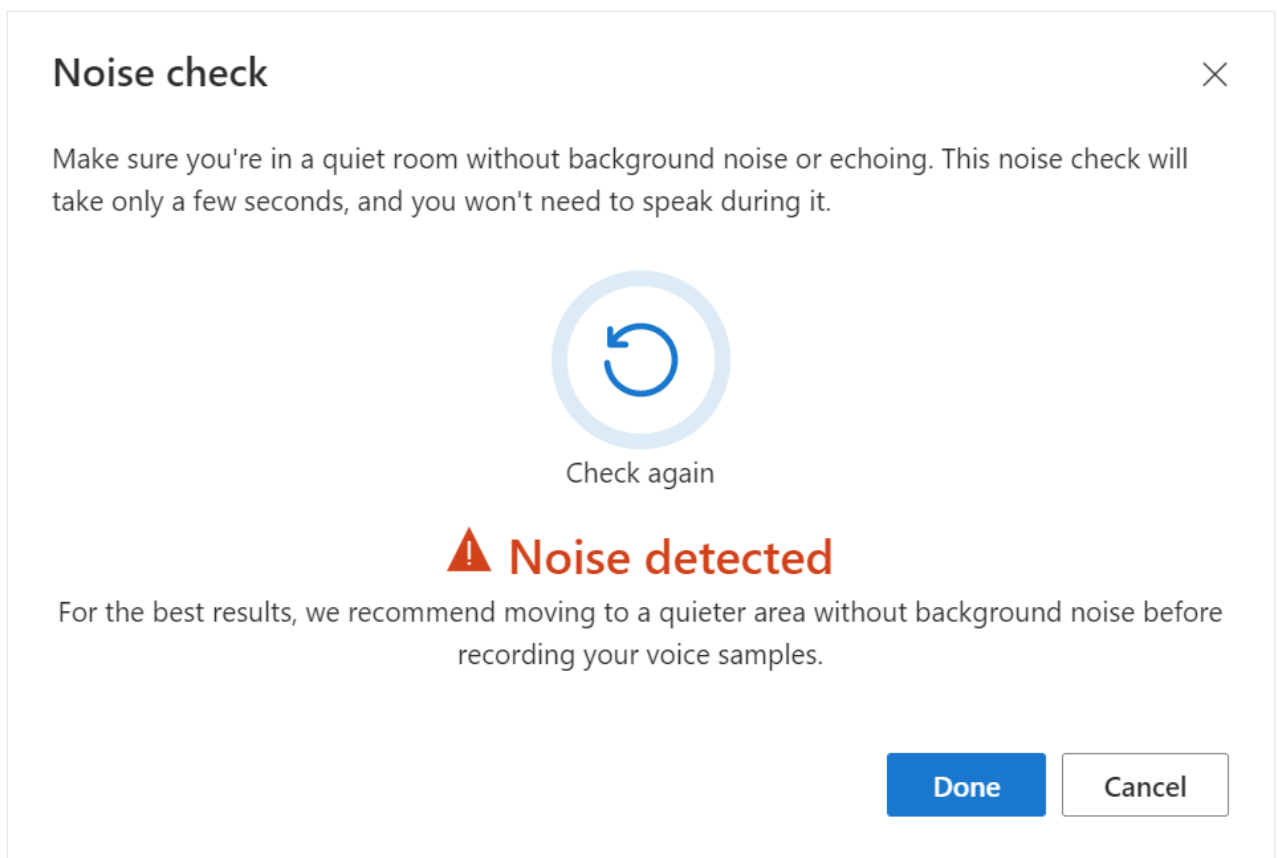
Custom voice training is currently only available in some regions. See footnotes in the [regions](#) table for more information.

Here are some tips to help you record your voice samples:

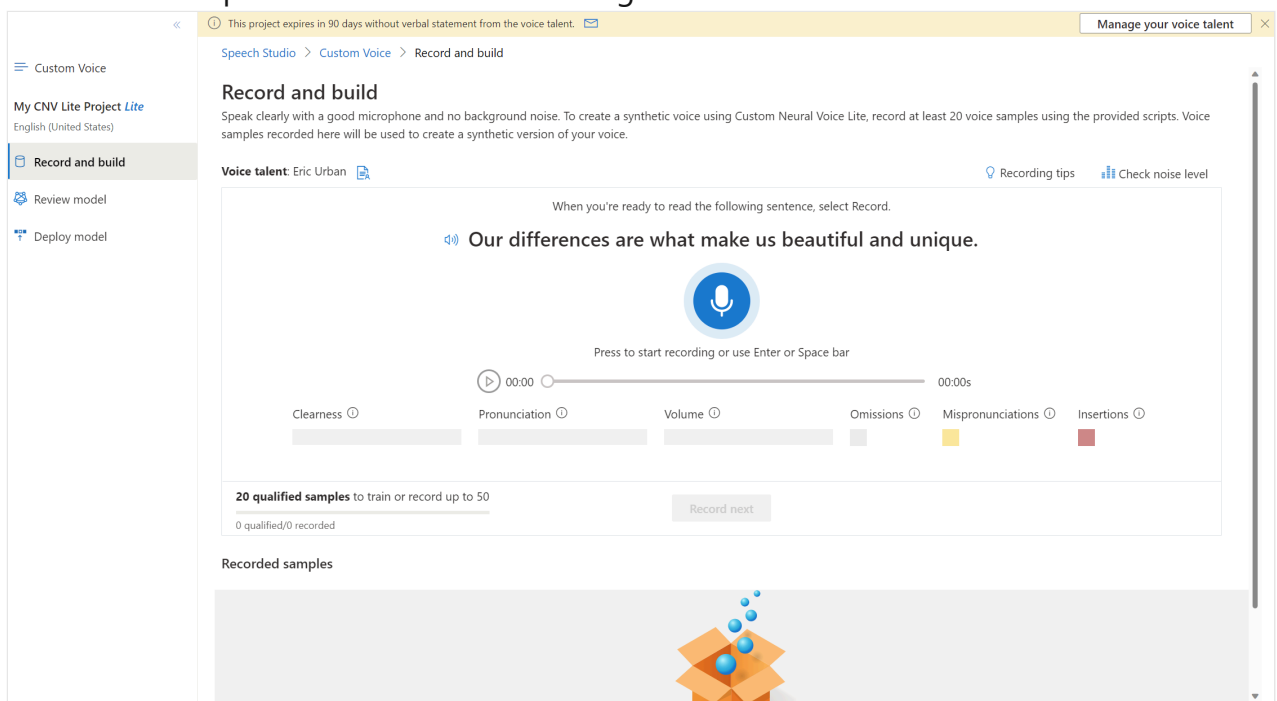
- Use a good microphone. Increase the clarity of your samples by using a high-quality microphone. Speak about 8 inches away from the microphone to avoid mouth noises.
- Avoid background noise. Record in a quiet room without background noise or echoing.
- Relax and speak naturally. Allow yourself to express emotions as you read the sentences.
- Record in one take. To keep a consistent energy level, record all sentences in one session.
- Pronounce each word correctly, and speak clearly.

To record and build a custom voice lite model, follow these steps in [Speech Studio](#):

1. Select **Custom voice** > Your project name > **Record and build**.
2. Select **Get started**.
3. Read the Voice talent terms of use carefully. Select the checkbox to acknowledge the terms of use.
4. Select **Accept**
5. Press the microphone icon to start the noise check. This noise check takes only a few seconds, and you don't need to speak during it.
6. If noise was detected, you can select **Check again** to repeat the noise check. If no noise was detected, you can select **Done** to proceed to the next step.



7. Review the recording tips and select **Got it**. For the best results, go to a quiet area without background noise before recording your voice samples.
8. Press the microphone icon to start recording.



9. Press the stop icon to stop recording.
10. Review quality metrics. After recording each sample, check its quality metric before continuing to the next one.
11. Record more samples. Although you can create a model with just 20 samples, it's recommended that you record up to 50 to get better quality.
12. Select **Train model** to start the training process.

The training process takes approximately one compute hour. You can check the progress of the training process in the **Review model** page.

Review model

To review the custom voice lite model and listen to your own synthetic voice, follow these steps:

1. Select **Custom voice** > Your project name > **Review model**. Here you can review the voice model name, model language, sample data size, and training progress. The voice name is composed of the word "Neural" appended to your project name.
2. Select the voice model name to review the model details and listen to the sample text to speech results.
3. Select the play icon to hear your voice speak each script.

Custom Voice

My CNV Lite Project *Lite*
English (United States)

Record and build

Review model

Deploy model

This project expires in 90 days without full access approval ([Learn more](#)) to Custom Neural Voice. [Apply for access](#)

Speech Studio > Custom Voice > Review model > My CNV Lite ProjectNeural

My CNV Lite ProjectNeural

Gender	Model language	Model type	Utterances ⓘ	Sampling rate	Status	Model ID	Created
Male	English (United States)	Neural	20	24000	⊙ Succeeded	ae6db871-18cd-42ab-856f-90282d9c5fb3	10/15/2022 8:18 AM

Sample output

Delete

Text input	Audio
✓ Congratulations! You have successfully created your synthetic voice with Custom Neural Voice Lite!	▶ ○
To deploy your voice model and use it in your business applications, you must get the full access to Custom Neural Voice and the explicit consent from your voice talent.	▶ ○
With the full access, you can create an even more natural voice by bringing your own data recorded in professional studios, using Custom Neural Voice Pro.	▶ ○

Submit verbal statement

A verbal statement recorded by the voice talent is required before you can [deploy the model](#) for your business use.

To submit the voice talent verbal statement, follow these steps in [Speech Studio](#) ↗:

1. Select **Custom voice** > Your project name > **Deploy model** > **Manage your voice talent**.

The screenshot shows the 'Manage your voice talent' page in the Speech Studio interface. The left sidebar contains a navigation menu with 'Custom Voice' selected, and a sub-menu with 'My CNV Lite Project Lite' (English (United States)), 'Record and build', 'Review model', and 'Deploy model'. The main content area has a breadcrumb trail: 'Speech Studio > Custom Voice > Voice talent statement'. Below this, there are two tabs: 'Deploy your model' and 'Manage your voice talent' (which is active). A warning banner at the top states: 'This project expires in 90 days without full access approval (Learn more) to Custom Neural Voice.' with an 'Apply for access' button. The 'Manage your voice talent' section includes a consent statement: 'Before you deploy the model, make sure your get meaningful consent from the voice talent to use the voice in your apps for the specific scenario defined. Learn more about Microsoft's processing, use and retention of voice talent data.' Below the consent statement are three input fields: 'Model *' (a dropdown menu showing 'Select model'), 'Voice talent name *' (a text box with placeholder '[state your first and last name]'), and 'Company name *' (a text box with placeholder '[state the name of the company]'). A large text box contains the statement to be read: 'I, [your first and last name], understand that the synthetic voice designated to sound like my voice created with Microsoft Speech technology will be used by [the name of the company] for its own business purposes.' Below the text box is a microphone icon with the instruction 'Press to start recording or use Enter or Space bar'. A progress bar shows '00:00' and '00:00s'. At the bottom, there are six performance metrics: 'Clearness', 'Pronunciation', 'Volume', 'Omissions', 'Mispronunciations', and 'Insertions', each with a corresponding colored bar. A 'Submit' button is located at the bottom right of the form.

2. Select the model.
3. Enter the voice talent name and company name.
4. Read and record the statement. Select the microphone icon to start recording. Select the stop icon to stop recording.
5. Select **Submit** to submit the statement.
6. Check the processing status in the script table at the bottom of the dashboard. Once the status is **Succeeded**, you can [deploy the model](#).

Deploy model

To deploy your custom voice lite model and use it in your applications, you must get the full access to custom voice. Request access on the [intake form](#). Within approximately 10 business days, you receive an email with the approval status. A [verbal statement](#) recorded by the voice talent is also required before you can deploy the model for your business use.

To deploy a custom voice lite model, follow these steps in [Speech Studio](#):

1. Select **Custom voice** > Your project name > **Deploy model** > **Deploy model**.
2. Select a voice model name and then select **Next**.
3. Enter a name and description for your endpoint and then select **Next**.
4. Select the checkbox to agree to the terms of use and then select **Next**.
5. Select **Deploy** to deploy the model.

From here, you can use the custom voice lite model similarly as you would use a professional voice model. For example, you can [suspend or resume](#) an endpoint after it's created, to limit spend and conserve resources that aren't in use. You can also access the voice in the [Audio Content Creation](#) tool in the [Speech Studio](#).

Next steps

- [Fine-tune a professional voice](#)
- [Try the text to speech quickstart](#)
- [Learn more about speech synthesis](#)

What is personal voice for text to speech?

With personal voice, you can enable your users to get AI generated replication of their own voices in a few seconds. With a verbal statement and a short speech sample as the audio prompt, you can create a personal voice for your users and allow them to generate speech in any of the more than 90 languages supported across more than 100 locales.

ⓘ Note

For the current list of regions that support personal voice, see the [Speech service regions table](#). For supported locales, see [personal voice language support](#).

The following table summarizes the difference between personal voice and professional voice.

 Expand table

Comparison	Personal voice	Professional voice
Target scenarios	Business customers to build an app to allow their users to create and use their own personal voice in the app.	Professional scenarios like brand and character voices for chat bots, or audio content reading.
Use cases	Restricted to limited use cases. See the transparency note .	
Training data	Make sure you follow the code of conduct.	Bring your own data. Recording in a professional studio is recommended.
Required data size	One minute of human speech.	300-2000 utterances (about 30 minutes to 3 hours of human speech).
Training time	Less than 5 seconds	Approximately 20-40 compute hours.
Voice quality	Natural	Highly natural
Multilingual support	Yes. The voice is able to speak about 100 languages, with automatic language detection enabled.	Yes. You need to select the "Neural – cross lingual" feature to train a model that speaks a different language from the training data.
Availability	The demo on Speech Studio  is available upon registration. Access to the API is restricted to eligible customers and approved use cases. Request access through the intake form.	You can only use professional voice fine-tuning after access is approved. Professional voice fine-tuning access is limited based on eligibility and usage criteria. Request access through the intake form.
Pricing	Check the pricing details here  ¹ .	Check the pricing details here  .

Comparison	Personal voice	Professional voice
Responsible AI requirements	Speaker's verbal statement required. No unapproved use case allowed.	Speaker's verbal statement required. No unapproved use case allowed.

¹ Note that personal voice pricing will only be visible for service regions where the feature is available. For the current list of supported regions, see the [Speech service regions table](#).

Try the demo

If you have an S0 resource, you can access the personal voice demo in Speech Studio. To use the personal voice API, you can apply for access [here](#).

1. Go to [Speech Studio](#).
2. Select the **Personal Voice** card.
3. You can record your own voice and try the voice output samples in different languages. The demo includes a subset of the languages supported by personal voice.

Try your own voice

Record your voice and create an AI replica in seconds. With each standard Speech resource, you can try up to five voices for free. If you want to use this service for your business, please apply for access through [this link](#).

Use with your resource

☒ I acknowledge that this application uses the resource **tianhua-speech-s0-eus** and will incur usage to my account. [Choose a different resource](#)

Voice list

- + New voice**
- ☒ **Jessa Smith_20231110_161816**

Detail

Jessa Smith_20231110_161816

Voice talent	Verbal statement	Source language
Jessa Smith		English (United States)

Generate audios using sample scripts

Output language English (United States)

- The two reservoirs fed by the Colorado River watershed, provide a critical supply of drinking water an...
- Let me give you the container number here.
- I created a ticket number for you as a reference to this call.
- I've never been there.
- The monkey was confident since he was good at imitating.

4/5 Voice

How to create a personal voice

To get started, here's a summary of the steps to create a personal voice:

1. [Create a project](#).
2. [Upload consent file](#). With the personal voice feature, it's required that every voice be created with explicit consent from the user. A recorded statement from the user is required acknowledging that the customer (Azure AI Speech resource owner) will create and use their voice.
3. [Get a speaker profile ID](#) for the personal voice. You get a speaker profile ID based on the speaker's verbal consent statement and an audio prompt. The user's voice characteristics are encoded in the `speakerProfileId` property that's used for text to speech.

Once you have a personal voice, you can [use it](#) to synthesize speech in any of the 91 languages supported across 100+ locales. A locale tag isn't required. Personal voice uses automatic language detection at the sentence level. For more information, see [use personal voice in your application](#).

Tip

Check out the code samples in the [Speech SDK repository on GitHub](#)  to see how to use personal voice in your application.

Reference documentation

[Custom voice REST API reference documentation](#)

Responsible AI

We care about the people who use AI and the people who will be affected by it as much as we care about technology. For more information, see the Responsible AI [transparency notes](#).

Next steps

- [Create a project](#).
- Learn more about custom voice in the [overview](#).
- Learn more about Speech Studio in the [overview](#).

Create a project for personal voice

08/07/2025

Personal voice projects contain the user consent statement and the personal voice ID. You can only create a personal voice project using the custom voice API. You can't create a personal voice project in the Speech Studio.

Create a project

To create a personal voice project, use the [Projects_Create](#) operation of the custom voice API. Construct the request body according to the following instructions:

- Set the required `kind` property to `PersonalVoice`. The kind can't be changed later.
- Optionally, set the `description` property for the project description. The project description can be changed later.

Make an HTTP PUT request using the URI as shown in the following [Projects_Create](#) example.

- Replace `YourResourceKey` with your Speech resource key.
- Replace `YourResourceRegion` with your Speech resource region.
- Replace `ProjectId` with a project ID of your choice. The case sensitive ID must be unique within your Speech resource. The ID will be used in the project's URI and can't be changed later.

Azure CLI

```
curl -v -X PUT -H "Ocp-Apim-Subscription-Key: YourResourceKey" -H "Content-Type: application/json" -d '{
  "description": "Project description",
  "kind": "PersonalVoice"
}' "https://YourResourceRegion.api.cognitive.microsoft.com/customvoice/projects/ProjectId?api-version=2024-02-01-preview"
```

You should receive a response body in the following format:

JSON

```
{
  "id": "ProjectId",
  "description": "Project description",
  "kind": "PersonalVoice",
  "createdDateTime": "2024-09-01T05:30:00.000Z"
}
```

You use the project `id` in subsequent API requests to [add user consent](#) and [get a speaker profile ID](#).

Next steps

Add user consent to the personal voice project.

Add user consent to the personal voice project

08/07/2025

With the personal voice feature, it's required that every voice be created with explicit consent from the user. A recorded statement from the user is required acknowledging that the customer (Azure AI Speech resource owner) will create and use their voice.

To add user consent to the personal voice project, you provide the prerecorded consent audio file [from a publicly accessible URL \(Consents_Create\)](#) or [upload the audio file \(Consents_Post\)](#).

Consent statement

You need an audio recording of the user speaking the consent statement.

You can get the consent statement text for each locale from the text to speech GitHub repository. See [verbal-statement-all-locales.txt](#) for the consent statement. Below is a sample for the `en-US` locale:

```
"I [state your first and last name] am aware that recordings of my voice will be used by [state the name of the company] to create and use a synthetic version of my voice."
```

Supported audio formats for consent audio

See the table below for the supported formats for consent audio files:

 Expand table

Format	Sample rate	Bit rate	Bit depth
mp3	16 kHz, 24 kHz, 44.1 kHz, 48 kHz	128 kbps, 192 kbps, 256 kbps, 320 kbps	/
wav	16 kHz, 24 kHz, 44.1 kHz, 48 kHz	/	16-bit, 24-bit, 32-bit

Add consent from a file

In this scenario, the audio files must be available locally.

To add consent to a personal voice project from a local audio file, use the `Consents_Post` operation of the custom voice API. Construct the request body according to the following instructions:

- Set the required `projectId` property. See [create a project](#).
- Set the required `voiceTalentName` property. The voice talent name can't be changed later.
- Set the required `companyName` property. The company name can't be changed later.
- Set the required `audiodata` property with the consent audio file.
- Set the required `locale` property. This should be the locale of the consent. The locale can't be changed later. You can find the text to speech locale list [here](#).

Make an HTTP POST request using the URI as shown in the following `Consents_Post` example.

- Replace `YourResourceKey` with your Speech resource key.
- Replace `YourResourceRegion` with your Speech resource region.
- Replace `JessicaConsentId` with a consent ID of your choice. The case sensitive ID will be used in the consent's URI and can't be changed later.

Azure CLI

```
curl -v -X POST -H "Ocp-Apim-Subscription-Key: YourResourceKey" -F
'description="Consent for Jessica voice"' -F 'projectId="ProjectId"' -F
'voiceTalentName="Jessica Smith"' -F 'companyName="Contoso"' -F
'audiodata=@"D:\PersonalVoiceTest\jessica-consent.wav"' -F 'locale="en-US"'
"https://YourResourceRegion.api.cognitive.microsoft.com/customvoice/consents/Jessi
caConsentId?api-version=2024-02-01-preview"
```

You should receive a response body in the following format:

JSON

```
{
  "id": "JessicaConsentId",
  "description": "Consent for Jessica voice",
  "projectId": "ProjectId",
  "voiceTalentName": "Jessica Smith",
  "companyName": "Contoso",
  "locale": "en-US",
  "status": "NotStarted",
  "createdDateTime": "2024-09-01T05:30:00.000Z",
  "lastActionDateTime": "2024-09-02T10:15:30.000Z"
}
```


The response header contains the `Operation-Location` property. Use this URI to get details about the `Consents_Post` operation. Here's an example of the response header:

HTTP

```
Operation-Location:
https://eastus.api.cognitive.microsoft.com/customvoice/operations/070f7986-ef17-41d0-ba2b-907f0f28e314?api-version=2024-02-01-preview
Operation-Id: 070f7986-ef17-41d0-ba2b-907f0f28e314
```

Add consent from a URL

In this scenario, the audio files must already be stored in an Azure Blob Storage container.

To add consent to a personal voice project from the URL of an audio file, use the `Consents_Create` operation of the custom voice API. Construct the request body according to the following instructions:

- Set the required `projectId` property. See [create a project](#).
- Set the required `voiceTalentName` property. The voice talent name can't be changed later.
- Set the required `companyName` property. The company name can't be changed later.
- Set the required `audioUrl` property. The URL of the voice talent consent audio file. Use a URI with the [shared access signatures \(SAS\)](#) token.
- Set the required `locale` property. This should be the locale of the consent. The locale can't be changed later. You can find the text to speech locale list [here](#).

Make an HTTP PUT request using the URI as shown in the following `Consents_Create` example.

- Replace `YourResourceKey` with your Speech resource key.
- Replace `YourResourceRegion` with your Speech resource region.
- Replace `JessicaConsentId` with a consent ID of your choice. The case sensitive ID will be used in the consent's URI and can't be changed later.

Azure CLI

```
curl -v -X PUT -H "Ocp-Apim-Subscription-Key: YourResourceKey" -H "Content-Type: application/json" -d '{
  "description": "Consent for Jessica voice",
  "projectId": "ProjectId",
  "voiceTalentName": "Jessica Smith",
  "companyName": "Contoso",
  "audioUrl": "https://contoso.blob.core.windows.net/public/jessica-consent.wav?mySasToken",
  "locale": "en-US"
} '
```

```
"https://YourResourceRegion.api.cognitive.microsoft.com/customvoice/consents/JessicaConsentId?api-version=2024-02-01-preview"
```

You should receive a response body in the following format:

JSON

```
{
  "id": "JessicaConsentId",
  "description": "Consent for Jessica voice",
  "projectId": "ProjectId",
  "voiceTalentName": "Jessica Smith",
  "companyName": "Contoso",
  "locale": "en-US",
  "status": "NotStarted",
  "createdDateTime": "2024-09-01T05:30:00.000Z",
  "lastActionDateTime": "2024-09-02T10:15:30.000Z"
}
```

The response header contains the `Operation-Location` property. Use this URI to get details about the `Consents_Create` operation. Here's an example of the response header:

HTTP

```
Operation-Location:
https://eastus.api.cognitive.microsoft.com/customvoice/operations/070f7986-ef17-41d0-ba2b-907f0f28e314?api-version=2024-02-01-preview
Operation-Id: 070f7986-ef17-41d0-ba2b-907f0f28e314
```

Next steps

Create a personal voice.

Get a speaker profile ID for the personal voice

08/07/2025

To use personal voice in your application, you need to get a speaker profile ID. The speaker profile ID is used to generate synthesized audio with the text input provided.

You create a speaker profile ID based on the speaker's verbal consent statement and an audio prompt (a clean human voice sample between 5 - 90 seconds). The user's voice characteristics are encoded in the `speakerProfileId` property that's used for text to speech. For more information, see [use personal voice in your application](#).

ⓘ **Note**

The personal voice ID and speaker profile ID aren't same. You can choose the personal voice ID, but the speaker profile ID is generated by the service. The personal voice ID is used to manage the personal voice. The speaker profile ID is used for text to speech.

You provide the audio files [from a publicly accessible URL](#) (`PersonalVoices_Create`) or [upload the audio files](#) (`PersonalVoices_Post`).

Prompt audio format

The supported formats for prompt audio files are:

[Expand table](#)

Format	Sample rate	Bit rate	Bit depth
mp3	16 kHz, 24 kHz, 44.1 kHz, 48 kHz	128 kbps, 192 kbps, 256 kbps, 320 kbps	/
wav	16 kHz, 24 kHz, 44.1 kHz, 48 kHz	/	16-bit, 24-bit, 32-bit

Create personal voice from a file

In this scenario, the audio files must be available locally.

To create a personal voice and get the speaker profile ID, use the [PersonalVoices_Post](#) operation of the custom voice API. Construct the request body according to the following instructions:

- Set the required `projectId` property. See [create a project](#).
- Set the required `consentId` property. See [add user consent](#).
- Set the required `audiodata` property. You can specify one or more audio files in the same request.

Make an HTTP POST request using the URI as shown in the following [PersonalVoices_Post](#) example.

- Replace `YourResourceKey` with your Speech resource key.
- Replace `YourResourceRegion` with your Speech resource region.
- Replace `JessicaPersonalVoiceId` with a personal voice ID of your choice. The case sensitive ID will be used in the personal voice's URI and can't be changed later.

Azure CLI

```
curl -v -X POST -H "Ocp-Apim-Subscription-Key: YourResourceKey" -F
'projectId="ProjectId"' -F 'consentId="JessicaConsentId"' -F
'audiodata=@"D:\PersonalVoiceTest\CNVSample001.wav"' -F
'audiodata=@"D:\PersonalVoiceTest\CNVSample002.wav"' "
https://YourResourceRegion.api.cognitive.microsoft.com/customvoice/personalvoices/
JessicaPersonalVoiceId?api-version=2024-02-01-preview"
```

You should receive a response body in the following format:

JSON

```
{
  "id": "JessicaPersonalVoiceId",
  "speakerProfileId": "3059912f-a3dc-49e3-bdd0-02e449df1fe3",
  "projectId": "ProjectId",
  "consentId": "JessicaConsentId",
  "status": "NotStarted",
  "createdDateTime": "2024-09-01T05:30:00.000Z",
  "lastActionDateTime": "2024-09-02T10:15:30.000Z"
}
```

Use the `speakerProfileId` property to integrate personal voice in your text to speech application. For more information, see [use personal voice in your application](#).

The response header contains the `Operation-Location` property. Use this URI to get details about the [PersonalVoices_Post](#) operation. Here's an example of the response header:

Operation-Location:

<https://eastus.api.cognitive.microsoft.com/customvoice/operations/1321a2c0-9be4-471d-83bb-bc3be4f96a6f?api-version=2024-02-01-preview>

Operation-Id: 1321a2c0-9be4-471d-83bb-bc3be4f96a6f

Create personal voice from a URL

In this scenario, the audio files must already be stored in an Azure Blob Storage container.

To create a personal voice and get the speaker profile ID, use the [PersonalVoices_Create](#) operation of the custom voice API. Construct the request body according to the following instructions:

- Set the required `projectId` property. See [create a project](#).
- Set the required `consentId` property. See [add user consent](#).
- Set the required `audios` property. Within the `audios` property, set the following properties:
 - Set the required `containerUrl` property to the URL of the Azure Blob Storage container that contains the audio files. Use [shared access signatures \(SAS\) SAS for a container](#) with both read and list permissions.
 - Set the required `extensions` property to the extensions of the audio files.
 - Optionally, set the `prefix` property to set a prefix for the blob name.

Make an HTTP PUT request using the URI as shown in the following [PersonalVoices_Create](#) example.

- Replace `YourResourceKey` with your Speech resource key.
- Replace `YourResourceRegion` with your Speech resource region.
- Replace `JessicaPersonalVoiceId` with a personal voice ID of your choice. The case sensitive ID will be used in the personal voice's URI and can't be changed later.

Azure CLI

```
curl -v -X PUT -H "Ocp-Apim-Subscription-Key: YourResourceKey" -H "Content-Type: application/json" -d '{
  "projectId": "ProjectId",
  "consentId": "JessicaConsentId",
  "audios": {
    "containerUrl": "https://contoso.blob.core.windows.net/voicecontainer?mySasToken",
    "prefix": "jessica/",
    "extensions": [
      ".wav"
    ]
  }
}
```

```

    ]
  }
} '
"https://YourResourceRegion.api.cognitive.microsoft.com/customvoice/personalvoices
/JessicaPersonalVoiceId?api-version=2024-02-01-preview"

# Ensure the `containerUrl` has both read and list permissions.
# Ensure the `.wav` files are located in the "jessica" folder within the
container. The `prefix` matches all `.wav` files in the "jessica" folder. If there
is no such folder, the prefix will match `.wav` files with names starting with
"jessica".

```

You should receive a response body in the following format:

JSON

```

{
  "id": "JessicaPersonalVoiceId",
  "speakerProfileId": "3059912f-a3dc-49e3-bdd0-02e449df1fe3",
  "projectId": "ProjectId",
  "consentId": "JessicaConsentId",
  "status": "NotStarted",
  "createdDateTime": "2024-09-01T05:30:00.000Z",
  "lastActionDateTime": "2024-09-02T10:15:30.000Z"
}

```

Use the `speakerProfileId` property to integrate personal voice in your text to speech application. For more information, see [use personal voice in your application](#).

The response header contains the `Operation-Location` property. Use this URI to get details about the [PersonalVoices_Create](#) operation. Here's an example of the response header:

HTTP

```

Operation-Location:
https://eastus.api.cognitive.microsoft.com/customvoice/operations/1321a2c0-9be4-
471d-83bb-bc3be4f96a6f?api-version=2024-02-01-preview
Operation-Id: 1321a2c0-9be4-471d-83bb-bc3be4f96a6f

```

Next steps

Use personal voice in your application.

Use personal voice in your application

08/07/2025

You can use the [speaker profile ID](#) for your personal voice to synthesize speech in any of the 91 languages supported across 100+ locales. A locale tag isn't required. Personal voice uses automatic language detection at the sentence level.

Integrate personal voice in your application

You need to use [speech synthesis markup language \(SSML\)](#) to use personal voice in your application. SSML is an XML-based markup language that provides a standard way to mark up text for the generation of synthetic speech. SSML tags are used to control the pronunciation, volume, pitch, rate, and other attributes of the speech synthesis output.

- The `speakerProfileId` property in SSML is used to specify the [speaker profile ID](#) for the personal voice.
- The voice name is specified in the `name` property in SSML. For personal voice, the voice name must be one of the supported base model voice names. To get a list of supported base model voice names, use the [BaseModels_List](#) operation of the custom voice API.

! Note

The voice names labeled with the `Latest`, such as `DragonLatestNeural` or `PhoenixLatestNeural`, will be updated from time to time; its performance may vary with updates for ongoing improvements. If you would like to use a fixed version, select one labeled with a version number, such as `PhoenixV2Neural`.

- `DragonLatestNeural` is a base model with superior voice cloning similarity compared to `PhoenixLatestNeural`. `PhoenixLatestNeural` is a base model with more accurate pronunciation and lower latency than `DragonLatestNeural`.
- For personal voice, you can use the `<lang xml:lang>` element to adjust the speaking language. It's the same as with multilingual voices. See [how to use the lang element to speak different languages](#).

Here's example SSML in a request for text to speech with the voice name and the speaker profile ID. The sample also demonstrates how to switch languages from `en-US` to `zh-HK` using the `<lang xml:lang>` element.

```
< speak version='1.0' xmlns='http://www.w3.org/2001/10/synthesis'
xmlns:mstts='http://www.w3.org/2001/mstts' xml:lang='en-US'>
  < voice name='DragonLatestNeural'>
    <mstts:ttseembedding speakerProfileId='your speaker profile ID here'>
      I'm happy to hear that you find me amazing and that I have made your
      trip planning easier and more fun.
      <lang xml:lang='zh-HK'>我很高興聽到你覺得我很了不起，我讓你的旅行計劃更輕
      鬆、更有趣。</lang>
    </mstts:ttseembedding>
  </voice>
</speak>
```

You can use the SSML via the [Speech SDK](#) or [REST API](#).

- **Real-time speech synthesis:** Use the [Speech SDK](#) or [REST API](#) to convert text to speech.
 - When you use Speech SDK, don't set Endpoint ID, just like prebuild voice.
 - When you use REST API, please use standard voices endpoint.

Supported and unsupported SSML elements for personal voice

For detailed information on the supported and unsupported SSML elements for Phoenix and Dragon models, refer to the following table. For instructions on how to use SSML elements, refer to the [SSML document structure and events](#).

 Expand table

Element	Description	Supported in Phoenix	Supported in Dragon
<voice>	Specifies the voice and optional effects (eq_car and eq_telecomhp8k).	Yes	Yes
<mstts:express-as>	Specifies speaking styles and roles.	No	No
<mstts:ttseembedding>	Specifies the speakerProfileId property for a personal voice.	Yes	Yes
<lang xml:lang>	Specifies the speaking language.	Yes	Yes
<prosody>	Adjusts pitch, contour, range, rate, and volume.		
pitch	Indicates the baseline pitch for the text.	No	No

Element	Description	Supported in Phoenix	Supported in Dragon
<code>contour</code>	Represents changes in pitch.	No	No
<code>range</code>	Represents the range of pitch for the text.	No	No
<code>rate</code>	Indicates the speaking rate of the text.	Yes	Yes
<code>volume</code>	Indicates the volume level of the speaking voice.	No	No
<code><emphasis></code>	Adds or removes word-level stress for the text.	No	No
<code><audio></code>	Embeds prerecorded audio into an SSML document.	Yes	No
<code><mstts:audioduration></code>	Specifies the duration of the output audio.	No	No
<code><mstts:backgroundaudio></code>	Adds background audio to your SSML documents or mixes an audio file with text to speech.	Yes	No
<code><phoneme></code>	Specifies phonetic pronunciation in SSML documents.		
<code>ipa</code>	One of the phonetic alphabets.	Yes	No
<code>sapi</code>	One of the phonetic alphabets.	No	No
<code>ups</code>	One of the phonetic alphabets.	Yes	No
<code>x-sampa</code>	One of the phonetic alphabets.	Yes	No
<code><lexicon></code>	Defines how multiple entities are read in SSML.	Yes	Yes (only support alias)
<code><say-as></code>	Indicates the content type, such as number or date, of the element's text.	Yes	Yes
<code><sub></code>	Indicates that the alias attribute's text value should be pronounced instead of the element's enclosed text.	Yes	Yes
<code><math></code>	Uses the MathML as input text to properly pronounce mathematical notations in the output audio.	Yes	No
<code><bookmark></code>	Gets the offset of each marker in the audio stream.	Yes	No

Element	Description	Supported in Phoenix	Supported in Dragon
<code><break></code>	Overrides the default behavior of breaks or pauses between words.	Yes	Yes
<code><mstts:silence></code>	Inserts pauses before or after text, or between two adjacent sentences.	Yes	No
<code><mstts:viseme></code>	Defines the position of the face and mouth while a person is speaking.	Yes	No
<code><p></code>	Denotes paragraphs in SSML documents.	Yes	Yes
<code><s></code>	Denotes sentences in SSML documents.	Yes	Yes

Supported and unsupported SDK features for personal voice

The following table outlines which SDK features are supported for Phoenix and Dragon models. For details on how to utilize these SDK features in your applications, refer to [Subscribe to synthesizer events](#).

 Expand table

SDK features	Description	Supported in Phoenix	Supported in Dragon
Word boundary	Signals that a word boundary was received during synthesis, providing precise word timing during the speech synthesis process.	Yes	No
Viseme events	Provides viseme (lips, jaw, and tongue movement) information during synthesis, allowing visual synchronization.	Yes	No

Reference documentation

[Custom voice REST API reference documentation](#)

Next steps

- Learn more about custom voice in the [overview](#).

- Learn more about Speech Studio in the [overview](#).

What is Text to speech avatar?

Text to speech avatar converts text into a digital video of a photorealistic human (either a standard avatar or a [custom text to speech avatar](#)) speaking with a natural-sounding voice. The text to speech avatar video can be synthesized asynchronously or in real time. Developers can build applications integrated with text to speech avatar through an API, or use a content creation tool on Speech Studio to create video content without coding.

With text to speech avatar's advanced neural network models, the feature empowers users to deliver life-like and high-quality synthetic talking avatar videos for various applications while adhering to [responsible AI practices](#).

Tip

To convert text to speech with a no-code approach, try the [Text to speech avatar tool in Speech Studio](#) .

Avatar capabilities

Text to speech avatar capabilities include:

- Converts text into a digital video of a photorealistic human speaking with natural-sounding voices powered by Azure AI text to speech.
- Provides a collection of standard avatars.
- Azure AI text to speech generates the voice of the avatar. For more information, see [Avatar voice and language](#).
- Synthesizes text to speech avatar video asynchronously with the [batch synthesis API](#) or in [real-time](#).
- Provides a [content creation tool](#)  in Speech Studio for creating video content without coding.
- Enables real-time avatar conversations through the [live chat avatar tool](#)  in Speech Studio.

With text to speech avatar's advanced neural network models, the feature empowers you to deliver lifelike and high-quality synthetic talking avatar videos for various applications while adhering to responsible AI practices.

Avatar voice and language

You can choose from a range of standard voices for the avatar. The language support for text to speech avatar is the same as the language support for text to speech. For details, see [Language and voice support for the Speech service](#). Standard text to speech avatars can be accessed through the [Speech Studio portal](#) or via API.

The voice in the synthetic video could be an Azure AI Speech standard voice or the [custom voice](#) of voice talent selected by you.

Avatar video output

Both batch synthesis and real-time synthesis resolution are 1920 x 1080, and the frames per second (FPS) are 25. Batch synthesis codec can be h264, hevc, or av1 if the format is `mp4` and can set codec as vp9 or av1 if the format is `webm`; only `vp9` can contain an alpha channel. Real-time synthesis codec is h264. Video bitrate can be configured for both batch synthesis and real-time synthesis in the request; the default value is 2000000; more detailed configurations can be found in the sample code.

 Expand table

	Batch synthesis	Real-time synthesis
Resolution	1920 x 1080	1920 x 1080
FPS	25	25
Codec	h264/hevc/vp9/av1	h264

Custom text to speech avatar

You can create custom text to speech avatars that are unique to your product or brand. All it takes to get started is taking 10 minutes of video recordings. If you're also fine-tuning a professional voice for the actor, the avatar can be highly realistic.

Voice sync for avatar is trained alongside the custom avatar utilizing audio from the training video. The voice is exclusively associated with the custom avatar and can't be independently used.

[Professional voice fine-tuning](#) and [custom text to speech avatar](#) are separate features. You can use them independently or together. If you plan to also use [professional voice fine-tuning](#) with a text to speech avatar, you need to deploy or [copy](#) your fine-tuned professional voice model to one of the [avatar supported regions](#).

For more information, see [What is custom text to speech avatar](#).

Sample code

Sample code for text to speech avatar is available on [GitHub](#). These samples cover the most popular scenarios:

- [Batch synthesis \(REST\)](#)
- [Real-time synthesis \(SDK\)](#)
- [Live chat with Azure OpenAI in behind \(SDK\)](#)
- To create a live chat APP with Azure OpenAI [On Your Data](#), you can refer to [this sample code](#) (search "On Your Data")

Pricing

- Throughout an avatar real-time session or batch content creation, the text to speech, speech to text, Azure OpenAI, or other Azure services are charged separately.
- Voice sync for avatar (via custom avatar training) is charged the same as a personal voice in terms of voice creation and synthesis. The storage of the voice is free.
- Refer to [text to speech avatar pricing note](#) to learn how billing works for the text-to-speech avatar feature.
- For the detailed pricing, see [Speech service pricing](#). Avatar pricing is only visible for service regions where the feature is available. For the current list of supported regions, see the [Speech service regions table](#).

Available locations

For the current list of regions that support text to speech avatar, see the [Speech service regions table](#).

Responsible AI

We care about the people who use AI and the people who will be affected by it as much as we care about technology. For more information, see the Responsible AI [transparency notes](#) and [disclosure for voice and avatar talent](#).

Next steps

- [Use batch synthesis for text to speech avatar](#)
 - [What is custom text to speech avatar](#)
-

Content credentials

08/07/2025

Content credentials help you verify that Azure text to speech avatar generated your video content. This transparency feature automatically adds tamper-evident information about your video's origin and creation history, following industry standards from the [Coalition for Content Provenance and Authenticity \(C2PA\)](#).

Use content credentials to:

- Confirm video authenticity for your audiences
- Meet transparency requirements for AI-generated content
- Build trust in your avatar-powered solutions

Content credentials are automatically applied to all supported video formats—no additional setup required.

What are content credentials?

Azure text to speech avatar adds a content credentials manifest to each avatar video, giving you information about the video's origin. A cryptographic signature secures the manifest, linking it back to Azure text to speech avatar.

The manifest has several key pieces of information:

 Expand table

Field name	Field content
"generator"	This field has a value of "Microsoft Azure Txt to Speech Avatar Service" for all applicable videos, confirming the AI-generated nature of the video.
"when"	The timestamp of when the content credentials were created.

Content credentials in Azure text to speech avatar can help people understand when video content is generated by the Azure text to speech avatar system. For more information on how to responsibly build solutions with text to speech avatar models, see the [Text to speech transparency note](#).

Limitations

The content credentials are only supported in video files generated by batch synthesis of text to speech avatar, and only `mp4` file format is supported.

Use content credentials in your solution

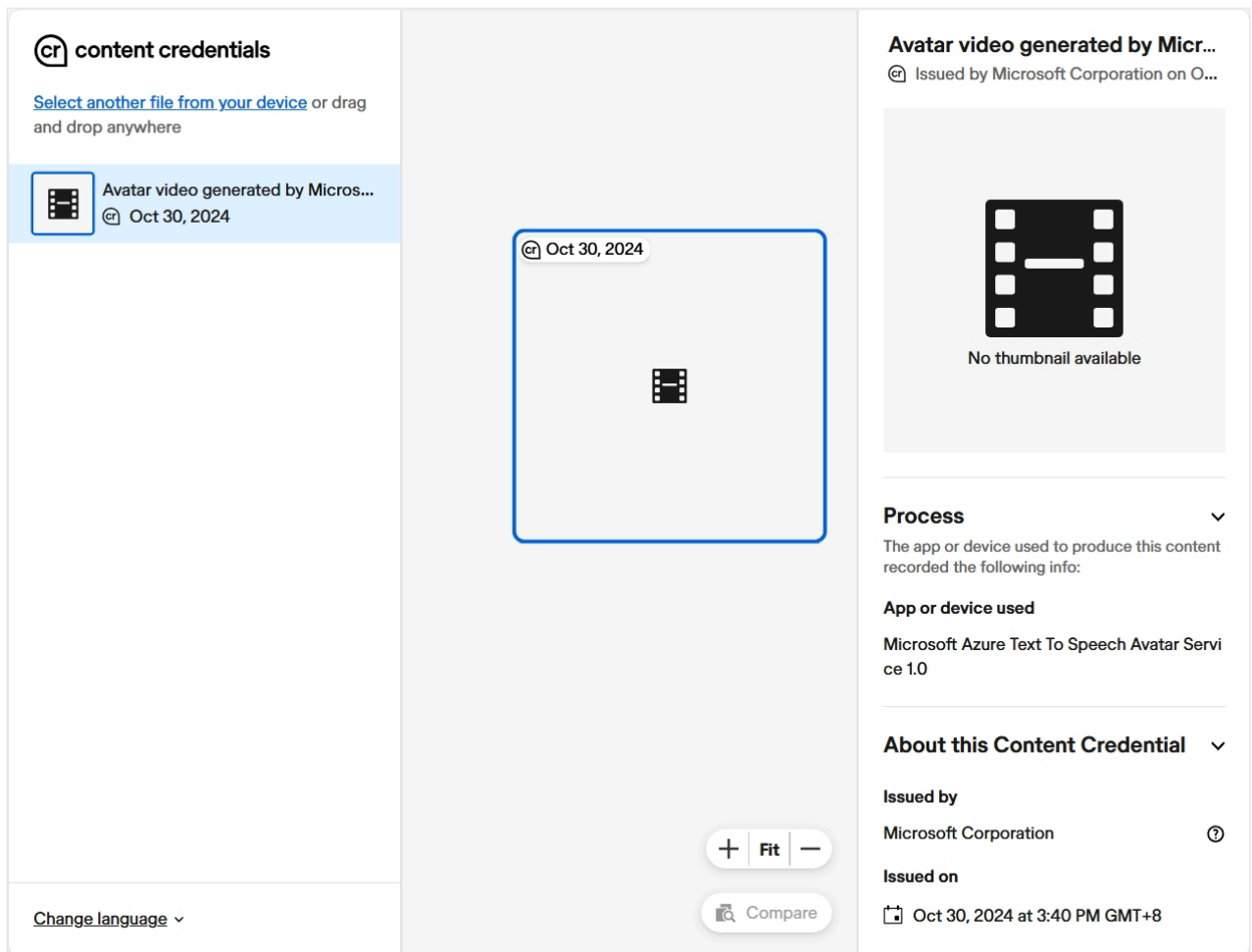
You can use content credentials by ensuring that your Azure text to speech avatar generated video files have content credentials.

No extra setup is necessary. Content credentials are automatically applied to all applicable videos generated by Azure text to speech avatar.

Check that a video file has content credentials

You can check the content credentials of a text to speech avatar using either of the following methods:

- **Content credentials verify webpage** (contentcredentials.org/verify): This tool lets users inspect the content credentials of content like a video. If an uploaded video of a text to speech avatar is created with the Speech service, the tool will show that and display the issued date and time of the content credentials. This screenshot shows an example avatar video upload result where the content credentials were issued by Microsoft.



This page shows that a video generated by Azure text to speech avatar has content credentials issued by Microsoft.

- **Content Authenticity Initiative (CAI) open-source tools:** The CAI provides multiple open-source tools that can be used to validate and display C2PA content credentials. Find the right tool for your application and [get started here](#).

Next steps

- [Use batch synthesis for text to speech avatar](#)

How to use text to speech avatar with real-time synthesis

This guide shows you how to use text to speech avatar with real-time synthesis. The avatar video is generated almost instantly after you enter text.

Prerequisites

You need:

- **Azure subscription:** [Create one for free](#) .
- **Speech resource:** [Create a speech resource](#) in the Azure portal. Select the **Standard S0** pricing tier to access avatars.
- **Speech resource key and region:** After deployment, select **Go to resource** to view and manage your keys.

Set up environment

To use real-time avatar synthesis, install the Speech SDK for JavaScript for your webpage. See [Install the Speech SDK](#).

Real-time avatar works on these platforms and browsers:

[Expand table](#)

Platform	Chrome	Microsoft Edge	Safari	Firefox	Opera
Windows	Y	Y	N/A	Y ¹	Y
Android	Y	Y	N/A	Y ¹²	N
iOS	Y	Y	Y	Y	Y
macOS	Y	Y	Y	Y ¹	Y

¹ Doesn't work with ICE server by Communication Service, but works with Coturn. ² Background transparency doesn't work.

Select text to speech language and voice

Speech service supports many [languages and voices](#). See the full list or try them in the [Voice Gallery](#) .

To match your input text and use a specific voice, set the `SpeechSynthesisLanguage` or `SpeechSynthesisVoiceName` properties in the `SpeechConfig` object:

JavaScript

```
const speechConfig = SpeechSDK.SpeechConfig.fromSubscription("YourSpeechKey",
  "YourSpeechRegion");
// Set either the `SpeechSynthesisVoiceName` or `SpeechSynthesisLanguage`.
speechConfig.speechSynthesisLanguage = "en-US";
speechConfig.speechSynthesisVoiceName = "en-US-AvaMultilingualNeural";
```

All neural voices are multilingual and fluent in their own language and English. For example, if you select **es-ES-ElviraNeural** and enter English text, the avatar speaks English with a Spanish accent.

If the voice doesn't support the input language, the Speech service doesn't create audio. See [Language and voice support](#) for the full list.

Default voice selection:

- If you don't set `SpeechSynthesisVoiceName` or `SpeechSynthesisLanguage`, the default voice in `en-US` is used.
- If you set only `SpeechSynthesisLanguage`, the default voice in that locale is used.
- If you set both, `SpeechSynthesisVoiceName` takes priority.
- If you use SSML to set the voice, both properties are ignored.

Select avatar character and style

See [supported avatar characters and styles](#).

Set avatar character and style:

JavaScript

```
const avatarConfig = new SpeechSDK.AvatarConfig(
  "lisa", // Set avatar character here.
  "casual-sitting", // Set avatar style here.
);
```

Set up connection to real-time avatar

Real-time avatar uses the WebRTC protocol to stream video. Set up the connection with the avatar service using a WebRTC peer connection.

First, create a WebRTC peer connection object. WebRTC is peer-to-peer and relies on an ICE server for network relay. Speech service provides a REST API to get ICE server info. We recommend fetching ICE server details from Speech service, but you can use your own.

Sample request to fetch ICE info:

HTTP

```
GET /cognitiveservices/avatar/relay/token/v1 HTTP/1.1
```

Host: westus2.tts.speech.microsoft.com

Ocp-Apim-Subscription-Key: YOUR_RESOURCE_KEY

Create the WebRTC peer connection using the ICE server URL, username, and credential from the previous response:

JavaScript

```
// Create WebRTC peer connection
peerConnection = new RTCPeerConnection({
  iceServers: [{
    urls: [ "Your ICE server URL" ],
    username: "Your ICE server username",
    credential: "Your ICE server credential"
  }]
})
```

ⓘ Note

The ICE server URL can start with `turn` (for example, `turn:relay.communication.microsoft.com:3478`) or `stun` (for example, `stun:relay.communication.microsoft.com:3478`). For `urls`, include only the `turn` URL.

Next, set up the video and audio player elements in the `ontrack` callback of the peer connection. This callback runs twice—once for video, once for audio. Create both player elements in the callback:

JavaScript

```
// Fetch WebRTC video/audio streams and mount them to HTML video/audio player elements
peerConnection.ontrack = function (event) {
  if (event.track.kind === 'video') {
    const videoElement = document.createElement(event.track.kind)
    videoElement.id = 'videoPlayer'
    videoElement.srcObject = event.streams[0]
```

```

        videoElement.autoplay = true
    }

    if (event.track.kind === 'audio') {
        const audioElement = document.createElement(event.track.kind)
        audioElement.id = 'audioPlayer'
        audioElement.srcObject = event.streams[0]
        audioElement.autoplay = true
    }
}

// Offer to receive one video track, and one audio track
peerConnection.addTransceiver('video', { direction: 'sendrecv' })
peerConnection.addTransceiver('audio', { direction: 'sendrecv' })

```

Then, use Speech SDK to create an avatar synthesizer and connect to the avatar service with the peer connection:

JavaScript

```

// Create avatar synthesizer
var avatarSynthesizer = new SpeechSDK.AvatarSynthesizer(speechConfig, avatarConfig)

// Start avatar and establish WebRTC connection
avatarSynthesizer.startAvatarAsync(peerConnection).then(
    (r) => { console.log("Avatar started.") }
).catch(
    (error) => { console.log("Avatar failed to start. Error: " + error) }
);

```

The real-time API disconnects after 5 minutes of idle or after 30 minutes of connection. To keep the avatar running longer, enable automatic reconnect. See this [JavaScript sample code](#) (search "auto reconnect").

Synthesize talking avatar video from text input

After setup, the avatar video plays in your browser. The avatar blinks and moves slightly, but doesn't speak until you send text input.

Send text to the avatar synthesizer to make the avatar speak:

JavaScript

```

var spokenText = "I'm excited to try text to speech avatar."
avatarSynthesizer.speakTextAsync(spokenText).then(
    (result) => {
        if (result.reason === SpeechSDK.ResultReason.SynthesizingAudioCompleted) {
            console.log("Speech and avatar synthesized to video stream.")
        } else {

```

```

        console.log("Unable to speak. Result ID: " + result.resultId)
        if (result.reason === SpeechSDK.ResultReason.Canceled) {
            let cancellationDetails =
SpeechSDK.CancellationDetails.fromResult(result)
            console.log(cancellationDetails.reason)
            if (cancellationDetails.reason ===
SpeechSDK.CancellationReason.Error) {
                console.log(cancellationDetails.errorDetails)
            }
        }
    })
}).catch((error) => {
    console.log(error)
    avatarSynthesizer.close()
});

```

Close the real-time avatar connection

To avoid extra costs, close the connection when you're done:

- Closing the browser releases the WebRTC peer connection and closes the avatar connection after a few seconds.
- The connection closes automatically if the avatar is idle for 5 minutes.
- You can close the avatar connection manually:

JavaScript

```
avatarSynthesizer.close()
```

Edit background

Set background color

Set the background color of the avatar video using the `backgroundColor` property of `AvatarConfig`:

JavaScript

```

const avatarConfig = new SpeechSDK.AvatarConfig(
    "lisa", // Set avatar character here.
    "casual-sitting", // Set avatar style here.
)
avatarConfig.backgroundColor = '#00FF00FF' // Set background color to green

```

ⓘ Note

The color string should be in the format `#RRGGBBAA`. The alpha channel (`AA`) is ignored—transparent backgrounds aren't supported for real-time avatar.

Set background image

Set the background image using the `backgroundImage` property of `AvatarConfig`. Upload your image to a public URL and assign it to `backgroundImage`:

JavaScript

```
const avatarConfig = new SpeechSDK.AvatarConfig(
  "lisa", // Set avatar character here.
  "casual-sitting", // Set avatar style here.
)
avatarConfig.backgroundImage = "https://www.example.com/1920-1080-image.jpg" // A
public accessible URL of the image.
```

Set background video

The API doesn't support background video directly, but you can customize the background on the client side:

- Set the background color to green (for easy matting).
- Create a canvas element the same size as the avatar video.
- For each frame, set green pixels to transparent and draw the frame to the canvas.
- Hide the original video.

This gives you a transparent avatar on a canvas. See [JavaScript sample code](#).

You can then place any dynamic content (like a video) behind the canvas.

Crop video

The avatar video is 16:9 by default. To crop to a different aspect ratio, specify the rectangle area using the coordinates of the top-left and bottom-right corners:

JavaScript

```
const videoFormat = new SpeechSDK.AvatarVideoFormat()
const topLeftCoordinate = new SpeechSDK.Coordinate(640, 0) // coordinate of top-left
vertex, with X=640, Y=0
```



```
const bottomRightCoordinate = new SpeechSDK.Coordinate(1320, 1080) // coordinate of
bottom-right vertex, with X=1320, Y=1080
videoFormat.setCropRange(topLeftCoordinate, bottomRightCoordinate)
const avatarConfig = new SpeechSDK.AvatarConfig(
  "lisa", // Set avatar character here.
  "casual-sitting", // Set avatar style here.
  videoFormat, // Set video format here.
)
```

For a full sample, see our [code example](#) and search for `crop`.

Code samples

Find text to speech avatar code samples in the Speech SDK GitHub repository. These samples show how to use real-time avatars in web and mobile apps:

- **Server + client**
 - [Python \(server\) + JavaScript \(client\)](#)
 - [C# \(server\) + JavaScript \(client\)](#)
- **Client only**
 - [JavaScript](#)
 - [Android](#)
 - [iOS](#)

Next steps

- [What is text to speech avatar](#)
- [Install the Speech SDK](#)

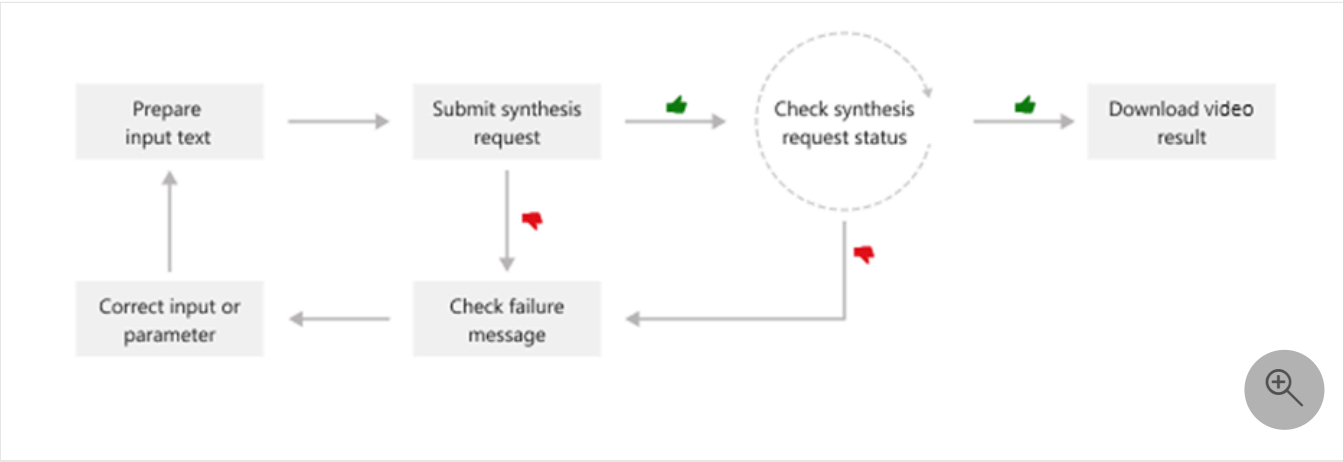
Last updated on 11/09/2025

Use batch synthesis for text to speech avatar

The batch synthesis API for text to speech avatar lets you synthesize text asynchronously into a talking avatar as a video file. Publishers and video content platforms can use this API to create avatar video content in a batch. That approach can be suitable for different use cases like training materials, presentations, or advertisements.

The synthetic avatar video will be generated asynchronously after the system receives text input. The generated video output can be downloaded in batch mode synthesis. You submit text for synthesis, poll for the synthesis status, and download the video output when the status shows success. The text input formats must be plain text or Speech Synthesis Markup Language (SSML) text.

This diagram provides a high-level overview of the workflow.



To run batch synthesis, you can use the following REST API operations.

[Expand table](#)

Operation	Method	REST API call
Create batch synthesis	PUT	avatar/batchsyntheses/{SynthesisId}?api-version=2024-08-01
Get batch synthesis	GET	avatar/batchsyntheses/{SynthesisId}?api-version=2024-08-01
List batch synthesis	GET	avatar/batchsyntheses/?api-version=2024-08-01
Delete batch synthesis	DELETE	avatar/batchsyntheses/{SynthesisId}?api-version=2024-08-01

You can refer to the code samples on [GitHub](#).

Create a batch synthesis request

Some properties in JSON format are required when you create a new batch synthesis job. Other properties are optional. The [batch synthesis response](#) includes other properties to provide information about the synthesis status and results. For example, the `outputs.result` property has the location from [where you can download a video file](#) containing the avatar video. From `outputs.summary`, you can get the summary and debug details.

To submit a batch synthesis request, construct the HTTP POST request body following these instructions:

- Set the required `inputKind` property.
- If the `inputKind` property is set to `PlainText`, you must also set the `voice` property in the `synthesisConfig`. In the following example, the `inputKind` is set to `SSML`, so the `speechSynthesis` isn't set.
- Set the required `SynthesisId` property. Choose a unique `SynthesisId` for the same speech resource. The `SynthesisId` can be a string of 3 to 64 characters, including letters, numbers, '-', or '_', with the condition that it must start and end with a letter or number.
- Set the required `talkingAvatarCharacter` and `talkingAvatarStyle` properties. You can find supported avatar characters and styles [here](#).
- Optionally, you can set the `videoFormat`, `backgroundColor`, and other properties. For more information, see [batch synthesis properties](#).

ⓘ Note

The maximum JSON payload size accepted is 500 kilobytes.

Each Speech resource can have up to 200 batch synthesis jobs running concurrently.

The maximum length for the output video is currently 20 minutes, with potential increases in the future.

To make an HTTP PUT request, use the URI format shown in the following example. Replace `YourSpeechKey` with your Speech resource key, `YourSpeechRegion` with your Speech resource region, and set the request body properties as described previously.

Azure CLI

```
curl -v -X PUT -H "Ocp-Apim-Subscription-Key: YourSpeechKey" -H "Content-Type: application/json" -d '{
  "inputKind": "SSML",
  "inputs": [
    {
      "content": "<speak version='\"'1.0'\"' xml:lang='\"'en-US'\"'><voice name='\"'en-US-AvaMultilingualNeural'\"'>The rainbow has seven colors.</voice>
```

```

</speak>"
    }
  ],
  "avatarConfig": {
    "talkingAvatarCharacter": "lisa",
    "talkingAvatarStyle": "graceful-sitting"
  }
}' "https://YourSpeechRegion.api.cognitive.microsoft.com/avatar/batchsyntheses/my-
job-01?api-version=2024-08-01"

```

You should receive a response body in the following format:

JSON

```

{
  "id": "my-job-01",
  "internalId": "5a25b929-1358-4e81-a036-33000e788c46",
  "status": "NotStarted",
  "createdDateTime": "2024-03-06T07:34:08.9487009Z",
  "lastActionDateTime": "2024-03-06T07:34:08.9487012Z",
  "inputKind": "SSML",
  "customVoices": {},
  "properties": {
    "timeToLiveInHours": 744,
  },
  "avatarConfig": {
    "talkingAvatarCharacter": "lisa",
    "talkingAvatarStyle": "graceful-sitting",
    "videoFormat": "Mp4",
    "videoCodec": "hevc",
    "subtitleType": "soft_embedded",
    "bitrateKbps": 2000,
    "customized": false
  }
}

```

The `status` property should progress from `NotStarted` status to `Running` and finally to `Succeeded` or `Failed`. You can periodically call the [GET batch synthesis API](#) until the returned status is `Succeeded` or `Failed`.

Get batch synthesis

To get the status of a batch synthesis job, make an HTTP GET request using the URI as shown in the following example.

Replace `YourSynthesisId` with your batch synthesis ID, `YourSpeechKey` with your Speech resource key, and `YourSpeechRegion` with your Speech resource region.

Azure CLI

```
curl -v -X GET  
"https://YourSpeechRegion.api.cognitive.microsoft.com/avatar/batchsyntheses/YourSynthesisId?api-version=2024-08-01" -H "Ocp-Apim-Subscription-Key: YourSpeechKey"
```

You should receive a response body in the following format:

JSON

```
{  
  "id": "my-job-01",  
  "internalId": "5a25b929-1358-4e81-a036-33000e788c46",  
  "status": "Succeeded",  
  "createdDateTime": "2024-03-06T07:34:08.9487009Z",  
  "lastActionDateTime": "2024-03-06T07:34:12.5698769",  
  "inputKind": "SSML",  
  "customVoices": {},  
  "properties": {  
    "timeToLiveInHours": 744,  
    "sizeInBytes": 344460,  
    "durationInMilliseconds": 2520,  
    "succeededCount": 1,  
    "failedCount": 0,  
    "billingDetails": {  
      "neuralCharacters": 29,  
      "talkingAvatarDurationSeconds": 2  
    }  
  },  
  "avatarConfig": {  
    "talkingAvatarCharacter": "lisa",  
    "talkingAvatarStyle": "graceful-sitting",  
    "videoFormat": "Mp4",  
    "videoCodec": "hevc",  
    "subtitleType": "soft_embedded",  
    "bitrateKbps": 2000,  
    "customized": false  
  },  
  "outputs": {  
    "result": "https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-output/xxxxx/xxxxx/0001.mp4?SAS_Token",  
    "summary": "https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-output/xxxxx/xxxxx/summary.json?SAS_Token"  
  }  
}
```

From the `outputs.result` field, you can download a video file containing the avatar video. The `outputs.summary` field lets you download the summary and debug details. For more information on batch synthesis results, see [batch synthesis results](#).

List batch synthesis

To list all batch synthesis jobs for your Speech resource, make an HTTP GET request using the URI as shown in the following example.

Replace `YourSpeechKey` with your Speech resource key and `YourSpeechRegion` with your Speech resource region. Optionally, you can set the `skip` and `top` (page size) query parameters in the URL. The default value for `skip` is 0, and the default value for `maxpagesize` is 100.

Azure CLI

```
curl -v -X GET
"https://YourSpeechRegion.api.cognitive.microsoft.com/avatar/batchsyntheses?
skip=0&maxpagesize=2&api-version=2024-08-01" -H "Ocp-Apim-Subscription-Key:
YourSpeechKey"
```

You receive a response body in the following format:

JSON

```
{
  "value": [
    {
      "id": "my-job-02",
      "internalId": "14c25fcf-3cb6-4f46-8810-ecad06d956df",
      "status": "Succeeded",
      "createdDateTime": "2024-03-06T07:52:23.9054709Z",
      "lastActionDateTime": "2024-03-06T07:52:29.3416944",
      "inputKind": "SSML",
      "customVoices": {},
      "properties": {
        "timeToLiveInHours": 744,
        "sizeInBytes": 502676,
        "durationInMilliseconds": 2950,
        "succeededCount": 1,
        "failedCount": 0,
        "billingDetails": {
          "neuralCharacters": 32,
          "talkingAvatarDurationSeconds": 2
        }
      },
      "avatarConfig": {
        "talkingAvatarCharacter": "lisa",
        "talkingAvatarStyle": "casual-sitting",
        "videoFormat": "Mp4",
        "videoCodec": "h264",
        "subtitleType": "soft_embedded",
        "bitrateKbps": 2000,
        "customized": false
      }
    },
  ],
}
```

```

        "outputs": {
            "result":
"https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-
output/xxxxx/xxxxx/0001.mp4?SAS_Token",
            "summary":
"https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-
output/xxxxx/xxxxx/summary.json?SAS_Token"
        }
    },
    {
        "id": "my-job-01",
        "internalId": "5a25b929-1358-4e81-a036-33000e788c46",
        "status": "Succeeded",
        "createdDateTime": "2024-03-06T07:34:08.9487009Z",
        "lastActionDateTime": "2024-03-06T07:34:12.5698769",
        "inputKind": "SSML",
        "customVoices": {},
        "properties": {
            "timeToLiveInHours": 744,
            "sizeInBytes": 344460,
            "durationInMilliseconds": 2520,
            "succeededCount": 1,
            "failedCount": 0,
            "billingDetails": {
                "neuralCharacters": 29,
                "talkingAvatarDurationSeconds": 2
            }
        },
        "avatarConfig": {
            "talkingAvatarCharacter": "lisa",
            "talkingAvatarStyle": "graceful-sitting",
            "videoFormat": "Mp4",
            "videoCodec": "hevc",
            "subtitleType": "soft_embedded",
            "bitrateKbps": 2000,
            "customized": false
        },
        "outputs": {
            "result":
"https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-
output/xxxxx/xxxxx/0001.mp4?SAS_Token",
            "summary":
"https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-
output/xxxxx/xxxxx/summary.json?SAS_Token"
        }
    }
],
"nextLink":
"https://YourSpeechRegion.api.cognitive.microsoft.com/avatar/batchsyntheses/?api-
version=2024-08-01&skip=2&maxpagesize=2"
}

```

From `outputs.result`, you can download a video file containing the avatar video. From `outputs.summary`, you can get the summary and debug details. For more information, see [batch](#)

[synthesis results](#).

The `value` property in the JSON response lists your synthesis requests. The list is paginated, with a maximum page size of 100. The `nextLink` property is provided as needed to get the next page of the paginated list.

Get batch synthesis results file

Once you get a batch synthesis job with `status` of "Succeeded", you can download the video output results. Use the URL from the `outputs.result` property of the [get batch synthesis](#) response.

To get the batch synthesis results file, make an HTTP GET request using the URI as shown in the following example. Replace `YourOutputsResultUrl` with the URL from the `outputs.result` property of the [get batch synthesis](#) response. Replace `YourSpeechKey` with your Speech resource key.

Azure CLI

```
curl -v -X GET "YourOutputsResultUrl" -H "Ocp-Apim-Subscription-Key: YourSpeechKey" > output.mp4
```

To get the batch synthesis summary file, make an HTTP GET request using the URI as shown in the following example. Replace `YourOutputsSummaryUrl` with the URL from the `outputs.summary` property of the [get batch synthesis](#) response. Replace `YourSpeechKey` with your Speech resource key.

Azure CLI

```
curl -v -X GET "YourOutputsSummaryUrl" -H "Ocp-Apim-Subscription-Key: YourSpeechKey" > summary.json
```

The summary file has the synthesis results for each text input. Here's an example `summary.json` file:

JSON

```
{
  "jobID": "5a25b929-1358-4e81-a036-33000e788c46",
  "status": "Succeeded",
  "results": [
    {
      "texts": [
        "<speak version='1.0' xml:lang='en-US'><voice name='en-US-AvaMultilingualNeural'>The rainbow has seven colors.</voice></speak>"
      ]
    }
  ]
}
```



```
    ],  
    "status": "Succeeded",  
    "videoFileName": "244a87c294b94ddeb3dbaccee8ffa7eb/5a25b929-1358-4e81-a036-  
33000e788c46/0001.mp4",  
    "TalkingAvatarCharacter": "lisa",  
    "TalkingAvatarStyle": "graceful-sitting"  
  }  
]  
}
```

Delete batch synthesis

After you get the audio output results and no longer need the batch synthesis job history, you can delete it. The Speech service keeps each synthesis history for up to 31 days or the duration specified by the request's `timeToLiveInHours` property, whichever comes sooner. The date and time of automatic deletion for synthesis jobs with a status of "Succeeded" or "Failed" is calculated as the sum of the `lastActionDateTime` and `timeToLive` properties.

To delete a batch synthesis job, make an HTTP DELETE request using the following URI format. Replace `YourSynthesisId` with your batch synthesis ID, `YourSpeechKey` with your Speech resource key, and `YourSpeechRegion` with your Speech resource region.

Azure CLI

```
curl -v -X DELETE  
"https://YourSpeechRegion.api.cognitive.microsoft.com/avatar/batchsyntheses/YourSynthesisId?api-version=2024-08-01" -H "Ocp-Apim-Subscription-Key: YourSpeechKey"
```

The response headers include `HTTP/1.1 204 No Content` if the delete request was successful.

Next steps

- [Batch synthesis properties](#)
- [Use batch synthesis for text to speech avatar](#)
- [What is text to speech avatar](#)

Batch synthesis properties for text to speech avatar

08/07/2025

Batch synthesis properties can be grouped as: avatar related properties, batch job related properties, and text to speech related properties, which are described in the following tables.

Some properties in JSON format are required when you create a new batch synthesis job. Other properties are optional. The batch synthesis response includes other properties to provide information about the synthesis status and results. For example, the `outputs.result` property contains the location from where you can download a video file containing the avatar video. From `outputs.summary`, you can access the summary and debug details.

Avatar properties

The following table describes the avatar properties.

 Expand table

Property	Description
<code>avatarConfig.talkingAvatarCharacter</code>	The character name of the talking avatar. For standard avatar, the supported avatar characters can be found here. For custom avatar, specify the avatar model name. This property is required.
<code>avatarConfig.talkingAvatarStyle</code>	The style name of the talking avatar. For standard avatar, the supported avatar styles can be found here. For custom avatar, this property should be omitted. This property is required for standard avatar.
<code>avatarConfig.customized</code>	A bool value indicating whether the avatar to be used is customized avatar or not. True for customized avatar, and false for standard avatar. This property is optional, and the default value is <code>false</code>.

Property	Description
avatarConfig.videoFormat	The format for output video file could be mp4 or webm. The <code>webm</code> format is required for a transparent background. This property is optional, and the default value is mp4.
avatarConfig.videoCodec	The codec for output video, could be h264, hevc, vp9 or av1. Vp9 is required for a transparent background. The synthesis speed is slower with the vp9 codec because vp9 encoding is slower. This property is optional, and the default value is hevc.
avatarConfig.bitrateKbps	The bitrate for output video, which is integer value, with unit kbps. This property is optional, and the default value is 2000.
avatarConfig.videoCrop	This property lets you crop the video output, which means to output a rectangle subarea of the original video. This property has two fields, which define the top-left vertex and bottom-right vertex of the rectangle. This property is optional, and the default behavior is to output the full video.
avatarConfig.videoCrop.topLeft	The top-left vertex of the rectangle for video crop. This property has two fields x and y, to define the horizontal and vertical position of the vertex. This property is required when properties.videoCrop is set.
avatarConfig.videoCrop.bottomRight	The bottom-right vertex of the rectangle for video crop. This property has two fields x and y, to define the horizontal and vertical position of the vertex. This property is required when properties.videoCrop is set.
avatarConfig.subtitleType	Type of subtitle for the avatar video file could be <code>external_file</code>, <code>soft_embedded</code>, <code>hard_embedded</code>, or <code>none</code>. This property is optional, and the default value is <code>soft_embedded</code>.
avatarConfig.backgroundImage	Add a background image using the <code>avatarConfig.backgroundImage</code> property. The value of the property should be a URL pointing to the desired image. This property is optional.
avatarConfig.backgroundColor	Background color of the avatar video, which is a string in #RRGGBBAA format. In this string: RR, GG, BB and AA mean the red, green, blue, and alpha channels, with hexadecimal value

Property	Description
	range 00~FF. Alpha channel controls the transparency, with value 00 for transparent, value FF for non-transparent, and value between 00 and FF for semi-transparent. This property is optional, and the default value is #FFFFFF (white).
avatarConfig.useBuiltInVoice	A boolean value indicating whether to use the voice sync for avatar as the synthesis voice. This property can only be used when using custom avatar trained with voice sync for avatar. When set to <code>true</code>, other voices specified in <code>synthesisConfig</code> or in SSML will be ignored. This property is optional, and the default value is <code>false</code>.
outputs.result	The location of the batch synthesis result file, which is a video file containing the synthesized avatar. This property is read-only.
properties.DurationInMilliseconds	The video output duration in milliseconds. This property is read-only.

Batch synthesis job properties

The following table describes the batch synthesis job properties.

[Expand table](#)

Property	Description
createdDateTime	The date and time when the batch synthesis job was created. This property is read-only.
description	The description of the batch synthesis. This property is optional.
ID	The batch synthesis job ID. This property is read-only.
lastActionDateTime	The most recent date and time when the status property value changed.

Property	Description
	This property is read-only.
properties	A defined set of optional batch synthesis configuration settings.
properties.destinationContainerUrl	The batch synthesis results can be stored in a writable Azure Blob Storage container. If you don't specify a container URI with shared access signatures (SAS) token, the Speech service stores the results in a container managed by Microsoft. SAS with stored access policies isn't supported. When the synthesis job is deleted, the result data is also deleted. This property is required when generating multiple videos in one job. For single video generation, this property is optional. This property is not included in the response when you get the synthesis job.
properties.destinationPath	The prefix path for storing batch synthesis results. If no prefix path is provided, a system-generated path will be used. This property is optional and can only be set when the <code>destinationContainerUrl</code> property is specified.
properties.timeToLiveInHours	A duration in hours after the synthesis job is created, when the synthesis results will be automatically deleted. The maximum time to live is 744 hours. The date and time of automatic deletion for synthesis jobs with a status of "Succeeded" or "Failed" is calculated as the sum of the lastActionDateTime and timeToLive properties. Otherwise, you can call the delete synthesis method to remove the job sooner.
status	The batch synthesis processing status. The status should progress from "NotStarted" to "Running", and finally to either "Succeeded" or "Failed". This property is read-only.

Text to speech properties

The following table describes the text to speech properties.

 Expand table

Property	Description
customVoices	A custom voice is associated with a name and its deployment ID, like this: "customVoices": {"your-custom-voice-name": "502ac834-6537-4bc3-9fd6-140114daa66d"} You can use the voice name in your <code>synthesisConfig.voice</code> when <code>inputKind</code> is set to "PlainText", or within SSML text of inputs when <code>inputKind</code> is set to "SSML". This property is required to use a custom voice. If you try to use a custom voice that isn't defined here, the service returns an error.
inputs	The plain text or SSML to be synthesized. When the <code>inputKind</code> is set to "PlainText", provide plain text as shown here: "inputs": [{"content": "The rainbow has seven colors."}]. When the <code>inputKind</code> is set to "SSML", provide text in the Speech Synthesis Markup Language (SSML) as shown here: "inputs": [{"content": "<speak version='1.0' xml:lang='en-US'><voice xml:lang='en-US' xml:gender='Female' name='en-US-AvaMultilingualNeural'>The rainbow has seven colors.</voice></speak>"}]. Include up to 1,000 text objects if you want multiple video output files. Here's example input text that should be synthesized to two video output files: "inputs": [{"content": "synthesize this to a file"}, {"content": "synthesize this to another file"}]. To generate multiple videos, the output must be stored in an Azure Blob Storage container by specifying <code>properties.destinationContainerUrl</code>. You don't need separate text inputs for new paragraphs. Within any of the (up to 1,000) text inputs, you can specify new paragraphs using the "\r\n" (newline) string. Here's example input text with two paragraphs that should be synthesized to the same audio output file: "inputs": [{"content": "synthesize this to a file\r\nsynthesize this to another paragraph in the same file"}]. This property is required when you create a new batch synthesis job. This property isn't included in the response when you get the synthesis job.
properties.billingDetails	The number of words that were processed and billed by <code>customNeural</code> (custom voice) versus <code>neural</code> (standard voice). This property is read-only.
synthesisConfig	The configuration settings to use for batch synthesis of plain text. This property is only applicable when <code>inputKind</code> is set to "PlainText".

Property	Description
synthesisConfig.pitch	The pitch of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when inputKind is set to "PlainText".
synthesisConfig.rate	The rate of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when inputKind is set to "PlainText".
synthesisConfig.style	For some voices, you can adjust the speaking style to express different emotions like cheerfulness, empathy, and calm. You can optimize the voice for different scenarios like customer service, newscast, and voice assistant. For information about the available styles per voice, see voice styles and roles. This optional property is only applicable when inputKind is set to "PlainText".
synthesisConfig.voice	The voice that speaks the audio output. For information about the available standard voices, see language and voice support. To use a custom voice, you must specify a valid custom voice and deployment ID mapping in the customVoices property. This property is required when inputKind is set to "PlainText".
synthesisConfig.volume	The volume of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when inputKind is set to "PlainText".
inputKind	Indicates whether the inputs text property should be plain text or SSML. The possible case-insensitive values are "PlainText" and "SSML". When the inputKind is set to "PlainText", you must also set the synthesisConfig voice property. This property is required.

Edit the background

The avatar batch synthesis API currently doesn't support setting background videos; it only supports static background images. But if you want to add a background for your video during post-production, you can generate videos with a transparent background.

To set a static background image, use the `avatarConfig.backgroundImage` property and specify a URL pointing to the desired image. Additionally, you can set the background color of the avatar video using the `avatarConfig.backgroundColor` property.

To generate a transparent background video, you must set the following properties to the required values in the batch synthesis request:

[Expand table](#)

Property	Required values for background transparency
<code>properties.videoFormat</code>	<code>webm</code>
<code>properties.videoCodec</code>	<code>vp9</code>
<code>properties.backgroundColor</code>	<code>#00000000</code> (or <code>transparent</code>)

Clipchamp is one example of a video editing tool that supports the transparent background video generated by the batch synthesis API.

Some video editing software doesn't support the `webm` format directly and only supports `.mov` format transparent background video input, like Adobe Premiere Pro. In such cases, you first need to convert the video format from `webm` to `.mov` with a tool like FFMPEG.

FFMPEG command line:

Bash

```
ffmpeg -vcodec libvpx-vp9 -i <input.webm> -vcodec png -pix_fmt rgba metadata:s:v:0 alpha_mode="1" <output.mov>
```

FFMPEG can be downloaded from ffmpeg.org. Replace `<input.webm>` and `<output.mov>` with your local path and file name in the command line.

Next steps

- [Create an avatar with batch synthesis](#)
- [What is text to speech avatar](#)

Customize text to speech avatar gestures with SSML

The [Speech Synthesis Markup Language \(SSML\)](#) with input text determines the structure, content, and other characteristics of the text to speech output. Most SSML tags also work in text to speech avatar. Furthermore, text to speech avatar batch mode provides avatar gesture insertion by using the SSML bookmark element with the format `<bookmark mark='gesture.*' />`.

A gesture starts at the insertion point in time. If the gesture takes more time than the audio, the gesture is cut at the point in time when the audio is finished.

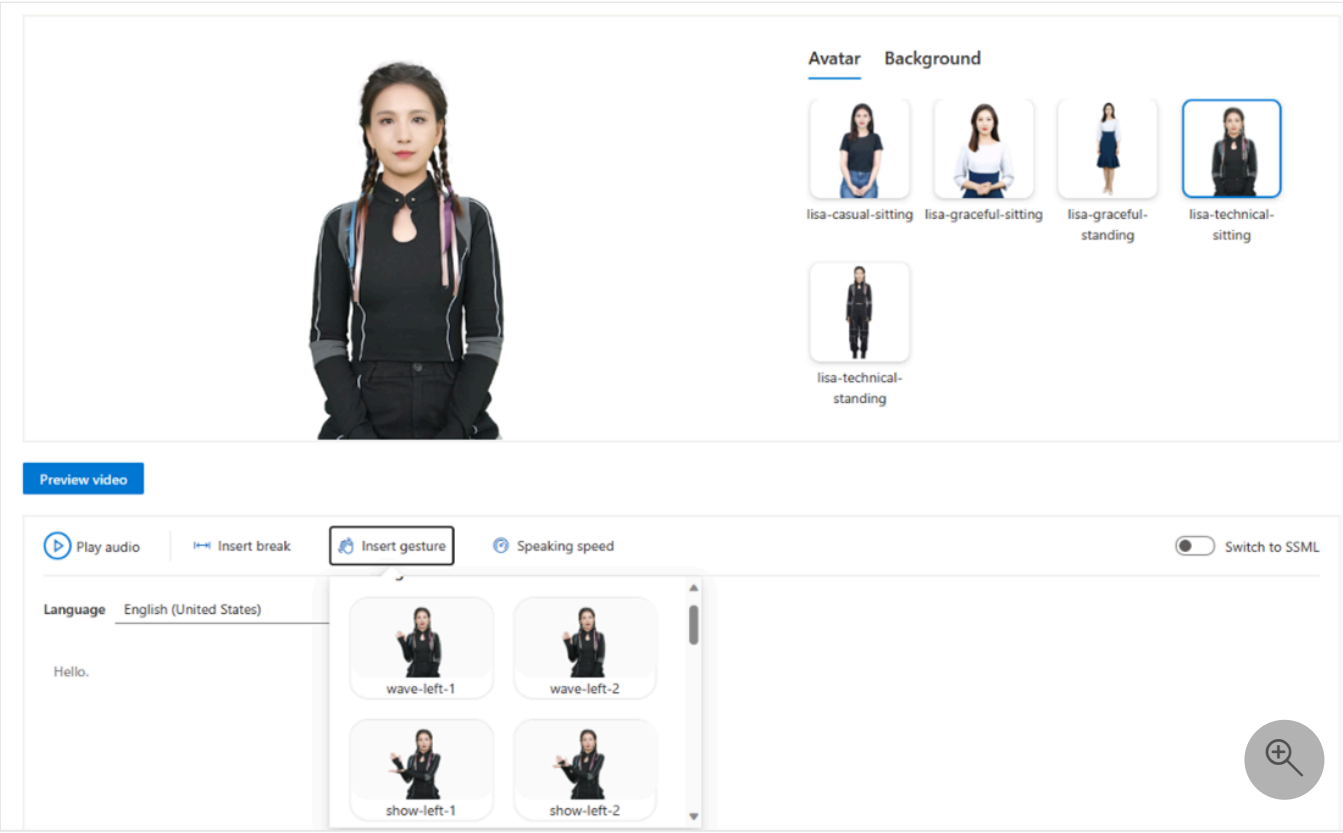
Bookmark example

The following example shows how to insert a gesture in the text to speech avatar batch synthesis with SSML.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
<voice name="en-US-AvaMultilingualNeural">
Hello <bookmark mark='gesture.wave-left-1' />, my name is Ava, nice to meet you!
</voice>
</speak>
```

In this example, the avatar starts waving their hand at the left after the word "Hello".



ⓘ Note

Gesture feature isn't currently supported when a voice sync for avatar is selected in a custom text to speech avatar.

Supported standard avatar characters, styles, and gestures

The full list of standard avatar supported gestures provided here can also be found in the text to speech avatar portal.

 Expand table

Characters	Styles	Gestures
Harry	business	123 calm-down come-on five-star-reviews good hello introduce invite

Characters	Styles	Gestures
		thanks welcome
Harry	casual	123 come-on five-star-reviews gong-xi-fa-cai good happy-new-year hello please welcome
Harry	youthful	123 come-on down five-star good hello invite show-right-up-down welcome
Jeff	business	123 come-on five-star-reviews hands-up here meddle please2 show silence thanks
Jeff	formal	123 come-on five-star-reviews lift please silence thanks very-good
Lisa	casual-sitting	numeric1-left-1 numeric2-left-1 numeric3-left-1 thumbsup-left-1 show-front-1 show-front-2

Characters	Styles	Gestures
		show-front-3 show-front-4 show-front-5 think-twice-1 show-front-6 show-front-7 show-front-8 show-front-9
Lisa	graceful-sitting	wave-left-1 wave-left-2 thumbsup-left show-left-1 show-left-2 show-left-3 show-left-4 show-left-5 show-right-1 show-right-2 show-right-3 show-right-4 show-right-5
Lisa	graceful-standing	
Lisa	technical-sitting	wave-left-1 wave-left-2 show-left-1 show-left-2 point-left-1 point-left-2 point-left-3 point-left-4 point-left-5 point-left-6 show-right-1 show-right-2 show-right-3 point-right-1 point-right-2 point-right-3 point-right-4 point-right-5 point-right-6
Lisa	technical-standing	
Lori	casual	123-left a-little

Characters	Styles	Gestures
		beg calm-down come-on five-star-reviews good hello open please thanks
Lori	graceful	123-left applaud come-on introduce nod please show-left show-right thanks welcome
Lori	formal	123 come-on come-on-left down five-star good hands-triangle hands-up hi hopeful thanks
Max	business	a-little-bit click-the-link display-number encourage-1 encourage-2 five-star-praise front-right good-01 good-02 introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 left lower-left number-one

Characters	Styles	Gestures
		press-both-hands-down-1 press-both-hands-down-2 push-forward raise-ones-hand right say-hi shrug-ones-shoulders slide-from-left-to-right slide-to-the-left thanks the-front top-middle-and-bottom-left top-middle-and-bottom-right upper-left upper-right welcome
Max	casual	a-little-bit applaud click-the-link display-number encourage-1 encourage-2 five-star-praise front-left good-1 good-2 hello introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 introduction-to-products-4 left length nodding number-one press-both-hands-down raise-ones-hand right right-front shrug-ones-shoulders slide-from-left-to-right slide-to-the-left thanks the-front upper-left

Characters	Styles	Gestures
		upper-right welcome
Max	formal	a-little-bit click-the-link display-number encourage-1 encourage-2 five-star-praise front-left front-right good-1 good-2 introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 left lower-left lower-right press-both-hands-down push-forward right say-hi shrug-ones-shoulders slide-from-left-to-right slide-to-the-left the-front top-middle-and-bottom-right upper-left upper-right
Meg	formal	a-little-bit click-the-link display-number encourage-1 encourage-2 five-star-praise front-left front-right good-1 good-2 hands-forward introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 left number-one press-both-hands-down-1

Characters	Styles	Gestures
		press-both-hands-down-2 right say-hi shrug-ones-shoulders slide-from-left-to-right the-front upper-left upper-right
Meg	casual	a-little-bit click-the-link cross-hand display-number encourage-1 encourage-2 five-star-praise front-left front-right good-1 good-2 handclap introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 left length lower-left lower-right number-one press-both-hands-down right say-hi shrug-ones-shoulders slide-from-right-to-left slide-to-the-left spread-hands the-front top-middle-and-bottom-left top-middle-and-bottom-right upper-left upper-right
Meg	business	a-little-bit encourage-1 encourage-2 five-star-praise front-left front-right

Characters	Styles	Gestures
		good-1 good-2 introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 left length number-one press-both-hands-down-1 press-both-hands-down-2 raise-ones-hand right say-hi shrug-ones-shoulders slide-from-left-to-right slide-to-the-left spread-hands thanks the-front upper-left

All styles except `lisa-graceful-sitting`, `lisa-graceful-standing`, `lisa-technical-sitting`, and `lisa-technical-standing` are supported via the real-time text to speech API. Gestures are only supported with the batch synthesis API and aren't supported via the real-time API.

Next steps

- [What is text to speech avatar](#)
- [Real-time synthesis](#)
- [Use batch synthesis for text to speech avatar](#)

What is custom text to speech avatar?

Custom text to speech avatar allows you to create a customized, one-of-a-kind synthetic talking avatar for your application. With custom text to speech avatar, you can build a unique and natural-looking avatar for your product or brand by providing video recording data of your selected actors. The avatar is even more realistic if you also use a [professional voice or voice sync for avatar](#) for the same actor.

Important

Custom text to speech avatar access is [limited](#) based on eligibility and usage criteria. Request access on the [intake form](#) .

How does it work?

Creating a custom text to speech avatar requires at least 10 minutes of video recording of the avatar talent as training data, and you must first get consent from the actor talent.

The custom avatar model can support:

- Video generation via the [batch synthesis API](#).
- Live chat via the [streaming synthesis API](#).

Before you get started, here are some considerations:

Your use case: Do you want to use the avatar to create video content such as training material or a product introduction? Do you want to use the avatar as a virtual salesperson in a real-time conversation with your customers? There are some recording requirements for different use cases.

The look of the avatar: The custom text to speech avatar looks the same as the avatar talent in the training data, and we don't support customizing the appearance of the avatar model, such as clothes, hairstyle, etc. So if your application requires multiple styles of the same avatar, you should prepare training data for each style, as each style of an avatar is considered as a single avatar model.

The voice of the avatar: The custom text to speech avatar can work with standard voice, professional voice, and voice sync for avatar.

- Voice sync for avatar: A synthetic voice resembling the avatar talent's voice is trained alongside the custom avatar utilizing audio from the training video.

- **Professional voice:** Fine-tune a professional voice with more training data, providing a premium voice experience for your avatar, including natural conversations, multi-style, and multilingual support.

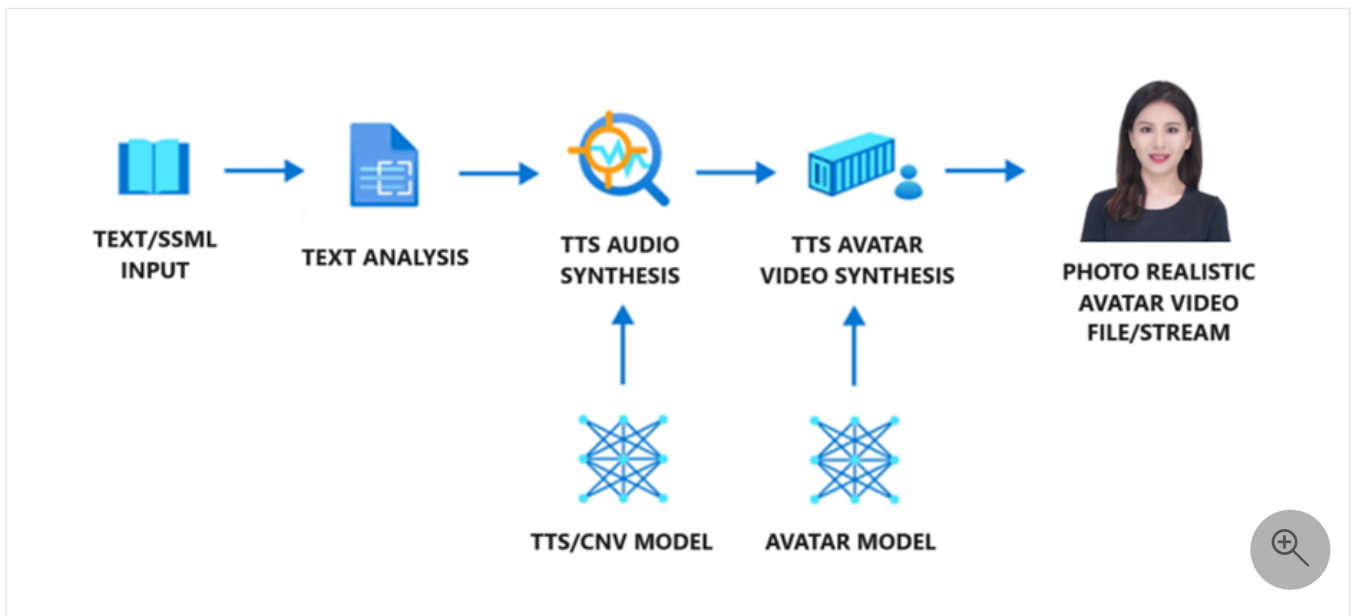
Here's an overview of the steps to create a custom text to speech avatar:

1. **Get consent video.** Obtain a video recording of the talent reading a consent statement. They must consent to the usage of their image and voice data to train a custom text to speech avatar model and a synthetic version of their voice.
2. **Prepare training data.** Ensure that the video recording is in the right format. It's a good idea to shoot the video recording in a professional-quality video shooting studio to get a clean background image. The quality of the resulting avatar heavily depends on the recorded video used for training. Factors like speaking rate, body posture, facial expression, hand gestures, consistency in the actor's position, and lighting of the video recording are essential to create an engaging custom text to speech avatar. See [how to prepare training data](#) for more details.
3. **Train the avatar model.** Once you have the data ready, upload your data to the [custom avatar portal](#) and start to train your model. Consent verification is conducted during the training. Make sure that you have access to the custom text to speech avatar feature before you can create a project.
4. **Deploy and use your avatar model in your applications.**

Components sequence

The custom text to speech avatar model contains three components: text analyzer, the text to speech audio synthesizer, and text to speech avatar video renderer.

- To generate an avatar video file or stream with the avatar model, text is first input into the text analyzer, which provides the output in the form of a phoneme sequence.
- The audio synthesizer synthesizes the speech audio for input text, and these two parts are provided by standard or custom voice models.
- Finally, the text to speech avatar model predicts the image of lip sync with the speech audio, so that the synthetic video is generated.



The text to speech avatar models are trained using deep neural networks based on the recording samples of human videos in different languages. All languages of standard voices and custom voices can be supported.

Available locations

For the current list of regions that support custom avatar training and usage, see the [Speech service regions table](#).

Custom voice and custom text to speech avatar

[Custom voice](#) and custom text to speech avatar are separate features. You can use them independently or together. If you're also creating a professional voice for the actor, the avatar can be highly realistic.

The custom text to speech avatar can work with a standard voice or custom voice as the avatar's voice. For more information, see [Avatar voice and language](#).

There are two kinds of custom voice for a custom avatar:

- **Voice sync for avatar:** When you enable the voice sync for avatar option during custom avatar training, a synthetic voice model using the likeness of the avatar talent is simultaneously trained with the avatar. This voice is exclusively associated with the custom avatar and can't be independently used. For supported regions, see the [Speech service regions table](#).
- **Professional voice:** You can fine-tune a professional voice. [Professional voice fine-tuning](#) and custom text to speech avatar are separate features. You can use them independently or together. If you choose to use them together, you need to apply for [professional voice](#)

[fine-tuning](#) and [custom text to speech avatar](#) separately, and you're charged separately for professional voice fine-tuning and custom text to speech avatar. For more information, see the [pricing page](#). Additionally, if you plan to use [professional voice fine-tuning](#) with a text to speech avatar, you need to deploy or [copy your custom voice model](#) to one of the [avatar supported regions](#).

If you fine-tune a professional voice and want to use it together with the custom avatar, pay attention to the following points:

- Ensure that the custom voice endpoint is created in the same Azure AI Foundry resource as the custom avatar endpoint. As needed, refer to [train your professional voice model](#) to copy the custom voice model to the same Azure AI Foundry resource as the custom avatar endpoint.
- You can see the custom voice option in the voices list of the [avatar content generation page](#) and [live chat voice settings](#).
- If you're using batch synthesis for avatar API, add the `"customVoices"` property to associate the deployment ID of the custom voice model with the voice name in the request. For more information, see the [text to speech properties](#).
- If you're using real-time synthesis for avatar API, refer to our sample code on [GitHub](#) to set the custom voice.

Related content

- [How to create a custom text to speech avatar](#)
- [How to prepare custom text to speech avatar training data](#)
- [Real-time synthesis for live chat avatar](#)
- [Batch synthesis for video creation](#)
- [Transparency note for text to speech](#)

Last updated on 10/24/2025

Record video samples for custom text to speech avatar

08/07/2025

This article shows you how to prepare high-quality video samples for creating a custom text to speech avatar.

Custom text to speech avatar model building requires training on a video recording of a real human speaking. This person is the avatar talent. You must get sufficient consent under all relevant laws and regulations from the avatar talent to create a custom avatar from their talent's image or likeness. To learn about requirements of the consent statement video, see [Get consent file from the avatar talent](#).

Recording environment

Record in a professional video recording studio or well-lit space.

Background requirements

For commercial, multi-scene avatars, use a clean, smooth, solid-colored background. A green screen works best.

If your avatar will only be used in a single scene, you can record in a specific location like your office, but you can't change the background later.

Follow these best practices when using a solid-colored background like a green screen:

- Position the green screen behind the actor. For full-body shots, place a green screen on the floor under the actor's feet. Connect the back and floor green screens seamlessly.
- Keep the green screen flat with uniform color.
- Maintain 0.5-1 meter distance between the actor and background.
- Light the green screen properly to prevent shadows.
- Keep the actor's full outline within the green screen edges.
- Don't let the actor stand too close to the green screen.
- Keep the actor's head and hands within the green screen when speaking.

Lighting requirements

- Use even, bright lighting on the actor's face. Avoid shadows on the face or reflections on glasses and clothing.

- Keep ambient lighting consistent. Turn off projectors, close curtains to avoid daylight changes, and use stable artificial light sources.

Equipment

- Camera: Minimum 1080p resolution and 25 FPS (frames per second).
- Keep lighting and camera positions fixed throughout recording.
- You can use a teleprompter during recording, but make sure it doesn't affect the actor's gaze toward the camera. Provide seating if the avatar needs to be in a sitting position.
- For half-length or seated avatars, provide seating for the actor. Choose an appropriate chair if you don't want it visible in the video.

Appearance of the actor

Custom text to speech avatar doesn't support customization of clothing or appearance. It's essential to carefully design and prepare the avatar's appearance when recording training data. Consider these tips:

 Expand table

Categories	Dos	Don'ts
Hair	- The actor's hair should have a smooth and glossy surface. - Even the actor's bangs or broken hair should have a clear and smooth border. - Choose a hairstyle that is easy to keep consistent during the whole video recording.	- Avoid messy hair or backgrounds showing through the hair. - Don't let hair block the eyes or eyebrows. - Avoid shadows on the face caused by hairstyle. - Avoid hair changes too much during speech and body gesture. For example, the high ponytail of an actor might appear, disappear, and swing during speaking.
Clothing	- Pay attention to clothing status and make sure no significant changes on the clothing during speaking.	- Avoid wearing clothing and accessories that are too loose, heavy, or complex, as they might affect the consistency of clothing status during speaking and body gesture. - Avoid wearing clothing that is too similar to the background color or reflective materials like white shirts or translucent materials. - Avoid clothing with obvious lines or items with logos and brand names you don't want to highlight. - Avoid reflective elements such as metal belts, shiny leather shoes, and leather pants.
Face	- Ensure the actor's face is clearly visible.	- Avoid face obscured by hair, sunglasses, or accessories.

What video clips to record

You need these types of video clips:

Consent Video (Required) The consent video is required for creating a custom avatar.

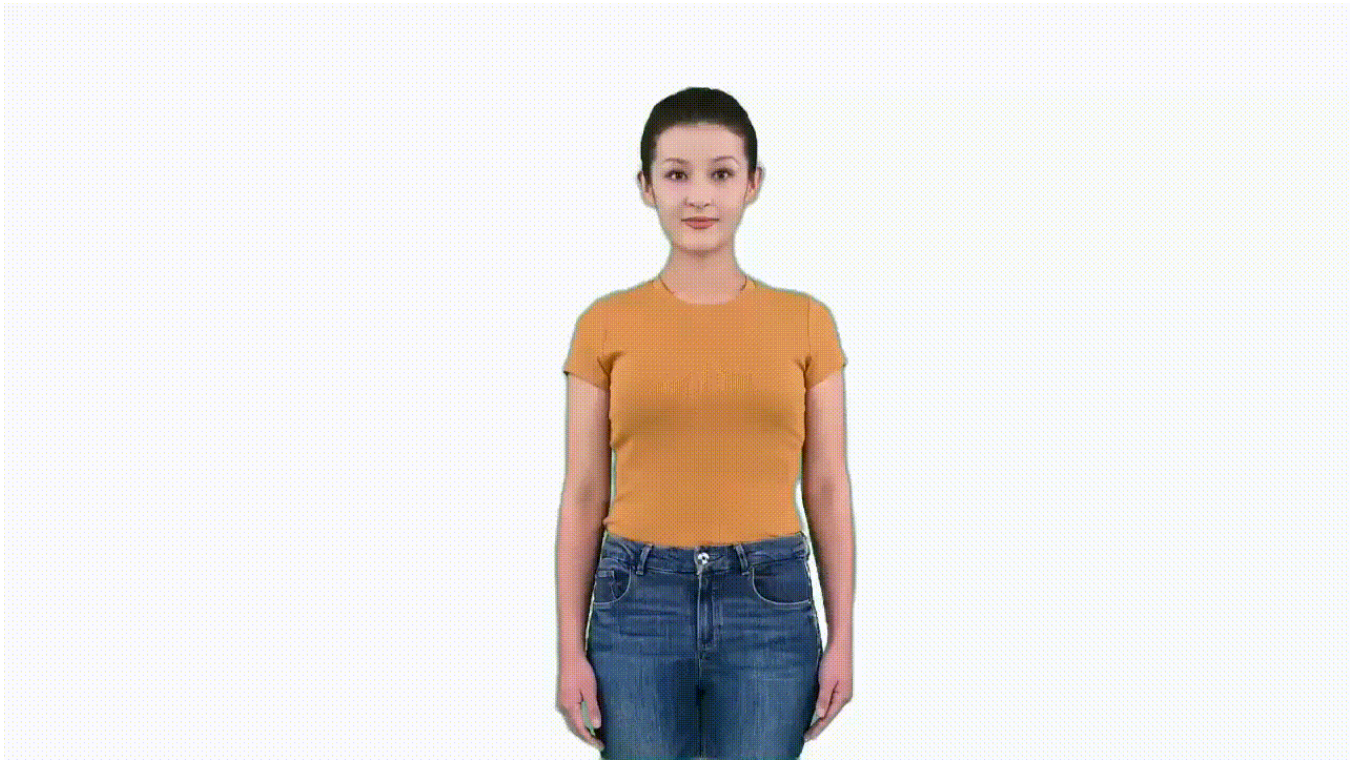
- The consent video must show the same avatar talent speaking and follow the consent statement requirements. Make sure the statement is recorded correctly with each word spoken clearly. You can use any supported language. To learn about consent statement video requirements, see [Get consent file from the avatar talent](#).
- The avatar talent should always face the camera without large movements.
- Record the video in a quiet environment with clear audio at reasonable volume. Keep the signal-to-noise ratio above 20. For voice recording guidance, see the [Recording custom voice samples](#) guide.
- Make sure the actor's head isn't blocked in any frame.
- Keep other objects out of the camera view, including filming equipment and mobile phones.

Status 0 speaking (Required for gestures) The status 0 speaking video clip is required for gestures with the avatar.

- Status 0 represents the posture you can naturally maintain most of the time while speaking. For example, arms crossed in front of the body or hanging naturally at the sides.
- Maintain a front-facing pose. The actor can move slightly to show a relaxed state, like moving the head or shoulder slightly, but don't move the body too much.
- Duration: 3-5 minutes of speaking in status 0.

Samples of status 0 speaking





Naturally speaking (Required) The naturally speaking video clip is required for the avatar to speak naturally.

- Actor speaks in status 0 but with natural hand gestures from time to time.
- Hands should start from status 0 and return after making gestures.
- Use natural and common gestures when speaking. Avoid meaningful gestures like pointing, applause, or thumbs up.
- Duration: Minimum 5 minutes, maximum 30 minutes total. At least one 5-minute continuous video recording is required. If recording multiple clips, keep each under 10 minutes.

Samples of natural speaking





Silent status (Required) The silent status video clip is required. It's important if you build a real-time conversation with the custom avatar. The video clip is used as the main template for both speaking and listening status for a chatbot.

- Maintain status 0, don't speak, but stay relaxed.
- Even while remaining in status 0, don't stay completely still. You can move slightly but not too much. Act like you're waiting.
- Maintain a smile as if listening or waiting patiently.
- Avoid nodding frequently.
- Duration: 1 minute.

Samples of silent status





Gestures (optional)

Gesture video clips are optional. If you need to insert certain gestures in the avatar speaking, follow this guideline to record gesture videos. Gesture insertion is only available for batch mode avatar; real-time avatar doesn't support gesture insertion. Each custom avatar model can support up to 10 gestures.

Gesture tips

- Each gesture clip should be within 10 seconds.
- Gestures should start from status 0 and end with status 0. It's essential that the character maintains the same position as in status 0, which is in the middle of the screen, throughout the gesture. Otherwise, the gesture clip can't be smoothly inserted into the avatar video.
- The gesture clip only captures the body gestures; the actor doesn't have to speak during making gestures.
- Design a list of gestures before recording. Here are some examples:

Samples of gesture

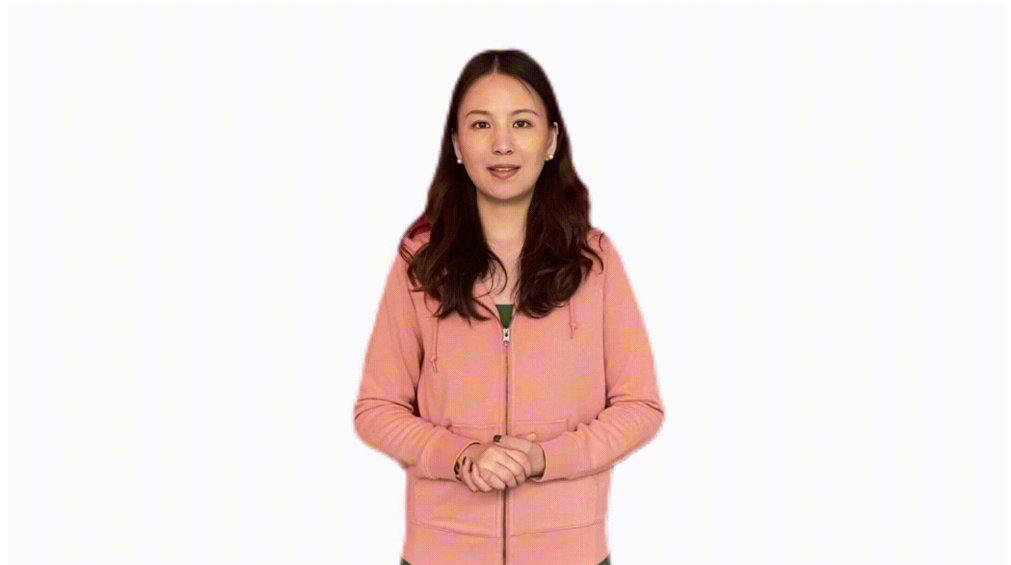
[Expand table](#)

Gestures**Samples**

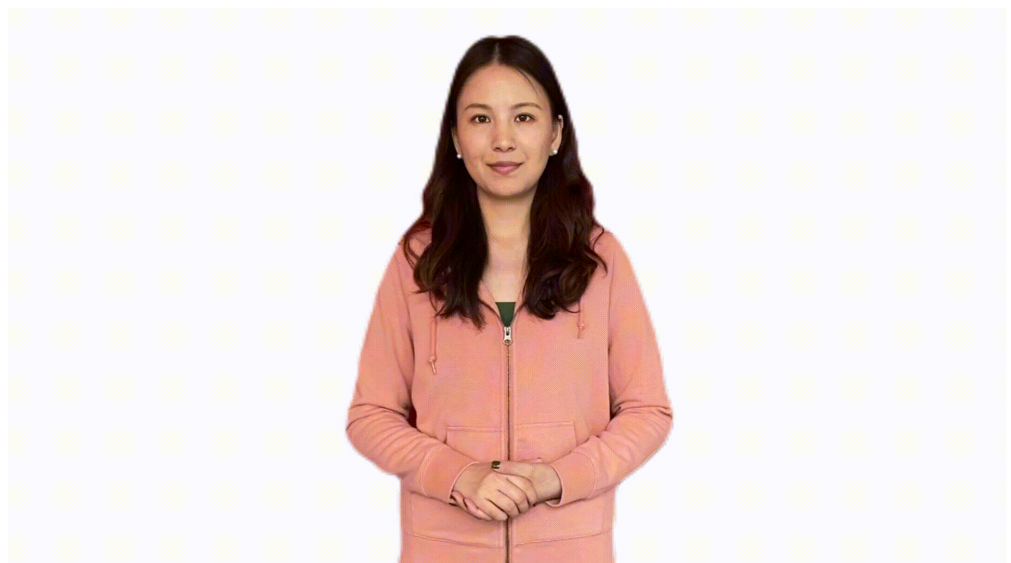
Delivering sell
link/promotion code



Praising the product



Introducing the
product



Gestures	Samples
Displaying the price (number from 1 to 10- fist-number with each hand)	Right hand  Left hand 

High-quality avatar models are built from high-quality video recordings, including audio quality. Here are more tips for actor's performance and recording video clips:

[Expand table](#)

Dos	Don'ts
- Ensure all video clips are taken in the same conditions. - During the recording process, design the size and display area of the character you need so that the character can be displayed on the screen appropriately. - Actor should be steady during the recording. - Mind facial expressions, which should be suitable for the avatar's use case. For example, look positive and smile if the custom text to speech avatar is used as customer service. Look professionally if the avatar is used	- Don't adjust the camera parameters, focal length, position, angle of view. Don't move the camera; keep the person's position, size, angle, consistent in the camera. - Characters that are too small might lead to a loss of image quality during post-processing. Characters that are too large might cause the screen to overflow during gestures and movements.

Dos	Don'ts
for news reporting. - Maintain eye gaze towards the camera, even when using a teleprompter. - Return your body to status 0 when pausing speaking. - Speak on a self-chosen topic, and minor speech mistakes like miss a word or mispronounced are acceptable. If the actor misses a word or mispronounces something, just go back to status 0, pause for 3 seconds, and then continue speaking. - Consciously pause between sentences and paragraphs. When pausing, go back to the status 0 and close your lips. - The audio should be clear and loud enough; bad audio quality impacts training result. - Keep the shooting environment quiet.	- Don't make too long gestures or too much movement for one gesture; for example, actor's hands are always making gestures and forget to go back to status 0. - The actor's movements and gestures must not block the face. - Avoid small movements of the actor like licking lips, touching hair, talking sideways, constant head shaking during speech, and not closing up after speaking. - Avoid background noise; staff should avoid walking and talking during video recording. - Avoid other people's voice recorded during the actor speaking.

How to prepare an interaction video clip

Creating a high-quality interaction video clip is essential if you're building a real-time conversation with a custom avatar. The clip should consist of a question-and-answer format, where a photographer asks a question, and the actor responds. Loop the question-answer pair until the conversation is complete. If you're filming alone, imagine someone else asking the questions during the asking phase.

Here are some tips for each phase:

Asking phase

- Maintain status 0, don't speak, but still feel relaxed.
- Even remaining in status 0, don't keep still. Perform like you're waiting.
- Maintain a smile as if listening or waiting patiently.
- Avoid nodding frequently.
- Length: Each asking slot should last around 3–5 seconds.

Answering phase

- Speak naturally with natural hand gestures from time to time.
- Use natural and common gestures when speaking. Avoid meaningful gestures like pointing, applause, or thumbs up.
- Begin gestures after starting to speak, and stop them before you finish.
- Length: Each answering slot should last around 5 seconds.

Total video length

- Aim for a total video length of 1–5 minutes.

Data requirements

Basic video processing helps improve model training efficiency:

- Keep the character centered on screen with consistent size and position throughout recording. Keep all video processing parameters like brightness and contrast consistent. The output avatar's size, position, brightness, and contrast will directly reflect those in the training data. We don't apply alterations during processing or model building.
- Start and end clips in status 0. Actors should close their mouths, smile, and look ahead. The video should be continuous, not abrupt.

Avatar training video recording file format: .mp4 or .mov.

Resolution: At least 1920x1080.

Frame rate per second: At least 25 FPS.

Related content

- [What is text to speech avatar](#)
- [What is custom text to speech avatar](#)

How to create a custom text to speech avatar

08/07/2025

Getting started with a custom text to speech avatar is a straightforward process. All it takes are a few video clips of your actor. If you'd like to train a [custom voice](#) for the same actor, you can do so separately.

ⓘ Note

Custom avatar access is limited based on eligibility and usage criteria. Request access on the [intake form](#) [↗].

Prerequisites

You need an Azure AI Foundry resource in one of the [regions that supports custom avatar training](#). Custom avatar only supports standard (S0) AI Foundry or Speech resources.

You need a video recording of the talent reading a consent statement acknowledging the use of their image and voice. You upload this video when you set up the avatar talent. For more information, see [Add avatar talent consent](#).

You need video recordings of your avatar talent as training data. You upload these videos when you prepare training data. For more information, see [Add training data](#).

Step 1: Start fine-tuning

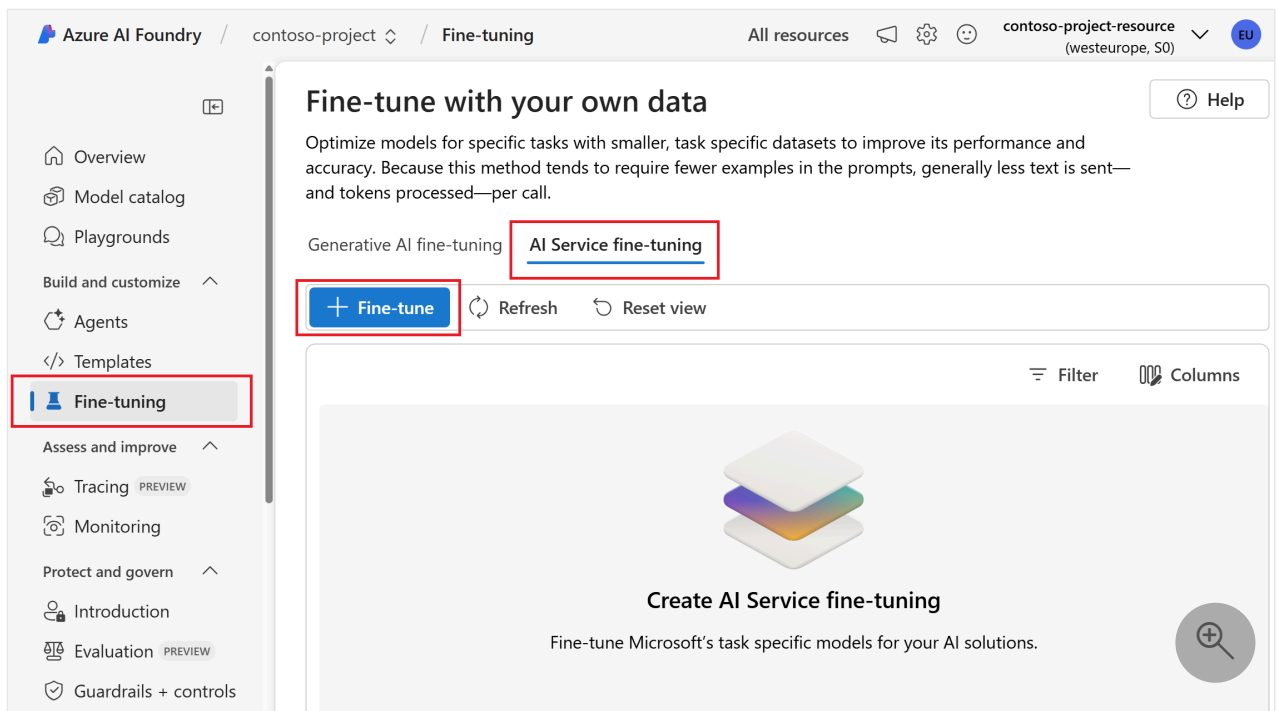
💡 Tip

Don't mix data for different avatars in one fine-tuning workspace. Each avatar must have its own fine-tuning workspace.

To fine-tune a custom avatar, follow these steps:

1. Go to your Azure AI Foundry project in the [Azure AI Foundry portal](#) [↗]. If you need to create a project, see [Create an Azure AI Foundry project](#).
2. Select **Fine-tuning** from the left pane.

3. Select **AI Service fine-tuning** > **+ Fine-tune**.



4. In the wizard, select **Custom avatar (text to speech avatar fine-tuning)**.

5. Select **Next**.

6. Follow the instructions provided by the wizard to create your fine-tuning workspace.

Step 2: Add avatar talent consent

An avatar talent is an individual or target actor whose video of speaking is recorded and used to create neural avatar models. You must obtain sufficient consent under all relevant laws and regulations from the avatar talent to use their video to create the custom text to speech avatar.

You must provide a video file with a recorded statement from your avatar talent, acknowledging the use of their image and voice. Microsoft verifies that the content in the recording matches the predefined script provided by Microsoft. Microsoft compares the face of the avatar talent in the recorded video statement file with randomized videos from the training datasets to ensure that the avatar talent in video recordings and the avatar talent in the statement video file are from the same person.

- If you want to create a voice sync for avatar during avatar training, a custom voice resembling your avatar is created alongside the custom avatar. The voice is used exclusively with the specified avatar. Your consent statement must include both the custom avatar and the voice sync for avatar. For an example of the consent statement for custom avatar with voice sync, see the *verbal-statement-voice-sync-for-avatar-all-locales.txt* file in the [Azure-Samples/cognitive-services-speech-sdk](#) GitHub repository.

- If you don't create a voice sync for avatar, only the custom avatar is trained, and your consent statement must reflect this scope. For an example of the consent statement for custom avatar only, see the *verbal-statement-all-locales.txt* file in the [Azure-Samples/cognitive-services-speech-sdk](#) [GitHub repository](#).

For more information about recording the consent video, see [How to record video samples](#) and [Disclosure for avatar talent](#).

To add an avatar talent profile and upload their consent statement in your project, follow these steps:

1. Sign in to the [Azure AI Foundry portal](#).
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the custom avatar fine-tuning task (by model name) that you [started as described in the previous section](#).
4. Select **Set up avatar talent > Upload consent video**.
5. On the **Upload consent video** page, follow the instructions to upload the avatar talent consent video you recorded beforehand.
 - Select the avatar type to build. Build a voice sync for avatar, which sounds like your avatar talent together with the avatar model, or build avatar without the voice sync for avatar. The option to build a voice sync for avatar is only available in the Southeast Asia, West Europe, and West US 2 regions.
 - Select the speaking language of the verbal consent statement recorded by the avatar talent.
 - Enter the avatar talent name and your company name in the same language as the recorded statement.
 - The avatar talent name must be the name of the person who recorded the consent statement.
 - The company name must match the company name that was spoken in the recorded statement.
 - You can choose to upload your data from local files, or from a shared storage with Azure Blob.
6. Select local files from your computer or enter the Azure Blob storage URL where your data is stored.
7. Select **Next**.
8. Review the upload details, and select **Upload**.

After the avatar talent consent upload is successful, you can proceed to train your custom avatar model.

Step 3: Add training data

The Speech service uses your training data to create a unique avatar tuned to match the look of the person in the recordings. After you train the avatar model, you can start synthesizing avatar videos or use it for live chats in your applications.

All data you upload must meet the requirements for the data type that you choose. To ensure that the Speech service accurately processes your data, it's important to correctly format your data before upload. To confirm that your data is correctly formatted, see [Data requirements](#).

Upload your data

When you're ready to upload your data, go to the **Prepare training data** tab to add your data.

To upload training data, follow these steps:

1. Sign in to the [Azure AI Foundry portal](#).
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the custom avatar fine-tuning task (by model name) that you [started as described in the previous section](#).
4. Select **Prepare training data > Upload data**.
5. In the **Upload data** wizard, choose a data type and then select **Next**. For more information about the data types (including **Naturally Speaking**, **Silent Status**, **Gesture**, and **Status 0 speaking**), see [what video clips to record](#).
6. Select local files from your computer or enter the Azure Blob storage URL where your data is stored.
7. Select **Next**.
8. Review the upload details, and select **Upload**.

Data files are automatically validated when you select **Upload**. Data validation includes series of checks on the video files to verify their file format, size, and total volume. If there are any errors, fix them and submit again.

After you upload the data, you can check the data overview, which indicates whether you provided enough data to start training.

Step 4: Train your avatar model

Important

All the training data in the project is included in the training. The model quality is highly dependent on the data you provided, and you're responsible for the video quality. Make sure you record the training videos according to the [how to record video samples guide](#).

To create a custom avatar in the Azure AI Foundry portal, follow these steps for one of the following methods:

1. Sign in to the [Azure AI Foundry portal](#) .
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the custom avatar fine-tuning task (by model name) that you [started as described in the previous section](#).
4. Select **Train model** > **+ Train model**.
5. Enter a Name to help you identify the model. Choose a name carefully. The model name is used as the avatar name in your synthesis request by the SDK and speech synthesis markup language (SSML) input. Only letters, numbers, hyphens, and underscores are allowed. Use a unique name for each model.

Important

The avatar model name must be unique within the same Speech or AI Services resource.

6. Select **Train** to start training the model.

Training duration varies depending on how much data you use. It normally takes 20-40 compute hours on average to train a custom avatar. Check the [pricing note](#)  on how training is charged.

Copy your custom avatar model to another project (optional)

Custom avatar training is currently only available in some regions. After your avatar model is trained in a supported region, you can copy it to an AI Services resource for Speech in another region as needed. For more information, see footnotes in the [regions table](#).

Note

You can only copy the voice sync for avatar model to the regions that support the voice sync for avatar feature, which are the same regions that support personal voice.

To copy your custom avatar model to another project:

1. On the **Train model** tab, select an avatar model that you want to copy, and then select **Copy to project**.
2. Select the subscription, region, AI Services resource for Speech, and project where you want to copy the model to. You must have an AI Services resource for Speech and project in the target region, otherwise you need to create them first.
3. Select **Submit** to copy the model.

Once the model is copied, you see a notification in the Azure AI Foundry portal.

Navigate to the project where you copied the model to deploy the model copy.

Step 5: Deploy and use your avatar model

After you successfully created and trained your avatar model, you deploy it to your endpoint.

To deploy your avatar:

1. Sign in to the [Azure AI Foundry portal](#).
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the custom avatar fine-tuning task (by model name) that you [started as described in the previous section](#).
4. Select **Deploy model** > **Deploy model**.
5. Select a model that you want to deploy.
6. Select **Deploy** to start the deployment.

Important

When a model is deployed, you pay for continuous up time of the endpoint regardless of your interaction with that endpoint. Check the pricing note on how model deployment is charged. You can delete a deployment when the model isn't in use to reduce spending and conserve resources.

After you deploy your custom avatar, it's available to use in the Azure AI Foundry portal or via API:

- The avatar appears in the avatar list of [text to speech avatar on Azure AI Foundry portal](#) [↗].
- The avatar appears in the avatar list of [live chat avatars via Azure AI Foundry portal](#) [↗].
- You can call the avatar from the SDK and SSML input by specifying the avatar model name. For more information, see the [avatar properties](#).

Remove a deployment

To remove your deployment, follow these steps:

1. Sign in to the [Azure AI Foundry portal](#) [↗].
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the custom avatar fine-tuning task (by model name) that you [started as described in the previous section](#).
4. Select the deployment on the **Deploy model** page. The model is actively hosted if the status is "Succeeded".
5. You can select the **Delete deployment** button and confirm the deletion to remove the hosting.

Tip

Once a deployment is removed, you no longer pay for its hosting. Deleting a deployment doesn't cause any deletion of your model. If you want to use the model again, create a new deployment.

Next steps

- [What is text to speech avatar](#)
- [How to record video samples](#)

Text to speech with the audio content creation tool

08/05/2025

You can use the audio content creation tool in [Azure AI Foundry portal](#) or [Speech Studio](#) for text to speech without writing any code.

Tip

Select **Foundry portal** or **Speech Studio** at the top of this article.

Build highly natural audio content for various scenarios, such as audiobooks, news broadcasts, video narrations, and chat bots. With audio content creation, you can efficiently fine-tune text to speech voices and design customized audio experiences.

The tool is based on [Speech Synthesis Markup Language \(SSML\)](#). It allows you to adjust text to speech output attributes in real-time or batch synthesis, such as voice characters, voice styles, speaking speed, pronunciation, and prosody.

- No-code approach: You can use the audio content creation tool for text to speech synthesis without writing any code. The output audio might be the final deliverable that you want. For example, you can use the output audio for a podcast or a video narration.
- Developer-friendly: You can listen to the output audio and adjust the SSML to improve speech synthesis. Then you can use the [Speech SDK](#) or [Speech CLI](#) to integrate the SSML into your applications.

You have easy access to a broad portfolio of [languages and voices](#). These voices include state-of-the-art standard voices and your custom voice, if you built one.

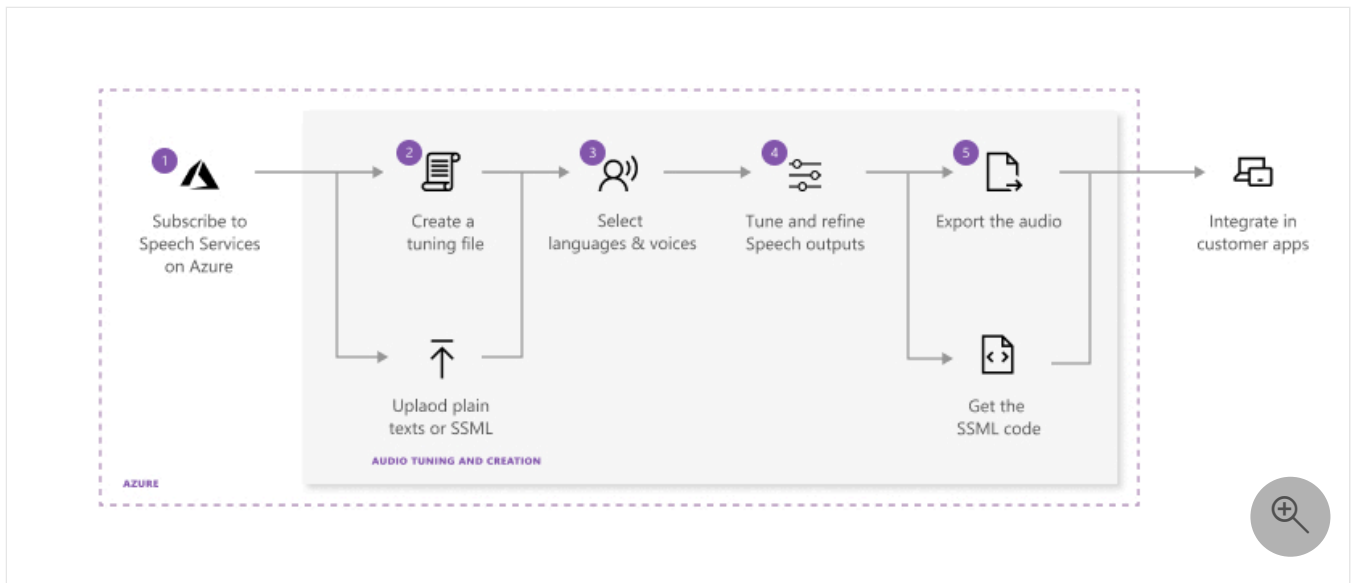
The audio content creation tool is free to access; you pay only for Speech service usage.

Prerequisites

- An active Azure subscription. [Create one for free](#).
- Permission to create resources in your subscription.
- An Azure AI Foundry project. For more information, see [Create an Azure AI Foundry project](#).

Use the audio content creation tool

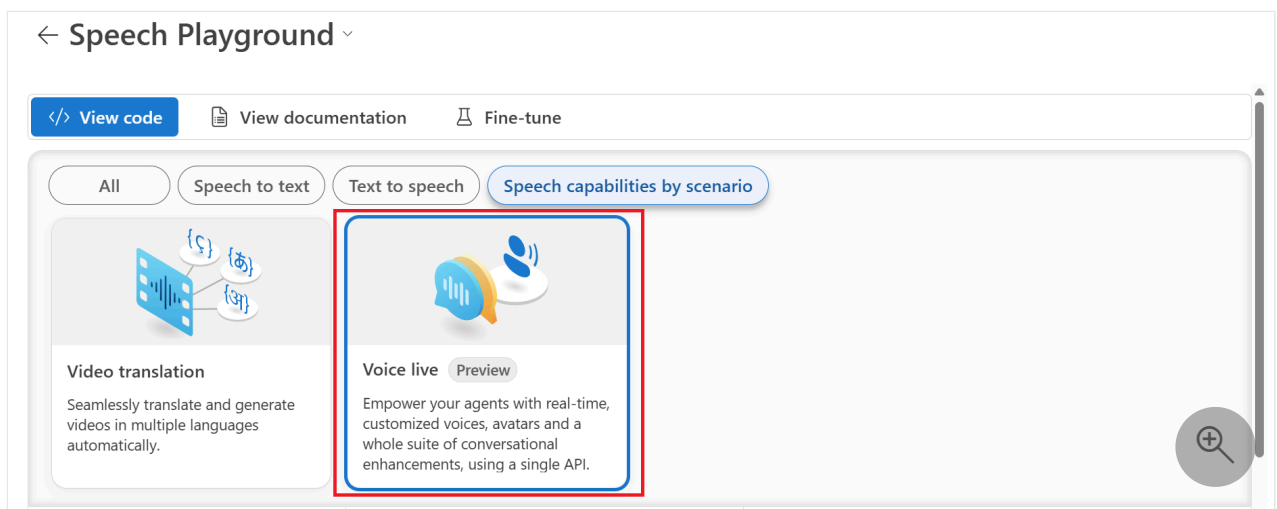
The following diagram displays the process for fine-tuning the text to speech outputs.



Access the tool

To access the audio content creation tool in Azure AI Foundry, follow these steps:

1. Go to your project in [Azure AI Foundry](#).
2. Select **Playgrounds** from the left pane.
3. In the **Speech playground** tile, select **Try the Speech playground**.
4. Select **Text to speech** > **Audio content creation**. You might need to scroll to find the tile.



Workflow overview

Once you have access to the tool, follow this general workflow:

1. [Create an audio tuning file](#) by using plain text or SSML scripts. Enter or upload your content into audio content creation.
2. Choose the voice and the language for your script content. Audio content creation includes all of the [standard text to speech voices](#). You can use standard voices or a custom voice.

ⓘ **Note**

Custom voice access is [limited](#) based on eligibility and usage criteria. Request access on the [intake form](#) [↗](#).

3. Select the content you want to preview, and then select **Play** (via the triangle icon) to preview the default synthesis output.

If you make any changes to the text, select the **Stop** icon, and then select **Play** again to regenerate the audio with changed scripts.

Improve the output by adjusting pronunciation, break, pitch, rate, intonation, voice style, and more. For a complete list of options, see [Speech Synthesis Markup Language](#).

4. Save and [export your tuned audio](#).

When you save the tuning track in the system, you can continue to work and iterate on the output. When you're satisfied with the output, you can create an audio creation task with the export feature. You can observe the status of the export task and download the output for use with your apps and products.

Create an audio tuning file

You can get your content into the audio content creation tool in either of two ways:

Option 1: Create a new audio tuning file

1. Select **New > Text file** to create a new audio tuning file.
2. Enter or paste your content into the editing window. The allowable number of characters for each file is 20,000 or fewer. If your script contains more than 20,000 characters, you can use Option 2 to automatically split your content into multiple files.
3. Select **Save**.

Option 2: Upload an audio tuning file

1. Select **Upload** > **Text file** to import one or more text files. Both plain text and SSML are supported.

If your script file is more than 20,000 characters, split the content by paragraphs, by characters, or by regular expressions.

2. When you upload your text files, make sure that they meet these requirements:

 Expand table

Property	Description
File format	Plain text (.txt) or SSML text (.txt) Zip files aren't supported.
Encoding format	UTF-8
File name	Each file must have a unique name. Duplicate files aren't supported.
Text length	Character limit is 20,000. If your files exceed the limit, split them according to the instructions in the tool.
SSML restrictions	Each SSML file can contain only a single piece of SSML.

Here's a plain text example:

txt

Welcome to use audio content creation to customize audio output for your products.

Here's an SSML example:

XML

```
<speak xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="http://www.w3.org/2001/mstts" version="1.0" xml:lang="en-US">
  <voice name="en-US-AvaMultilingualNeural">
    Welcome to use audio content creation <break time="10ms" />to customize
    audio output for your products.
  </voice>
</speak>
```

Export tuned audio

After you review your audio output and are satisfied with your tuning and adjustment, you can export the audio.

1. Select **Export** to create an audio creation task.

We recommend **Export to Audio library** to easily store, find, and search audio output in the cloud. You can better integrate with your applications through Azure blob storage. You can also download the audio to your local disk directly.

2. Choose the output format for your tuned audio. The **supported audio formats and sample rates** are listed in the following table:

 Expand table

Format	8 kHz sample rate	16 kHz sample rate	24 kHz sample rate	48 kHz sample rate
wav	riff-8khz-16bit-mono-pcm	riff-16khz-16bit-mono-pcm	riff-24khz-16bit-mono-pcm	riff-48khz-16bit-mono-pcm
mp3	N/A	audio-16khz-128kbitrate-mono-mp3	audio-24khz-160kbitrate-mono-mp3	audio-48khz-192kbitrate-mono-mp3

3. To view the status of the task, select the **Task list** tab.

If the task fails, see the detailed information page for a full report.

4. When the task is complete, your audio is available for download on the **Audio library** pane.
5. Select the file you want to download and **Download**.

Now you're ready to use your custom tuned audio in your apps or products.

Related content

- [Speech Synthesis Markup Language \(SSML\)](#)
- [Batch synthesis](#)

What are OpenAI text to speech voices?

Like Azure AI Speech voices, OpenAI text to speech voices deliver high-quality speech synthesis to convert written text into natural sounding spoken audio. This unlocks a wide range of possibilities for immersive and interactive user experiences.

OpenAI text to speech voices are available via two model variants: **Neural** and **NeuralHD**.

- **Neural**: Optimized for real-time use cases with the lowest latency, but lower quality than **NeuralHD**.
- **NeuralHD**: Optimized for quality.

Available text to speech voices in Azure AI services

You might ask: If I want to use an OpenAI text to speech voice, should I use it via the Azure OpenAI in Azure AI Foundry Models or via Azure AI Speech? What are the scenarios that guide me to use one or the other?

Each voice model offers distinct features and capabilities, allowing you to choose the one that best suits your specific needs. You want to understand the options and differences between available text to speech voices in Azure AI services.

You can choose from the following text to speech voices in Azure AI services:

- OpenAI text to speech voices in [Azure OpenAI](#). For the current list of supported regions, see the [Speech service regions table](#).
- OpenAI text to speech voices in [Azure AI Speech](#). For the current list of supported regions, see the [Speech service regions table](#).
- Azure AI Speech service [text to speech voices](#). Available in dozens of regions. See the [region list](#).

OpenAI text to speech voices via Azure OpenAI or via Azure AI Speech?

If you want to use OpenAI text to speech voices, you can choose whether to use them via [Azure OpenAI](#) or via [Azure AI Speech](#). You can visit the [Voice Gallery](#) to listen to samples of Azure OpenAI voices or synthesize speech with your own text using the [Audio Content Creation](#). The audio output is identical in both cases, with only a few feature differences between the two services. See the table below for details.

Here's a comparison of features between OpenAI text to speech voices in Azure OpenAI and OpenAI text to speech voices in Azure AI Speech.

 Expand table

Feature	Azure OpenAI (OpenAI voices)	Azure AI Speech (OpenAI voices)	Azure AI Speech voices
Region	North Central US, Sweden Central	North Central US, Sweden Central	Available in dozens of regions. See the region list .
Voice variety	6	12	More than 500
Multilingual voice number	6	12	49
Max multilingual language coverage	57	57	77
Speech Synthesis Markup Language (SSML) support	Not supported	Support for a subset of SSML elements .	Support for the full set of SSML in Azure AI Speech.
Development options	REST API	Speech SDK, Speech CLI, REST API	Speech SDK, Speech CLI, REST API
Deployment option	Cloud only	Cloud only	Cloud, embedded, hybrid, and containers.
Real-time or batch synthesis	Real-time	Real-time	Real-time and batch synthesis
Latency	greater than 500 ms	greater than 500 ms	less than 300 ms
Sample rate of synthesized audio	24 kHz	8, 16, 24, and 48 kHz	8, 16, 24, and 48 kHz
Speech output audio format	opus, mp3, aac, flac	opus, mp3, pcm, truesilk	opus, mp3, pcm, truesilk

There are additional features and capabilities available in Azure AI Speech that aren't available with OpenAI voices. For example:

- OpenAI text to speech voices in Azure AI Speech [only support a subset of SSML elements](#). Azure AI Speech voices support the full set of SSML elements.
- Azure AI Speech supports [word boundary events](#). OpenAI voices don't support word boundary events.

Available OpenAI text to speech voices

The available OpenAI voices in Azure OpenAI are:

- alloy
- echo
- fable
- onyx
- nova
- shimmer

The available OpenAI voices in Azure AI Speech are:

- en-US-AlloyMultilingualNeural
- en-US-EchoMultilingualNeural
- en-US-FableMultilingualNeural
- en-US-OnyxMultilingualNeural
- en-US-NovaMultilingualNeural
- en-US-ShimmerMultilingualNeural
- en-US-AlloyMultilingualNeuralHD
- en-US-EchoMultilingualNeuralHD
- en-US-FableMultilingualNeuralHD
- en-US-OnyxMultilingualNeuralHD
- en-US-NovaMultilingualNeuralHD
- en-US-ShimmerMultilingualNeuralHD

SSML elements supported by OpenAI text to speech voices in Azure AI Speech

The [Speech Synthesis Markup Language \(SSML\)](#) with input text determines the structure, content, and other characteristics of the text to speech output. For example, you can use SSML to define a paragraph, a sentence, a break or a pause, or silence. You can wrap text with event tags such as bookmark or viseme that can be processed later by your application.

The following table outlines the Speech Synthesis Markup Language (SSML) elements supported by OpenAI text to speech voices in Azure AI speech. Only the following subset of SSML tags are supported for OpenAI voices. See [SSML document structure and events](#) for more information.

SSML element name	Description
<code><say-as></code>	Encloses the entire content to be spoken. It's the root element of an SSML document.
<code><voice></code>	Specifies a voice used for text to speech output.
<code><sub></code>	Indicates that the alias attribute's text value should be pronounced instead of the element's enclosed text.
<code><say-as></code>	Indicates the content type, such as number or date, of the element's text. All of the <code>interpret-as</code> property values are supported for this element except <code>interpret-as="name"</code>. For example, <code><say-as interpret-as="date" format="dmy">10-12-2016</say-as></code> is supported, but <code><say-as interpret-as="name">ED</say-as></code> isn't supported. For more information, see pronunciation with SSML.
<code><s></code>	Denotes sentences.
<code><lang></code>	Indicates the default locale for the language that you want the neural voice to speak.
<code><break></code>	Use to override the default behavior of breaks or pauses between words.

Related content

- [Try the text to speech quickstart in Azure AI Speech](#)
- [Try the text to speech via Azure OpenAI](#)

Last updated on 10/24/2025

Text to speech FAQ

This article answers commonly asked questions about the text to speech (TTS) capability. If you can't find answers to your questions here, check out [other support options](#).

General

How does the billing work for text to speech?

Text to speech usage is billed per character. Check the definition of billable characters in the [pricing note](#).

What is the rate limit for the text to speech synthesis requests?

The text to speech synthesis rate scales automatically as it receives more requests. A default rate limit is set per speech resource. The rate is adjustable with business justifications and no extra charges are incurred for rate limit increase. Check more details in [Speech service quotas and limits](#).

How would we disclose to the end user that the voice is a synthetic voice?

We recommend that every user should follow our [code of conduct](#) when using the text to speech capability. There are several ways to disclose the synthetic nature of the voice including implicit and explicit byline. Refer to [Disclosure design guidelines](#).

How can I reduce the latency for my voice app?

We provide several tips for you to lower the latency and bring the best performance to your users. See [Lower speech synthesis latency using Speech SDK](#).

What output audio formats does text to speech support?

Azure AI text to speech supports various streaming and non-streaming audio formats, with the commonly used sampling rates. All TTS standard voices are created to support high-fidelity audio outputs with 48 kHz and 24 kHz. The audio can be resampled to support other rates as needed. See [Audio outputs](#).

Can the voice be customized to stress specific words?

Adjusting the emphasis is supported for some voices depending on the locale. See the [emphasis tag](#).

Can we have multiple strength for each emotion, like sad, slightly sad, and so on, in?

Adjusting the style degree is supported for some voices depending on the locale. See the [mstts:express-as tag](#).

Is there a mapping between Viseme IDs and mouth shape?

Yes. See [Get facial position with viseme](#).

Why am I getting HTTP 429 (Too Many Requests) errors when using Text-to-Speech Standard Voice?

The default quota of 200 transactions per second (TPS) for Text-to-Speech (TTS) Standard Voice is designed to accommodate the needs of most customers. In most cases, HTTP 429 errors are not caused by quota limitations, but by insufficient backend service capacity in the selected region for the specified voice. Increasing the quota does not resolve these capacity constraints. To address this issue effectively:

- Use native regions: Deploy the voice in a region where it is natively supported and better resourced (for example, use the Japan region for Japanese voices).
- Select popular voices: Choose a more commonly used voice within your current region to reduce the likelihood of hitting capacity limits.

Audio Content Creation

How can I reference a lexicon file that I created on the Audio Content Creation platform in my code?

First, you can open the lexicon file on the Audio Content Creation and obtain the lexicon file ID, which is located before "?fileKind=CustomLexiconFile" in the file path. For example, if the file

path is

`https://speech.microsoft.com/portal/d391a094f76846acbcd11dc2ba835f4f/audiocontentcreation/file/6cbc2527-8d57-4c1b-b9d9-3ea6d13ca95c?fileKind=CustomLexiconFile`, the lexicon file ID is `6cbc2527-8d57-4c1b-b9d9-3ea6d13ca95c`. Then, switch a file referencing this lexicon to SSML format on the Audio Content Creation. In the SSML file, locate the `<!--ID=FCB` xml node, where you can find the URI of the lexicon file based on the mentioned file ID. Finally, reference the lexicon file URI link using the SSML lexicon element in your code. For instance, if you locate the XML node `<!--ID=FCB5B6FB566-33CA-4B68-BEAF-B013C53B3368;Version=1|{"Files":{"6cbc2527-8d57-4c1b-b9d9-3ea6d13ca95c":`
`{"FileKind":"CustomLexiconFile","FileSubKind":"CustomLexiconFile","Uri":"https://cvoiceprodwus2.blob.core.windows.net/acc-public-files/d391a094f76846acbcd11dc2ba835f4f/e9a6a5a2-9cef-47f4-b961-d175be75d92f.xml"}}}`, you can obtain the lexicon file URI `https://cvoiceprodwus2.blob.core.windows.net/acc-public-files/d391a094f76846acbcd11dc2ba835f4f/e9a6a5a2-9cef-47f4-b961-d175be75d92f.xml`.

Professional voice fine-tuning

How much data is required for professional voice fine-tuning?

You need training data of at least 300 lines of recordings (or approximately 30 minutes of speech) for professional voice fine-tuning. We recommend 2,000 lines of recordings (or approximately 2-3 hours of speech) to create a voice for production use. For the script selection criteria, see [Record custom voice samples](#).

Can we include duplicate text sentences in the same set of training data?

No. The service will flag the duplicate sentences and just keep the first imported one. For the script selection criteria, see [Record custom voice samples](#).

Can we include multiple styles in the same set of training data?

We recommend that you keep the style consistent in one set of training data. If the styles are different, put them into different training sets. In this case, consider using the multi-style training method of professional voice fine-tuning. For the script selection criteria, see [Record custom voice samples](#).

Does switching styles via SSML work for custom voices?

Switching styles via SSML works for both multi-style standard voices and multi-style custom voices. With multi-style training, you can create a voice that speaks in different styles, and you can also adjust these styles via SSML.

How does cross-lingual voice work with languages that have different pronunciation structure and assembly?

Sentence structure and pronunciation naturally vary across languages such as English and Japanese. Each neural voice is trained with audio data recorded by native speaking voice talent. For [cross lingual](#) voice, we transfer the major features like timbre to sound like the original speaker and preserve the right pronunciation. For example, a cross-lingual voice uses the native way to speak Japanese and still sounds similar (but not exactly) like the original English speaker.

Can I use professional voice fine-tuning to customize pronunciation for my domain?

Professional voice fine-tuning enables you to create a brand voice for your business. You can optimize it for your domain as well. We recommend you include domain-specific samples in your training data for higher naturalness. However, the pronunciation is defined by the Speech service by default. We don't support pronunciation customization with professional voice fine-tuning. If you want to customize pronunciation for your voice, use SSML. See [Pronunciation with Speech Synthesis Markup Language \(SSML\)](#).

After one training can I train my voice again?

You can train again. Each training creates a new voice model. You are charged for each training.

Is the model version the same as the engine version?

No. The model version is different from the engine version. The model version means the version of the training recipe for your model and varies by the features supported and model training time. Azure AI services text to speech engines are updated from time to time to capture the latest language model that defines the pronunciation of the language. After you've trained your voice, you can apply your voice to the new language model by updating to the